

Apuntes de Algoritmia

Erwin Meza Vega

Índice general

Índice general	I
Índice de figuras	III
Índice de código	V
Prefacio	IX
Introducción	XIII
1 Conceptos básicos	1
1.1. Concepto de algoritmo	1
1.2. Tipos de algoritmos	2
1.3. Representación de algoritmos	4
1.4. Programa de computador	6
1.5. La complejidad en el desarrollo de software	7
1.6. Principios para la construcción de software	11
1.7. Pasos para la construcción de un algoritmo	12
1.8. Lecturas recomendadas	23
2 Estructuras básicas para construcción de algoritmos	25
2.1. Instrucciones	25
2.2. Variables, tipos de datos y representación	32
2.3. Condiciones	41
2.4. Elementos de control de flujo	43
3 Ejercicios de fundamentación	73
3.1. Instrucciones y estructuras de decisión	73
3.2. Estructuras repetitivas y estructuras anidadas	86

3.3.	Subrutinas	109
3.4.	Ejercicios propuestos	140
4	Arreglos	143
4.1.	Inicialización de arreglos	143
4.2.	Búsqueda y selección de elementos	148
4.3.	Inserción de elementos	158
4.4.	Eliminación de elementos	164
4.5.	Ejercicios propuestos	167
5	Cadenas de caracteres	171
5.1.	Inicialización de cadenas	171
5.2.	Consulta, selección y extracción de datos	177
5.3.	Algoritmos generales	187
5.4.	Ejercicios propuestos	196
6	Ordenamiento	199
6.1.	Ordenamiento por selección (Selection sort)	200
6.2.	Ordenamiento por inserción (Insertion Sort)	202
6.3.	Shellsort	205
6.4.	Ordenamiento por mezcla (Mergesort)	208
6.5.	Ordenamiento rápido (Quicksort)	216
6.6.	Ejercicios propuestos	222
	Bibliografía	225
	Índice analítico	227

Índice de figuras

1.1. Diagrama conceptual de un algoritmo	1
1.2. Diagrama de flujo suma de dos números	18
1.3. Diagrama N-S suma de dos números	18
1.4. Compilar el programa en lenguaje C	20
1.5. Compilar el programa en C usando un comando	21
1.6. Archivo <i>fuentes</i> y <i>ejecutable</i> de un programa	21
1.7. Iniciar ejecución del programa	21
1.8. Programa con interfaz gráfica	22
1.9. Ejecución del programa en Python	23
2.1. Instrucción de salida por pantalla	26
2.2. Hola mundo en diagrama de flujo	27
2.3. Instrucción de lectura de datos	28
2.4. Entrada y salida de datos	29
2.5. Diagrama de flujo asignaciones	31
2.6. Representación de los tipos simples de datos	33
2.7. Arreglo unidimensional de números enteros	36
2.8. Arreglo bidimensional de números enteros	37
2.9. Estructura de decisión en diagrama de flujo	45
2.10. Estructura de decisión anidada	47
2.11. Estructura de selección	49
2.12. Estructura <i>mientras hacer</i>	54
2.13. Estructura <i>hacer mientras</i>	55
2.14. Estructura <i>para hacer</i>	59
3.1. Intercambio de dos valores	75
3.2. Distancia euclidiana entre dos puntos	139
4.1. Arreglo con índices almacenados en su posición	144

4.2. Comparación de todos los elemento de un arreglo	156
4.3. Insertar un elemento al final de un arreglo	159
4.4. Insertar en una posición dada	162
4.5. Eliminar un elemento de un arreglo	165
5.1. Construir la cadena inversa	187
5.2. Dividir una cadena por un delimitador	192
6.1. Ordenamiento de datos	199
6.2. Ordenamiento por selección - primera iteración	200
6.3. Ordenamiento por selección - segunda iteración	200
6.4. Todas las iteraciones del ordenamiento por selección	201
6.5. Funcionamiento general de <i>insertion sort</i>	203
6.6. Primera iteración de <i>insertion sort</i>	203
6.7. Segunda iteración de <i>insertion sort</i>	203
6.8. Tercera iteración de <i>insertion sort</i>	204
6.9. Quinta iteración de <i>insertion sort</i>	204
6.10. Primera pasada de <i>shellsort</i>	207
6.11. Mezcla de dos secuencias ordenadas	208
6.12. Mezcla de dos subsecuencias	210
6.13. Pivote en <i>quicksort</i>	216
6.14. Movimientos de los índices en la partición	217
6.15. Intercambio de valores en la partición	217
6.16. Siguiente movimiento de los índices en la partición	218
6.17. Siguiente intercambio de valores en la partición	218
6.18. Cruce de índices en la partición	218
6.19. Mover pivote a su posición	219
6.20. Mover pivote a su posición con un intercambio	219

Índice de código

1.1.	Suma de dos números	16
1.2.	Ejemplo de programa en lenguaje C	20
1.3.	Suma de dos números en C#	22
1.4.	Ejemplo de programa en Python	23
2.1.	Instrucción de salida por pantalla	26
2.2.	Algoritmo Hola mundo	26
2.3.	Instrucción para lectura de datos	27
2.4.	Ejemplo de entrada de datos	28
2.5.	Ejemplos de asignación	30
2.6.	Ejemplo de incremento	31
2.7.	Ejemplo de decremento	32
2.8.	Definición de un tipo compuesto de datos	39
2.9.	Instrucciones de manejo de referencias	40
2.10.	Forma general de la estructura de decisión	45
2.11.	Verificar si un número es mayor que cero	45
2.12.	Estructura de decisión anidada	48
2.13.	Estructura de decisión con una sola secuencia	48
2.14.	Forma general de la estructura de selección	49
2.15.	Ejemplo de estructura de selección	50
2.16.	Forma general de la estructura <i>mientras hacer</i>	50
2.17.	Imprimir los números de 1 a 10	51
2.18.	Imprimir números de 1 a 10, pre-incremento	53
2.19.	Forma general de la estructura <i>hacer mientras</i>	53
2.20.	Validar entrada de datos	55
2.21.	Estructura <i>repetir hasta</i> para validar datos	56
2.22.	Forma general de la estructura <i>para hacer</i>	57
2.23.	Imprimir números de 1 a 10 usando <i>para hacer</i>	57
2.24.	Ejemplo de estructuras repetitivas anidadas	59
2.25.	Ejemplo de uso de la instrucción cancelar	60

2.26. Ejemplo de uso de la instrucción terminar	61
2.27. Ejemplo de uso instrucción continuar	63
2.28. Ejemplo de definición y uso de subrutinas	66
2.29. Ejemplo de subrutina para sumar dos números	68
2.30. Paso de parámetros por valor	70
2.31. Paso de parámetros por referencia	71
3.1. Intercambio de valores	75
3.2. Función para intercambio de valores	76
3.3. Comparación entre dos números	78
3.4. Determinar si tres números son iguales, condiciones simples	79
3.5. Determinar si tres números son iguales, condiciones compuestas	80
3.6. Promedio de dos números	81
3.7. Determinar si un número es divisor de otro	83
3.8. Determinar si un número es par	84
3.9. Solución alternativa para verificar si un número es par	84
3.10. Valor de la compra	86
3.11. Secuencia de números desde 1 hasta n	87
3.12. Suma de los primeros n números naturales	89
3.13. Leer n valores usando una estructura repetitiva	92
3.14. Determinar si n números son iguales	94
3.15. Determinar si n números son iguales usando variable booleana	95
3.16. Verificar si n números son pares	97
3.17. Verificar si un número dado es primo	99
3.18. Primeros 20 números de la sucesión de Fibonacci	101
3.19. Promedio de n números	103
3.20. Calculadora simple	106
3.21. Solo mayores de edad	107
3.22. Suma de números pares	109
3.23. Factorial iterativo	110
3.24. Factorial iterativo, usando funciones	111
3.25. Factorial usando función recursiva	111
3.26. Potencia entera positiva usando iteraciones	113
3.27. Potencia usando una función recursiva	114
3.28. Cálculo iterativo de π	116
3.29. Cálculo iterativo de π usando una función	116
3.30. Máximo común divisor	119
3.31. Máximo común divisor usando funciones	120
3.32. Algoritmo de Euclides	121

3.33. Algoritmo de Euclides en versión recursiva	121
3.34. Cálculo de la nota definitiva de un estudiante	123
3.35. Cálculo de la nota definitiva de varios estudiantes	124
3.36. Cálculo de la nota definitiva, sin validar datos	125
3.37. Validar una nota ingresada por el usuario	125
3.38. Cálculo de la nota definitiva, validando datos	126
3.39. Estudiantes con nota final menor que el promedio	134
3.40. Estudiantes con alguna nota menor que el promedio	137
3.41. Distancia euclidiana entre dos puntos	140
4.1. Secuencia de asignaciones elemento en su posición	144
4.2. Cada índice en su posición de un arreglo	144
4.3. Estructura repetitiva anidada para inicializar una matriz	147
4.4. Estructura repetitiva simple para inicializar una matriz	147
4.5. Inicializar una matriz con una sola estructura repetitiva	148
4.6. Búsqueda de un elemento en un arreglo	149
4.7. Primera posición de un elemento en un arreglo	150
4.8. Imprimir el mayor y el menor dato en un arreglo	152
4.9. Promedio de los datos de un arreglo	153
4.10. Imprimir el par de puntos con menor distancia entre sí	157
4.11. Insertar un elemento al final del arreglo	159
4.12. Generar un arreglo de números proporcionados por el usuario	159
4.13. Insertar al final de un arreglo de tamaño limitado	161
4.14. Insertar un elemento en una posición de un arreglo	164
4.15. Eliminar un elemento de un arreglo	166
5.1. Saludar al usuario	172
5.2. Crea una cadena filtrando caracteres	174
5.3. Concatenar dos cadenas	176
5.4. Calcular la longitud de una cadena terminada en nulo	178
5.5. Primera ocurrencia de un caracter en una cadena	180
5.6. Encontrar la primera ocurrencia de una subcadena	183
5.7. Encontrar subcadena	183
5.8. Verificar paréntesis balanceados	186
5.9. Función para obtener la cadena inversa	188
5.10. Eliminar espacios iniciales y finales	190
5.11. Dividir una cadena por un delimitador	193
5.12. Transformar un número en cadena de caracteres	195
6.1. Ordenamiento por selección	201
6.2. Algoritmo <i>insertion sort</i>	205

6.3. Paso de salto para <i>shellsort</i>	206
6.4. Pseudocódigo de shellsort	207
6.5. Mezcla de dos secuencias ordenadas	209
6.6. Mezcla de subsecuencias en el mismo arreglo	210
6.7. Pseudocódigo recursivo para mergesort	211
6.8. Mergesort iterativo	214
6.9. Función de particionado para quicksort	220
6.10. Pseudocódigo para el algoritmo quicksort	221

Prefacio

*No escuchar no es tan bueno como escuchar,
escuchar no es tan bueno como observar.
Observar no es tan bueno como saber,
saber no es tan bueno como actuar.*

Sólo cuando algo produce una acción, se puede decir que ha sido verdaderamente aprendido.
- Traducción libre de un proverbio Chino atribuido a Xunzi, aprox. 310 - 220 A.C.

La algoritmia, definida como el *estudio sistemático del diseño y análisis de algoritmos*, es uno de los elementos fundamentales en la computación y la informática. Es tal su importancia que en la mayoría de planes de estudio de Informática, Ingeniería de Sistemas o Computación se incluyen varias asignaturas con el propósito de presentar los conceptos y las herramientas que permiten a los estudiantes desarrollar sus habilidades para analizar, diseñar y construir algoritmos, y en instancias posteriores, para construir sistemas software en los cuales se usan estos algoritmos para solucionar problemas complejos.

Sin embargo, existe una paradoja en su enseñanza. Como norma general, los libros de texto de algoritmia están dirigidos a personas que han cursado por lo menos una asignatura de programación, e incluso que hayan desarrollado programas en lenguajes como C o Java. Es decir, ¿primero debemos aprender a programar, para luego aprender a solucionar problemas mediante algoritmia? Otros textos mezclan al mismo tiempo conceptos de algoritmia con aspectos propios de un lenguaje de programación, lo cual implica que el estudiante debe aprender dos cosas a la vez: los fundamentos de algoritmia, y los fundamentos del lenguaje de programación elegido por el autor.

La competencia para diseñar e implementar un algoritmo es mucho más compleja que saber usar un computador, y conocer un lenguaje de programación. Como veremos más adelante, al crear un algoritmo se está definiendo *una estrategia* o un *plan* para solucionar un problema. El *pensamiento algorítmico*, expresado como la capacidad de vislumbrar el plan, debe ser *adquirido*, no puede ser *enseñado*. Muchos estudiantes toman equivocadamente la ruta que los lleva a dominar un *lenguaje de programación*, y no se detienen a pensar en su verdadero propósito: expresar el algoritmo (el plan) en un programa (la implementación del plan) que puede ser ejecutado por un computador.

Este texto busca que el estudiante se enfoque en desarrollar primero sus habilidades para diseñar y analizar algoritmos, los cuales, si se encuentran bien estructurados, se pueden codificar sin mayor dificultad en diferentes lenguajes de programación (llámese C, Java, Python, etc.). Para este propósito se usa una la siguiente estrategia: al inicio se presentan todos los conceptos y las estructuras algorítmicas que se usarán a lo largo del texto, y posteriormente se explican con mayor detalle cuando son usadas en la solución de los problemas planteados.

Los algoritmos que dan solución a los problemas se representan usando pseudocódigo. Si bien esto puede ser visto como una carga adicional o un retraso aparente debido a la necesidad de usar otros recursos de estudio para aprender a codificar los algoritmos en algún lenguaje, el pseudocódigo ofrece la libertad al lector de *pensar en forma algorítmica*, en vez de concentrarse en aspectos como la interfaz de usuario del programa, o los elementos propios del lenguaje de programación.

El texto está escrito en un lenguaje coloquial, sin la formalidad que se podría encontrar en otros materiales. El autor no ha tenido la intención de ofender al lector por la aparente trivialidad de los ejemplos, o subestimar sus conocimientos previos. Se trata de un material que busca acompañarlo en los *primeros pasos* del aprendizaje de la algoritmia. Le sugiero darle una oportunidad al documento y revisarlo completamente, sin omitir secciones o ejemplos, ¡incluídas estas líneas! Sólo después de haber revisado todas las secciones y los ejercicios, podrá decidir si le fue de utilidad o si perdió su tiempo.

Aunque el autor ha dedicado tiempo y esfuerzo considerables para hacer que el texto cuente con la calidad necesaria para ser usado como material de estudio, se ofrece *sin ningún tipo de garantía* implícita o explícita de estar libre de errores o imprecisiones. Por lo tanto, es conveniente que se revisen en detalle los conceptos descritos, y se realicen pruebas de escritorio de los algoritmos presentados.

Algunos enunciados de los ejemplos y ejercicios que se incluyen pueden ser encontrados en otros textos, bien sea con el mismo planteamiento o con una idea similar. No es el objetivo del autor tomarse el crédito por dichos enunciados. Un gran cantidad de problemas que se estudian hoy en día se remontan a los inicios de la informática, pero debido a su importancia en el aprendizaje de la algoritmia y la programación se incluyen una y otra vez en los libros de texto. De otro lado, las soluciones presentadas se han construido a partir del análisis de los problemas, y por su estructura pueden coincidir con las que presentan otros autores.

Si este material le ha sido útil, por favor hágalo saber al autor, quien estará motivado para completarlo, corregirlo y agregarle nuevas secciones o complementar las existentes con sus valiosos aportes. Por otro lado, si piensa usar este material para otros fines diferentes al estudio personal, le solicito cordialmente que otorgue el crédito correspondiente al autor. Puede contactar al autor por medio de las direcciones de correo electrónico

emezav@unicauca.edu.co y emezav@gmail.com. Sus observaciones y aportes serán bien recibidos.

Introducción

El libro se divide en secciones, que en conjunto representan los temas que se abordan en un curso introductorio de algoritmia y programación. Deberían ser revisadas en secuencia, pero pueden ser consultadas por separado para revisar algún concepto o algoritmo específico.

La sección *Conceptos básicos* presenta una breve reseña de los aspectos más importantes que debemos conocer para dominar el *lenguaje algorítmico*, y para comprender el rol que juega la algoritmia y la programación en el desarrollo de software. Se incluye el concepto de algoritmo, sus diferentes formas de representación, el concepto de pruebas y programas de computador. También se ofrece una breve descripción del problema de la complejidad inherente tanto al software como a su construcción, se enumeran las tareas generales que se deben llevar a cabo para construir sistemas complejos, y se explica el papel que cumple la algoritmia y programación en este proceso.

La siguiente sección presenta las estructuras básicas que necesitaremos para resolver los problemas mediante algoritmos. Se definen los conceptos de instrucciones, variables, tipos de datos, así como el concepto de *condición* y las diferentes estructuras que permiten controlar el *flujo de la ejecución* de un algoritmo.

La sección *Ejercicios de fundamentación* sirve como punto de partida para introducir los tipos básicos de problemas que pueden ser resueltos mediante algoritmos. En los ejercicios presentados se ilustra el uso general de variables, estructuras de decisión y repetición, así como los primeros ejemplos de uso de arreglos, tipos compuestos de datos, subrutinas y recursividad.

La sección *Arreglos* presenta algunos ejercicios relacionados con el uso de los arreglos, en los cuales se estudian aspectos como la inicialización e inserción de datos, la inserción, búsqueda y eliminación de datos, además de algunos algoritmos de uso general. En esta sección se aumenta el uso de *subrutinas* para estructurar los algoritmos de una forma más adecuada.

La siguiente sección incluye problemas relacionados con la construcción y manipulación de *cadenas de caracteres*. Estos problemas, aunque simples, frecen la oportunidad para estudiar los algoritmos más comunes usados en el manejo de cadenas y para ver cómo

se aplican los elementos presentados hasta ahora. En varios de los algoritmos presentados se omitirá el algoritmo principal, y solo se presentará la subrutina o las subrutinas que permiten solucionar el problema planteado.

La última sección se dedica al problema de *ordenamiento* de datos, el cual nos ofrece de nuevo la oportunidad de apreciar la aplicación de los diferentes elementos algorítmicos. Se estudiarán algunos de los algoritmos de ordenamiento más comunes, en los cuales se usan arreglos, estructuras de decisión, estructuras repetitivas, subrutinas y recursividad.

Conceptos básicos

En este capítulo se presenta una breve introducción a los conceptos básicos fundamentales de algoritmia, que incluyen el concepto de algoritmo, los tipos de algoritmos y el concepto de programa de computador. Posteriormente se describe de forma breve el papel que cumple la algoritmia en la construcción de sistemas software complejos, y las fases generales que se deben llevar a cabo para construcción de algoritmos.

1.1. Concepto de algoritmo

Un algoritmo es un método o proceso para resolver un problema. Según la RAE (2015), se define como un *Conjunto ordenado y finito de operaciones que permite hallar la solución a un problema*. Un algoritmo que se diseña para ser ejecutado por un computador usualmente implica la repetición de operaciones, llamadas también *iteraciones*, las cuales, si fueran realizadas por una persona, tomarían mucho tiempo.

La Figura 1.1 ilustra el funcionamiento conceptual de un algoritmo. A uno o más *datos de entrada* se les aplica un *proceso* que involucra una serie de operaciones, y como resultado se obtienen *datos de salida*.

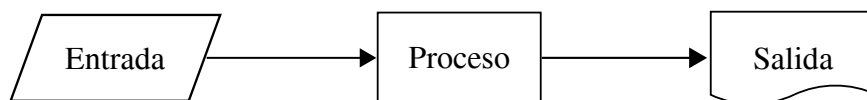


Figura 1.1: Diagrama conceptual de un algoritmo

Una característica fundamental de un algoritmo es que puede ser ejecutado tantas veces como sea necesario. Si se proporcionan los mismos datos de entrada, y el algoritmo está

correctamente construido, deberá producir los mismos datos de salida cada vez que se ejecuta. Si los datos de entrada cambian, el algoritmo deberá producir otros datos de salida de acuerdo con el proceso realizado.

1.2. Tipos de algoritmos

Es necesario considerar que cada problema (y por tanto cada algoritmo) tiene unas condiciones únicas que deben ser consideradas para hallar una solución *correcta y completa*. De forma general, la mayoría de algoritmos pueden ser clasificados en clases o categorías, por ejemplo algoritmos *de decisión*, *de producción*, *de transformación* *de búsqueda*, *clasificación* o *selección*, y *de optimización*.

En problemas complejos un algoritmo toma características de varios tipos, por lo cual será difícil clasificarlo en un solo tipo. No obstante, en su funcionamiento general, se acercará en mayor proporción a uno de los tipos mencionados.

Algoritmos de decisión

Los algoritmos de decisión tratan de dar solución a problemas en los cuales se desea obtener como respuesta *Sí* o *No*. Es necesario definir una *secuencia lógica* de *decisiones* que permitan determinar si uno o varios datos de entrada cumplen con determinada característica, o presentan una propiedad especial. Para este propósito se deben usar estructuras de decisión (*if - else*) o selección (*switch - case*), las cuales permiten al algoritmo *tomar las decisiones* requeridas.

Algunos ejemplos sencillos de algoritmos para solucionar problemas de decisión son:

- Determinar si un número es par.
- Determinar si un número es primo.
- Determinar si un número x es solución de una ecuación cuadrática.

Algoritmos para la construcción o producción de datos

Estos algoritmos permiten obtener un dato o un conjunto de datos que deben cumplir con determinadas condiciones, propiedades o restricciones. Los algoritmos de producción pueden ser usados como fuente de datos de entrada para otros algoritmos más complejos, como algoritmos de transformación, clasificación y optimización en los cuales se requiere muchos datos, o en caso que no sea conveniente o práctico esperar que el usuario ingrese todos los datos.

Para llevar a cabo la construcción es necesario aplicar decisiones sobre un conjunto de reglas o restricciones que deben cumplir los datos de salida, y posiblemente clasificación y transformación de datos.

Por ejemplo, se pueden construir algoritmos que permitan:

- Generar una secuencia de números naturales o reales que serán usados para cálculos aritméticos.
- Generar un número *aleatorio* (al azar) o una secuencia de números aleatorios que serán usadas para alguna simulación.

Algoritmos de transformación de datos

Estos algoritmos tienen el propósito de obtener nuevos datos a partir de datos de entrada, o de datos generados por el propio algoritmo. Algunas transformaciones simples que se pueden realizar sobre un conjunto de datos son:

- Calcular la suma o el promedio de un conjunto numérico de datos.
- Calcular la suma de los datos menores que determinado valor.
- Calcular el factorial de un número dado.
- Calcular el resultado de elevar un número x a la potencia y .
- Calcular el resultado de evaluar una función sobre un número o una secuencia de números enteros o reales.

Los algoritmos de transformación pueden tomar decisiones para *clasificar* o *discriminar* los datos, para luego transformarlos. También pueden usar estructuras de datos y arreglos, y por lo general requieren realizar cálculos a partir de los datos de entrada (sumatorias, promedios, productos, etc.)

El resultado de un algoritmo de transformación es un dato o un conjunto de datos que cumplen con determinada condición y que se obtienen a partir de ciertos datos de entrada.

Algoritmos de clasificación, búsqueda o selección

El siguiente tipo de algoritmos corresponde a aquellos que tratan de dar solución a problemas en los cuales se debe *clasificar* o *agrupar* los datos de entrada, cumpliendo acuerdo con uno o más *parámetros* o *restricciones*. Bajo esta perspectiva, los datos se pueden *agrupar* o *clasificar* en conjuntos mutuamente excluyentes.

También se puede incluir los algoritmos en los cuales se desea *encontrar* los datos dentro de una *colección* que cumplan con determinadas condiciones o restricciones, por lo cual se deben tomar decisiones sobre los datos de entrada.

Por ejemplo, podemos realizar una clasificación para determinar los números dentro de un conjunto son pares, primos, menores que determinado número, etc, y se debe o bien imprimirlos, o usarlos para cálculos posteriores.

En los algoritmos de clasificación también que se debe tomar algún tipo de decisión para determinar el conjunto en el cual deberá incluirse cada número. Por lo general los algoritmos de este tipo operan sobre más de un dato, por lo cual con frecuencia se deben usar *estructuras repetitivas* para llevar a cabo la tarea de clasificación, y estructuras complejas de datos y arreglos para almacenar las colecciones obtenidas.

El resultado de un algoritmo de clasificación es un dato o un conjunto de datos, posiblemente menor al conjunto de entrada, que contiene sólo aquellos datos que cumplen con las condiciones requeridas para incluirlos en alguno de los conjuntos que se desean obtener.

Algoritmos de optimización

Este es uno de los tipos de algoritmos más interesantes en las ciencias de la computación, que buscan ofrecer soluciones a problemas complejos de la realidad. Son algoritmos que deben ser diseñados y contruidos siguiendo indicaciones muy precisas, que permitan obtener *la mejor solución* o una *solución aceptable* que se aproxima a la solución *ideal*.

Debido a su complejidad, los algoritmos de optimización se estudian poco a nivel de pregrado, pero son muy importantes a nivel de posgrado. De hecho se desarrollan trabajos de maestría o doctorado orientados a proponer algoritmos que ofrezcan una solución que mejore en algún aspecto las soluciones proporcionadas por los algoritmos existentes hasta el momento.

Los algoritmos de optimización pueden requerir el uso de una gran cantidad de estructuras de datos complejas, arreglos unidimensionales y bidimensionales (matrices), para almacenar *cálculos intermedios*, y procesos complejos para tomar las decisiones que les permita encontrar o acercarse a la solución óptima.

1.3. Representación de algoritmos

Los algoritmos se pueden representar de diferentes formas, entre las que se encuentran:

- **Texto libre:** Es un texto en el cual se describe el comportamiento general del algoritmo. Tiene como ventaja que puede ser transmitido en lenguaje natural, por voz, o por texto en prosa, entre personas. Si se encuentra descrito con un cierto nivel de detalle, será posible reproducir su comportamiento. Sin embargo, un algoritmo representado en texto libre es susceptible de contener instrucciones *ambiguas*, es decir, que dan lugar a múltiples interpretaciones.
- **Dibujo libre o esquema:** Otra alternativa consiste en usar un dibujo libre o alguna clase de esquema para expresar el funcionamiento del algoritmo. Puede ser realizado en papel o mediante alguna herramienta de dibujo. Al igual que el texto libre, puede contener elementos ambiguos. Si no se usa un formato definido, los elementos pueden ser interpretados de forma diferente por otras personas.
- **Pseudocódigo:** Es un texto con un formato más estricto, en el que se trata de eliminar la ambigüedad, y que puede ser codificado con relativa facilidad en algún lenguaje de programación. En el pseudocódigo se omiten detalles propios de los lenguajes para evitar el *ruido*, es decir, elementos que nada tienen que ver con el algoritmo en sí. Será la representación usada en este texto.
- **Diagrama de flujo:** Es una representación gráfica del *flujo de ejecución* del algoritmo. Al iniciar el algoritmo, se tiene un único punto de partida, y al finalizar se tiene un único punto de llegada. La representación por diagrama de flujo tiene la ventaja que se puede *apreciar* visualmente cómo se comporta el algoritmo ante determinados datos de entrada, es decir, qué las decisiones y repeticiones se toman partiendo del inicio para llegar al fin. Sin embargo, en algoritmos complejos, la representación por medio de diagrama de flujo puede resultar también demasiado complicada.
- **Programa:** Si el algoritmo es lo suficientemente sencillo, podemos construir un *programa de computador* usando un *lenguaje de programación*. El programa tiene como ventaja que es posible *ejecutar* el algoritmo tantas veces como se desee, y se tendrá inmediatamente el resultado de su ejecución sin necesidad de hacer el seguimiento *a mano*. Sin embargo, al codificarlo mediante un lenguaje de programación determinado, nos veremos obligados a incluir aspectos propios del lenguaje que no tienen nada que ver con el algoritmo. Con relativa frecuencia nos encontraremos en la situación en la cual debemos *analizar* un programa para *construir* el algoritmo, debido a que los creadores originales de programa no proporcionan el algoritmo correspondiente o no se preocuparon por construirlo antes de escribir el código.

1.4. Programa de computador

Sin importar el mecanismo de representación usado para construir el algoritmo, en última instancia se deberá crear un *programa de computador*. Un programa es el resultado de *codificar* el algoritmo usando un *lenguaje de programación*, que se escribe como texto usando *sintaxis* específica. Una sintaxis define cómo se deben transformar las instrucciones del algoritmo a instrucciones en el lenguaje de programación seleccionado, que luego se transformarán en instrucciones que pueden ser ejecutadas por un *sistema computacional*.

Existe una gran cantidad de lenguajes de programación en los cuales pueden ser codificados los algoritmos, cada uno de ellos con sus fortalezas y debilidades. Históricamente uno de los lenguajes más usados para crear programas de computador es el Lenguaje C, aunque en la actualidad ha dado paso a lenguajes como C++, Java, C#, Python y recientemente JavaScript. No obstante, el lenguaje C sigue siendo usado como base en situaciones de desempeño crítico, como en la construcción de sistemas operativos o de programas para manejar dispositivos de hardware, sistemas de tiempo real, etc.

Una vez que el algoritmo se ha *codificado* en archivos de texto en el lenguaje seleccionado, se toma el archivo (o los archivos) y por medio de un programa llamado *compilador*, se crea un *programa en código ejecutable*, (o simplemente *programa ejecutable*), que es la traducción de las instrucciones escritas en un lenguaje de programación a instrucciones que pueden ser comprendidas y ejecutadas por el computador. Los programas en lenguaje C, por ejemplo, deben ser compilados para crear programas ejecutables.

Otros lenguajes de programación no requieren que los programas sean *compilados*. En estos lenguajes se utiliza un programa específico llamado *intérprete*, el cual ejecuta directamente las instrucciones del programa. Los lenguajes Python y JavaScript se encuentran en esta categoría. Como es de esperarse, el desempeño de estos programas no es igual al de un programa compilado, ya que no son ejecutados directamente por el computador, sino por el intérprete. No obstante, se pueden acercar al rendimiento ofrecido por los programas compilados a código ejecutable.

Existen lenguajes de programación que no son *interpretados*, pero en los cuales los programas tampoco se compilan los programas en código ejecutable. En su lugar, los programas son compilados a un *código intermedio*, que es compilado a código ejecutable en el momento justo de ejecutarlos¹. Este es el caso de lenguajes como Java y C#, cuyos programas en código intermedio son compilados en tiempo de ejecución por la *Máquina Virtual de Java* (Java Virtual Machine) y el *Entorno en Tiempo de Ejecución de Lenguaje Común* (CLR) respectivamente.

¹Esto es conocido como *compilación en tiempo de ejecución* o compilación *Just in Time*.

Cada lenguaje de programación trae consigo una ola de popularidad, motivada por su facilidad de uso, la *eficiencia* del lenguaje con respecto a los resultados que se pueden alcanzar por otros, o simplemente por *moda*. A medida que pasa el tiempo, o que aparecen problemas para los cuales el nuevo lenguaje no se desempeña tan bien como se espera, el interés empieza a decaer y aparecerá un nuevo lenguaje de programación, el *chico nuevo*.

Sin embargo, los algoritmos pueden sobrevivir el surgimiento y la desaparición de infinidad de lenguajes de programación, ya que son especificados usando un lenguaje abstracto que puede ser codificado con relativa facilidad. De hecho, en la actualidad aún usamos algoritmos que fueron concebidos hace décadas y que han sido implementados en los lenguajes de programación actuales. Cuando aparezca un nuevo lenguaje de programación, seguramente alguien codificará el algoritmo usando el nuevo lenguaje. Por tal razón, antes de aprender a *programar*, debemos aprender a diseñar y construir algoritmos para solucionar problemas.

1.5. La complejidad en el desarrollo de software

A medida que los problemas que deseamos resolver se acercan a la *escala humana*, crece la complejidad de los algoritmos que debemos construir. De hecho, para solucionar problemas complejos, en la mayoría de los casos (por no decir en todos) es necesario expresarlos en términos de problemas más simples (*subproblemas*), los cuales a su vez pueden ser solucionados mediante algoritmos más simples (*subalgoritmos*).

Por consiguiente, la solución al problema original no está dada por un solo algoritmo sino por un conjunto de algoritmos, posiblemente agrupados en diferentes programas, que mediante una ejecución coordinada permiten obtener las salidas que el *cliente* desea². A este conjunto de programas que trabajan de forma coordinada lo llamaremos *sistema software*, el cual, de acuerdo a su tamaño, complejidad y propósito recibe nombres como *aplicaciones*, *conjuntos de aplicaciones* (*suites*), *aplicaciones empresariales*, *aplicaciones científicas*, etc.

La construcción de un sistema software es una actividad que rara vez puede ser abordada por una sola persona. Esto se debe a que *en la vida real*, los problemas que pretendemos solucionar abarcan un conjunto de *necesidades* de uno o varios clientes, quienes esperan contar con el sistema funcionando *adecuadamente* en el menor tiempo posible. Por tanto, es necesario integrar grupos de trabajo, cada uno de los cuales se encarga de solucionar un *subproblema*, y luego las soluciones se *integran* en un *producto final*.

²Existen otras estrategias para descomponer un problema (por ejemplo descomposición estructural o descomposición orientada a objetos), las cuales están fuera del alcance de este texto.

1. CONCEPTOS BÁSICOS

Por ser una labor realizada por seres humanos, la construcción de algoritmos, programas y por tanto sistemas software no está libre de errores. Por tal razón es necesario seguir un proceso estructurado, al que llamaremos *proceso de desarrollo de software*. Un proceso de desarrollo de software abarca no solamente el diseño de algoritmos y la codificación de programas, también incluye actividades orientadas a garantizar que el producto obtenido satisface adecuadamente los *requisitos* establecidos por el cliente, se desempeña de forma adecuada, no presenta fallos, etc.

Se puede pensar en un proceso de desarrollo (de software) como un conjunto de tareas que son llevadas a cabo de forma *sistemática* y organizada, guiadas por un marco de trabajo que permite obtener sistemas software. Por ser un proceso complejo, se debe dividir en tareas que posiblemente son desarrolladas por diferentes personas o equipos de trabajo, en algunos casos *en sitios diferentes*. Algunas de estas tareas que se deben realizar son (Bell, 2005):

- Estudio de viabilidad: Antes de emprender el desarrollo de un software se debe establecer si es conveniente llevar a cabo el proyecto. Puede que el sistema no sea realmente necesario, o que su construcción y puesta en marcha impliquen una gran cantidad de riesgos e inconvenientes que no tendremos la certeza si pueden ser solucionados. Por supuesto, como profesionales creemos que siempre es posible llevar a cabo cualquier tarea, pero debemos afrontar el proceso con la ética y la responsabilidad necesarias para decir “no” en el momento adecuado y así evitar invertir tiempo y esfuerzo tanto del equipo de desarrollo como del cliente en construir un sistema que no se pueda usar.
- Especificación y análisis de requisitos: En esta tarea se busca establecer los *requisitos del cliente*, es decir, definir desde un principio las necesidades y las condiciones que se espera sean satisfechas por el software. En un escenario ideal, estas necesidades no cambian mientras se está llevando a cabo el desarrollo, pero la realidad muestra que a medida que avanza el proyecto, pueden aparecer necesidades o problemas que no fueron considerados.
- Diseño de la arquitectura: En la actualidad los marcos de trabajo de desarrollo de software se fundamentan en la idea de construir primero un diseño general del sistema antes de iniciar su construcción. Una arquitectura software define la estructura y el funcionamiento general del sistema a construir, el cual, por ser complejo, estará compuesto por varios *componentes* que operan de forma coordinada.
- Diseño detallado: A partir del diseño general se crea una serie de diseños detallados de cada uno de los componentes definidos en la arquitectura. Se debe usar diferentes

técnicas y lenguajes para crear estos diseños, buscando que puedan ser comprendidos por todos los involucrados en el proyecto.

- **Diseño de interfaces de usuario:** Por lo general, los usuarios interactúan con el software por medio de *interfaces* que se operan con el mouse y/o el teclado. Se debe diseñar interfaces *usables*, es decir, que sean fáciles de usar y aprender, y también cumplan con el propósito para el cual fueron creadas.
- **Diseño de interfaces con otros sistemas:** En la mayoría de proyectos complejos, el software que vamos a desarrollar deberá operar en conjunto con otros sistemas que posee el cliente, o que son proporcionados por terceros. Por lo tanto, será necesario definir los *mecanismos y protocolos* que se usarán para permitir el paso de datos desde y hacia nuestro sistema, es decir, las *interfaces* entre nuestro sistema y las otras aplicaciones.
- **Construcción (programación):** La siguiente tarea consiste en obtener el código que implementa los diseños realizados. En esta etapa se hace uso de algoritmos y lenguajes de programación para construir los diferentes componentes del sistema. Con frecuencia, el desarrollador deberá decidir entre diferentes algoritmos para solucionar los problemas, de acuerdo con los requisitos y la arquitectura establecida.
- **Integración:** Una vez contruidos los diferentes componentes, debemos *integrarlos* o *ensamblarlos*, de forma que funcionando en conjunto ofrezcan la *funcionalidad* esperada. Al realizar la integración también será necesario hacer ajustes y mejoras a los componentes para que operen apropiadamente en conjunto. Tanto la construcción del sistema como la integración de componentes son procesos *iterativos*, en los cuales cada *iteración* o *ciclo* permite obtener una *versión* del sistema que implementa progresivamente las funcionalidades requeridas.
- **Verificación:** Esta tarea tiene como propósito garantizar que el producto obtenido cumple con las especificaciones definidas. La verificación se lleva a cabo por los *desarrolladores*, quienes tienen conocimiento de la estructura interna del sistema y pueden realizar los ajustes y los cambios necesarios para que funcione de la forma esperada. La verificación debe ser un proceso estructurado e *iterativo* que se lleva a cabo desde etapas tempranas de la construcción, para evitar que los errores no resueltos se *manifiesten* al momento de realizar la integración de los diferentes componentes.
- **Validación:** Busca establecer si el sistema *cumple* con los requisitos y las condiciones establecidas por el cliente, ya que no tiene sentido construir un software que

1. CONCEPTOS BÁSICOS

opere correctamente, pero que no realice las tareas para las cuales fue concebido. De nuevo será necesario realizar ajustes, buscando que éstos no sean significativos y costosos en términos de trabajo y tiempo.

- **Implantación:** En esta tarea se toma el software construido, se *instala* en el sitio que lo requiere el cliente y se pone en operación. Algunos autores denominan esta tarea la *puesta en producción* del sistema. Se deberá capacitar a los usuarios para que puedan usarlo de forma efectiva.
- **Mantenimiento:** Un sistema en producción puede requerir cambios o ajustes, a medida que aparecen situaciones que no fueron previstas por el cliente o por el equipo que produjo el software. En ocasiones puede surgir la necesidad de *sacar el sistema* de operación para realizar ajustes mayores, lo cual deberá ser planeado cuidadosamente para evitar traumatismos en las actividades de los clientes.
- **Documentación:** Durante todas las etapas de construcción, puesta en marcha y mantenimiento del sistema, se deben generar documentos en los cuales se explican los procesos usados para la construcción, la estructura, el funcionamiento y el mantenimiento del sistema, y por supuesto documentos que permitan a nuevos usuarios *aprender* a usar el sistema por su cuenta. La documentación también es *dinámica*, es decir que *evoluciona* a medida que se avanza en el proceso. Esta es una de las tareas que con frecuencia no se realiza de forma adecuada, y esto causa que la experiencia y las lecciones aprendidas no puedan ser usadas en proyectos posteriores.
- **Gestión del proyecto:** Dado que el desarrollo de software implica tiempos, costos, personal, entre otros factores, es necesario llevar a cabo actividades orientadas a garantizar que el *proyecto de desarrollo de software* marcha como debería (es decir, que no tome tiempo y esfuerzo más allá de lo planeado), y tomar las acciones necesarias para remediar situaciones que pueden poner en riesgo el éxito de cada una de las tareas que se deben realizar.

En resumen, podemos decir que el desarrollo de software es un proceso complejo que involucra diferentes etapas y tareas que deben ser llevadas a cabo por uno o varios equipos de personas que operan de forma coordinada. La algoritmia y programación son elementos fundamentales en este proceso, ya que de nada sirve un buen diseño si no se implementa en un sistema funcional que pueda ser entregado a los clientes. Por tanto, debemos adquirir habilidades para descomponer problemas complejos en problemas más simples que pueden ser resueltos mediante algoritmos, los cuales a su vez son implementados en programas de computador.

Por lo anterior, nos enfocaremos en desarrollar las habilidades iniciales para resolver problemas usando algoritmia, sin considerar algunos aspectos que serían importantes en el marco de un proyecto de desarrollo de software. No obstante, aplicaremos los *principios* y usaremos algunas de las *etapas* que se deberían considerar en proyectos de mayor complejidad, de forma que sea más fácil la transición hacia este tipo de proyectos.

1.6. Principios para la construcción de software

Desde los inicios de la informática y la computación se han identificado algunos *principios fundamentales* que nos permiten abordar problemas complejos y construir software para solucionarlos. Estos principios se presentan a continuación.

Abstracción

La abstracción es un concepto ampliamente usado en la ciencia, que se puede definir como el proceso mediante el cual se *identifican* los aspectos importantes de un fenómeno, ignorando sus detalles. Es decir, nos *alejamos* un poco de la realidad del fenómeno, *identificamos* los *elementos* generales que lo componen y las relaciones que existen entre ellos, para estar en capacidad de *analizarlos* buscando *explicar* el fenómeno general en términos de los elementos y sus relaciones. Si realizamos la abstracción correctamente, la explicación del fenómeno *abstracto* podrá ser usada directamente o *extendida* para explicar el fenómeno real.

Puede ocurrir que las *abstracciones* realizadas no sean suficientes para *comprender* completamente el problema, de forma que se deben realizar nuevas abstracciones sobre cada uno de los elementos identificados. Es decir, creamos diferentes *niveles de abstracción*, en cada uno de los cuales nos *enfocamos* en determinadas características del problema o de sus *componentes*.

La abstracción es una herramienta fundamental en algoritmia, y en mayor grado en las diferentes tareas involucradas en un proceso de desarrollo de software, específicamente en el análisis del problema y el diseño de la arquitectura. Aplicando este principio, podemos crear *modelos* que expresan la estructura y el comportamiento del sistema *antes de construirlo*, es decir, podremos explicar *qué hará el sistema*, sin necesidad de esperar hasta que se tenga el producto final.

Modularidad

La modularidad se encuentra estrechamente relacionada con la abstracción. Se basa en la idea de *dividir* un sistema complejo en *módulos* o *componentes* que realizan tareas con-

cretas, las cuales, si son coordinadas adecuadamente, permitirán llevar a cabo los *procesos* que se espera sean realizados por el sistema completo.

Para llevar a cabo la división de los componentes del sistema con frecuencia es necesario usar diferentes *niveles de abstracción*, cada uno de los cuales se ocupa de un aspecto específico del problema a solucionar. Al definir una arquitectura, primero se identificarán los componentes o módulos generales, los cuales a su vez estarán compuestos por otros módulos, y así sucesivamente hasta encontrar *unidades funcionales* que pueden ser tratadas con un nivel de dificultad manejable.

El diseño de los componentes es también iterativo, es decir, se realizan varias *iteraciones* o *ciclos* para identificar y definir los componentes de acuerdo con los diferentes niveles de abstracción establecidos. Durante este proceso pueden surgir cambios en cuanto a la estructura y la organización de los componentes, lo cual se deberá reflejar en la documentación generada.

La modularidad tiene una consecuencia interesante: Si se definen adecuadamente los módulos *antes de construirlos*, es decir, se establece de forma precisa las entradas y las salidas y *qué hace* cada uno de ellos, es posible *reutilizar* módulos construidos en otros proyectos e incluso módulos desarrollados por terceros, con lo cual podemos reducir el costo en tiempo y dinero que deberíamos invertir al crearlos desde cero.

Ocultamiento de información

Los módulos deben ser considerados como *entidades abstractas*, de las cuales sólo se conoce el formato de sus entradas, el formato de las salidas y las funciones específicas que realizan (qué hacen). Si están correctamente definidos, no será necesario conocer los *procesos internos* que realizan para ofrecer los resultados esperados.

El principio de ocultamiento de información implementa en la práctica en forma de *interfaces*, las cuales pueden ser vistas como los *contratos* que definen los módulos en cuanto a la estructura y el formato de las entradas y las salidas, que si se cumplen, permitirán obtener la respuesta esperada. Por lo tanto, si al usar un módulo *cumplimos* con los contratos establecidos, no debemos preocuparnos acerca de *cómo* el módulo realiza su tarea.

1.7. Pasos para la construcción de un algoritmo

De forma similar al proceso de construir un software complejo, construir un algoritmo también implica llevar a cabo un proceso *sistemático* y ordenado, compuesto por una serie de pasos, etapas, o actividades cuyo resultado final es el algoritmo (y posiblemente el código ejecutable) que soluciona el problema.

A continuación se presentan los pasos generales que se llevan a cabo para construir un algoritmo. Se debe tener en cuenta que si bien los pasos se presentan uno tras otro, con frecuencia se realizan *en paralelo* y de forma *iterativa*, es decir, se repiten varias veces hasta obtener el resultado deseado.

Para explicar mejor cada uno de los pasos, vamos a partir del siguiente problema:

Construir un algoritmo que dados dos números, calcule y muestre el resultado de sumarlos.

Análisis del problema

Por definición, un algoritmo es creado para solucionar un problema. Por tal motivo, el primer paso lógico consiste en analizar el problema o la necesidad planteada, con el propósito de identificar los elementos que debemos resolver para ofrecer una solución algorítmica.

El problema presentado puede parecer *trivial*, pero no lo es en lo absoluto. Por el contrario, si quisiéramos, podemos realizar un análisis y un diseño tan detallado de la solución que involucraría no solamente aspectos de *software*, sino que deberíamos llegar al punto de especificar el computador sobre el cual se ejecutará este algoritmo, y luego adentrarnos en conceptos de arquitectura de computadores, electrónica, física y matemática. Todo esto para resolver un problema tan simple³.

Para llevar a cabo el análisis debemos usar el principio de *abstracción*, el cual nos permite enfocarnos en los aspectos *importantes* y necesarios para resolver el problema, dejando momentáneamente un lado elementos que desde luego son importantes, pero que no aportan al diseño general de la solución.

El planteamiento menciona que los datos a sumar son *dados* al algoritmo, esto quiere decir, de alguna forma son introducidos por el usuario. Estos serán los datos de entrada.

El proceso a realizar será la operación aritmética de sumar los dos datos de entrada, y el resultado deberá ser almacenado en algún sitio para posteriormente ser mostrado al usuario.

Entonces, aplicando el concepto general de algoritmo, el problema consiste en recibir dos datos (entrada), aplicarles un proceso (sumarlos) y generar una salida (el resultado de la suma).

La suma de dos números es una operación aritmética simple, que para números relativamente pequeños, puede ser realizada sin problemas por un ser humano. Entonces, ¿para qué molestarnos en construir un algoritmo que sume dos números? Por supuesto, para realizar el ejercicio académico. Sigamos la corriente, para ver hasta dónde nos lleva.

³En las lecturas recomendadas se incluye una referencia a un texto que aborda este problema.

1. CONCEPTOS BÁSICOS

Podemos proponer una primera solución al problema, usando *lenguaje natural*:

Primero, se debe solicitar al usuario (la persona) que ingrese dos números, usando el teclado o algún tipo de interfaz gráfica. Luego, debemos sumar estos dos números, y finalmente le mostramos al usuario, por ejemplo en la pantalla del computador, el resultado de la suma.

Este algoritmo es válido y a nivel general soluciona el problema, pero es necesario precisar algunos detalles que permitan evitar la ambigüedad. Por ejemplo, ¿qué significa que los datos se deben ingresar usando el teclado o algún tipo de interfaz gráfica? ¿vamos a proporcionar un conjunto de *botones* que el usuario debe pulsar para introducir los números? Todos estos son aspectos que tienen poco que ver con el algoritmo, pero serán importantes si deseamos llevar el algoritmo a un programa.

De otro lado, es importante considerar que los sistemas computacionales tienen algunas limitaciones inherentes a su naturaleza, que les permite sólo operar sobre números de una magnitud limitada. Por ejemplo, en sistemas computacionales de 32 *bits*, el mayor valor entero que se puede operar aritméticamente se acerca a los 4.000.000.000, número fácilmente alcanzable cuando se habla por ejemplo de la cantidad de correos electrónicos enviados diariamente a nivel mundial o el número de búsquedas de internet.

Esta limitante no existe *en el mundo* de la matemática, pero sí *en el mundo* de la computación. Vamos a realizar una *abstracción*, considerando que nuestro algoritmo sólo realizará la operación aritmética simple para sumar dos números enteros, y el tamaño exacto de los números a sumar estará determinado por las capacidades del computador en el cual se ejecutará el programa. De esta forma, la suma se reduce a la siguiente expresión:

$$\text{suma} = a + b$$

En donde *a* y *b* son *variables* que almacenan los valores (datos) que deseamos sumar y *suma* almacena el resultado. Por ahora no vamos a detenernos en el concepto de variable, sólo lo usaremos para desarrollar el ejemplo presentado. Estos conceptos se presentarán más adelante.

Este problema sencillo no requiere de muchas operaciones, o de secuencias de operaciones que se repiten una y otra vez, por lo cual no hay espacio para aplicar modularización. En su momento, en los ejercicios en los cuales se presente la oportunidad, aplicaremos este concepto.

Diseño del algoritmo

Nuestro algoritmo consistirá en una *secuencia* de operaciones que permita *obtener* y sumar dos valores *dados*. Pero, ¿de dónde provienen los valores? ¿en dónde se mostrará el resultado de la suma? Vamos a esbozar la segunda aproximación a la solución, que se puede expresar de la siguiente forma, en la cual se *formaliza* un poco el lenguaje usado anteriormente:

1. Obtener los dos números y almacenarlos en las variables a y b
2. Realizar la operación aritmética: $\text{suma} = a + b$
3. Mostrar el resultado almacenado en la variable suma

Ya tenemos un *conjunto ordenado de operaciones* que solucionan el problema, es decir que ¡ya tenemos un algoritmo! Sin embargo, este algoritmo aún es susceptible a diversas interpretaciones.

Acudiendo de nuevo al principio de abstracción, supongamos que *existe* una *instrucción leer* que le solicita al sistema que *obtenga* un dato y lo almacene en una *variable*. De esta forma, se cuenta con una alternativa para que el usuario *proporcione* los datos sobre los cuales se realizarán las operaciones, posiblemente usando el teclado del computador. También vamos a suponer que los valores se almacenan directamente como números, por lo cual no tendremos que realizar ningún tipo de transformación de los datos leídos.

Entonces, ya tenemos casi solucionado el problema. No se usarán números para indicar el orden de las operaciones, ya que su especificación en secuencia es suficiente para saber cuál debe ser ejecutada antes y cuál después.

```
leer a
leer b
suma = a + b
mostrar el resultado obtenido
```

Ahora supongamos que *existe* otra instrucción llamada *imprimir*, que permite *mostrar* el dato almacenado en una variable. En este punto estamos dando por hecho que las instrucciones *leer* e *imprimir* existen y pueden ser usadas. Pero, ¿quién las proporciona? No importa en este momento. Este problema deberá ser resuelto al *codificar* nuestro algoritmo en algún lenguaje de programación, el cual, dependiendo de sus capacidades, nos permitirá usar el teclado o el mouse para ingresar los datos, y una *consola* o una *interfaz gráfica* para presentar la salida del algoritmo.

```
leer a
leer b
```

```
suma = a + b
imprimir suma
```

Ya contamos con un algoritmo, representado como un texto con un nivel aceptable de *formalidad*. Nos encontramos en la frontera entre *diseño* y *construcción*. De hecho, en este caso fue posible obtener un *prototipo* del algoritmo mientras diseñábamos la solución.

Construcción del algoritmo

Con base en el diseño obtenido, podemos *construir* o *implementar* el algoritmo, usando alguna de las alternativas mencionadas anteriormente. Dado que el ejemplo es lo suficientemente sencillo, construiremos el algoritmo usando varias representaciones: pseudocódigo, un diagrama estructurado N-S y un diagrama de flujo. Podemos construir un programa, lo cual efectivamente haremos, sólo con el ánimo de ilustrar el proceso completo. Sin embargo, no se explicará cada representación en detalle.

El pseudocódigo permite expresar el funcionamiento del algoritmo, sin preocuparnos por los aspectos descritos en el análisis y en el diseño. También usa expresiones del lenguaje natural, pero define una estructura más formal que puede ser codificada con relativa facilidad en un lenguaje de programación. Además se puede incluir definiciones y expresiones similares a ecuaciones matemáticas llamadas *asignaciones*. Veamos la representación del algoritmo en el Código 1.1.

Código 1.1: Suma de dos números

```
1  /*
2  * Suma de dos números enteros
3  * Entrada: Enteros a y b, leídos por teclado
4  * Proceso: Calcula la suma de los números ingresados
5  * Salida: Suma de los datos leídos, por pantalla
6  */
7  algoritmo sumaDeDosNumeros
8  entero a, entero b, entero suma
9  inicio
10 //Entrada de datos
11 leer a
12 leer b
13 //Operación
14 suma = a + b
15 //Salida
16 imprimir suma
17 fin
```

El pseudocódigo es prácticamente similar al prototipo obtenido mientras se realizaba el diseño, pero incluye nuevos elementos que se explican de forma breve a continuación.

La estructura del pseudocódigo tiene una *sintaxis* específica para escribir el algoritmo, la cual contiene los siguientes elementos:

- Descripción general del algoritmo (documentación), como un bloque de texto
- Nombre del algoritmo
- Definición de las variables que se usarán y el tipo de datos que se almacenará en cada una de ellas
- Una marca (*etiqueta*) de inicio, que define el comienzo del algoritmo en sí.
- La secuencia de operaciones del algoritmo, en medio de las cuales también se puede incluir documentación (texto) adicional.
- Una marca de finalización del algoritmo.

Es importante observar que las operaciones que se encuentran entre las marcas de inicio y fin se han desplazado hacia la derecha para indicar que se encuentran *contenidas* en el algoritmo. A esta práctica se le conoce como *indentación*⁴. Tanto el pseudocódigo como el código deben ser indentados de forma adecuada, para mejorar su estética y legibilidad.

La *documentación* puede ser definida como la información textual que describe la estructura y el funcionamiento del algoritmo, la cual es útil para que las personas que no lo hayan construido lo puedan comprender, y realizar los cambios si es el caso (los cuales también deberán ser documentados). Se debería consignar en documentos externos, pero se considera una *buen práctica* incluirla directamente en el pseudocódigo y el código, con lo cual, al ser compartidos entre varias personas, permitirá tener acceso inmediato a ella.

En la actualidad existen herramientas que permiten analizar un texto (bien sea un pseudocódigo o el código generado en algún lenguaje de programación) y *generar automáticamente* la documentación en forma de páginas web o documentos, que luego pueden ser completados para producir la documentación formal que se entrega al cliente denominada *manuales*.

La documentación se incluye en el pseudocódigo (y posteriormente en el código de los programas) en forma de *comentarios*, que son bloques de texto encerrados entre caracteres especiales llamados *delimitadores*. Cada lenguaje algorítmico y cada lenguaje de programación usa diferentes caracteres como delimitadores de comentarios, pero nosotros usaremos aquellos que se usan con mayor frecuencia en la actualidad para incluir comentarios de múltiples líneas (al inicio del pseudocódigo), y de una sola línea (antes de cada bloque de operaciones). Usaremos la siguiente convención para la documentación:

⁴Esta palabra es una traducción de la palabra inglesa *indentation*. La palabra reconocida por la RAE es *sangrado*.

1. CONCEPTOS BÁSICOS

```
1 /*  
2  * Comentario de varias líneas  
3  * de texto  
4  */  
5  
6 //Comentario de una línea de texto  
7  
8 //Otro comentario de una línea
```

Ahora se presentará una de las variantes gráficas más usadas para representar un algoritmo: el *diagrama de flujo* o *flujograma*. La Figura 1.2 muestra el diagrama de flujo para la solución al problema planteado.

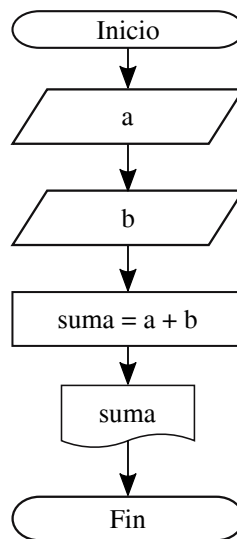


Figura 1.2: Diagrama de flujo suma de dos números

Los *diagramas estructurados* o diagramas Nassi and Shneiderman (1973) son otra forma de representar gráficamente un algoritmo, en los cuales las operaciones se disponen una tras otra en bloques. La Figura 1.3 presenta el diagrama N-S para el algoritmo obtenido.

Pruebas

La prueba de un algoritmo se puede realizar de diversas formas, por ejemplo:

- A mano: Se realiza usando una hoja de papel, una pizarra o algún elemento en el cual se puedan tomar notas. Se comienza por el inicio del algoritmo, y se van ejecutando las instrucciones *una a una*, de la misma forma como lo haría un computador. Se debe hacer seguimiento y actualización de las variables, de acuerdo con instrucciones que se van ejecutando. A este tipo de prueba se le llama *prueba de escritorio*.

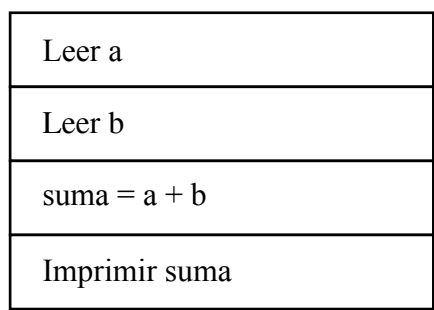


Figura 1.3: Diagrama N-S suma de dos números

- Mediante un programa: Se deberá *codificar* el algoritmo en un programa, que deberá ser *compilado y ejecutado* o *interpretado* (Dependiendo del lenguaje seleccionado). El programa tomará los datos de entrada de alguna fuente (el usuario, un archivo, otro programa, etc..) y ofrecerá algún dato de salida. Entonces el usuario deberá verificar que los datos de salida obtenidos sean consistentes con los datos de entrada y el proceso realizado. Al probar un programa se debe buscar la menor interacción posible con el usuario, de modo que el proceso de prueba pueda ser *automatizado*, es decir, repetido muchas veces con diferentes datos de entrada⁵.

En una prueba de escritorio se ejecutan *en secuencia* las instrucciones que contiene el algoritmo, realizando las acciones y tomando las decisiones que tomaría el computador de acuerdo con los datos ingresados. El registro de la ejecución se puede realizar en papel y organizar en una tabla en la cual se presentan las instrucciones, los datos de entrada y las salidas que se enviarán a la pantalla como se muestra en la Tabla 1.1.

Tabla 1.1: Prueba de escritorio suma de dos números

Instrucción	a	b	suma	Salida
inicio				
leer a	5			
leer b		2		
suma = a + b			7	
imprimir				7
fin				

Observe que en esta prueba de escritorio se especificó 5 como valor de a y el número 2 como el valor de b. Por lo tanto, la suma es 7.

Un programa que implementa el algoritmo en el lenguaje de programación “C” se muestra a en el Código 1.2.

⁵Por supuesto, si lo que se está probando es precisamente la interacción con el usuario, no sería conveniente automatizar el proceso.

Código 1.2: Ejemplo de programa en lenguaje C

```
1 //Incluir las librerías necesarias
2 #include <stdio.h>
3 #include <stdlib.h>
4 //Inicio del programa principal
5 int main(int argc, char * argv[]) {
6     int a; int b; //Declaración de las variables
7     int suma;
8
9     //scanf: Leer número entero y almacenarlo en la variable a
10    scanf("%d", &a);
11
12    //scanf: Leer número entero y almacenarlo en la variable b
13    scanf("%d", &b);
14
15    //Operación
16    suma = a + b;
17
18    //Imprimir el resultado
19    printf("%d\n", suma);
20 }
```

Luego de ser escrito en un archivo de texto usando algún editor o un *Entorno Integrado de Desarrollo* (IDE), el programa se *compila* para crear un *archivo ejecutable*. En la Figura 1.4 se muestra la opción para compilar el programa usando el entorno de desarrollo Dev-C++.

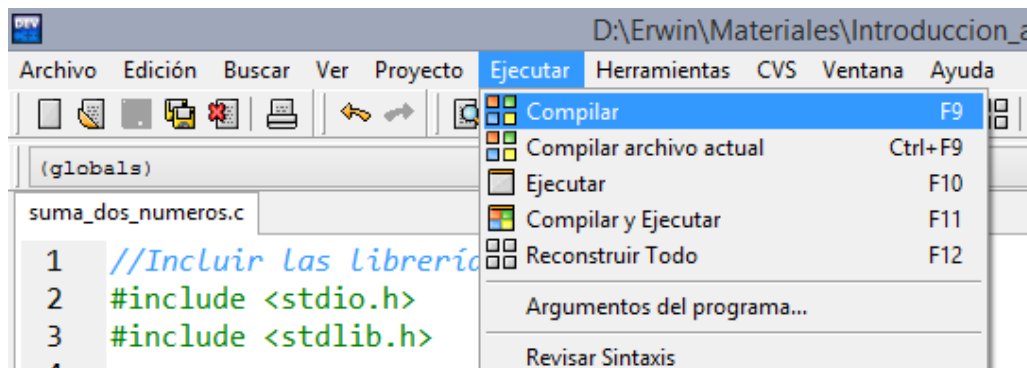
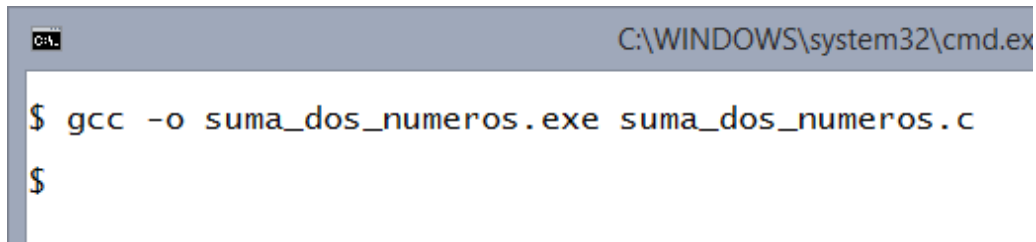


Figura 1.4: Compilar el programa en lenguaje C

Es posible que al momento de codificar el algoritmo se incluyan *errores* de sintaxis, en cuyo caso la compilación fallará. Se debe corregir los errores y compilar hasta que sea posible obtener el archivo ejecutable.

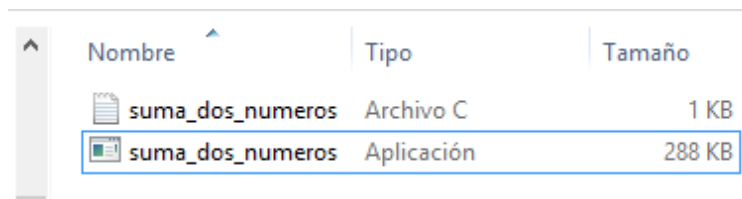
Los *IDE* facilitan el proceso de editar, compilar y ejecutar los programas. Sin embargo, es posible compilar *manualmente* el programa usando *comandos*, como se muestra en la Figura 1.5. Ahora se tienen dos archivos: el archivo de código fuente, el cual puede ser

modificado y compilado de nuevo, y el archivo ejecutable o *aplicación*, la cual debemos *iniciar* o *ejecutar* cuantas veces sea necesario (Ver Figura 1.6).



```
C:\WINDOWS\system32\cmd.exe
$ gcc -o suma_dos_numeros.exe suma_dos_numeros.c
$
```

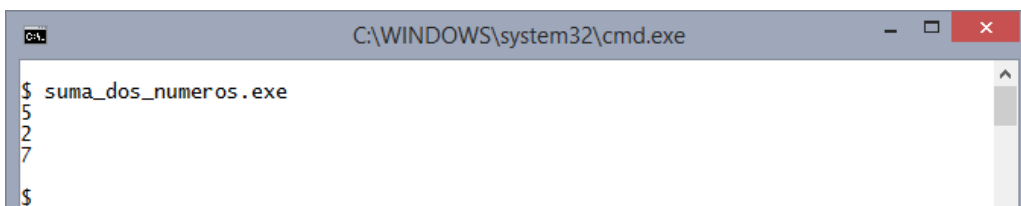
Figura 1.5: Compilar el programa en C usando un comando



Nombre	Tipo	Tamaño
suma_dos_numeros	Archivo C	1 KB
suma_dos_numeros	Aplicación	288 KB

Figura 1.6: Archivo *fuentes* y *ejecutable* de un programa

Por lo general un programa básico de C no presenta una *interfaz gráfica*, por lo cual debe ser ejecutado desde una *terminal* o *consola* ⁶. La figura 1.7 muestra la ejecución del programa, en el cual se proporcionan los mismo valores a sumar, 5 y 2. Dado que el programa no incluye mensajes hacia el usuario, inmediatamente se deberán proporcionar dos los valores a sumar, pulsando la tecla ENTER después de ingresar cada número. El programa ofrecerá la salida esperada, es decir, 7.



```
C:\WINDOWS\system32\cmd.exe
$ suma_dos_numeros.exe
5
2
7
$
```

Figura 1.7: Iniciar ejecución del programa

También es posible crear un programa que presente una *interfaz gráfica* al usuario, la cual tendrá elementos denominados *controles*, que permitirán interactuar con el programa de forma más amigable. La Figura 1.8 muestra la *interfaz* de un programa escrito en el lenguaje *C Sharp* (C#) que implementa el mismo algoritmo. Este lenguaje ofrece

⁶Sin embargo, usando *librerías* de terceros o del sistema operativo, es posible crear interfaces de escritorio en C.

1. CONCEPTOS BÁSICOS

los elementos necesarios para construir *aplicaciones de escritorio*, es decir, aplicaciones que cuentan con los elementos comunes en los programas que usamos diariamente como ventanas, cuadros de texto, botones, etc.

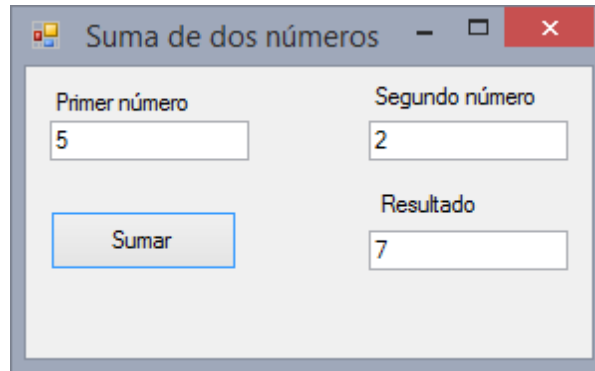


Figura 1.8: Programa con interfaz gráfica

En este programa las *abstracciones de leer e imprimir* se materializan de forma diferente. Se usan tres *cajas de texto*, dos de ellas para los datos de entrada y otra para mostrar el resultado de la suma. La acción de sumar los números se encuentra implementada como un *evento* que se inicia cuando el usuario *hace click* sobre el botón “Sumar”, pero en esencia la lógica de programación es la misma que se especificó en el algoritmo.

La parte del programa en C# que se ejecuta al *hacer click* sobre el botón “sumar” se muestra en el Código 1.3.

Código 1.3: Suma de dos números en C#

```
private void btnSumar_Click(object sender, EventArgs e)
{
    int a;
    int b;
    int suma;
    a = int.Parse(txtA.Text);
    b = int.Parse(txtB.Text);
    suma = a + b;
    txtResultado.Text = suma.ToString();
}
```

Observe que se deben usar instrucciones específicas del lenguaje para *leer* los datos almacenados en las cajas de texto llamadas `txtA` y `txtB`, transformar estos datos en números enteros, realizar la suma y luego transformar el resultado en un texto que se muestra en la caja de texto `txtResultado`.

Los detalles de este programa están fuera del alcance de este libro, pero de nuevo confirmamos que aparte de las transformaciones necesarias para manejar los elementos gráficos, el algoritmo es el mismo sin importar el lenguaje de programación.

Es comprensible que tengamos *la tentación* de aprender inmediatamente algún lenguaje, pero debemos tratar *primero* de desarrollar el *pensamiento algorítmico* y después las *habilidades* para codificar los algoritmos. Si cree que puede hacer las dos cosas al tiempo, elija un buen manual de programación y trate de implementar uno a uno los algoritmos que se presentan en este libro ¡sólo después de entender por completo el problema y el algoritmo!

Para terminar esta sección, el Código 1.4 muestra el mismo algoritmo codificado en el lenguaje Python. Al igual que en lenguaje C, los programas básicos desarrollados en este lenguaje no presentan una interfaz gráfica, aunque es posible usar *librerías* especializadas para crear aplicaciones de escritorio.

Código 1.4: Ejemplo de programa en Python

```
1 #Lee dos números por la entrada estándar (teclado)
2 #calcula e imprime la suma por la salida estándar (pantalla)
3 a=int(input())
4 b=int(input())
5 suma = a + b
6 print(suma)
```

Los programas en Python pueden ser guardados en archivos o ejecutados directamente en el *intérprete* de Python, como se muestra en la Figura 1.9.

```
$ python
Python 3.4.0 (v3.4.0:04f714765c13, Mar 16 2014, 19:24:06) [MSC v.1600
tel)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> a=int(input())
5
>>> b=int(input())
2
>>> suma = a + b
>>> print(suma)
7
```

Figura 1.9: Ejecución del programa en Python

1.8. Lecturas recomendadas

Una descripción de los orígenes del concepto de algoritmo y sus características se puede encontrar en el clásico volumen 1 de la serie de algoritmia de Knuth (1997), considerado *una biblia* de la computación. El autor presenta una solución al algoritmo de máximo común divisor que ha existido desde la antigua grecia, y aprovecha este ejercicio para introducir el concepto de algoritmo y sus características.

Una introducción más *amigable* al concepto de algoritmo se encuentra en el libro de Brassard and Bratley (2006), en su capítulo 1. En este capítulo también se incluyen elementos de notación matemática, teoría de conjuntos y algunas técnicas de demostración que servirán de base para cursos avanzados de algoritmia.

Un texto obligado en hispanoamérica por su historia, su amplio uso y la preocupación del autor por incluir nuevos temas, lenguajes de programación y metodologías actuales es el de *Fundamentos de Programación* (Joyanes Aguilar, 2008). El libro de Joyanes es uno de los materiales más completos en cuanto a los fundamentos de algoritmia y programación, dado que incluye una gran cantidad de ejemplos y ejercicios propuestos. Puede parecer denso al principio, pero si se aborda con un objetivo específico con seguridad encontrará lo que necesita.

Otro libro recomendado es *Diseño estructurado de algoritmos* (Oviedo F., 2004). Es un texto relativamente corto, pero que presenta de una forma concreta y amigable los fundamentos del diseño de algoritmos e incluye una buena cantidad de ejemplos.

También sería conveniente revisar el libro *Lógica de programación orientada a objetos* (Oviedo Regino, 2015), el cual presenta, además de los conceptos básicos de algoritmia, las bases fundamentales de la Programación Orientada a Objetos. Este es el paradigma de programación dominante en la actualidad, que generalmente es abordado después de los cursos introductorios de algoritmia.

El problema de crear un sistema computacional ha sido abordado de forma brillante por Nisan and Schocken (2005), quienes argumentan que la forma de comprender el funcionamiento de los computadores es construyendo uno desde cero. Para lograr su propósito, desarrollan una serie de proyectos, que permiten obtener la especificación y la implementación de un sistema computacional llamado *Hack* sobre el cual funciona un sistema operativo llamado *Jack OS*, que puede ser ejecutado usando un simulador. En su sitio web (Nisan and Schocken, 2015), ofrecen la posibilidad de descargar el software y los materiales necesarios para usar el sistema implementado.

Estructuras básicas para construcción de algoritmos

El paradigma de la programación imperativa se fundamenta en que es necesario *indicar* al sistema computacional las operaciones que debe realizar. En otras palabras, cuando se implementa un programa, *se le está diciendo a la máquina* qué debe hacer y cómo. Esto se realiza mediante **instrucciones** que se especifican una tras otra formando *secuencias*. La ejecución de estas secuencias se puede controlar mediante **estructuras de control de flujo**, que se agrupan en estructuras de decisión y selección, estructuras repetitivas y subrutinas.

2.1. Instrucciones

Las instrucciones son operaciones que debe realizar la máquina (el computador), e indican que se debe realizar una acción. En un algoritmo, la *secuencia* de las instrucciones especificada es importante, ya que se ejecutan *una a una, en el orden que fueron especificadas*. Es decir, que hasta que no se haya completado una instrucción actual que está procesando la máquina, no se ejecuta la siguiente instrucción¹.

¹Gracias a la evolución de los computadores, en la actualidad es posible ejecutar instrucciones y secuencias *en paralelo*, pero eso está fuera del alcance de este texto.

Instrucción de salida por pantalla

Esta instrucción ofrece la posibilidad de informar al usuario que algo ha sucedido, o que el algoritmo (y el programa resultante) requiere su atención. Esto se realiza mediante una instrucción *imprimir* (*print* en inglés), la cual permite enviar un mensaje a la pantalla, o el dispositivo de salida configurado en el programa.

Aplicando el principio de abstracción, vamos a suponer que la instrucción *imprimir* permite enviar los mensajes a la pantalla, pero en realidad pueden ser enviados a otros dispositivos como una impresora, un archivo, o incluso a otro programa. Esto se deberá decidir al momento de codificar el algoritmo.

La instrucción básica de salida por pantalla se escribe como se muestra en el Código 2.1.

Código 2.1: Instrucción de salida por pantalla

```
//Instrucción para imprimir  
imprimir mensaje
```

En donde *mensaje* es una cadena de caracteres (un conjunto de caracteres rodeado por comillas dobles), o una expresión que puede ser evaluada como una cadena de caracteres.

La representación en diagrama de flujo de la instrucción de salida por pantalla se muestra en la Figura 2.1.

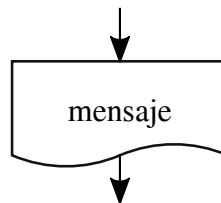


Figura 2.1: Instrucción de salida por pantalla

Por ejemplo, el Código 2.2 permite mostrar por pantalla la secuencia de caracteres *Hola, mundo*, y luego termina su ejecución.

Código 2.2: Algoritmo Hola mundo

```
/*  
 * Imprime la cadena de caracteres "Hola mundo" por pantalla  
 */  
algoritmo HolaMundo  
inicio  
    imprimir "Hola, mundo"  
fin
```

La representación en diagrama de flujo del algoritmo anterior se muestra en la Figura 2.2.

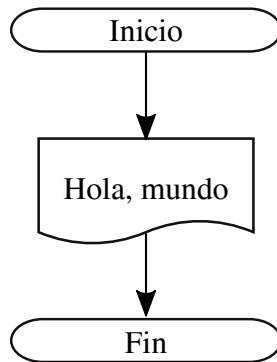


Figura 2.2: Hola mundo en diagrama de flujo

Tabla 2.1: Prueba de escritorio hola mundo

Instrucción	Salida
inicio	
imprimir	Hola, mundo
fin	

La prueba de escritorio del algoritmo se presenta en la Tabla 2.1.

Tenga en cuenta que las comillas se usan en el algoritmo (y el respectivo programa) para *delimitar* la cadena de caracteres, es decir, para indicar el comienzo y el fin de la secuencia de caracteres que la conforma. Estas no aparecerán en la pantalla.

Instrucción para lectura de datos

Esta instrucción le permite al usuario introducir un dato al algoritmo, que será usado posteriormente para realizar cálculos o tomar decisiones. Por defecto, se asume que el usuario ingresará los datos usando el teclado del computador.

El formato general de una instrucción de lectura de datos se muestra en el Código 2.3.

Código 2.3: Instrucción para lectura de datos

```
leer var
```

En donde *var* es el nombre de una variable dentro de nuestro algoritmo, en el cual quedará almacenado el dato leído. Para seguir adecuadamente las normas básicas de la algoritmia, la variable deberá ser *definida* al inicio del algoritmo.

Una instrucción *leer* suspende la ejecución del programa correspondiente hasta que el usuario ha especificado el valor. Cuando estamos probando un algoritmo, al llegar a una instrucción *leer* se deberá proporcionar un valor, que se almacenará en la variable

2. ESTRUCTURAS BÁSICAS PARA CONSTRUCCIÓN DE ALGORITMOS

correspondiente. De esta forma, es posible probar el algoritmo varias veces, con diferentes datos de entrada.

La instrucción de lectura se representa en diagrama de flujo como se muestra en la Figura 2.3.

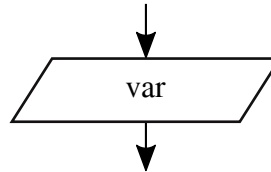


Figura 2.3: Instrucción de lectura de datos

Como ejemplo, vamos a construir un algoritmo para solucionar el siguiente problema:

Construir un algoritmo que solicite al usuario un número entero, el cual deberá ser mostrado de nuevo por pantalla.

Para este ejercicio no se requiere un análisis detallado. Simplemente debemos realizar la siguiente secuencia de operaciones: Mostrar un mensaje por pantalla solicitando un número, leer el número que se almacenará en una variable (que llamaremos *n*) y luego mostrar el valor que se encuentra almacenado en dicha variable.

El Código 2.4 muestra el algoritmo representado en pseudocódigo.

Código 2.4: Ejemplo de entrada de datos

```
1 /*
2  * Imprime un mensaje solicitando un número entero, lee el número
3  * y lo muestra por pantalla acompañado de un mensaje.
4  */
5 algoritmo EntradaSalida
6 entero n
7 inicio
8   imprimir "Por favor ingrese un número "
9   leer n
10  imprimir "Usted ingresó ", n
11 fin
```

La solución a este problema representado en diagrama de flujo se puede apreciar en la Figura 2.4, y en la Tabla 2.2 se realiza la respectiva prueba de escritorio. Para esta prueba se proporciona el valor 5 cuando se llega a la instrucción *leer*.

Instrucciones de asignación

Una instrucción de asignación permite establecer el dato que se almacena en la variable (su valor), a partir de un valor constante, el valor actual de otra variable, una expresión

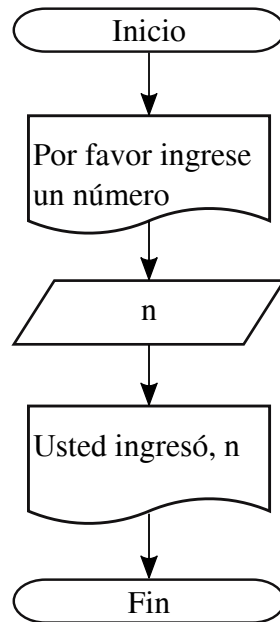


Figura 2.4: Entrada y salida de datos

Tabla 2.2: Prueba de escritorio entrada y salida

Instrucción	n	Salida
inicio		
imprimir		Por favor ingrese un número
leer n	5	
imprimir		Usted ingresó 5
fin		

matemática o el resultado de una operación más compleja.

Una asignación siempre *sobreescribe* el valor que se encontraba almacenado en la variable, es decir, lo borra y almacena un valor nuevo. Puede ser vista como un tipo especial de ecuación, en el cual a la izquierda del igual sólo aparece una variable, y a la derecha aparece un valor constante, otra variable, o una operación más compleja.

Las asignaciones son un elemento fundamental en la programación imperativa, dado que permiten cambiar los datos almacenados en las variables. Sin esta capacidad, sería prácticamente imposible construir algoritmos, ya que el uso de las diferentes estructuras algorítmicas (decisión y repetición, funciones), basan gran parte de su funcionamiento en la capacidad de modificar los valores almacenados en variables.

En una asignación se define colocando a la izquierda del igual *una sola variable*, y a la derecha del igual un valor constante, el nombre de otra variable o una expresión más compleja. Si en la expresión de asignación (a la derecha del igual) aparecen una o más

2. ESTRUCTURAS BÁSICAS PARA CONSTRUCCIÓN DE ALGORITMOS

variables, se debe *evaluar* la expresión, es decir, *reemplazar* las variables con el dato que contienen *en este momento* de ejecución del algoritmo.

Por ejemplo, en el Código 2.5 realiza una serie de asignaciones que permiten establecer el valor de algunas variables. Antes de finalizar, muestra por pantalla el último valor que se almacenó en cada una de ellas.

Código 2.5: Ejemplos de asignación

```
1 /* Asignaciones de diferentes tipos a las variables */
2 algoritmo asignaciones
3 entero a, b, c
4 inicio
5   //Asignar valores constantes
6   a = 5
7   b = 2
8   //Asignar resultados de operaciones
9   c = a + 2
10  c = b //Sobreescribe el valor almacenado
11  a = c + 1
12  c = a + b
13  //Imprimir los valores almacenados luego de las asignaciones
14  imprimir "El valor de a es ", a
15  imprimir "El valor de b es ", b
16  imprimir "El valor de c es ", c
17 fin
```

En este ejemplo, la variable *a* se asigna (y por lo tanto, se modifica) dos veces. Inicialmente se asigna el valor de 5, y posteriormente se almacena en esta variable el resultado de la operación $c + 1$, la cual, con el valor actual de *c* (2), toma el valor de 3.

La variable *b* se asigna una sola vez, por lo tanto su valor final será 2.

La variable *c*, por su parte, se asigna varias veces: Primero recibe el valor resultante de sumar el valor **actual** de la variable *a* (5 en ese momento) más 2, por lo tanto en la primera asignación tomará el valor resultante de la suma, 7. Después se le asigna el valor que se encuentre almacenado *b*, por lo cual el valor actual de *c* (7) se borrará y en su lugar se almacena 2.

Finalmente, en *c* se almacenará el resultado de sumar los valores *actuales* de *a* y *b*, por lo cual *c* terminará con valor de 5.

La representación en diagrama de flujo de la secuencia de asignaciones anterior se muestra en la Figura 2.5. Cuando se tienen instrucciones seguidas del mismo tipo (asignaciones, entrada y salida por pantalla), se acostumbra *agruparlas* para evitar que el diagrama se complique innecesariamente.

La prueba de escritorio del algoritmo se muestra en la Tabla 2.3.

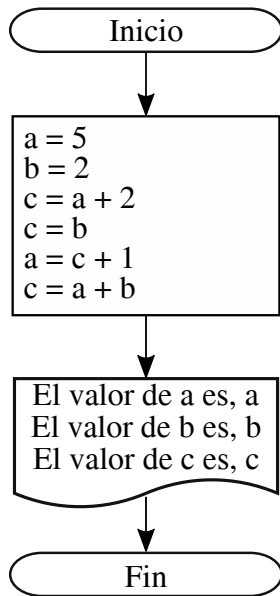


Figura 2.5: Diagrama de flujo asignaciones

Tabla 2.3: Prueba de escritorio asignaciones

Instrucción	a	b	c	Salida
inicio				
a = 5	5			
b = 2		2		
c = a + 2			7	
c = b			2	
a = c + 1	3			
c = a + b			5	
imprimir				El valor de a es 3
imprimir				El valor de b es 2
imprimir				El valor de c es 5
fin				

Incremento y decremento

El incremento es un tipo especial de asignación que se usa sobre variables que almacenan números. Al incrementar la variable, se almacena en ella misma el valor que tenía originalmente, aumentado en 1.

El Código 2.6 muestra el uso del incremento sobre una variable *i*, cuyo valor inicial es 1. Al realizar el incremento, su valor cambiará a 2. En el ejemplo se imprime dos veces, para verificar los valores almacenados.

Código 2.6: Ejemplo de incremento

1	<code>//Prueba de operación incremento</code>	31
2	<code>i = 1</code>	
3	<code>imprimir i</code>	
4	<code>i = i + 1</code>	
5	<code>imprimir i</code>	

La prueba de escritorio del pseudocódigo se presenta en la Tabla 2.4

Tabla 2.4: Prueba de escritorio incremento

Instrucción	i	Salida
-------------	---	--------

2. ESTRUCTURAS BÁSICAS PARA CONSTRUCCIÓN DE ALGORITMOS

El decremento es la operación contraria, es decir, restar 1 al valor de una variable y almacenarlo en la misma variable. La siguiente secuencia de mostrada en el Código 2.7 presenta esta operación.

Código 2.7: Ejemplo de decremento

```
1 //Prueba de operación decremento
2 i = 100
3 imprimir i
4 i = i - 1
5 imprimir i
```

La prueba de escritorio para el decremento se muestra en la Tabla 2.5.

Tabla 2.5: Prueba de escritorio decremento

Instrucción	i	Salida
i = 100	100	
imprimir i		100
i = i - 1	99	
imprimir i		99

La instrucción de decremento toma el valor actual de la variable, le resta el valor de 1, y luego almacena el resultado en la misma variable.

2.2. Variables, tipos de datos y representación

Una variable es un espacio de memoria, en el cual se almacena de forma *temporal*² un dato que necesita el algoritmo para ejecutarse (un *dato de entrada*), un dato que se obtiene durante el proceso del algoritmo o un resultado que se desea proporcionar como producto final (un *dato de salida*) del algoritmo.

Por lo general, los datos almacenados en las variables *cambian* a medida que se lleva a cabo el algoritmo (de ahí su nombre: variable - que varía, que cambia). Existen algunas variables especiales denominadas *constantes*, cuyo valor no cambia durante la ejecución del algoritmo.

Las variables sólo existen en el contexto de ejecución del algoritmo, es decir, entre el punto de inicio y el punto de finalización. Por lo tanto, si se ejecuta de nuevo el algoritmo, se deben *inicializar* de nuevo las variables o solicitar que el usuario ingrese los valores.

En un algoritmo, es necesario definir *todas* las variables que se van a usar, y el tipo de datos que almacenará cada una de ellas. Una variable ser de diferente *tipo*, entre los que

²Es decir, que solo existe durante la ejecución del algoritmo.

se encuentran los *tipos simples* o *tipos primitivos*, los *arreglos*, los *tipos compuestos* y las *referencias*, también llamadas *apuntadores*.

Tipos simples de datos

Un tipo simple (tipo base o tipo primitivo) representa el elemento básico de datos que puede manejar un algoritmo, y en consecuencia, un programa. Los tipos simples incluyen:

- Números enteros (números enteros)
- Números con decimales, llamados *números de punto flotante*.
- Caracteres (una letra, un espacio, un símbolo de puntuación, interrogación, admiración, paréntesis, etc.)
- Arreglos: Un arreglo una clase especial de variable, que permite almacenar más de un dato *del mismo tipo*. Inicialmente se puede pensar en un arreglo como un *estante*, en las cuales en cada posición es posible almacenar un *elemento* (dato). En la mayoría de lenguajes de programación, los arreglos también son considerados un *tipo primitivo*.
- Cadenas de caracteres: Son secuencias de caracteres, que representan un dato que puede contener letras, números, signos de puntuación, etc. Por ejemplo, es posible almacenar los nombres de una persona, una dirección de correo electrónico o un fragmento de un texto en una cadena de caracteres. En algunos lenguajes de programación, una cadena de caracteres se maneja de forma similar a un arreglo de caracteres.

La Figura 2.6 muestra diferentes variables, cada una de un tipo simple diferente.

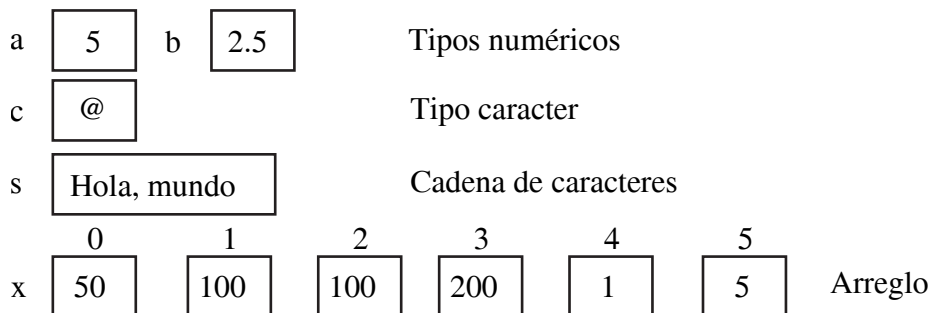


Figura 2.6: Representación de los tipos simples de datos

Tipos numéricos de datos

Un tipo numérico permite representar un dato que puede ser usado en cálculos matemáticos. En un sistema computacional, generalmente los números se diferencian en dos grandes clases:

- **Números enteros:** En una variable de tipo entero se puede almacenar cualquier número natural, negativo o cero. En algunos lenguajes de programación además es posible definir enteros *sin signo*, es decir, tipos de variables que sólo almacenan números naturales y el cero.
- **Números de punto flotante:** Incluyen los números enteros y los números racionales, es decir, aquellos que tienen cifras decimales. Debido a que la capacidad de un sistema computacional es limitada, existen tipos de número de tipo flotante de diferente *precisión*, para almacenar números más grandes con una mayor cantidad de decimales.

Caracteres

Este es un tipo especial que permite almacenar un dato como una letra, un dígito de un número, un espacio, un signo de puntuación, etc. A nivel computacional, este tipo de datos se representa como un número cuyo valor se encuentra entre 0 y 255, y a cada valor se le asigna un caracter.

En los años 60 se definió un estándar denominado ASCII (American Standard Code for Information Interchange), el cual asignaba un caracter específico a cada número de 1 a 127. Los primeros 33 caracteres se llaman *caracteres de control*, y permitían controlar algunos aspectos de los dispositivos de hardware (ver Tabla 2.6). La mayoría de los caracteres de control no tienen sentido práctico hoy en día, pero siguen existiendo por razones históricas. Sin embargo, algunos de estos caracteres, como el caracter de *escape* (127), *backspace* (8) *tabulador* (9) los caracteres *nueva línea* (10) y *retorno de carro* (13), además del caracter *nulo* (0) siguen siendo ampliamente usados.

Tabla 2.6: Caracteres de control ASCII

0 (Null)	8 BS (Backspace)	16 DLE	24 CAN (Cancel)
1 SOH	9 HT (Tab)	17 DC1	25 EM
2 STX	10 LF (Line Feed)	18 DC2	26 SUB
3 ETX	11 VT	19 DC3	27 ESC (Escape)
4 EOT	12 FF	20 DC4	28 FS
5 ENQ	12 CR (Retorno de carro)	21 NAK	29 GS
6 ACK	14 SO	22 SYN	30 RS

2.2. Variables, tipos de datos y representación

7 BEL	15 SI	23 ETB	31 US 127 DEL (Delete)
-------	-------	--------	---------------------------

A partir del número 32 hasta el 126 se encuentran definidos los caracteres *imprimibles*, como se muestra en la Tabla 2.7. El caracter 32 corresponde a un espacio.

Tabla 2.7: Caracteres imprimibles ASCII

32	␣	!	"	#	\$	%	&	'
40	(	)	*	+	,	-	.	/
48	0	1	2	3	4	5	6	7
56	8	9	:	;	<	=	>	?
64	@	A	B	C	D	E	F	G
72	H	I	J	K	L	M	N	O
80	P	Q	R	S	T	U	V	W
88	X	Y	Z	[	\	]	^	_
96	`	a	b	c	d	e	f	g
104	h	i	j	k	l	m	n	o
112	p	q	r	s	t	u	v	w
120	x	y	z	{		}	~	

En esta tabla se puede apreciar la disposición de los caracteres alfabéticos y alfanuméricos. Por ejemplo, al caracter @ le corresponde el número 64 en decimal, y al caracter A (a mayúscula) le corresponde el número 65.

Posteriormente se asignaron los 127 números restantes (del 128 al 255), dentro de los cuales incluyen los caracteres latinos con acentos como á, é, etc.). La tabla ASCII completa puede ser consultada en internet ASCII (2015).

Dado que una variable de tipo caracter almacena el número correspondiente a su código ASCII, se puede realizar operaciones aritméticas sobre ésta. Por ejemplo, si en una variable se almacena el caracter A (es decir, el número 65 en decimal), al sumarle 1 se obtendrá el caracter B, cuyo número asignado es 66.

Arreglos

A medida que los algoritmos se hacen más complejos, y aumenta el número de datos, se hace necesario usar uno o más arreglos para solucionar el problema. Estos se usan para almacenar los datos de entrada, los datos resultantes del proceso del algoritmo o los datos que se ofrecerán como salida. A los arreglos que almacenan datos simples se les denomina *arreglos unidimensionales*.

La Figura 2.7 muestra un arreglo que llamaremos *x*, el cual almacena un número entero en cada posición. A cada posición en el arreglo le corresponde un *índice*, representado

2. ESTRUCTURAS BÁSICAS PARA CONSTRUCCIÓN DE ALGORITMOS

como un número entero que indica exactamente a cual elemento (dato almacenado) estamos haciendo referencia. Los índices, por lo general, se enumeran desde 0, aunque existen lenguajes de programación en los cuales se enumeran desde 1.

0	1	2	3	4	5	6
50	100	100	100	200	1	5

Figura 2.7: Arreglo unidimensional de números enteros

De este arreglo podemos decir que:

- Contiene seis (6) elementos (6 datos).
- Almacena números de tipo entero.
- El número 50 se encuentra en la posición 0.
- Los elementos (datos almacenados) en la posición 1 y 2 de arreglo son iguales.
- El elemento en la posición 3 del arreglo es el mayor de todos.
- El elemento en la posición 4 es el menor de todos.
- El elemento en la posición 5 (el último) es el número 5, lo cual es una simple coincidencia para este caso.

El arreglo anterior también puede ser representado conceptualmente con una secuencia de elementos, en donde el orden de los elementos señala su posición dentro del arreglo:

$x = [50, 100, 100, 200, 1, 5]$

Una característica importante del uso de arreglos consiste en que se deben usar estructuras repetitivas para almacenar y extraer sus datos, debido a que el acceso a ellos se realiza por medio de índices. Por lo anterior, si necesitamos llenar un arreglo de n elementos, se deberá usar una estructura repetitiva que realice n asignaciones, una para cada posición.

La implementación de los arreglos varía de acuerdo con el lenguaje de programación. Algunas características que se pueden encontrar (o no) de acuerdo con el lenguaje elegido son:

- Los índices no necesariamente deben ser numéricos.
- No es necesario definir con anticipación el tamaño máximo del arreglo, es decir, el máximo número de elementos que puede almacenar. Dicho en otras palabras, el arreglo puede *crecer* o *contraerse* dinámicamente.

- No se requiere una variable adicional para llevar la cuenta de los elementos almacenados en el arreglo.

A no ser que se diga lo contrario, en los ejemplos presentados se asumirá que:

- Los subíndices de los arreglos son numéricos e inician en cero.
- Los arreglos pueden crecer y contraerse dinámicamente, pero se deberá llevar la cuenta del tamaño actual del arreglo.

Por definición, un arreglo es una colección de datos. Pero, ¿qué sucede si cada uno de los datos almacenados dentro de cada posición no es un dato simple, sino un arreglo que almacena datos simples? Tendremos entonces un *arreglo de arreglos*, llamado *arreglo bidimensional*, o *matriz*, como lo muestra la Figura 2.8.

	0	1	2	3	4	5
0	-1	1	2	3	4	5
1	6	7	8	9	10	11
2	-1	13	-1	-1	-1	-1
3	-1	19	-1	-1	-1	-1
4	-1	25	-1	-1	-1	-1
5	-1	31	-1	-1	-1	-1

Figura 2.8: Arreglo bidimensional de números enteros

Ahora se necesitan dos índices para acceder los datos del arreglo. El primer índice representará la *fila*, y el segundo la *columna*. Del arreglo presentado, se puede decir que:

- Tiene 6 filas y 6 columnas, por lo cual almacenará hasta 36 datos.
- Tiene datos mayores o iguales que cero en las dos primeras filas (filas numeradas con índice 0 y 1) y en la segunda columna (columna numerada con índice 1).
- La segunda fila contiene los datos 6, 7, 8, 9, 10 y 11.
- La segunda columna contiene los datos 1, 7, 13, 19, 25, 31.
- El elemento en la posición [2, 1] es 13.
- El elemento en la posición [1, 4] es 10.

La matriz anterior puede ser representada de la siguiente forma:

```
x = [ [-1, 1, 2, 3, 4, 5],      x[0] = [-1, 1, 2, 3, 4, 5]
      [6, 7, 8, 9, 10, 11],    x[1] = [6, 7, 8, 9, 10, 11]
      [-1, 13, -1, -1, -1, -1], x[2] = [-1, 13, -1, -1, -1, -1]
      [-1, 19, -1, -1, -1, -1], x[3] = [-1, 19, -1, -1, -1, -1]
      [-1, 25, -1, -1, -1, -1], x[4] = [-1, 25, -1, -1, -1, -1]
      [-1, 31, -1, -1, -1, -1]  x[5] = [-1, 31, -1, -1, -1, -1] ]
```

Las matrices no necesariamente deben ser *cuadradas*, es decir, no es obligatorio que tengan el mismo número de filas y columnas. De forma general, se dice que una matriz tiene n filas por m columnas, y por lo tanto almacenará hasta $n * m$ datos.

Cadenas de caracteres

Las cadenas de caracteres son un tipo de datos usado con mucha frecuencia en diferentes problemas, ya que nos permiten almacenar datos cuya representación es más cercana a los datos que se manejan a diario. Las cadenas de caracteres son un tipo flexible de datos que nos permite almacenar un nombre, un apellido, un correo electrónico, la dirección de una página web, etc. en una sola variable.

Una cadena de caracteres (o simplemente cadena) puede ser definida como una secuencia de *caracteres* (letras, números, signos de puntuación, etc.). En algunos lenguajes de programación, se debe adicionar al final de la cadena un caracter especial, llamado **caracter nulo** o simplemente **nulo** (null en inglés). A este caracter le corresponde el primer código ASCII, es decir, el número 0, y se representa con el caracter `\0`. Si bien en los algoritmos se almacenan *caracteres* en las cadenas, los programas resultantes realmente se están almacenando los códigos ASCII correspondientes.

En este texto se asumirá que las cadenas de caracteres deben estar terminadas con el caracter nulo. Esto simplifica la construcción de los algoritmos, ya que podemos determinar con exactitud en dónde terminan.

Las cadenas de caracteres pueden ser vistas como *arreglos de caracteres*, o también como arreglos de números cuyo valor se encuentra entre 0 y 255, en las cuales el último elemento siempre es el carácter NUL (nulo). Esto permite que en algunos lenguajes de programación el uso de las cadenas de caracteres sea similar al uso de los arreglos.

Sin embargo, lo contrario no siempre se cumple. Es decir, una cadena de caracteres puede verse como un arreglo de caracteres, pero un arreglo de caracteres puede contener cero, una o más cadenas.

Tipos compuestos de datos

A medida que aumenta la cantidad de datos que debe procesar un algoritmo, se hace necesario *organizarlos* usando alguna estrategia que permita gestionar varios datos en conjunto. Por ejemplo, en un algoritmo en el cual se debe procesar datos de muchas personas, sería conveniente *agrupar* los datos para cada persona (sus nombres, apellidos, teléfono, dirección, etc.). En algunos lenguajes de programación a los tipos compuestos se les denomina *estructuras de datos*, o simplemente *estructuras*.

En algoritmia también podemos definir tipos compuestos de datos, que permiten agrupar diferentes atributos posiblemente de diferentes tipos. Para el ejemplo anterior, podemos definir el tipo *persona*, que contendrá diferentes *atributos* para almacenar los nombres (cadena de caracteres), apellidos (cadena de caracteres), teléfono (número o posiblemente cadena de caracteres) y dirección (cadena de caracteres). El tipo definido se presenta en el Código 2.8.

Código 2.8: Definición de un tipo compuesto de datos

```
1 /* Tipo compuesto de datos Persona*/  
2 tipo persona  
3   cadena nombres  
4   cadena apellidos  
5   entero telefono  
6   cadena direccion  
7 fin tipo
```

Una vez definido este nuevo tipo compuesto, podemos definir nuevas variables de este tipo (*persona*, en el ejemplo). En cada variable se almacenarán los datos completos para una persona, con lo cual no tendremos necesidad de usar variables diferentes para cada uno de sus atributos.

Referencias o Apuntadores

Son un tipo especial de variables, las cuales no almacenan un dato en sí, sino una referencia a otra variable. Los apuntadores (también llamados *punteros*) son un aspecto más de implementación que de algoritmia, pero son muy importantes en situaciones en las cuales se desea controlar de forma explícita la cantidad de memoria usada por un programa, y también para facilitar de cierta forma el acceso y la manipulación de algunas estructuras de datos como arreglos o tipos complejos.

Es importante tener en cuenta que en algunos lenguajes de programación *no existe* el concepto de apuntador, por lo cual no debemos depender totalmente de ellos para solucionar un problema. De hecho, muchas personas recomiendan no usarlos, ya que argumentan que son *peligrosos* y pueden causar fallas *catastróficas* en un programa e incluso en el

2. ESTRUCTURAS BÁSICAS PARA CONSTRUCCIÓN DE ALGORITMOS

sistema. Esto se debe a que al manipular apuntadores se está manipulando directamente la memoria del computador *sin protección*, lo cual, si no se realiza correctamente, puede causar problemas. No obstante, la mayoría de sistemas operativos actuales ofrecen mecanismos automáticos para *controlar* los accesos ilegales a memoria y terminar los programas que han causado estos fallos.

Si son usados correctamente, los apuntadores son una herramienta muy flexible y poderosa para el programador que le permite simplificar sus algoritmos, y por tanto, sus programas. Además, si el programador conoce el modelo de organización de memoria del sistema, puede usarla de forma más eficiente. Esto puede ser crucial en aplicaciones de desempeño crítico o en sistemas con memoria limitada, por ejemplo en *sistemas empotrados*.

En el Código 2.9 se presentan algunas instrucciones que ilustran el manejo de referencias. La sintaxis usada es similar al uso de apuntadores en lenguaje C.

Código 2.9: Instrucciones de manejo de referencias

```
1 /*
2  * Prueba de uso de referencias
3  */
4 algoritmo referencias
5 entero a
6 entero b
7 ref entero c //Declaración del apuntador
8 inicio
9   a = 5 c = ref a //Almacenar la referencia a la variable en el
10  apuntador
11  imprimir *c
12  *c = 10
13  imprimir *c
14  a = 20
15  imprimir *c
16  imprimir a
17 fin
```

En la Tabla 2.8 se puede apreciar la prueba de escritorio del algoritmo. En el caso del apuntador *c*, almacenará un número “X” que no interesa en el algoritmo, pero que corresponderá a la dirección de memoria de la variable *a* cuando se ejecute el respectivo programa.

Tabla 2.8: Prueba de escritorio referencias

Instrucción	a	b	c	Salida
inicio				
a = 5	5			
c = ref a			X	
imprimir *c				5

Continued on next page

Tabla 2.8: Prueba de escritorio referencias

Instrucción	a	b	c	Salida
*c = 10	10			
imprimir *c				10
a=20	20			
imprimir *c				20
imprimir *a				20
fin				

2.3. Condiciones

En algoritmia, una *condición* es una expresión *lógica* que se utiliza generalmente en las estructuras de decisión y repetición. Las expresiones involucran cero o más variables y algunas lógicas entre ellas.

Las condiciones con frecuencia incluyen *operadores relacionales*, que permiten realizar *comparaciones* entre dos datos para determinar si son iguales, diferentes, o uno es menor que otro, mayor o igual, etc. Es posible comparar variables con otras variables, o con valores constantes. Cuando una variable aparece en una condición, el algoritmo *evalúa* la condición usando el valor actual almacenado en la variable.

También es posible definir *condiciones compuestas*, es decir, condiciones en las cuales aparecen varias operaciones lógicas encadenadas mediante los operadores lógicos y (AND), y o (OR).

Por ejemplo, las siguientes expresiones se evaluarán a verdadero o falso dependiendo del valor que se encuentre almacenado en las variables en ese punto del algoritmo. La sintaxis usada dependerá de las convenciones establecidas en el algoritmo.

```

a = 5           Verdadero si el valor actual de a es 5
a = b           Verdadero si el valor actual de a es igual
                 al valor actual de b
a < b           Verdadero si a es menor que b a >= b
Verdadero si a es mayor o igual que b
a != b         Verdadero si a es diferente de b (a > b) y (a < c)
Verdadero si el valor de a está entre b y c ! (a > b)      Negación:
verdadero si a es menor o igual que b
not (a > b)      Negación: verdadero si a es menor o igual que b

```

Operador lógico y (AND)

El operador lógico *AND* representa una *conjunción* lógica, en la cual para que el resultado de la expresión sea verdadero, tanto el elemento de la izquierda como el de la derecha deben ser evaluados a verdadero. La Tabla 2.9 muestra el resultado del operador *AND* sobre dos variables.

Tabla 2.9: Operador *AND*

A	B	A y B
V	V	V
V	F	F
F	V	F
F	F	F

A continuación se presentan algunas condiciones en las cuales se usa el operador lógico *AND*.

(a = 5) y (b = 2) (a < 10) y (a > b)
(a > b) y (b > a) Cuidado cuidado!

La última condición *nunca* se evaluará a verdadero, debido a que si el valor actual de a es mayor que b, entonces b no puede ser mayor que a. En este caso, *se está cometiendo un error de lógica de programación*. Si este algoritmo se lleva a un programa, el compilador posiblemente no realice ninguna advertencia y el programa tendrá un funcionamiento diferente al esperado.

Operador lógico o (OR)

El operador lógico *OR* representa una *disyunción* lógica, en la cual para que el resultado de la expresión sea verdadero, cualquiera de los dos elementos (o ambos) deben ser verdaderos. La Tabla 2.10 muestra la operación *OR* sobre dos variables.

Tabla 2.10: Operador *OR*

A	B	A o B
V	V	V
V	F	V
F	V	V
F	F	F

Por ejemplo, el operador *OR* se puede usar de la siguiente forma:

```
(a > b) o (a > c)    #+end_example
```

En este caso, /aún/ no se puede determinar si el valor actual de `~a~` es mayor que el valor actual de `~b~` y también que el valor actual de `~c~`. Si esto es necesario, se debe cambiar la condición por:

```
#+begin_example  
(a > b) y (a > c)
```

Esta condición sólo será verdadera si el valor actual de `a` es mayor que `b` y también es mayor que `c`.

Condiciones compuestas

Una condición compuesta se usa para evaluar de una sola vez varias condiciones simples, que se encuentran unidas mediante los operadores lógicos *AND* y *OR*. Considere las siguientes expresiones que pueden surgir del planteamiento de un problema:

Si la temperatura se encuentra entre 32 y 34 grados, inclusive Si la nota del estudiante se encuentra entre 0 y 5, inclusive Si la nota del estudiante no se encuentra entre 0 y 5

Las condiciones correspondientes pueden ser:

```
(temp >= 32) y (temp <= 43)  
(nota >= 0.0) y (nota <= 5.0)  
(nota < 0) o (nota > 5)
```

2.4. Elementos de control de flujo

En la programación imperativa, el algoritmo *define* el orden en el cual se ejecutan las instrucciones. Además, es necesario que el algoritmo incluya algunos mecanismos para permitir que:

- Se ejecuten secuencias diferentes con base en una condición o el valor de una variable para lo cual se usan *estructuras de decisión o selección*.
- Se ejecute una misma secuencia cero, una o más veces, con base en una condición, para lo cual se usan *estructuras de repetición*, también llamadas *estructuras repetitivas o ciclos*.

- Se pueda *cancelar* una iteración, o se pueda *omitir* parte de una secuencia dentro de una estructura repetitiva y realizar una nueva iteración.
- Se ejecute una secuencia antes de poder continuar, para lo cual se usan *subrutinas*.
- Se pueda *terminar* el algoritmo en un punto determinado, sin ejecutar todas las instrucciones del mismo.

Una estructura de decisión permite que un algoritmo ejecute una o dos secuencias de instrucciones *excluyentes entre sí*³. Para lograr este propósito, se debe especificar una *condición*, que en caso de cumplirse, hará que el algoritmo ejecute una determinada secuencia de instrucciones, y en caso contrario, pueda ejecutar otra secuencia diferente.

La estructura de selección puede ser vista como un caso especial de una estructura de decisión, en el cual *no se usan condiciones* para determinar la secuencia a ejecutar, sino que la decisión se toma dependiendo *del valor actual de una variable*. Además, a diferencia de la estructura de decisión, la estructura de selección permite ejecutar más de dos secuencias diferentes.

Las estructuras repetitivas permiten ejecutar la misma secuencia de instrucciones por *cero, una o más veces* (es decir, ejecutar repetidamente una secuencia, o *iterar*). Estas estructuras, al igual que la estructura de decisión, se basan en *condiciones* que se deben cumplir para que se repita la secuencia definida. Cada lenguaje de programación tiene sus propias versiones de las estructuras repetitivas, pero a nivel conceptual se pueden agrupar en tres grandes grupos: *mientras hacer* (while do), *hacer mientras* (do while) y *para hacer* (for).

Debido a que este tipo de estructuras *repiten* la secuencia especificada con base en una *condición*, y generalmente en una condición participan una o más variables, *es necesario* alterar de alguna forma las variables que participan en la condición para garantizar que en algún momento *se va a terminar la iteración*. En caso contrario, el algoritmo nunca terminará.

En los siguientes apartados se describen y presentan algunos ejemplos de las estructuras de control de flujo de ejecución de un algoritmo.

Estructura de decisión *si - sino* (if - else)

Una estructura de decisión permite que un algoritmo *se comporte* de forma diferente, dependiendo del resultado de evaluar una condición, es decir, que ejecute secuencias de instrucciones diferentes con base en una *decisión lógica*.

La forma general de una estructura de decisión se presenta en el Código 2.10.

³Es decir, que se ejecuta solo una de las dos secuencias de instrucciones.

Código 2.10: Forma general de la estructura de decisión

```

1 secuencia A
2 si COND
3   secuencia B
4 sino
5   secuencia C
6 fin si
7 secuencia D

```

En donde *secuencia* corresponde a una secuencia de cero o más instrucciones. El mismo algoritmo representado en diagrama de flujo se muestra en la Figura 2.9.

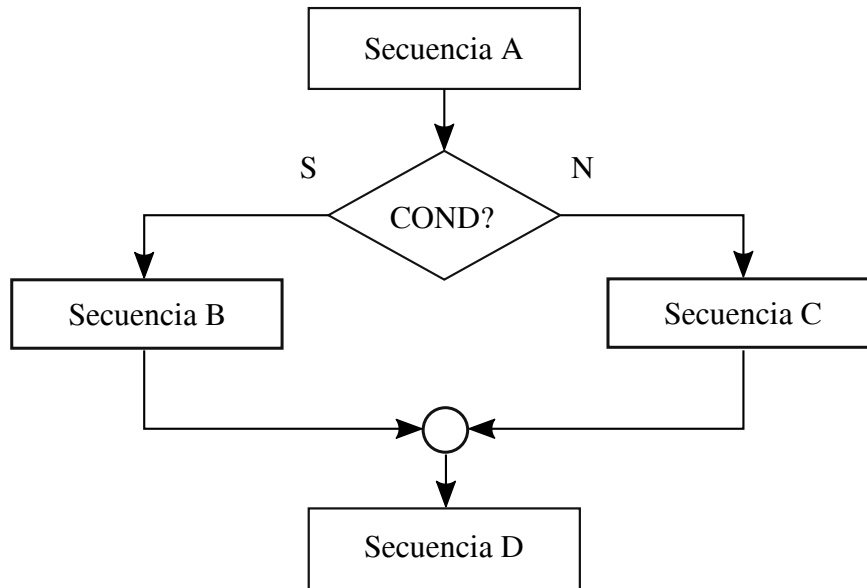


Figura 2.9: Estructura de decisión en diagrama de flujo

Al ejecutar esta sección de un algoritmo, inicialmente se ejecuta la *secuencia A*. Luego se evalúa la condición *COND*. Si se obtiene como resultado **verdadero** (es decir, la condición se cumple), se ejecuta *secuencia B*. En caso contrario, si al evaluar *COND* se obtiene como resultado **falso**, se ejecutará *secuencia C*. Luego de la decisión, sin importar cual *secuencia* se haya ejecutado, se ejecutará *secuencia D*. Por ejemplo, el algoritmo presentado en el Código 2.11 permite verificar si un número ingresado por el usuario es mayor que cero mediante una estructura de decisión.

Código 2.11: Verificar si un número es mayor que cero

```

1 /* Lee un número entero y verifica si es mayor que cero */
2 algoritmo esMayorQueCero
3 entero x
4 inicio
5   imprimir "Por favor ingrese un número"

```

2. ESTRUCTURAS BÁSICAS PARA CONSTRUCCIÓN DE ALGORITMOS

```
6 leer x
7 si x > 0
8     imprimir "El número ", x, " es mayor que cero"
9 sino
10    imprimir "El número ", x, " no es mayor que cero"
11 fin si
12 imprimir "Fin del algoritmo"
13 fin
```

Al ejecutar el algoritmo, las instrucciones que se ejecutan dependerán del valor que ingrese el usuario. La prueba de escritorio se presenta en la Tabla 2.11, en la cual se asigna el valor de 10 en la instrucción de lectura. Para las instrucciones *imprimir* solo se escribe esta instrucción y en la columna de salida por pantalla se muestra el mensaje. En cada condición, se coloca al frente si se evalúa a verdadero (V) o falso (F).

Tabla 2.11: Prueba de escritorio con dato mayor que cero

Instrucción	x	Salida
inicio		
imprimir		Por favor ingrese un número
leer x	10	
si x >0 : V		
imprimir		El número 10 es mayor que cero
fin si		
imprimir		Fin del algoritmo
fin		

Por el contrario, si el dato proporcionado por el usuario (y asignado en el algoritmo) es menor o igual que cero, el algoritmo producirá el resultado que se muestra en la Tabla 2.12 diferente:

Tabla 2.12: Prueba de escritorio con dato menor o igual que cero

Instrucción	x	Salida
inicio		
imprimir		Por favor ingrese un número
leer x	-5	
x >0 : F		
imprimir		El número -5 no es mayor que cero
fin si		
imprimir		Fin del algoritmo
fin		

En ambos casos se imprimirá el mensaje *Fin del algoritmo*, debido a que se encuentra *después* de la estructura de decisión.

Dentro de la secuencia de instrucciones de una estructura de decisión (y de cualquier estructura algorítmica) se puede usar otras estructuras. En este caso se dirá que las estructuras se encuentran *anidadas*, es decir, contenidas una dentro de las otras.

La representación en diagrama de flujo de una estructura de decisión anidada dentro de otra se aprecia con mayor facilidad en un diagrama de flujo como el que se muestra en la Figura 2.10.

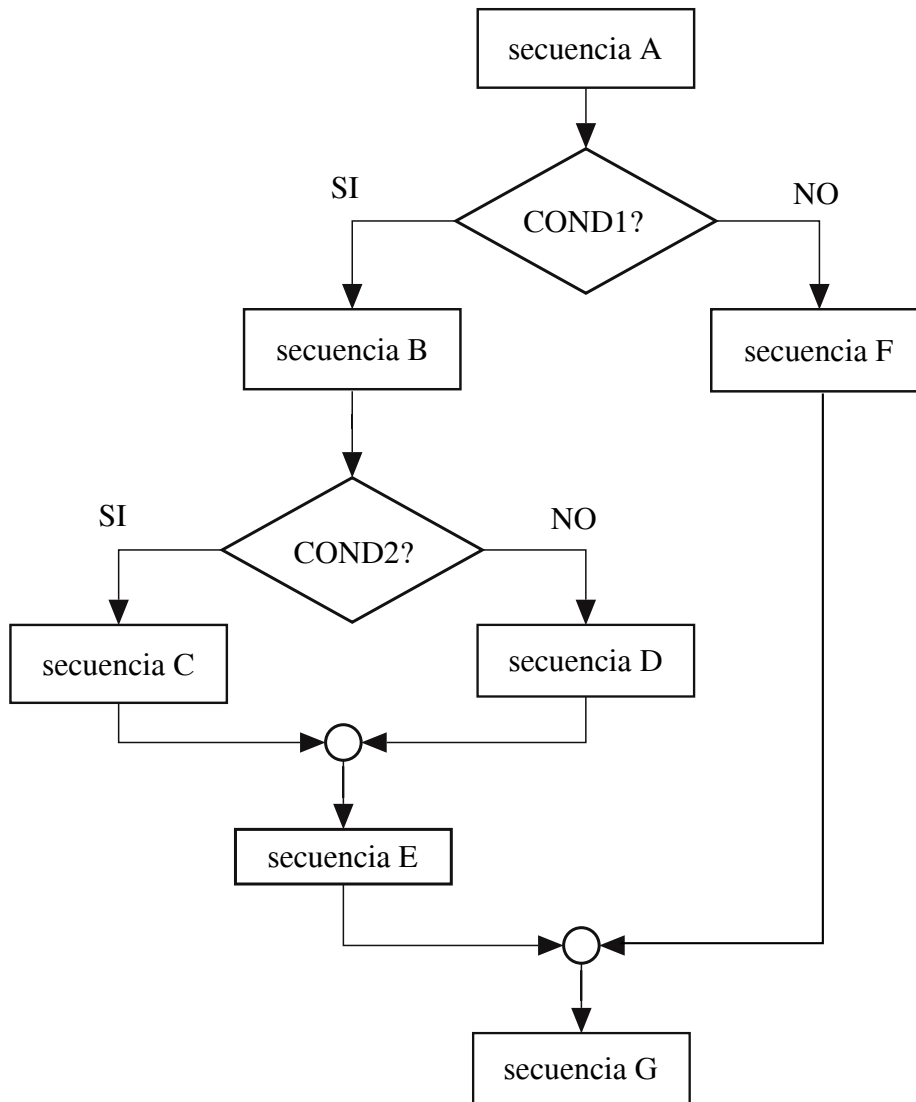


Figura 2.10: Estructura de decisión anidada

El funcionamiento de la estructura es el siguiente:

2. ESTRUCTURAS BÁSICAS PARA CONSTRUCCIÓN DE ALGORITMOS

- Inicialmente se ejecutará secuencia A.
- Si COND1 es verdadera, se ejecutará secuencia B y luego se tomará otra decisión:
 - Si COND2 es verdadera (se cumple), se ejecutará secuencia C.
 - Si COND2 es falsa (no se cumple), se ejecutará secuencia D.
- Si se cumplió COND1, se ejecutará secuencia E sin importar si antes se ejecutó secuencia C o secuencia D.
- Si por el contrario COND1 no se cumplió, se ejecutará solamente secuencia F.
- Finalmente se ejecutará secuencia G, sin importar cual condición se cumplió.

El Código 2.12 implementa la misma estructura. Observe que el primer `sino` (línea 6) corresponde a la estructura anidada que comienza en la línea 4, y el `fin si` de la línea 8 cierra esta estructura. El `sino` en la línea 10 corresponde a la estructura externa (que inicia en la línea 3, igual que el `fin si` de la línea 12). Por lo tanto, podemos concluir que una estructura anidada se debe encontrar *completamente* contenida dentro de otra estructura.

Código 2.12: Estructura de decisión anidada

```
1 //Estructuras de decisión anidadas
2 secuencia A
3 si COND1
4     secuencia B si COND2
5         secuencia C
6     sino
7         secuencia D
8     fin si
9     secuencia E
10 sino
11     secuencia F
12 fin si
13 secuencia G
```

En ocasiones es necesario ejecutar una secuencia de instrucciones sólo si se cumple (o si no se cumple) determinada condición. Ante esta situación se puede usar una estructura de decisión en la cual solo se define una de las dos secuencias como se muestra en el Código 2.13.

Código 2.13: Estructura de decisión con una sola secuencia

```
1 secuencia A
2 si COND secuencia B
3 fin si
4 secuencia C
```

En este caso, inicialmente se ejecutará secuencia A. secuencia B sólo se ejecutará si se cumple COND, y la secuencia C se ejecutará sin importar si COND se cumple o no.

Estructura de selección (switch - case)

Una estructura de selección permite ejecutar diferentes secuencias de instrucciones *dependiendo del valor que tenga una variable*. Su representación general en diagrama de flujo se muestra en la Figura 2.11.

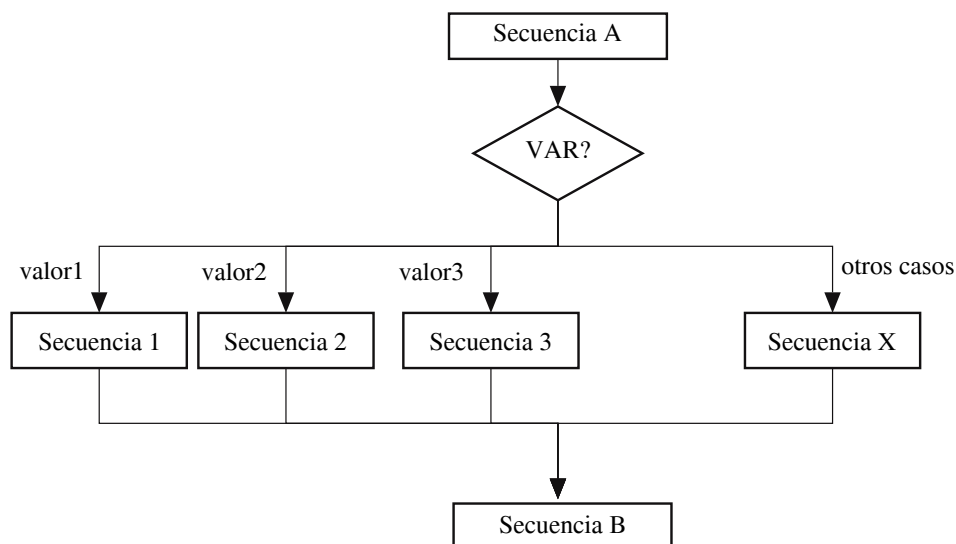


Figura 2.11: Estructura de selección

La misma estructura se implementa en el Código 2.14.

Código 2.14: Forma general de la estructura de selección

```

1 secuencia A
2 seleccionar var entre
3 caso (valor1):
4     secuencia 1
5 caso (valor2):
6     secuencia 2
7 caso (valor3):
8     secuencia 3
9 otros casos:
10    secuencia x
11 fin seleccion
12 secuencia B
  
```

El funcionamiento de la estructura es como se explica a continuación: Inicialmente se ejecutará secuencia A. Luego, dependiendo del valor que se encuentre almacenado en var, se ejecutará la secuencia correspondiente. Si en var se encuentra almacenado un dato diferente a valor1, valor2 y valor3, se ejecutará secuencia 3. La secuencia definida como caso alternativo (secuencia x), se ejecutará si la var no tiene ninguno de los va-

2. ESTRUCTURAS BÁSICAS PARA CONSTRUCCIÓN DE ALGORITMOS

lores anteriores. Finalmente, sin importar cual secuencia se haya ejecutado, se ejecutará secuencia B.

Podemos usar la estructura de selección para determinar si el usuario ha ingresado ciertos números enteros, como se muestra en el Código 2.15.

Código 2.15: Ejemplo de estructura de selección

```
1 /* Lee un número y verifica si se ingresó 1, 2 o 3*/
2 algoritmo seleccionarNumeros
3 entero x
4 inicio
5     imprimir "Ingrese un número"
6     leer x
7     seleccionar x entre
8         caso 1:
9             imprimir "Usted ingresó 1"
10        caso 2:
11            imprimir "Usted ingresó 2"
12        caso 3:
13            imprimir "Usted ingresó 3"
14        otros casos:
15            imprimir "Usted no ingresó ni 1 2 o 3"
16 fin seleccion
17 fin
```

Si se realiza la prueba de escritorio con un valor de 2 para la variable x, se obtendrá el resultado que se muestra en la Tabla 2.13.

Tabla 2.13: Prueba de escritorio seleccionar números

Instrucción	x	Salida
inicio		
imprimir		Ingrese un número
leer x	2	
seleccionar x		
caso 2		
imprimir		Usted ingresó 2
fin selección		
fin		

Estructura *mientras hacer* (While Do)

La estructura *mientras hacer* (o simplemente *mientras*) permite repetir una misma secuencia de instrucciones durante cero, una o más veces. Su formato general se presenta en el Código 2.16.

Código 2.16: Forma general de la estructura *mientras hacer*

```

1 secuencia 1
2 mientras COND hacer
3   secuencia 2
4 fin mientras
5 secuencia 3

```

Inicialmente se ejecutará secuencia 1. Luego, se evaluará COND. Si se cumple, se ejecutará secuencia 2, *mientras* se siga cumpliendo la condición COND. Cuando COND deja de cumplirse, es decir, se evalúa a falso, se ejecutará secuencia 3.

El número de veces que se repite secuencia 2 está determinada por dos elementos:

- La condición COND: Si se desea que secuencia 2 se ejecute *por lo menos* una vez, debemos garantizar que secuencia 1 establezca los valores de las variables que participan en la condición de forma que ésta *se cumpla*, es decir, se evalúe a verdadero. En caso contrario, si COND inicialmente no se cumple, la ejecución del algoritmo *omitirá* la estructura *mientras* y ejecutará inmediatamente secuencia 3.
- Las modificaciones sobre las variables que participan en COND que se realizan en secuencia 2. Si secuencia 2 se ejecuta al menos una vez, *se debe garantizar* que contenga instrucciones que modifiquen las variables que participan en la condición COND de forma que en algún momento esta se evalúe a falso. En caso contrario, secuencia 2 se ejecutará de forma indefinida y el algoritmo no terminará.

Por lo general, las instrucciones para modificar las variables que participan en la condición se especifican *justo antes* del *fin mientras*. Dicho de otra forma, los últimos pasos antes de repetir la secuencia de instrucciones se encargan de modificar las variables que participan en la condición que controla la estructura repetitiva.

Podemos usar una estructura repetitiva para imprimir los números de 1 a 10 sin necesidad de especificar 10 instrucciones imprimir. De hecho, este algoritmo permitirá imprimir la secuencia de números enteros desde 1 hasta cualquier número arbitrario, ya que solo es necesario cambiar el valor constante que controla la condición (10, en este caso). Este uso de la estructura repetitiva se muestra en el Código 2.17. Se puede observar la forma general de uso de la estructura: Inicialización de la variable que participa en la condición *antes* de la estructura y *modificación* de la variable dentro de la estructura.

Código 2.17: Imprimir los números de 1 a 10

```

1 /* Imprime los números de 1 a 10 */
2 algoritmo numerosUnoADiez
3 entero i
4 inicio
5   i = 1
6   mientras i <= 10
7     imprimir i

```

2. ESTRUCTURAS BÁSICAS PARA CONSTRUCCIÓN DE ALGORITMOS

```
8   i = i + 1
9   fin mientras
10 fin
```

Dado que el valor inicial establecido es 1, se cumple la condición $i \leq 10$. Así que entramos a ejecutar la secuencia de instrucciones dentro de la estructura repetitiva. La última instrucción antes de `fin mientras` garantiza que la variable i se modifica (incrementa), por lo cual llegará a tomar el valor de 11. En ese momento se dejará de cumplir la condición, por lo cual la estructura repetitiva terminará. La prueba de escritorio de este algoritmo se puede apreciar la Tabla 2.14.

Tabla 2.14: Prueba de escritorio imprimir números de 1 a 10

Instrucción	i	Salida
inicio		
i = 1	1	
mientras i <= 10 : V		
imprimir i		1
i = i + 1	2	
fin mientras		
mientras i <= 10 : V		
imprimir i		2
i = i + 1	3	
fin mientras		
(Iteraciones)		
mientras i <= 10 : V		
imprimir i		10
i = i + 1	11	
fin mientras		
mientras i <10 : F		
fin mientras		
fin		

Este algoritmo implementa lo explicado anteriormente:

- Inicialmente se almacena el valor de 1 en la variable i , que será usada en la estructura repetitiva.
- La condición $i \leq 10$ se cumple, debido a que la variable i se estableció en 1.
- Dentro de la estructura repetitiva, se imprime el valor actual de i (inicialmente 1).
- La última instrucción dentro de la estructura repetitiva *modifica* el valor almacenado en la variable i , para evitar que la estructura se repita indefinidamente. Al incremen-

tar *i*, garantizamos que la condición *dejará de cumplirse en algún momento* (cuando *i* tome el valor de 11).

Observe que la variable *i* se imprime *antes* de ser incrementada, debido a que *antes* de la estructura repetitiva se asignó el valor de 1 y este valor debe ser mostrado por pantalla. A esta estrategia se le conoce como *post-incremento*: Primero se realiza la acción (imprimir, en este caso), y después se incrementa la variable.

Es posible construir otra solución, en la cual la variable *i* se establece en 0 inicialmente. En este caso, se deberá incrementar la variable y luego imprimir su valor, como se muestra en el Código 2.18. La estrategia de incrementar antes de realizar una acción se denomina *pre-incremento*.

Código 2.18: Imprimir números de 1 a 10, pre-incremento

```

1  /* Imprime los números de 1 a 10 */
2  algoritmo numerosUnoADiez
3  entero i
4  inicio
5      i = 0
6      mientras i < 10
7          i = i + 1
8          imprimir i
9      fin mientras
10 fin

```

En este caso, se debe cambiar la condición, para evitar que también se imprima el valor de 11. Se deja como ejercicio la prueba de escritorio de este algoritmo.

La representación en diagrama de flujo de la estructura *mientras* se muestra en la Figura 2.12. Tenga en cuenta que es *similar* a una estructura de decisión, con la diferencia que se tiene in *ciclo* o *bucle*, es decir, luego de la secuencia de instrucciones se evalúa de nuevo la condición para determinar si se debe continuar repitiendo la secuencia o se debe pasar a ejecutar otra secuencia.

Estructura *hacer mientras* (Do while)

Esta estructura es similar a la estructura *mientras hacer*, con la diferencia que permite repetir una misma secuencia de instrucciones por *una o más veces*, mientras se cumpla la condición especificada.

Este comportamiento se logra evaluando la condición *después* de ejecutar la secuencia. Si la condición se cumple, se repite la secuencia. En caso contrario, no se realiza repetición. La forma general de la estructura *hacer mientras* se presenta en el Código 2.19.

Código 2.19: Forma general de la estructura *hacer mientras*

```

1  secuencia 1

```

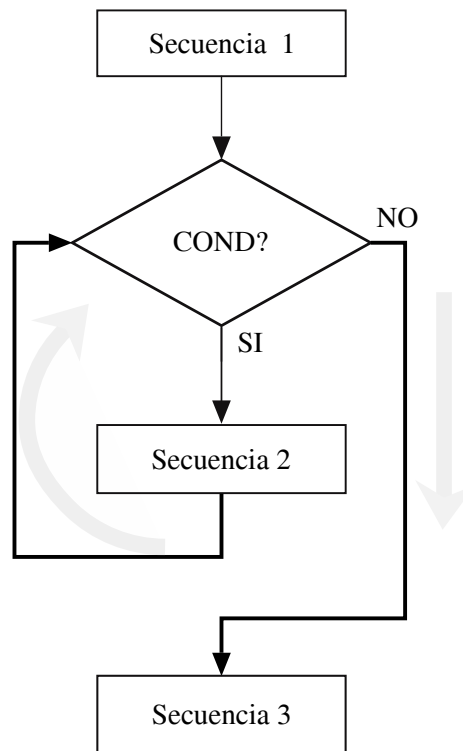


Figura 2.12: Estructura *mientras hacer*

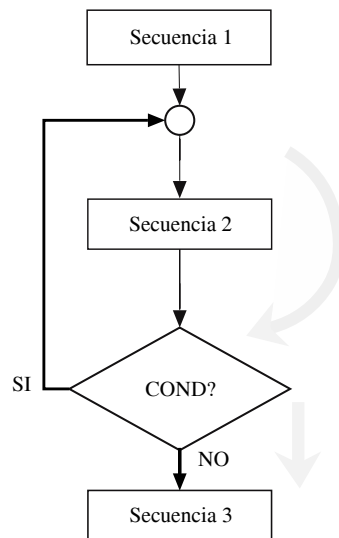
```
2 hacer
3   secuencia 2
4 mientras COND
5   secuencia 3
```

Inicialmente se ejecutará secuencia 1. Luego, se ejecutará secuencia 2, y después de ejecutarla se verifica COND. Si es verdadera, se repite secuencia 2. Cuando COND se evalúe a falso, se terminará la repetición y se ejecutará secuencia 3.

A diferencia de la estructura *mientras hacer*, secuencia 2 se ejecutará *por lo menos una vez* (es decir, se ejecutará por lo menos *una iteración*, debido a que la condición se evalúa *después* de haberla ejecutado). De nuevo, se debe garantizar que dentro de secuencia 2 se incluya una o más instrucciones que modifiquen las variables que participan en la condición COND.

La representación en diagrama de flujo de la estructura *hacer mientras* se muestra en la Figura 2.13. Al igual que en la estructura *mientras*, también existe un *ciclo* o *bucle* que se repetirá mientras la condición se siga cumpliendo.

En algunos textos se usa una estructura repetitiva llamada *repetir hasta*, la cual es básicamente similar a una estructura *hacer mientras* con la condición contraria (es decir,

Figura 2.13: Estructura *hacer mientras*

la negación de la condición que se usaría en una estructura *hacer mientras*).

Podemos usar una estructura *hacer mientras* para *validar* un dato de entrada, es decir, *obligar* al usuario a proporcionar un dato que cumpla con determinadas características. El Código 2.20 implementa un algoritmo que permita leer un número entero, y validar que sea mayor que 0 y menor o igual que 5:

Código 2.20: Validar entrada de datos

```

1 /* * Leer un número entero y repetir la lectura mientras que
2 * el valor leído no esté entre 0 y 5
3 */
4 algoritmo entreCeroYCinco
5 real x
6 inicio
7   hacer
8     imprimir "Ingrese un número entre 0 y 5"
9     leer x
10  mientras x < 0 o x > 5
11  imprimir "Usted ingresó ", x
12 fin

```

La prueba de escritorio del algoritmo se muestra en la Tabla 2.15.

Tabla 2.15: Prueba de escritorio validar entrada de datos

Instrucción	x	Salida
inicio		
imprimir		Ingrese un número entre 0 y 5

Continued on next page

2. ESTRUCTURAS BÁSICAS PARA CONSTRUCCIÓN DE ALGORITMOS

Tabla 2.15: Prueba de escritorio validar entrada de datos

Instrucción	x	Salida
hacer		
leer x	10	
mientras x <0 o x >5 : V		
hacer		
leer x	-1	
mientras x <0 o x >5 : V		
(Iteraciones)		
hacer		
leer x	3	
mientras x <0 o x >5 : F		
imprimir		Usted ingresó 3
fin		

Observe que la condición de la estructura *hacer mientras* se encuentra *negada* con respecto a lo que se pide en el problema. Dado que *mientras* el valor de x no sea válido (sea menor que 0 o mayor que 5), se debe continuar con la lectura del dato. Cuando el usuario ingrese un valor en el rango de 0 a 5, la estructura dejará de repetirse y la ejecución continuará.

Si se utiliza una estructura *repetir hasta*, el algoritmo sería el que se presenta en el Código 2.21.

Código 2.21: Estructura *repetir hasta* para validar datos

```
1 /* * Leer un número entero y repetir la lectura hasta que
2 * el valor leído esté entre 0 y 5
3 */
4 algoritmo entreCeroYCinco
5 real x
6 inicio
7   repetir
8     imprimir "Ingrese un numero entre 0 y 5"
9     leer x
10    hasta x >= 0 y x <= 5
11    imprimir "Usted ingreso ", x
12 fin
```

En esta estructura la condición sí es la planteada en el problema original, que la nota debe estar entre 0.0 y 5.0.

Estructura *para hacer* (for)

La estructura *para hacer* puede ser vista como una estructura *mientras hacer* más compacta, en la cual se especifica la inicialización, la condición y la modificación de las

variables en una sola instrucción.

En el Código 2.22 se muestra la forma general de la estructura *para hacer*.

Código 2.22: Forma general de la estructura *para hacer*

```

1 secuencia A
2 para INICIALIZACIÓN, CONDICIÓN, ACTUALIZACIÓN
3   secuencia B
4 fin para
5 secuencia C

```

Su funcionamiento es como se describe a continuación

- Primero se ejecutará secuencia A.
- Luego se realizará INICIALIZACIÓN *una sola vez*, es decir, la primera vez que se ejecute la estructura repetitiva. En esta inicialización generalmente se asignan los valores iniciales a alguna de las variables que participan en CONDICIÓN, por lo general buscando que ésta se evalúe a verdadero para permitir que secuencia B se ejecute por lo menos una vez.
- Se verifica CONDICIÓN. Si se evalúa a verdadero, se ejecutará secuencia B una vez. De lo contrario, se ejecutará secuencia C inmediatamente.
- Luego se realizará ACTUALIZACIÓN. En ella se deberá modificar alguna de las variables que participan en CONDICIÓN.
- Ahora viene la repetición: se verifica si CONDICIÓN se sigue cumpliendo. En caso afirmativo, se ejecuta de nuevo secuencia B y se realiza ACTUALIZACIÓN. En caso contrario, se termina la estructura repetitiva y se ejecutará secuencia C.

El Código 2.23 muestra cómo se implementaría el algoritmo para imprimir los números de 1 a 10 usando la estructura *para hacer*.

Código 2.23: Imprimir números de 1 a 10 usando *para hacer*

```

1 /* Imprimir los números de 1 a 10 con una estructura para hacer */
2 algoritmo numerosUnoADiez
3   entero i
4   inicio
5     para i = 1, i <= 10, i = i + 1
6       imprimir i
7     fin para
8   fin

```

La prueba de escritorio del algoritmo se muestra en la Tabla 2.16.

Tabla 2.16: Prueba de escritorio estructura *para hacer*

Instrucción	i	Salida
inicio		
para : (i = 1)	1	
imprimir		1
fin para : (i = i + 1)	2	
para: (i <= 10)? : V		
imprimir		2
fin para : (i = i + 1)	3	
(Iteraciones)		
fin para : (i = i + 1)	10	
para: (i <= 10)? : V		
imprimir		10
fin para : (i = i + 1)	11	
para : (i <= 10)? : F		
fin para		
fin		

Podemos observar que la variable *i* se inicializa en 1, y la condición se evalúa de forma similar a una estructura *mientras*. La modificación es en este caso un *incremento*, y se realiza al terminar la estructura repetitiva. Este incremento causará que en alguna de las siguientes repeticiones se deje de cumplir la condición $i \leq 10$.

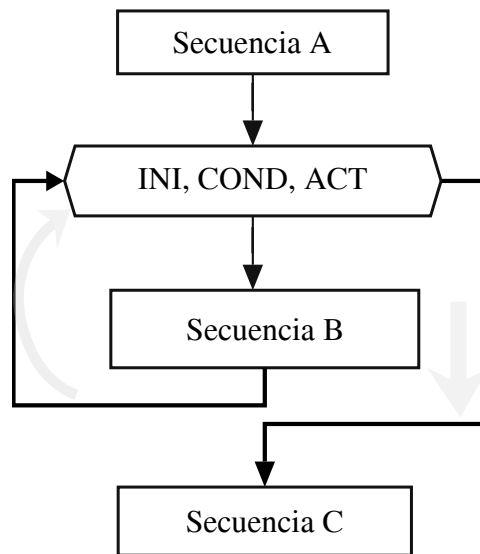
La representación en diagrama de flujo de la estructura *para hacer* se muestra en la Figura 2.14.

La estructura *para hacer* se prefiere cuando se sabe de antemano el número de veces que se debe realizar la repetición de una secuencia. También, la instrucción de actualización usualmente consiste en incrementar la variable que participa en la condición. Recuerde que la inicialización solo se realiza una vez, y que si la condición de la estructura repetitiva se evalúa inicialmente a falso, no se realizará ninguna iteración. Por último tenga en cuenta que la modificación se realiza *automáticamente después* de ejecutar toda la secuencia especificada en la estructura repetitiva, y antes de verificar la condición.

Estructuras repetitivas anidadas

Al igual que las demás estructuras algorítmicas, una estructura repetitiva puede contener otra estructura repetitiva dentro de su secuencia de instrucciones. En este caso también tendremos *anidamiento*.

Muchos elementos humanos están controlados o representados por elementos repetitivos *anidados*. Considere por ejemplo la noción de tiempo. Un año tiene varios meses, un mes tiene varios días, un día tiene varias horas, y así sucesivamente. Si visualizamos estos

Figura 2.14: Estructura *para hacer*

elementos como ruedas o piñones concéntricos, nos damos cuenta que por cada *paso* de una rueda externa, la rueda interna da una vuelta completa. Por ejemplo, para cada año, se recorren los meses de enero a diciembre. Para cada mes, se recorren los días de 1 a 30 (suponiendo que todos los meses tuvieran 30 días). Para cada día, se recorren las horas de 0 a 23. Para cada hora se recorren los minutos, para cada minuto los segundos, y así sucesivamente.

Vamos a expresar la relación entre horas y minutos de un día mediante estructuras repetitivas. Claramente nos damos cuenta que para cada día se deben generar 24 horas (de 0 a 23), y a su vez para cada hora se deben generar 60 minutos (de 0 a 59). Es decir, debemos tener una estructura repetitiva para las horas, y dentro de esta tendremos una estructura repetitiva *anidada* para los minutos. El Código 2.24 presenta esta relación.

Código 2.24: Ejemplo de estructuras repetitivas anidadas

```

1 /* * Generar e imprimir las horas y minutos de un día */
2 algoritmo generarMinutos
3 entero h, entero m
4 inicio
5   para h = 0, h < 24, h = h + 1 //Ciclo para las horas
6     imprimir "Hora ", h //(1)
7     para m = 0, m < 60, m = m + 1 //Para cada hora (h), generar los minutos
8       imprimir h, m //(2)
9     fin para
10  fin para
11 fin

```

Se deja como ejercicio realizar la prueba de escritorio del algoritmo. Encontraremos

que el mensaje (1) se imprimirá exactamente 24 veces (una vez por cada repetición controlada por h), y el mensaje para los minutos (2) se imprimirá exactamente 1440 veces (una vez por cada repetición de m , multiplicada por el número de repeticiones de h , $60 * 24$).

Si bien las estructuras repetitivas pueden ser útiles para solucionar un problema, deben ser usadas con cautela. Si tenemos una estructura que se ejecuta m veces dentro de una estructura que se ejecuta n veces, las instrucciones dentro de estructura anidada se ejecutarán exactamente $n * m$ veces. Por tal razón, en la práctica es poco frecuente encontrar algoritmos en los cuales se tengan más de dos niveles de anidamiento de estructuras repetitivas.

Instrucción *cancelar*

Esta instrucción especial permite terminar la ejecución de una estructura repetitiva sin completar la secuencia dentro de ésta. Puede ser usada para *romper* la repetición en el momento en que se presente determinada situación, aún si la condición que controla la estructura repetitiva se sigue cumpliendo.

Tenga en cuenta que si se usa la instrucción *cancelar* en una estructura repetitiva que se encuentra *anidada* dentro de otra, sólo se cancelará la estructura repetitiva interna.

El Código 2.25 ilustra el uso de la instrucción *cancelar* para validar un número entero proporcionado por el usuario.

Código 2.25: Ejemplo de uso de la instrucción *cancelar*

```
1  /*
2  * Ilustra el uso de la instrucción cancelar para romper
3  * la ejecución de una estructura repetitiva
4  */
5
6  algoritmo validarConCancelar
7  entero x
8  inicio
9    hacer
10     imprimir "Ingrese un número mayor que cero"
11     leer x
12     si x > 0
13       cancelar
14     fin si
15     imprimir "El número ingresado no es válido"
16   mientras x <= 0
17   imprimir "Usted ingresó ", x
18 fin
```

Si el usuario ingresa un valor mayor que 0, *se cancelará* la estructura repetitiva e inmediatamente se mostrará el valor ingresado al usuario. Por el contrario, si el valor ingresado es menor o igual que cero, se mostrará el mensaje de error y se repetirá la estructura, para imprimir el primer mensaje y solicitar el nuevo número.

La Tabla 2.17 presenta una prueba de escritorio en la cual primero se ingresa 0, lo cual causa que se ejecute toda la secuencia dentro de la estructura *hacer mientras*. Luego se ingresa un valor mayor que cero, lo que permite que se rompa el ciclo con la instrucción *cancelar*.

Tabla 2.17: Prueba de escritorio instrucción cancelar

Instrucción	x	Salida
inicio		
imprimir		Ingresa un número mayor que cero
leer x	0	
si x > 0 : F		
fin si		
imprimir		El número ingresado no es válido
mientras x <= 0 : V		
imprimir		Ingresa un número mayor que cero
leer x	5	
si x > 0 : V		
cancelar		
imprimir		Usted ingresó 5
fin		

Al llegar a una instrucción *cancelar*, el algoritmo *rompe* inmediatamente el ciclo y continúa en la instrucción que se encuentra inmediatamente después de la estructura repetitiva.

Instrucción *terminar*

Esta instrucción permite terminar inmediatamente el algoritmo, sin importar si faltan instrucciones por ejecutar o si nos encontramos dentro de alguna estructura de decisión o de repetición. Se usa en casos en los cuales *no es posible* continuar con la ejecución del algoritmo, porque ocurrió una situación en la cual no se debe continuar de ninguna manera.

Por ejemplo, si se construye un algoritmo que calcule la división de dos números enteros, se debe verificar que el denominador no sea cero. Esto causaría la falla del programa, ya que la división por cero da como resultado indeterminado. Por lo tanto, si el denominador especificado es cero, se deberá terminar el algoritmo inmediatamente.

El Código 2.26 presenta un posible uso de la instrucción *terminar*, para finalizar el algoritmo debido a que se presentó una *condición de error* (división por cero).

Código 2.26: Ejemplo de uso de la instrucción terminar

2. ESTRUCTURAS BÁSICAS PARA CONSTRUCCIÓN DE ALGORITMOS

```
1 /*
2  * Calcula la división entre dos números, validando si el
3  * denominador es cero
4  */
5 algoritmo division
6 entero a, b
7 real c
8 inicio
9   imprimir "Ingrese el numerador"
10  leer a
11  imprimir "Ingrese el denominador"
12  leer b
13  si b = 0
14     imprimir "Imposible dividir, el denominador es cero"
15     terminar
16  fin si
17  c = a / b
18  imprimir a, "entre " , b , " es ", c
19 fin
```

La Tabla 2.18 presenta la prueba de escritorio cuando el usuario ha ingresado 0 para el denominador.

Tabla 2.18: Prueba de escritorio instrucción terminar

Instrucción	a	b	Salida
inicio			
imprimir			Ingrese el numerador
leer a	5		
imprimir			Ingrese el denominador
leer b	0		
si b = 0 : V			
imprimir			Imposible dividir, el denominador es cero
terminar			
fin			

Por el contrario, si el usuario ingresa un denominador válido, el algoritmo se ejecutará como se muestra en la Tabla 2.19

Tabla 2.19: Prueba de escritorio instrucción terminar

Instrucción	a	b	c	Salida
inicio				
imprimir				Ingrese el numerador
leer a	5			
imprimir				Ingrese el denominador
leer b		2		
si b = 0 : F				

Continued on next page

Tabla 2.19: Prueba de escritorio instrucción terminar

Instrucción	a	b	c	Salida
fin si				
c = a / b			2.5	
imprimir				5 entre 2 es 2.5
terminar				
fin				

La instrucción *terminar* nos permite *descartar* primero aquellas situaciones en las cuales el programa *no debe* continuar con su ejecución, o dicho de otra forma, debe *terminar temprano* debido a que la situación presentada no permite que continúe la ejecución.

Instrucción *continuar*

Esta instrucción permite *omitir* parte de una secuencia que se encuentra dentro de una estructura repetitiva, y comenzar una nueva iteración evaluando de nuevo la condición de la estructura. En caso de ser usada en una estructura *para hacer*, se realizará la modificación especificada antes de evaluar de nuevo la condición.

El Código 2.27 muestra cómo podemos usar esta estructura en un algoritmo que permita leer 10 números, y calcule e imprima su valor absoluto.

Código 2.27: Ejemplo de uso instrucción continuar

```

1 /*
2  * Prueba de la instrucción continuar para saltar secciones
3  * de una estructura repetitiva
4  */
5 algoritmo valorAbsoluto
6 entero x, entero i
7 inicio
8   para i=1, i <= 10, i = i + 1
9       imprimir "Ingrese un numero"
10      leer x
11      si x > 0
12          imprimir "El valor absoluto es ", x
13          continuar
14      fin si
15      x = x * -1
16      imprimir "El valor absoluto es ", x
17 fin para
18 fin

```

En este caso se podría implementar el algoritmo usando *sino* en la estructura de decisión, pero veamos cómo funciona el algoritmo tal como está implementado.

2. ESTRUCTURAS BÁSICAS PARA CONSTRUCCIÓN DE ALGORITMOS

Si el número leído es mayor que cero, no necesitamos cambiarle de signo. Por tal razón, se imprime el mensaje, y se continúa con la siguiente iteración incrementando el valor de i y evaluando de nuevo la condición de la estructura repetitiva.

En caso contrario, es decir si el número es menor que 0, debemos cambiarle de signo antes de imprimir el mensaje. Luego continuamos con la siguiente iteración, incrementando i y evaluando de nuevo la condición.

La Tabla 2.20 muestra la ejecución del algoritmo cuando el usuario ingresa valores mayores y menores que cero.

Tabla 2.20: Prueba de escritorio instrucción continuar

Instrucción	i	x	Salida
inicio			
para : (i = 1)	1		
imprimir			Ingrese un número
leer x		-1	
si x >0 : F			
fin si			
x = x * -1		1	
imprimir			El valor absoluto es 1
fin para : (i = i + 1)	2		
para : (i <= 10)? : V			
imprimir			Ingrese un número
leer x		5	
si x >0 : V			
imprimir			El valor absoluto es 5
continuar			
fin si			
fin para : (i = i + 1)	3		
(Iteraciones)			
fin para : (i = i + 1)	11		
para : (i <= 10)? : F			
fin			

Subrutinas

El último elemento de control de flujo de ejecución de un algoritmo, y uno de los más importantes a medida que aumenta la complejidad del problema a solucionar son las subrutinas. Una subrutina es básicamente una *secuencia independiente de instrucciones*, que ejecutadas en conjunto realizan una tarea específica. Cuenta con las siguientes características:

- Se define por separado al *algoritmo principal*.
- Tiene un nombre único en el contexto de la solución del problema.
- Puede ser usada cuantas veces sea necesario y en diferentes lugares, incluso dentro de otras subrutinas.
- La secuencia de instrucciones puede ser simple, como una instrucción para imprimir un mensaje o realizar una operación aritmética, o tan compleja que incluye estructuras de decisión y/o estructuras repetitivas, posiblemente anidadas. Es decir, una subrutina puede llegar a ser tan compleja que por sí misma constituya un algoritmo.
- Puede recibir datos de entrada, y de ser necesario, proporcionar un dato de salida. A los datos de entrada se les denomina *argumentos* o *parámetros*.
- También puede modificar los datos de entrada, sin ofrecer una salida.
- Los datos de entrada pueden ser simples o compuestos.

El concepto de subrutina surge de la idea de **reutilizar** una secuencia concreta de instrucciones que realizan una tarea determinada. Dependiendo de la forma como se defina e implemente una subrutina, puede incluso ser usada en otros algoritmos y programas.

Por ejemplo, es posible definir una rutina sumatoria, la cual toma como entrada un conjunto de números y permite obtener la suma de éstos. Esta rutina podría ser usada en un infinito de algoritmos. Observe que desde esta perspectiva, se puede ver a un algoritmo como una subrutina o un conjunto de subrutinas que operan de forma coordinada para dar solución a un problema. De hecho, puede ser posible que un algoritmo se convierta en una subrutina de otro algoritmo de mayor complejidad.

En la literatura se distinguen dos clases de subrutinas:

- **Funciones:** Reciben cero, uno o más datos de entrada, realizan un proceso y proporcionan un único dato de salida. Tanto los datos de entrada como el dato de salida pueden ser de diferentes tipos, simples o compuestos.
- **Procedimientos:** También pueden recibir cero, uno o más datos de entrada, pero no proporcionan datos de salida. En cambio, pueden modificar los datos de entrada o realizar un proceso que modifica la ejecución del algoritmo.

En este texto no se tendrá en cuenta esta clasificación, y se usará sólo el término *función* para definir una subrutina. Su carácter de *función* o *procedimiento* lo dará el hecho de retornar o no un dato como resultado de su ejecución.

Una subrutina se especifica de forma similar a un algoritmo. Es necesario definir:

2. ESTRUCTURAS BÁSICAS PARA CONSTRUCCIÓN DE ALGORITMOS

- Nombre de la subrutina.
- Tipo de dato de retorno (para el caso de una función).
- Variables que se usarán dentro de la subrutina.
- Secuencia de instrucciones y elementos de control de flujo que componen la subrutina.

La estructura general y la forma de uso de las subrutinas se muestra en el Código 2.28.

Código 2.28: Ejemplo de definición y uso de subrutinas

```
1 algoritmo miAlgoritmo
2 entero x
3 entero y
4 inicio // Secuencia principal de instrucciones del algoritmo
5 leer x
6 y = calcularAlgoCon(x) //Usar la función
7 hacerAlgoCon(y) //Usar la función
8 fin
9 //Calcula algo con el dato recibido y retorna el resultado
10 funcion calcularAlgoCon(entero x) : entero
11 entero resultado
12 inicio
13     secuencia A
14     resultado = OPERACION //Realizar alguna operación
15     retornar resultado
16 fin funcion
17 //Realiza una secuencia de acciones con el dato recibido. No retorna valor.
18 funcion hacerAlgoCon(entero y)
19 inicio
20     secuencia C
21 fin funcion
```

En este caso, el algoritmo hace uso de dos subrutinas, llamadas `calcularAlgoCon` y `hacerAlgoCon`. Cuando se está ejecutando este algoritmo, la instrucción de la línea 6 causa que la ejecución de la secuencia principal de instrucciones del algoritmo *suspenda* su ejecución, y *se pase el control* a la subrutina `calcularAlgoCon` (línea 10). Esta subrutina *recibe* como parámetro el valor leído en la variable `x`. La subrutina realizará un proceso, que puede ser una simple operación, o una secuencia compleja que puede incluir estructuras de decisión, ciclos o llamada a otras subrutinas.

Al terminar secuencia de la subrutina, se calcula un resultado, el cual *es devuelto* (retornado) al punto del algoritmo en el cual *se suspendió* la ejecución (línea 6). Es decir, el valor *devuelto* por la subrutina será *asignado* a la variable `y` y en el algoritmo principal, y continuará la ejecución de dicho algoritmo.

Al regresar al algoritmo principal, la instrucción en la línea 7 causa que se *suspenda de nuevo* su ejecución para dar paso a la subrutina `hacerAlgoCon` (línea 18). A esta subrutina

también se le envía un parámetro, esta vez el valor almacenado en la variable *y* que fue calculado anteriormente por la otra subrutina.

La subrutina *hacerAlgoCon* realiza alguna secuencia simple o compleja de instrucciones, y al finalizar *regresa* al punto en el cual continúa el algoritmo principal (línea 7), esta vez sin *devolver* un valor.

Se debe considerar que:

- Cada subrutina debe tener un nombre diferente, de modo que el algoritmo (y por supuesto el programa) puedan determinar cual subrutina se va a usar.
- Una subrutina puede recibir cero, uno o más parámetros de diferente tipo, de acuerdo con lo que debe realizar.
- Se debe respetar el número, el tipo y el orden de los parámetros de las subrutinas. Esto generalmente es verificado por el *compilador*, cuando se codifica el algoritmo en un lenguaje de programación.

Cuando los programas son muy complejos, puede ser necesario crear muchas subrutinas. Estas se pueden agrupar en **módulos**, en los cuales las subrutinas se organizan de forma lógica de acuerdo con el papel que desempeñan para solucionar el problema. Algunos módulos pueden ser reutilizados incluso por otros programas. En este caso se les denomina *librerías*.

Un buen ejemplo de librerías es la Librería Estándar de C (ANSI), la cual contiene definiciones de datos y de subrutinas que pueden ser usadas para diferentes propósitos, como entrada y salida de datos, operaciones sobre números y cadenas de caracteres, funciones aritméticas, etc.

Revisemos el problema de la suma de dos números enteros:

Construir un algoritmo que dados dos números enteros, calcule la suma entre ellos.

Este problema, que ya solucionamos sin necesidad de usar subrutinas, nos da la posibilidad de comenzar a apreciar su potencial.

Aplicando el principio de *modularidad*, podemos definir una subrutina llamada *suma*, que *reciba como entrada* dos números enteros, y proporcione como *dato de salida* la suma de ellos. Esta subrutina podrá ser usada muchas veces dentro del mismo algoritmo, e incluso en otros algoritmos.

El algoritmo principal junto con la definición de la subrutina se presenta en el Código 2.29.

2. ESTRUCTURAS BÁSICAS PARA CONSTRUCCIÓN DE ALGORITMOS

Código 2.29: Ejemplo de subrutina para sumar dos números

```
1 //Calcula la suma de dos números usando funciones
2 algoritmo suma
3 entero a, entero b, entero c
4 inicio
5     leer a
6     leer b
7     c = sumar(a, b) //Llamado a la función sumar
8     imprimir c
9 fin
10 //Recibe dos valores enteros y retorna la suma de ellos
11 función sumar(entero x, entero y): entero
12     entero resultado
13     inicio
14         resultado = x + y
15         retornar resultado
16 fin función
```

En este caso la subrutina sumar se puede clasificar como una *función*, dado que devuelve un valor al sitio en el algoritmo en el cual se *invoca* (se usa), para ser almacenado en una variable.

En las Tablas 2.21 y 2.22 podemos apreciar la prueba de escritorio del algoritmo. Debido a que una función puede ser *invocada* muchas veces con diferentes parámetros, es necesario realizar la prueba de escritorio de cada llamada.

Tabla 2.21: Prueba de escritorio con dato uso de funciones

Instrucción	a	b	c	Salida
inicio				
leer a	10			
leer b		5		
c = sumar(a, b)			sumar(10, 5)	
c = sumar(a, b)			15 ←	
imprimir				15
fin				

La prueba de escritorio para la llamada sumar(10, 5) se muestra en la Tabla 2.22. Observe que en la prueba de escritorio no se especifica columna para salida por pantalla, debido a que la función sumar realiza el cálculo y retorna el resultado. Si la función enviara algún mensaje por pantalla, se deberíamos adicionar la columna correspondiente.

De igual forma, las variables x e y se establecen al inicio de la función, debido a que son los parámetros de entrada enviados por el algoritmo principal.

Tabla 2.22: Prueba de escritorio llamada a sumar

Instrucción	x	y	resultado	Retorno
inicio	10	5		
resultado = x + y			15	
retornar resultado				15 $\Rightarrow$

También es necesario tener en cuenta que en cualquier momento que se llegue a una instrucción *terminar*, la función terminará inmediatamente y devolverá el valor especificado al punto en el cual fue invocada.

Desde luego, una simple operación matemática no amerita implementar una subrutina. Así que debemos encontrar un balance entre qué se implementa como subrutina y qué se define dentro del algoritmo principal. La capacidad para lograr este balance sólo se desarrolla con la experiencia en algoritmia. En algunos ejemplos posteriores veremos que en ocasiones es más *eficiente* no usar subrutinas, para evitar llamadas innecesarias a las mismas o repetir muchas veces las mismas operaciones.

Subrutinas recursivas

Las subrutinas recursivas son una de las herramientas más poderosas y a la vez peligrosas en algoritmia y programación, debido a que ofrecen la posibilidad de construir soluciones *elegantes* desde el punto de vista matemático y computacional. Sin embargo, si no se definen correctamente pueden *salirse de control* y hacer que el algoritmo siga ejecutándose de forma indefinida.

Por definición, /una subrutina recursiva es aquella que dentro de su secuencia de instrucciones se invoca a sí misma/.

Considere el siguiente problema:

Construir un algoritmo que permita calcular la sumatoria de los números enteros entre 1 y un número n dado.

Vamos a encontrar la solución *recursiva* a este problema. Esta solución será de agrado de los matemáticos, debido a que se puede plantear como una *ecuación de recurrencia*:

```
sumatoria(n) = 1, si n = 1 //Caso base
              n + sumatoria(n - 1), si n > 1 //Cálculo recursivo
```

Por ejemplo, si queremos calcular la sumatoria de 1 hasta 5, debemos aplicar la anterior definición:

2. ESTRUCTURAS BÁSICAS PARA CONSTRUCCIÓN DE ALGORITMOS

<code>sumatoria(5) = 5 + sumatoria(4)</code>	debemos calcular <code>sumatoria(4)</code>
<code>sumatoria(4) = 4 + sumatoria(3)</code>	debemos calcular <code>sumatoria(3)</code>
<code>sumatoria(3) = 3 + sumatoria(2)</code>	debemos calcular <code>sumatoria(2)</code>
<code>sumatoria(2) = 2 + sumatoria(1)</code>	debemos calcular <code>sumatoria(1)</code>
<code>sumatoria(1) = 1</code>	por definición es 1

Inmediatamente nos damos cuenta de la situación: Para calcular la sumatoria de un número n , primero debemos calcular la sumatoria de $n - 1$, $n - 2$, etc., hasta llegar al *caso base*, cuando $n = 1$. Este valor se toma para calcular la sumatoria de 2, cuyo resultado se toma para calcular la sumatoria de 3, hasta llegar a calcular la sumatoria de 5.

El caso $n = 1$ *base*, porque se puede tomar como *punto de partida* (o *punto de llegada*), y al llegar a este punto *no se depende* de ningún otro parámetro para realizar la operación (en este caso el cálculo). Si no existiera este caso base, el algoritmo no se detendrá jamás.

Más adelante en el texto se presentará el algoritmo que soluciona el problema de la sumatoria de 1 hasta n . Por ahora solo necesitamos conocer la definición de subrutina recursiva y su relación con las ecuaciones de recurrencia.

Parámetros por valor y por referencia

Como se mencionó anteriormente, una subrutina puede recibir cero, uno o más *argumentos*, también llamados *parámetros*. Por defecto los parámetros se pasan a una subrutina *por valor*, lo que significa que los datos de las variables que se envían a la subrutina son *copiados* a las variables que ésta tiene definida para recibirlos, llamadas *variables locales*. La subrutina puede modificar sus variables locales, es decir, las copias de los datos que recibió, y definir nuevas variables, pero no puede modificar los datos almacenados en las variables originales.

El algoritmo implementado en el Código 2.30 ilustra el concepto de paso de parámetros por valor.

Código 2.30: Paso de parámetros por valor

```
1 //Prueba de paso de parámetros por valor
2 algoritmo pasoPorValor
3 entero x
4 inicio
5   x = 5
6   imprimir "Dato original: ", x
7   incrementarEImprimir(x) //Llamar la subrutina
8   imprimir "Dato después de la llamada: ", x
9 fin
10 //Incrementa el valor recibido e imprime el resultado
11 funcion incrementarEImprimir(entero n)
12 inicio
13   n = n + 1 //Modifica la copia local
```

```

14 imprimir "Dentro de la rutina: ", n //Imprime la copia local
15 fin

```

Las Tablas 2.23 y 2.24 presentan la prueba de escritorio del algoritmo principal y la función invocada.

Tabla 2.23: Prueba de escritorio paso por referencia

Instrucción	x	Salida
inicio		
x = 5	5	
imprimir		Dato original: 5
incrementarEImprimir(x)		Dentro de la rutina: 6
imprimir		Dato después de la llamada: 5
fin		

La prueba de escritorio de la función incrementarEImprimir se presenta en la Tabla 2.24.

Tabla 2.24: Llamada incrementarEImprimir, paso por valor

Instrucción	n = $\overleftarrow{x}$	Salida
inicio	5	
*n = *n + 1	6	
imprimir		Dentro de la rutina: 6
fin		

En este caso, la variable x se pasó por valor a la función incrementarEImprimir, por lo cual el valor que tenía x al momento de realizar la llamada (5) se copió dentro de la variable *local* n de la subrutina. La variable n puede ser modificada dentro de la subrutina, pero estos cambios no se reflejan en la variable x .

En algunos lenguajes de programación es posible pasar parámetros *por referencia*. En este caso, la función no recibe un dato sino una referencia a la variable original. Al modificar el dato almacenado dentro de esta referencia, esta modificación será reflejada en la variable referenciada.

Una forma de implementar el paso de parámetros por referencia consiste en usar *apuntadores*, los cuales como se explicó anteriormente no almacenan datos sino referencias a otras variables. El Código 2.31 muestra el mismo ejemplo anterior, esta vez usando paso por referencia.

Código 2.31: Paso de parámetros por referencia

```

1 //Prueba de paso de parámetros por referencia
2 algoritmo pasoPorReferencia

```

2. ESTRUCTURAS BÁSICAS PARA CONSTRUCCIÓN DE ALGORITMOS

```
3 entero x
4 inicio
5   x = 5
6   imprimir "Dato original: ", x
7   incrementarEImprimir(ref x) //Llamar la subrutina, paso por referencia
8   imprimir "Dato después de la llamada: ", x
9 fin
10 //Incrementa el valor almacenado en la variable referenciada
11 funcion incrementarEImprimir(ref entero n)
12 inicio
13   *n = *n + 1 //Modifica el dato de la variable referenciada
14   imprimir "Dentro de la rutina: ", *n //Imprime el dato
15 fin
```

Las Tablas 2.25 y 2.26 presentan la prueba de escritorio del algoritmo principal y la función invocada.

Tabla 2.25: Prueba de escritorio paso por referencia

Instrucción	x	Salida
inicio		
x = 5	5	
imprimir		Dato original: 5
incrementarEImprimir($\overleftarrow{x}$)	$\overleftarrow{6}$	Dentro de la rutina: 6
imprimir		Dato después de la llamada: 6
fin		

La prueba de escritorio de la función incrementarEImprimir se presenta en la Tabla 2.26.

Tabla 2.26: Llamada incrementarEImprimir, paso por referencia

Instrucción	n = $\overleftarrow{x}$	Salida
inicio	5	
*n = *n + 1	6	
imprimir		Dentro de la rutina: 6
fin		

Ejercicios de fundamentación

En este capítulo se presenta una serie de ejercicios cuya solución permitirá afianzar las bases fundamentales de la algoritmia. Comenzamos con ejercicios sencillos, que aumentan su nivel de complejidad y requieren el uso de los diferentes recursos del pensamiento algorítmico. Se espera que al terminar esta sección se cuente con la capacidad de explicar aplicar los conceptos de *variables*, *estructuras de decisión* y *repetición*, así como de algunos elementos importantes entre los que se encuentran las *banderas* o *centinelas*, *condiciones simples* y *compuestas*, y *subrutinas*, entre otros.

En cada ejercicio se presenta el planteamiento del problema, el análisis, el diseño y el algoritmo representado en pseudocódigo. En el primer ejercicio también se incluye una prueba de escritorio. Se deja al lector la tarea de realizar las respectivas pruebas de escritorio de los demás ejercicios, y de ser necesario, la codificación en algún lenguaje.

3.1. Instrucciones y estructuras de decisión

En este apartado se presentarán ejercicios relacionados con el uso de *instrucciones*, *secuencias de instrucciones* y *estructuras de decisión*.

Intercambio de valores

Una de las primitivas básicas en algoritmia es la capacidad de recibir datos por parte del usuario, manipularlos y luego ofrecer una salida. El siguiente problema sencillo muestra esta situación, y además se ilustra el uso básico de variables.

Construir un algoritmo que lea dos números enteros en dos variables, intercambie los valores almacenados en ellas y luego imprima los los valores resultantes.

Análisis

Este es un problema sencillo en el cual se tienen los procesos básicos sobre los datos: entrada, transformación y salida. Para la entrada se puede usar dos instrucciones de lectura, que almacenarán los valores proporcionados por el usuario en dos variables que llamaremos a y b. El proceso consistirá en *intercambiar* los valores almacenados en las variables. Finalmente se deberá imprimir los valores almacenados en las variables, que deberán estar intercambiados.

Una primera aproximación a la solución del problema puede ser la siguiente:

```
leer a y b
imprimir a y b
intercambiar los valores de a y b
imprimir a y b
```

Diseño

El intercambio consiste en almacenar en cada variable el valor de la otra, es decir, necesitamos almacenar el valor actual de la variable a en la variable b, y el valor actual de la variable b en la variable a. Esto se puede llevar a cabo en dos pasos:

```
a = b //Tomar el valor de b y llevarlo a a
b = a //Tomar el valor de a y llevarlo a b ?????
```

Pero, surge un problema: Al realizar la primera asignación, el valor que estaba almacenado en la variable a se pierde (Figura 3.1 (a)). Si por el contrario, asignamos a b el valor de a, el valor que estaba almacenado en b se pierde (Figura 3.1 (b)). Por lo tanto, debemos *almacenar* el valor de a en *otra* variable, la cual llamaremos *variable auxiliar*. Allí permanecerá *a salvo*, para luego almacenarlo en la variable b (Figura 3.1 (c)).

El pseudocódigo correspondiente será el que se muestra a continuación:

```
//Intercambio de los valores de a y b
aux = a
a = b
b = aux
```

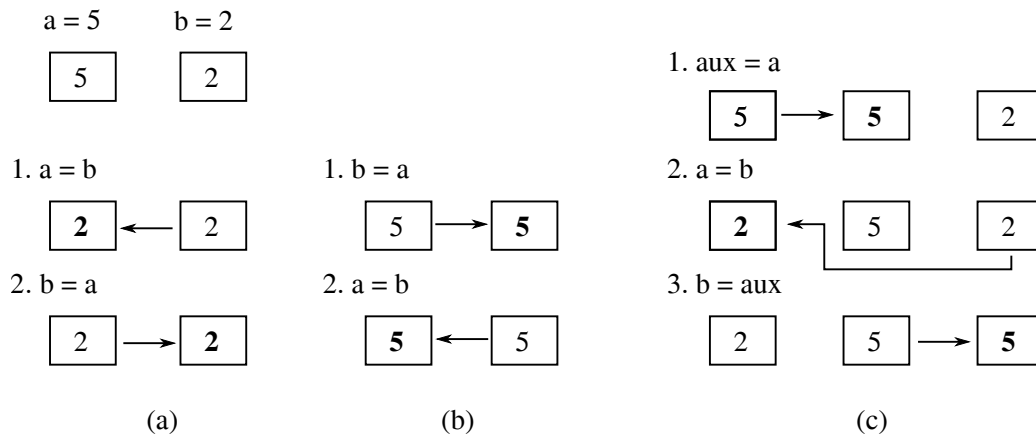


Figura 3.1: Intercambio de dos valores

Algoritmo

El Código 3.1 muestra la implementación del algoritmo para solucionar el problema.

Código 3.1: Intercambio de valores

```

1 //Intercambia los valores leídos de a y b
2 algoritmo intercambio
3 entero a, entero b, entero aux
4 inicio
5   leer a
6   leer b
7   imprimir a, b
8   aux = a
9   a = b
10  b = aux
11  imprimir a, b
12 fin
    
```

La prueba de escritorio del algoritmo implementado se presenta en la Tabla 3.1.

Tabla 3.1: Prueba de escritorio intercambio de valores

Instrucción	a	b	aux	Salida
inicio				
leer a	3			
leer b		8		
imprimir				3 8
aux = a			3	
a = b	8			
b = aux		3		
imprimir				8 3

Continued on next page

3. EJERCICIOS DE FUNDAMENTACIÓN

Tabla 3.1: Prueba de escritorio intercambio de valores

Instrucción	a	b	aux	Salida
fin				

También se puede salvar el valor almacenado en la variable b, en cuyo caso el algoritmo se codificará de la siguiente forma. Por lo general se prefiere almacenar el valor de la primera variable, pero es igualmente correcto usar la variable auxiliar para almacenar el valor de la segunda variable.

```
1 //Intercambia los valores leídos de a y b
2 algoritmo intercambio
3 entero a, entero b, entero aux
4 inicio
5   leer a
6   leer b
7   imprimir a, b
8   aux = b
9   b = a
10  a = aux
11  imprimir a, b
12 fin
```

¿Qué tal si aprovechamos este ejemplo para aplicar los conceptos de funciones y paso de parámetros por referencia?. En este problema se necesita intercambiar los valores almacenados en las variables a y b. Entonces, se puede definir una función llamada intercambio, la cual reciba como parámetro las referencias a las dos variables, y modifique los datos almacenados en las variables referenciadas. La nueva versión del algoritmo se puede ver en el Código 3.2.

Código 3.2: Función para intercambio de valores

```
1 //Intercambia los valores leídos de a y b
2 algoritmo intercambio
3 entero a, entero b
4 inicio
5   leer a
6   leer b
7   imprimir a, b
8   intercambio(ref a, ref b)
9   imprimir a, b
10 fin
11 /*
12  * Intercambia los valores almacenados en las referencias recibidas
13  */
14 funcion intercambio(ref entero x, ref entero y)
15   entero aux
16   inicio
17     aux = *x
18     *x = *y
19     *y = aux
```

20 `fin funcion`

Se deja como ejercicio construir la prueba de escritorio. Puede guiarse por la prueba realizada cuando se presentó el concepto de referencias, y la prueba de escritorio de la función `incrementarEImprimir` (Tabla 2.26).

Comparación de dos números

El siguiente ejercicio tiene como propósito presentar el uso de las estructuras de decisión simples.

Construir un algoritmo que dados dos números, determine si son iguales.

Análisis

Este es un problema de decisión muy sencillo, en el cual se trata de determinar si dos números proporcionados por el usuario son iguales o no. Obviamente el usuario sabe si los números que ingresa son iguales, pero el propósito de este algoritmo es mostrar cómo se puede implementar una *lógica computacional* (es decir, una serie de instrucciones ejecutada por una máquina) que permita determinar si dos números son iguales.

La solución general será la siguiente:

```
leer a y b
si a es igual a b, imprimir "son iguales"
en caso contrario, imprimir "no son iguales"
```

Diseño

El algoritmo para solucionar este problema primero deberá leer dos valores, (usando instrucciones de lectura de datos), los cuales serán almacenados en dos variables diferentes debido a que necesitamos compararlos. Luego se deberá usar una estructura de decisión para verificar si los valores almacenados en las dos variables son iguales, es decir, para compararlos. En cada caso (tanto si son iguales o no) se deberá informar al usuario mediante una instrucción de salida la condición de igualdad entre los números.

Algoritmo

El Código 3.3 presenta una solución del problema planteado. Las instrucciones *leer* almacenan los valores ingresados en dos variables, y luego se usa una estructura de decisión para comparar los números e imprimir los mensajes correspondientes.

Código 3.3: Comparación entre dos números

```
1  /*
2  Lee dos números y determina si son iguales
3  */
4  algoritmo sonIguales
5  entero a, entero b
6  inicio
7    leer a
8    leer b
9    si a = b entonces
10     imprimir "Los numeros son iguales"
11  sino
12     imprimir "Los numeros no son iguales"
13  fin si
14  fin
```

Con este algoritmo podemos construir un programa que *puede determinar* si dos números proporcionados son iguales o no.

Comparación de tres números

En el siguiente ejercicio clásico presentado en Sedgewick and Wayne (2011) y también en Joyanes Aguilar and Zahonero (2005), se ilustrará el uso de estructuras de decisión anidadas y condiciones compuestas.

Construir un algoritmo que dados tres números, determine si son iguales.

Análisis

En este caso no se puede aplicar una decisión simple. Se tienen dos alternativas:

- Usar condiciones simples y estructuras de decisión *anidadas*. Primero se comparan dos números, y si son iguales, se compara uno de los dos primeros con el tercer número. Si son iguales, todos ellos son iguales.
- Usar una *condición compuesta*. En este caso se comparan los tres números en una sola condición.

Diseño

La comparación usando condiciones simples y estructuras anidadas tendrá la siguiente estructura:

```

leer a, b, y c
si a = b
    si a = c
        //Los tres son iguales
    sino
        //no son iguales
    fin si
sino
    //no son iguales, no hay necesidad de verificar más
fin si

```

La segunda estrategia parte del hecho que las condiciones pueden ser compuestas, es decir, pueden estar formadas por varias condiciones simples que se evalúan en conjunto enlazándolas con los operadores lógicos y (and) y o (or).

Es decir, se puede crear una condición como la siguiente:

```

leer a, b, c
si a = b y a = c
    //Los tres son iguales
sino
    //No son iguales
fin si

```

Algoritmo

En el Código 3.4 en la cual se usan estructuras anidadas. Cada estructura *si - sino* evalúa una condición simple (en este caso, compara dos números). Si la condición se cumple, entonces se define otra estructura de decisión, que permitirá verificar si el tercer número es igual a cualquiera de los dos números comparados anteriormente, si son iguales.

Código 3.4: Determinar si tres números son iguales, condiciones simples

```

1  /*
2  * Lee y compara los tres números para determinar si son iguales
3  */
4  algoritmo tresIguales
5  entero a, entero b, entero c
6  inicio
7      leer a
8      leer b
9      leer c
10
11     si a = b
12         si a = c
13             imprimir "Los tres numeros son iguales"

```

3. EJERCICIOS DE FUNDAMENTACIÓN

```
14     sino
15         imprimir "Los tres números no son iguales"
16     fin si
17     sino
18         imprimir "Los tres números no son iguales"
19     fin si
20 fin
```

Observe que para que el algoritmo sea completo, se debe imprimir el mensaje cuando los números no son iguales en dos sitios diferentes del algoritmo. Si al comparar a y b no son iguales, el algoritmo no continúa por el camino del `si` y se debe agregar el `si no`. Dentro del `si no` se debe imprimir el mensaje.

Ahora bien, si a y b son iguales, se debe comparar uno de ellos (cualquiera, porque son iguales) con c . Si al comparar por ejemplo a con c son iguales, se debe imprimir el mensaje correspondiente. Pero en caso contrario, también se debe imprimir el mensaje que indica que no son iguales.

El Código 3.5 muestra la solución usando la condición compuesta.

Código 3.5: Determinar si tres números son iguales, condiciones compuestas

```
1  /*
2  * Lee y compara los tres números para determinar si son iguales
3  * usando condiciones compuestas
4  */
5  algoritmo tresIguales
6  entero a, entero b, entero c
7  inicio
8      leer a
9      leer b
10     leer c
11     si a = b y a = c
12         imprimir "Los tres números son iguales"
13     sino
14         imprimir "Los números no son iguales"
15     fin si
16 fin
```

Promedio de dos números

Pasamos ahora a un problema de transformación simple, en el cual se toman dos datos de entrada, se aplica un proceso y se proporciona una salida.

Construir un algoritmo que dados dos números a y b , calcule e imprima el valor de su promedio.

Análisis

En este caso, vemos que se trata de un problema de transformación de datos. Se nos dan dos datos, y debemos calcular su promedio. Por definición, el promedio de dos números se calcula como la suma de ellos dividida por 2.

Diseño

Simplemente debemos leer dos valores enteros y realizar la operación de promedio como se muestra a continuación.

```
leer a
leer b
promedio = (a + b) / 2
```

Algoritmo

El Código 3.6 implementa la solución al problema planteado.

Código 3.6: Promedio de dos números

```
1 /*
2  *Calcula e imprime el promedio de dos números leídos
3  */
4 algoritmo promedio
5 entero a, entero b, real promedio
6 inicio
7     leer a
8     leer b
9     promedio = (a + b) / 2
10    imprimir promedio
11 fin
```

Determinar si un número es divisor entero de otro

A continuación se presentará un problema que implica realizar una serie de cálculos matemáticos y almacenar el resultado en variables para luego tomar decisiones de acuerdo con los valores obtenidos.

Construir un algoritmo que dados dos números a y b, permita determinar si b es divisor entero de a.

Análisis

Este es un típico problema de decisión. En este caso, se busca que el algoritmo pueda determinar si para dos números de entrada a y b , b es divisor entero de a .

Se considera que b es divisor entero de a , solo cuando al dividir a entre b se obtiene un cociente entero y un residuo de cero. El residuo se puede calcular mediante la siguiente fórmula:

```
cociente = a / b
residuo = a - (cociente * b)
```

En donde la variable `cociente` almacena el resultado de la división entera entre a y b , sin redondear. El residuo se obtiene entonces al multiplicar este cociente por b , y luego restarlo al valor de a .

Por ejemplo, para calcular el residuo entre 10 y 3:

```
cociente = a / b = 10 / 3 = 3
residuo = a - ( cociente * b ) = 10 - ( 3 * 3 ) = 1
```

En computación existe la operación *modulo* (`mod`), la cual permite encontrar el residuo de la división entera entre dos números. La operación modulo se especifica de la siguiente forma:

```
residuo = a mod b
```

Diseño

Para el ejemplo anterior, si $a = 10$ y $b = 3$, residuo tomará el valor de 1, que es el resto obtenido de la división entera entre 10 y 3. Si se toma $a = 10$ y $b = 2$, el residuo será cero, con lo cual podemos afirmar que 2 es divisor entero de 10. Entonces el algoritmo deberá realizar un proceso como el siguiente:

```
residuo = a mod b
si residuo = 0
    //b es divisor de a
sino
    //b no es divisor de a
fin si
```

Algoritmo

Con esta información ya podemos construir el algoritmo. Para saber si b es divisor de a , solo basta con calcular el residuo de la división entera entre a y b . Si este residuo es cero, b es divisor de a .

La solución completa se puede apreciar en el Código 3.7.

Código 3.7: Determinar si un número es divisor de otro

```

1 /*
2  * Lee dos números a y b y determina si b es divisor de a
3  */
4 algoritmo esDivisor
5 entero a, entero b, entero residuo
6 inicio
7   leer a
8   leer b
9   residuo = a mod b
10  si residuo = 0
11    imprimir b, " es divisor de ", a
12  sino
13    imprimir b, " no es divisor de ", a
14  fin si
15 fin

```

Determinar si un número es par

Con frecuencia es necesario saber si un dato, en este caso un número, cumple con una condición determinada.

Construir un algoritmo que dado un número, determine si es par o no.

Análisis

La noción de número par es sencilla: Si el número es divisible por 2, se considera par. En caso contrario, se considera impar. Por lo tanto, se deberá usar la operación **modulo**, la cual calcula el residuo de la división entera entre un número a y otro número b .

Diseño

Para averiguar si un número es par o no, solo es necesario hallar el residuo de dividir el número dado entre 2. Si este residuo es cero, el número es par. En caso contrario, es impar. Entonecs, para solucionar el problema necesitamos una estructura de decisión (ver Código 3.8).

3. EJERCICIOS DE FUNDAMENTACIÓN

Código 3.8: Determinar si un número es par

```
leer n
residuo = n mod 2
si residuo = 0 entonces
    imprimir "El numero ", n , " es par"
sino
    imprimir "El numero ", n, "es impar"
fin si
```

La estructura de decisión if - else permite evaluar el resultado de una operación directamente, por lo cual se puede evitar el uso de la variable *residuo*.

Algoritmo

La solución se presenta en el Código 3.9. En este caso se realiza la operación de módulo directamente dentro de la condición, y se verifica si el resultado de la operación es cero.

Código 3.9: Solución alternativa para verificar si un número es par

```
1 /*
2  * Verifica si un número dado es par
3  */
4 algoritmo verificarSiEsPar
5 entero n
6 inicio
7     leer n
8     si (n mod 2) = 0 entonces
9         imprimir "El numero ", n , " es par"
10    sino
11        imprimir "El numero ", n, "es impar"
12    fin si
13 fin
```

Valor de la compra

Veamos un problema en el cual se toman datos de entrada, se aplica un proceso y se genera un dato de salida.

Construir un algoritmo que dado el valor de un artículo y la cantidad de artículos a comprar, determine el valor total a pagar de acuerdo con las siguientes condiciones:

- Si se compran entre 5 y 10 artículos, se aplica un descuento del 10% sobre el total a pagar.
- Si se compran más de 10 artículos, se aplica un descuento del 20% sobre el total a pagar.

- En cualquier otro caso no se aplica descuento.

Análisis

La solución de este problema es relativamente sencilla, ya que el mismo enunciado nos está indicando las decisiones y los cálculos que debemos realizar. El esquema general de solución será el siguiente:

```
leer precio
leer cantidad
si cantidad se encuentra entre 5 y 10
    calcular el total
    aplicar el descuento del 10 por ciento sobre el total
sino
    si cantidad es mayor que 10
        calcular el total
        aplicar el descuento del 20 por ciento sobre el total
    sino
        calcular el total
        (no aplicar descuento)
    fin si
fin si
imprimir el total
```

Diseño

Vamos a refinar el esquema básico obtenido en el análisis. Podemos partir de un *caso base*, el cual es calcular el valor total de la compra. El descuento se aplicará *sólo* si se cumple la condición correspondiente.

```
leer precio
leer cantidad
total = cantidad * precio
si cantidad >= 5 y cantidad <= 10
    total = total * 0.1 //Aplicar descuento 10 por ciento
sino
    si cantidad > 10
        total = total * 0.2 //Aplicar descuento 20 por ciento
    fin si fin si
```

Algoritmo

El algoritmo se obtiene de completar el diseño realizado. El Código 3.10 muestra el algoritmo completo.

Código 3.10: Valor de la compra

```
1  /*
2  * Calcula el valor de la compra, aplicando un descuento
3  * del 10 por ciento si la cantidad de artículos se encuentra entre
4  * 5 y 10 y un 20 por ciento si la cantidad es mayor que 10
5  */
6  algoritmo valorCompra
7  entero cantidad
8  real precio, real total
9  inicio
10 leer precio
11 leer cantidad
12 total = cantidad * precio
13 si cantidad >= 5 y cantidad <= 10
14     total = total * 0.1
15 sino
16     si cantidad > 10
17         total = total * 0.2
18     fin si
19 fin si
20 imprimir total
21 fin
```

3.2. Estructuras repetitivas y estructuras anidadas

A medida que los problemas crecen en complejidad, las instrucciones, secuencias y estructuras de decisión no son suficientes. Es necesario usar estructuras de repetitivas para ejecutar la misma secuencia de instrucciones varias veces. Ahora bien, si la secuencia de instrucciones dentro de una estructura repetitiva contiene estructuras de decisión, el algoritmo ejecutará secuencias diferentes de acuerdo con las condiciones que se evalúen.

Secuencia de números de 1 hasta n

En este problema necesitamos generar una secuencia simple de números.

Construir un algoritmo que imprima por pantalla la secuencia de números enteros entre 1 y un número entero $n > 0$.

Análisis

Este es un ejemplo básico de un algoritmo de producción de datos. Inicialmente, solo se requiere que el usuario especifique el valor de n , y con esto el algoritmo ya tiene toda la información necesaria para producir (*generar*) todos los números naturales de 1 a n e imprimirlos.

Diseño

Para solucionar este problema se puede usar una variable cuyo valor inicial se establece en 1, y mediante una estructura repetitiva (*mientras - hacer*, *hacer mientras*, o *para hacer*), se imprime su valor y se incrementa, mientras su valor sea menor o igual a n . Veamos el proceso con una estructura *hacer mientras*.

```

leer n
i = 1
mientras i <= n
    //i toma los valores 1, 2, 3, 4 .. n
    //En este punto se puede imprimir el valor de i
    imprimir i
    i = i + 1
    //i toma los valores 2, 3, 4, ... n + 1
    //No es buen lugar para imprimir i!
fin mientras
//i termina almacenando n + 1

```

Cuando hacemos uso de una variable de la forma como hicimos con i , suele denominarse *contador*. Los contadores (dependiendo del problema) se inicializan en 0 o 1, y *después* se incrementan usando una estructura repetitiva. Se debe tener en cuenta que valor final que se almacena en contador dependerá de la condición que controla dicha estructura.

Puede realizar la prueba de escritorio para verificar que la variable i toma los valores de 1, 2, 3, hasta $n + 1$, momento en el cual termina de ejecutarse la estructura repetitiva.

Algoritmo

El algoritmo que soluciona al problema se presenta en el Código 3.11, ya que en el diseño realizado identificamos la posición en la cual se debe imprimir el valor de la variable i .

Código 3.11: Secuencia de números desde 1 hasta n

```

1 /*
2  * Genera e imprime la secuencia de números naturales entre 1 y un número
3  * n dado
4  */
5 algoritmo secuenciaEntera

```

3. EJERCICIOS DE FUNDAMENTACIÓN

```
6 entero i, n
7 inicio
8 leer n
9 i = 1
10 mientras i <= n
11     imprimir i
12     i = i + 1
13 fin mientras
14 fin
```

Sumatoria de n primeros números naturales

Comenzamos a estudiar el uso de estructuras repetitivas con el siguiente ejercicio:

Construir un algoritmo que dado un número natural n , calcule la sumatoria de los números naturales de 1 a n .

Análisis

En este problema se tiene un solo dato de entrada, un número natural n . El algoritmo entonces deberá calcular la sumatoria de los números enteros hasta n , de la siguiente forma:

$$\text{suma} = 1 + 2 + 3 + \dots + n - 1 + n$$

A partir de este análisis se pueden identificar dos subproblemas: primero, debemos *generar* los números naturales de 1 a n , y luego debemos sumarlos. El primero de ellos fue solucionado en el ejemplo anterior, de forma que sólo nos queda realizar la suma de los números generados.

Diseño

Si analizamos de nuevo el algoritmo que soluciona el problema anterior obtenemos la solución inmediata, ya que tenemos dos lugares en los cuales podemos *usar* el valor del contador para calcular la sumatoria:

```
leer n
suma = 0
i = 1
mientras i <= n
    //i toma los valores 1, 2, 3, ... n
    //realizar la suma aquí
    suma = suma + i
    i = i + 1
```



```
//i toma los valores 2, 3, ... n + 1
//sería equivocado realizar la suma aquí
fin mientras
```

La variable *suma* se usa como un *acumulador*, que de forma progresiva almacena la suma de 1 hasta *n*. Observe que es similar a un *contador*, pero en vez de sumar 1 cada vez se suma el valor almacenado en *i* (1, 2, 3, hasta *n*).

Tenga en cuenta al igual que el contador, la inicialización del acumulador se debe realizar *antes* de iniciar la estructura repetitiva que lo modifica. Si se inicializa dentro de la estructura repetitiva, *se perdería* lo acumulado.

También es posible solucionar este problema inicializando el contador en 0, en cuyo caso debemos *modificar la condición* y realizar la suma *después* del incremento:

```
leer n
i = 0
mientras i < n
    //i toma los valores 0, 1, 2, ... n - 1
    i = i + 1
    //i toma los valores 1, 2, 3, ... n
    //realizar la suma aquí
    suma = suma + i
fin mientras
```

Por último podemos usar una estructura *para hacer*, la cual, definida correctamente nos permite sumar exactamente los números que necesitamos:

```
leer n
suma = 0
para i = 1, i <= n, i = i + 1
    //i toma los valores 1, 2, 3, 4 .. n
    suma = suma + i
fin para
```

Algoritmo

La solución se obtiene al completar el diseño obtenido, como se muestra en el Código 3.12.

Código 3.12: Suma de los primeros *n* números naturales

```
1 /*
2  * Calcula la suma de los números naturales desde 1 a un
3  * número n especificado
4  */
5 algoritmo sumaNaturales
6 entero n
7 entero i
8 inicio
9     leer n
```

3. EJERCICIOS DE FUNDAMENTACIÓN

```
10 suma = 0
11 para i = 1, i <= n, i = i + 1
12     suma = suma + i
13 fin para
14 imprimir suma
15 fin
```

Determinar si todos los números son iguales

Regresamos de nuevo al punto de partida, un algoritmo de decisión que permite comparar números. Ya vimos cómo se puede construir algoritmos que permitan comparar dos o tres números, para determinar si son iguales o no. Pero, qué pasa si no sabemos de antemano cuantos números vamos a comparar? o dicho de otra forma, *¿si quien establece cuantos datos debe comparar no es el algoritmo, sino el usuario?*

Construir un algoritmo que lea n números y determine si todos ellos son iguales.

Análisis

Comprender la solución de este problema nos permitirá dar un paso significativo en el desarrollo del *pensamiento algorítmico*, también llamado *pensamiento computacional*.¹. De hecho, es *necesario* comprender el análisis de este problema y su solución, antes de estar en capacidad de abordar problemas más complejos.

Inicialmente, el algoritmo deberá leer un valor n que representa la cantidad de números que se van a leer y verificar si son todos iguales. Con el valor de n definido, el algoritmo deberá obtener del usuario exactamente n datos, que representan los números a leer y comparar.

```
leer n
```

En un algoritmo *se debe* conocer de antemano el número de variables que se van a usar. Eso representa un problema, ya que en este caso *es el usuario* quien define el valor de n. Entonces, primero debemos solucionar el problema de leer n números, antes siquiera de intentar compararlos para determinar si son iguales. Este problema puede ser solucionado usando dos estrategias diferentes que se explican a continuación.

¹Es decir, la *secuencia de razonamiento* y las acciones que debería usar un sistema computacional para resolver un problema

Si el valor de n se conoce de antemano, y es pequeño, entonces se definirán en el algoritmo n variables y se ejecutará una *secuencia* de n instrucciones de lectura, en las cuales cada instrucción almacena el valor leído en una variable diferente.

Por ejemplo, si se desea leer dos números ($n = 2$), se definirán dos variables (cuyos nombres dependerán del problema, pero podemos llamarlas a y b) y se ejecutarán dos instrucciones de lectura, la primera para leer el valor de a y la segunda para leer el valor de b . Si ($n = 3$), se usarán tres variables (a , b y c , por ejemplo), y tres instrucciones de lectura. Esta fue la estrategia usada en los algoritmos anteriores.

```
1 algoritmo leerTresNumeros
2 entero a, entero b, entero c
3 inicio
4
5 leer a
6 leer b
7 leer c
8
9 //TODO comparación
10
11 fin
```

En el pseudocódigo presentado, al llegar a la línea 9 ya se cuenta con los valores necesarios para realizar la comparación. Para llevar a cabo este proceso, se puede usar estructuras de decisión con condiciones simples o compuestas, o estructuras de decisión anidadas.

Regresemos al caso base². Ya sabemos cómo comparar dos números, para determinar si son iguales o no. Debemos extender esta solución, *pensando de forma algorítmica*. Ahora bien, ¿cómo se pueden comparar cuatro números a , b , c y d para determinar si son iguales? ¿y cinco números? ¿y n números?

Vamos a extender el caso base en *un paso*, es decir, comparar tres números. Para ello reutilizamos el algoritmo desarrollado anteriormente.

```
1 leer a
2 leer b
3 leer c
4 si a = b y a = c
5 imprimir "Los números son iguales"
6 sino
7 imprimir "Los números no son iguales"
8 fin si
```

Comparar cuatro números ya no es tan sencillo, debido a que la condición dentro de la estructura *si - sino* se complica. Suponiendo que los valores se leen en cuatro variables a , b , c y d , la estructura básica sería:

²Un caso base es una condición o situación inicial que es conocida, y sobre la cual podemos partir para avanzar en la solución del problema.

3. EJERCICIOS DE FUNDAMENTACIÓN

```
leer a
leer b
leer c
leer d
si a = b y a = c y a = d
  //Son iguales
sino
  //No son iguales
fin si
```

Y si fueran cinco números, la condición, usando cinco variables diferentes, sería:

```
leer a, b, c, d, e
si a = b y a = c y a = d y a = e
```

Y si fueran n variables?

```
leer a, b, c, d, e, f, g, ,Z
si a = b y a = c y a = d y a = e y a = f y a = g y a = Z
```

¡Rápidamente nos quedaríamos sin variables!.

Si el valor de n es muy grande, o no se conoce de antemano porque es un parámetro que ingresa el usuario, se debe usar una estrategia diferente. En este caso, no se definen n variables y n instrucciones de lectura, y no se usan estructuras de decisión anidadas o condiciones complejas. En vez de ello, *se puede definir una variable, y repetir una lectura sobre esta variable n veces*. En otras palabras, no se usará una secuencia de n instrucciones de lectura, sino que se repetirá una instrucción de lectura n veces.

Diseño

En el análisis encontramos que no es viable usar un número predefinido de variables para almacenar los datos leídos, y que en su lugar debemos usar alguna estructura repetitiva. La decisión de cual de ellas usar dependerá del problema que se esté solucionando. Pero, ¿cómo controlamos el número de repeticiones que debemos realizar? es decir, si el usuario ingresa por ejemplo 5 como valor de n, entonces debemos *repetir* una lectura de datos 5 veces, si el usuario ingresa 10 para n, debemos *repetir* una lectura 10 veces, y así sucesivamente.

Podemos hacer uso de un contador, como se mostró en el ejercicio anterior. En el Código 3.13 se muestra un esquema el cual usa un contador para controlar el número de veces que se lee un dato en la variable x, dependiendo del valor de n proporcionado por el usuario.

Código 3.13: Leer n valores usando una estructura repetitiva

```
1 leer n
```

```
2 para i = 1, i<=n, i = i + 1
3   leer x
4 fin para
```

Ya tenemos una estrategia básica para *leer n números*, pero ocurre algo interesante. Observe que la lectura del dato a procesar se lleva a cabo dentro de una estructura que se repite exactamente *n* veces. Por lo cual, cada vez que se repita esta estructura se almacenará en la variable (*x* en este caso) el valor obtenido, y se *reemplazará* el valor que estaba almacenado en la iteración anterior.

Esto tiene una consecuencia interesante: en cualquier momento de la ejecución de la estructura repetitiva, *sólo se cuenta* con el último valor leído. Si se necesitara contar con todos los valores, se debería *salvar* cada uno de ellos en otro sitio (en otra variable), antes de iterar para realizar la siguiente lectura y evitar que el valor anterior *se pierda*.

Pero, por las características de este problema *no necesitamos* almacenar *todos* los valores para compararlos posteriormente. Podemos usar un razonamiento más simple: Si todos los números son iguales, ¡entonces todos los números a partir del segundo son iguales al primer número leído!

Por lo tanto podemos leer el primer número y almacenar este valor en una variable. Después leemos los números restantes (*n - 1*), y comparamos cada uno de ellos con el primero, como se muestra en el siguiente fragmento.

```
leer n
leer primero //Leer primer número
//leer los demás y comparar con el primero
para i = 1, i <= n - 1, i = i + 1
  leer x
  si x != primero
    //Encontramos un número diferente!
  fin si
fin para
```

Con este fragmento podemos determinar cuando algún número es diferente del primero, y por tanto, podemos decir que todos los números no son iguales.

Debido a que el problema nos pide leer *n* números, no podemos *cancelar* el ciclo al encontrar el primer dato diferente. Pero debemos *marcar* de alguna manera que se ha presentado esta situación.

Para este propósito podemos usar una *bandera*, la cual no es más que una que dependiendo de su valor, nos permite determinar si se ha cumplido una condición específica. Cuando la condición se cumple (o no, dependiendo del caso) *movemos la bandera*, es decir, **cambiamos su valor**, para indicar que se presentó una situación que debemos conocer. Las banderas también se conocen como *centinelas*, porque *avisan* que algo importante ha sucedido.

```
leer n
```

3. EJERCICIOS DE FUNDAMENTACIÓN

```
leer primero
iguales = 1 //inicializar la bandera
para i = 1, i <= n - 1, i = i + 1
    leer x
    si x != primero
        iguales = 0 (moverBandera)
    fin si
fin para
```

En primer lugar, se supone que todos los valores son iguales, condición se seguirá cumpliendo *mientras los valores leídos sean iguales*, o dicho de otra forma, *hasta que el nuevo valor leído sea diferente al primer valor leído*. En este caso *moveremos la bandera* (línea moverBandera) para indicar que *se encontró un dato que no es igual a los demás*.

Observe que en este caso la bandera se mueve *cada vez* que se encuentra un dato diferente. Al final lo que interesa es si ha sido movida, no importa cuantas veces.

Algoritmo

El Código 3.14 presenta una implementación del algoritmo que soluciona el problema combinando los elementos explicados anteriormente. Además se valida si el número n ingresado es menor o igual que cero, en cuyo caso el algoritmo *termina temprano*.

Código 3.14: Determinar si n números son iguales

```
1  /*
2  * Determina si n números leídos son iguales
3  */
4  algoritmo todosSonIguales
5  entero n, entero x, entero i
6  entero primero
7  entero iguales
8  inicio
9      leer n
10     si n <= 0
11         terminar // Validar el valor de n proporcionado
12     fin si
13     leer primero // Primera lectura
14     iguales = 1
15     para i = 1, i <= n - 1, i = i + 1
16         leer x // Leer los demás valores usando la misma variable
17         si x != primero
18             iguales = 0 //Mover la bandera!
19         fin si
20     fin para
21     si iguales = 1
22         imprimir "Todos los números leídos son iguales"
23     sino
24         imprimir "No todos los números leídos son iguales"
25     fin si
26 fin
```

Con este algoritmo aún no podemos saber cual es el número diferente, de hecho ese problema es aún más interesante.

En el algoritmo implementado, la bandera usada la variable `iguales` era de tipo entero, con lo cual podía tomar cualquier valor numérico. No obstante, sólo nos interesa que tome dos valores, el valor de **1** si los números son iguales, y **0** si los números no son iguales.

Existe un tipo de datos especial, llamado **booleano**, el cual es usado con frecuencia para implementar las *banderas*. Este tipo se expresa en dos valores, verdadero y falso, y en algunos lenguajes de programación toman valores de 1 y 0, respectivamente³. De esta forma, podemos cambiar nuestro algoritmo anterior, para que la bandera sea una variable booleana como se muestra en el Código 3.15.

Código 3.15: Determinar si n números son iguales usando variable booleana

```

1  /*
2  * Determina si n números leídos son iguales
3  */
4  algoritmo todosSonIguales
5  entero n, entero x, entero i
6  entero primero
7  booleano iguales
8  inicio
9  leer n
10 si n <= 0
11     terminar // Validar el valor de n proporcionado
12 fin si
13 leer primero // Primera lectura
14 iguales = verdadero
15 para i = 1, i <= n - 1, i = i + 1
16     leer x // Leer los demás valores usando la misma variable
17     si x != primero
18         iguales = falso //Mover la bandera!
19     fin si
20 fin para
21 si iguales = verdadero
22     imprimir "Todos los números leídos son iguales"
23 sino
24     imprimir "No todos los números leídos son iguales"
25 fin si
26 fin

```

Observe que en este caso, en el pseudocódigo se usan las palabras verdadero y falso. Al codificar este pseudocódigo en un lenguaje de programación, podemos codificarlo de diferentes formas: Si el lenguaje incluye soporte para tipos booleanos, se usará la palabra `true` o `TRUE` para verdadero, y `false` o `FALSE` para falso.

³Dependiendo del lenguaje de programación la definición de verdadero y falso puede ser diferente. Una definición alternativa considera falso como 0, y verdadero como cualquier valor diferente de cero

3. EJERCICIOS DE FUNDAMENTACIÓN

Debido a que las estructuras condicionales *evalúan* la condición para determinar si es verdadera o falsa, es posible abreviar la condición cuando se usa una variable booleana. En vez de:

```
si iguales = verdadero
```

Simplemente se puede escribir:

```
si iguales
```

Debido a que la variable `iguales` es de tipo booleano, y almacenará verdadero o falso, dependiendo del resultado del proceso anterior.

Todos son pares?

Ahora vamos a resolver un problema en el cual se busca verificar si *todos* los datos cumplen con determinada característica.

Construir un algoritmo que dados n números, determine si todos ellos son pares o no.

Análisis

Al igual que en el ejercicio anterior, necesitamos primero leer el valor de n . Este valor representa la cantidad de números que debemos leer. Se puede hacer uso de una estructura repetitiva para leer los n números, y de nuevo suponemos que los números a leer son pares. Si encontramos que un número que no es par, debemos *anotar* esta situación, y continuar con la lectura hasta completar los n datos. Al final, debemos verificar si en algún momento se encontró un número que no fuera par.

Diseño

Para solucionar este problema podemos usar de nuevo una **bandera**, la cual nos permitirá almacenar un valor: verdadero si los números leídos son pares, y falso si alguno de los números leídos no es par. Al final del algoritmo, se verificará el valor de la bandera y se imprimirá el mensaje correspondiente.

```
leer n
para i = 1, i <= n, i = i + 1
  leer x
  si x mod 2 != 0
    //x no es par, mover la bandera
  fin si
```



```
fin para
//Se movió la bandera?
```

Algoritmo

Partiendo del diseño realizado, podemos construir el algoritmo que permita verificar si todos los números leídos son pares (Ver Código 3.16).

Código 3.16: Verificar si n números son pares

```
1 /*
2  * Verifica si n números dados son pares
3  */
4 algoritmo todosSonPares
5 entero n, entero i, entero x
6 booleano todospares
7 inicio
8   leer n
9   todospares = verdadero
10  para i=1, i<=n, i = i + 1
11     leer x
12     si x mod 2 != 0
13         todospares = falso
14     fin si
15  fin para
16
17  si todospares
18     imprimir "Todos los numeros leidos son pares"
19  sino
20     imprimir "No todos los numeros leidos son pares"
21  fin si
22 fin
```

Observe que la variable `todospares` se inicializa en `verdadero`, por lo cual estamos asumiendo que los datos de entrada son pares. En cuanto un dato leído no es par, su valor se actualiza a `falso` (se mueve la bandera). Si varios números leídos no son pares, este valor se establece de nuevo a `falso`.

Dicho de otra forma, los únicos casos en que la variable `todospares` termina siendo verdadera al final del algoritmo son:

- El valor de `n` es cero (o menor que cero), por lo cual no hay datos de entrada. Esto se puede validar con una condición simple, que se omite en el algoritmo.
- El valor de `n` es mayor que cero (hay datos de entrada), y la condición nunca se cumple, por lo cual todos los datos leídos son pares.

Determinar si un número es primo

Los números primos son usados ampliamente no solo en matemática. Su aplicación se extiende a la ciencia y la tecnología, incluso los algoritmos para cifrar datos usualmente hacen uso de las propiedades especiales de esta clase de números. Este problema se encuentra especificado en varios libros de texto, entre los que se incluye Joyanes Aguilar and Zahonero (2005).

Construir un algoritmo que dado un número n , determine si es primo o no.

Análisis

Por definición, un número natural entero mayor que 1 se considera primo sólo si es divisible por sí mismo y por la unidad. Esta definición tiene algunos aspectos interesantes:

- Ni los números negativos, así como 0 y 1 se consideran primos.
- El 2 es un caso especial, porque es par (divisible por 2) y primo.

En los demás casos, para determinar si un número N es primo, se puede usar un proceso relativamente sencillo pero poco eficiente: dividir sucesivamente por 2, 3, 4, 5, hasta $N - 1$. Si el residuo de alguna de estas divisiones es cero, hemos encontrado un divisor entero, por lo tanto el número no es primo. Podemos *terminar* la verificación en este punto, debido a que hemos encontrado un divisor.

Diseño

Para solucionar el problema podemos usar una estructura repetitiva entre los números 2 y $N - 1$ para verificar si alguno de ellos es divisor el número dado:

```
leer n
para i = 2 hasta n - 1
  si n mod i = 0
    //i es divisor de n, por lo tanto n no es primo
    //terminar!! encontramos un divisor
  fin si
fin para
```

Algoritmo

El algoritmo que permite determinar si un número dado es primo se presenta en el Código 3.17.

Código 3.17: Verificar si un número dado es primo

```

1  /*
2  * Determina si un número n dado es primo
3  */
4  algoritmo esPrimo
5  entero n, entero i
6  inicio
7  leer n
8  si n <= 1
9  imprimir "El numero no es primo"
10  terminar
11 fin si
12 si n = 2
13 imprimir "El numero es primo"
14 terminar
15 fin si
16 para i = 2, i < n, i = i + 1
17 si n mod i = 0
18 imprimir "El numero no es primo"
19 terminar
20 fin para
21 imprimir "El numero es primo"
22 fin

```

En esta solución se hace uso de la estrategia *terminar temprano*. Esta estrategia consiste en verificar primero los denominados *casos de frontera*, es decir, aquellos casos para los cuales no tiene sentido continuar ejecutando el algoritmo.

Para el ejercicio de verificación de un número primo, existen los siguientes casos para terminar temprano:

- Primero descartamos todos los números negativos, el 0 y el 1. Estos números no se consideran primos. Se debe terminar.
- El número 2 también es un caso especial, lo tratamos como tal. Es un número primo, por lo cual ya debemos terminar.

De otro lado, en el momento en que se encuentre el primer divisor, el algoritmo termina, ya que lo único que se está pidiendo es verificar si el número es primo o no.

¿Cómo se podría implementar este algoritmo sin aplicar la estrategia de *terminar temprano*? una posible solución sería usando *banderas*.

Sucesión de Fibonacci

Esta secuencia ha sido estudiada por siglos y se le atribuyen propiedades interesantes. Se dice que la secuencia fue descubierta al estudiar el problema de la cría de conejos, para determinar el número de animales que se tendría luego de un número determinado de generaciones.

3. EJERCICIOS DE FUNDAMENTACIÓN

Construir un algoritmo que presente los primeros 20 números de la sucesión de Fibonacci.

Análisis

La sucesión de Fibonacci es una secuencia infinita de números naturales, que inicia con los números 1, 1 y de ahí en adelante cada nuevo número se calcula como *la suma de los dos números anteriores*.

Los primeros números de la sucesión de Fibonacci son:

1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, etc.

Con esta información podemos diseñar un algoritmo, el cual a partir de dos valores, calcula el siguiente valor de la sucesión. En este problema se deberá generar e imprimir los primeros 20 números de la sucesión (es decir, los primeros 18 porque ya tenemos los dos iniciales).

Diseño

En la situación inicial (el *caso base*), la secuencia comienza con 1 y 1. Estos valores se pueden almacenar en dos variables, `a` y `b`, e imprimirlos inmediatamente. Además se almacenará el valor de `b` (1) en una variable adicional, llamada `resultado`.

En el siguiente paso, se deben tomar los *dos valores anteriores* para calcular el nuevo valor. Para ello, se debe realizar la siguiente secuencia:

```
a = b
b = resultado
resultado = a + b
```

- El valor de `b` ahora debe ser almacenado en la variable `a`.
- El nuevo valor de `b` es el resultado de la suma anterior, es decir, el el valor que se encuentra almacenado en la variable `resultado`.
- Ahora se calcula de nuevo la suma entre `a` y `b`, que se almacenará de nuevo en la variable `resultado`.

Partiendo del caso inicial `a = 1` y `b = 1`, la secuencia hará que ahora la variable `a` tome el valor de `b` (1), `b` tomará el valor almacenado en `resultado` (1), y ahora `resultado` se calcula como la suma de `a` y `b`, es decir, 2.

Si ejecutamos de nuevo la misma secuencia, ahora a tomará el valor de b (1), y b tomará el valor de la suma anterior (2). Se calcula de nuevo resultado como la suma de a y b, que da como resultado 3.

Esta secuencia de instrucciones puede ser repetida tantas veces como se desee, y permitirá calcular el siguiente valor de la secuencia. En este caso, sólo se necesitan los primeros 20 números.

Algoritmo

Una posible implementación del algoritmo para generar los primeros 20 números de la sucesión de fibonacci se presenta en el Código 3.18.

Código 3.18: Primeros 20 números de la sucesión de Fibonacci

```
1 /*
2  Genera los 20 primeros números de la sucesión de Fibonacci
3  */
4 algoritmo fibonacci
5 entero a, entero b
6 entero resultado
7 entero i, entero limite
8 inicio
9   a = 1
10  b = 1
11  imprimir a
12  imprimir b
13  i = 2
14  limite = 20
15  resultado = a
16  mientras i <= limite
17     a = b
18     b = resultado
19     resultado = a + b
20     imprimir resultado
21     i = i + 1
22 fin mientras
23 fin
```

Promedio de n números

Vamos a extender el problema del promedio de dos números, para calcular el promedio de una cantidad inicialmente indeterminada de elementos.

Construir un algoritmo que dada una serie de números, calcule su promedio. Suponga que los números son enteros y positivos. El algoritmo deberá terminar cuando el número leído sea menor o igual que cero.

Análisis

Este problema es interesante, ya que no se sabe de antemano cuantos datos hay que leer. En los problemas anteriores, primero se preguntaba al usuario la cantidad de datos a leer, pero en este caso se debe repetir el algoritmo hasta que el número leído sea menor o igual que cero, o dicho de otra forma, mientras sea mayor que cero. En otras palabras, la cantidad de datos *es indeterminada*, y se deberá continuar leyendo dato por dato hasta que se cumpla la condición especificada.

Para calcular el promedio de un conjunto de números, se deben sumar y dividir entre la cantidad de números en el conjunto. Por ejemplo, para el siguiente conjunto:

5 3 4 7 19 20 2 11

El promedio se calculará así:

$\text{promedio} = (5 + 3 + 4 + 7 + 19 + 20 + 2 + 11) / 8 = 8.875.$

Entonces debemos primero *sumar* y *contar* todos los datos, para luego *calcular* el promedio como la suma entre el número de datos leídos.

Diseño

Para leer los datos se puede usar una estructura repetitiva *hacer mientras*, cuya condición permitirá verificar si el siguiente valor leído es mayor que cero. En caso afirmativo, continuará la lectura. En caso contrario se romperá el ciclo.

```
algoritmo leerNumeros
entero x
inicio
  leer x
  mientras x > 0
    //demás instrucciones
    leer x
  fin mientras
fin
```

En este caso, se realiza una lectura *antes* de iniciar el ciclo. De esta forma, sólo se entra al ciclo si el valor leído cumple con la condición requerida (en este caso, debe ser diferente de cero). Luego se ejecuta una secuencia de instrucciones, y *justo antes* de cerrar la estructura repetitiva, se lee un nuevo dato y se almacena en la variable. Luego, debemos retornar al inicio del ciclo, y evaluar de nuevo la condición.

El algoritmo deberá entonces usar la estructura repetitiva presentada anteriormente para leer un número a la vez, y sumarlo a una variable que inicia con valor 0 y que al final del ciclo contendrá la suma de los datos leídos.

```
1 suma = 0
2 leer x
3
4 mientras x > 0
5     suma = suma + x
6     leer x
7 fin mientras
```

Adicionalmente se deberá usar otra variable para llevar la cuenta de la cantidad de números leídos, es decir, un *contador*.

```
1 suma = 0
2 n = 0
3 leer x
4 mientras x > 0
5     n = n + 1
6     suma = suma + x
7     leer x
8 fin mientras
```

Después de ejecutar la estructura repetitiva, se tendrá en la variable *suma* la sumatoria de los números ingresados, y en otra variable *n* se almacenará la cuenta de los números leídos. Con esto ya se puede calcular el promedio de los datos.

Algoritmo

Antes de calcular el promedio se debe realizar una *validación* adicional: qué sucede si el primer dato ingresado es cero?. En este caso, no se ejecuta la estructura repetitiva, y por tal razón tanto *suma* como *n* terminan con valor de cero. Si dividimos *suma* entre *n*, tendremos **indeterminado**, debido a que la división por cero es indeterminada en matemáticas. Por esta razón, solo se debe calcular el promedio si *n* es mayor que cero.

Esto se logra con una asignación inicial para la variable *promedio* en cero, y una estructura condicional que solo calcula el promedio si *n* es mayor que cero, es decir, si se leyó al menos un dato. El algoritmo que implementa la solución del problema se presenta en el Código 3.19.

Código 3.19: Promedio de n números

```
1 /*
2  * Lee n números y calcula su promedio
3  */
4 algoritmo promedio
5 entero x
6 entero suma, entero n
7 real promedio
8 inicio
9
10 promedio = 0
```

3. EJERCICIOS DE FUNDAMENTACIÓN

```
11 suma = 0
12 n = 0
13 leer x
14 mientras x > 0
15     n = n + 1
16     suma = suma + x
17     leer x
18 fin mientras
19
20 si n > 0
21     promedio = suma / n
22 fin si
23 imprimir promedio
24 fin
```

Calculadora simple

Ahora vamos a presentar un ejemplo sencillo del uso de una estructura repetitiva combinada con una estructura de selección.

Construir un algoritmo que implemente una calculadora simple. Para ello, deberá tomar como entrada dos números reales (a y b), y un caracter que representa la operación a realizar: suma (+), resta (-), multiplicación (*) o división (/). El algoritmo deberá calcular e imprimir el resultado de la operación correspondiente. Luego deberá preguntar al usuario si desea continuar, a lo cual el usuario responderá con el caracter 's' o 'n'. El algoritmo deberá terminar cuando el usuario responda 'n'.

Análisis

Este algoritmo es relativamente sencillo. No obstante, tiene una complicación relacionada con el mecanismo para continuar ejecutándose hasta que el usuario ingrese 'n' (sin las comillas). Resolveremos este problema primero, y luego resolveremos el problema de los cálculos que se deben realizar.

En los algoritmos anteriores se usaron tipos de datos numéricos (enteros y/o reales). Otro tipo usado en los programas es el tipo **caracter**. Este tipo de datos permite almacenar un único caracter (letra, número, signo de puntuación, espacio, etc.) en una variable.

Diseño

Se definirán dos variables de tipo caracter. La primera llamada `operacion` contendrá la operación a realizar, y la otra variable llamada `c` permitirá determinar si se debe continuar leyendo datos o terminar.

El esquema de algoritmo para realizar un proceso hasta que el usuario especifique 'n' es una repetición simple, de la siguiente forma:

```
algoritmo hastaQueNo
caracter c
inicio
  c = 's'
  mientras c != 'n'
    //Proceso
    imprimir "Desea continuar? s/n"
    leer c
  fin mientras
fin
```

Observe que en este caso, es necesario *establecer* la variable de tipo de caracter antes de iniciar el ciclo, para que se entre a éste por lo menos una vez. Esto se puede evitar usando una estructura *hacer mientras*. Esta estructura permite leer primero el dato, y luego verificar si debe repetir la secuencia o continuar:

```
algoritmo hastaQueNo
caracter c

inicio
  hacer
    //proceso
    imprimir "Desea continuar? s/n"
    leer c
  mientras c != 'n'
fin
```

Se puede usar una estructura de selección para realizar una operación diferente de acuerdo con el dato leído:

```
leer operacion
seleccionar operacion entre
  caso '+':
    //Suma
  caso '-':
    //Resta
  caso '*':
    //Multiplicación
  caso '/':
    //División
fin seleccion
```

Algoritmo

Con este diseño básico podemos implementar el algoritmo de la calculadora, validando los casos respectivos y usando una estructura de selección sobre el valor del operador leído. La solución se presenta en el Código 3.20.

3. EJERCICIOS DE FUNDAMENTACIÓN

Código 3.20: Calculadora simple

```
1  /*
2  * Calcula el resultado de una operación aritmética dada sobre dos números
3  */
4  algoritmo calculadora
5  real a, real b
6  caracter operacion
7  real resultado
8  caracter c
9  inicio
10 hacer
11     leer a
12     leer b
13     leer operacion
14     seleccionar operacion entre
15         caso '+':
16             resultado = a + b
17             imprimir resultado
18         caso '-':
19             resultado = a - b
20             imprimir resultado
21         caso '*':
22             resultado = a * b
23             imprimir resultado
24         caso '/':
25             si b != 0
26                 resultado = a / b
27                 imprimir resultado
28             sino
29                 imprimir "Indeterminado"
30         fin si
31     otros casos:
32         imprimir "Operacion invalida, debe ser +, -, * o /"
33     fin seleccion
34     imprimir "Desea continuar? s/n"
35     leer c
36     mientras c != 'n'
37 fin
```

Sólo los mayores de edad

Vamos a solucionar un problema en el cual se obtiene un conjunto de datos y se usa una estructura de decisión para seleccionar algunos de ellos y mostrarlos en la pantalla.

Construir un algoritmo que dados la edad y el nombre de n personas, imprima por pantalla solo el nombre y la edad de las personas mayores de edad. Se considera que una persona es mayor de edad si tiene 18 años o más.

Análisis

Este problema es relativamente sencillo. Se requiere leer los datos para n personas, de forma similar a los problemas anteriores. Primero se leerá el valor n , que indica el número de personas. Para cada persona se debe leer su nombre y su edad, y solo si la edad leída es 18 o más, imprimir los datos por pantalla.

Diseño

Vamos a usar una estructura repetitiva, en la que cada iteración permitirá leer los datos (nombre y edad) de una persona de la siguiente forma:

```
leer n
para i = 1, i <= n, i = i + 1
    leer edad
    leer nombre
fin para
```

Ahora necesitamos una estructura de decisión que nos permita imprimir el nombre y la edad de cada persona *sólo* si su edad es 18 o más:

```
leer edad
leer nombre
si edad >= 18
    imprimir nombre, edad
fin si
```

En este caso particular no se requiere usar la posible secuencia alternativa sino, ya que los datos de las personas con edad inferior a 18 años *se deben omitir*, es decir, no se debe mostrar por pantalla.

Algoritmo

El Código 3.21 presenta una posible solución al problema, integrando los elementos descritos en el diseño.

Código 3.21: Solo mayores de edad

```
1  /*
2  * Lee la edad y el nombre de n personas, imprime por
3  * pantalla solo los datos de los mayores de edad
4  */
5  algoritmo soloMayores
6  entero n, entero edad, cadena nombre
7  inicio leer n
8  para i=1, i<=n, i = i + 1
9      leer edad
10     leer nombre
```

3. EJERCICIOS DE FUNDAMENTACIÓN

```
11  si edad >= 18
12      imprimir nombre, edad
13  fin si
14  fin para
15  fin
```

Suma de pares

Ahora vamos a solucionar un problema de transformación de n datos de entrada en un dato de salida. En este caso sólo se tiene en cuenta en el cálculo del dato de salida los datos de entrada que cumplen con una característica específica.

Construir un algoritmo que dados n números, calcule e imprima la suma de los números pares.

Análisis

Este problema es relativamente sencillo. Se necesitará un acumulador para almacenar la suma de los números, pero antes de sumar cada número se debe verificar si es par.

Diseño

La estructura básica para leer los datos es la misma usada en ejercicios anteriores:

```
leer n
para i = 1, i <= n, i = i + 1
    leer x
fin para
```

Para decidir si el número debe ser sumado o no, se puede verificar si es par o no mediante la operación módulo. Sólo si el residuo de la operación es 0, se suma al total.

```
leer x
si x mod 2 = 0
    suma = suma + x
fin si
```

La variable suma deberá ser inicializada en 0 antes de iniciar la lectura. Al terminar la estructura repetitiva, almacenará la suma de los números pares.

Algoritmo

El Código 3.22 muestra una implementación del algoritmo.

Código 3.22: Suma de números pares

```

1  /*
2  * Lee n números y calcula la suma de aquellos que sean pares.
3  */
4  algoritmo sumaPares
5  entero n, entero i, entero x
6  entero suma
7  inicio
8  leer n
9  suma = 0
10 para i=1, i<=n, i = i + 1
11     leer x
12     si x mod 2 = 0
13         suma = suma + x
14     fin si
15 fin para
16 imprimir suma
17 fin

```

3.3. Subrutinas

En esta sección se presentará una serie de problemas que pueden ser solucionados usando estructuras repetitivas y estructuras de decisión, pero para ofrecer una solución más *modular*, se definen y usan *subrutinas*.

Factorial de un número

Este problema permite estudiar un concepto matemático muy importante que tiene múltiples usos en matemática, estadística e ingeniería, entre otros campos.

Construir un algoritmo que dado un número entero mayor o igual que cero, calcule e imprima su factorial.

Análisis

El factorial de un número entero n , representado como $n!$, se define como la multiplicación de todos los números desde 1 hasta n , incluido. Esta definición implica que podemos calcular el factorial *progresivamente* (hacia adelante), realizando el siguiente producto:

$$n! = 1 * 2 * 3 * 4 * 5 * 6 = 720$$

También podemos plantear la siguiente *ecuación de recurrencia* para calcular el factorial de otra forma, partiendo del hecho que el factorial del número 0 es 1:

3. EJERCICIOS DE FUNDAMENTACIÓN

```
factorial(0) = 1
factorial(1) = 1 * factorial(0) = 1 * 1 = 1
factorial(2) = 2 * factorial(1) = 2 * 1 * factorial (0) = 2 * 1 * 1
...
factorial(n) = n * factorial(n - 1)
              = n * n - 1 * factorial(n - 2)
              = n * n - 1 * n - 2 * ... * 1 * 1
```

Diseño

De la definición de factorial se puede ver que necesitamos usar una estructura repetitiva, que nos permita multiplicar sucesivamente una variable que comienza en 1, luego se multiplica por 2, este resultado se multiplica por 3, 4, hasta llegar a multiplicar por n. El factorial será entonces el resultado de esta multiplicación.

```
resultado = 1
para i = 2, i <= n, i = i + 1
    resultado = resultado * i
fin para
```

También es posible realizar el proceso inverso, es decir, comenzar por n y multiplicar hacia atrás:

```
resultado = 1
i = n
mientras i > 1
    resultado = resultado * i
    i = i - 1
fin mientras
```

Observe que en ambos casos se omite la multiplicación por 1, ya que sobra.

Algoritmo

El Código 3.23 presenta una implementación del algoritmo para calcular el factorial de un número entero.

Código 3.23: Factorial iterativo

```
1 /*
2  * Calcula el factorial de un número n de forma iterativa
3  */
4 algoritmo factorial
5 entero n, entero resultado
6 inicio
7     leer n
8     resultado = 1
```

```

9  para i = 2, i <= n, i = i + 1
10     resultado = resultado * i
11  fin para
12  imprimir resultado
13  fin

```

La variable `resultado` inicia en 1, y permite almacenar la multiplicación sucesiva desde 2 hasta n , la cual es precisamente el factorial de n .

Observe lo que sucede si el usuario ingresa el valor de 0 o 1. La estructura repetitiva no se ejecuta, y la variable `resultado` termina con valor de 1. Por feliz coincidencia, estos son precisamente los factoriales de 0 o 1, así que no necesitamos validar los dos *casos base*.

En el Código 3.24 se modifica la solución anterior, implementando el cálculo del factorial como una *función*. Esta función recibirá como parámetro el número al cual deseamos calcular el factorial, y *devolverá* (retornará) el valor del factorial del parámetro recibido.

Código 3.24: Factorial iterativo, usando funciones

```

1  /*
2  * Calcula el factorial de un número n por medio de una función
3  */
4  algoritmo factorial
5  entero n, entero f
6  inicio
7  leer n
8  f = factorial(n) //Llamada a la función
9  imprimir f
10 fin
11 //Calcula y retorna el factorial de un número entero
12 funcion factorial(entero n):entero
13 entero resultado, entero i
14 inicio
15     resultado = 1
16     para i = 2, i <= n, i = i + 1
17         resultado = resultado * i
18     fin para
19     retornar resultado
20 fin funcion

```

También podemos implementar de nuevo el algoritmo, esta vez usando una función recursiva como se muestra en el Código 3.25. Esta solución recursiva calcula el factorial comenzando en n hasta llegar a 1.

Código 3.25: Factorial usando función recursiva

```

1  /*
2  * Calcula el factorial de un número n usando una función recursiva
3  */
4  algoritmo factorial
5  entero n, entero f
6  inicio

```

3. EJERCICIOS DE FUNDAMENTACIÓN

```
7 leer n
8 si n <= 0
9     imprimir "Dato invalido"
10    terminar
11 fin si
12 f = factorial(n) //Llamada a la función
13 imprimir f
14 fin
15 //Calcula y retorna el factorial de un número entero
16 funcion factorial(entero n):entero
17 inicio
18     si n = 0 o n = 1 //Dos casos base
19         retornar 1
20     sino
21         retornar n * factorial(n - 1) //Cálculo recursivo
22     fin si
23 fin funcion
```

En esta función se tienen dos casos base, que son manejados con una sola condición compuesta. Observe que también se debe validar el dato de entrada, para evitar que la función recursiva se ejecute de forma indefinida.

Cálculo de la potencia entera positiva

El siguiente problema es similar al cálculo del factorial, y puede ser solucionado usando estructuras repetitivas o una función recursiva.

Construir un algoritmo que dados un número real x y un número entero $y \geq 0$, calcule el valor de la potencia x^y .

Análisis

Por definición, el resultado de elevar un número x a la y -ésima potencia se calcula multiplicando y veces el valor de x . El caso especial se presenta cuando $y = 0$, en el cual el resultado es 1 sin importar el valor de x .

Diseño

Aplicando esta definición, se puede usar una estructura repetitiva para calcular la potencia de cualquier número elevado a una potencia mayor o igual que cero.

```
leer x
leer y
resultado = 1
para i = 1, i <= y, i = i + 1
```



```

    resultado = resultado * x
fin para

```

Observe que el cálculo realizado es *similar* al factorial, pero no es el mismo. En el factorial se multiplican los valores que se almacenan en la variable que controla la estructura repetitiva `--i--`, mientras que en la potencia se repite la multiplicación del número `x` leído.

Algoritmo

La solución se obtiene directamente del diseño realizado, usando una estructura repetitiva y una variable que almacena el resultado de las multiplicaciones sucesivas como se muestra en el Código 3.26.

Código 3.26: Potencia entera positiva usando iteraciones

```

1  /*
2  * Lee dos números x e y, y calcula el valor de x elevado a la
3  * y-esima potencia
4  */
5  algoritmo potenciaPositiva
6  real x, entero y, entero i, real resultado
7  inicio
8
9  leer x
10 leer y
11
12 resultado = 1
13
14 //Solucionar primero el caso base
15 si y = 0
16     imprimir resultado
17     terminar
18 fin si
19
20 //Demás casos
21 para i = 1, i <= y, i = i + 1
22     resultado = resultado * x
23 fin para
24
25 imprimir resultado
26 fin

```

Se debe descartar el caso base $y = 0$, ya que el resultado será 1 sin importar el valor de x . Luego, el algoritmo realiza las multiplicaciones sucesivas, guardando el resultado en la variable del mismo nombre. Finalmente se imprime el resultado obtenido.

Este problema también se puede solucionar usando una ecuación de recurrencia, de la siguiente forma:

3. EJERCICIOS DE FUNDAMENTACIÓN

$$x^y = 1, \text{ si } y = 0$$

Y su ecuación complementaria:

$$x^y = x * x^{y-1}, \text{ si } y > 0$$

A partir de esta ecuación de recurrencia podemos definir una función recursiva que soluciona el problema de una forma *elegante* como se muestra en el Código 3.27.

Código 3.27: Potencia usando una función recursiva

```
1 /*
2  * Lee dos números x e y, y calcula el valor de x elevado a la
3  * y-ésima potencia usando una función recursiva
4  */
5
6 algoritmo potenciaPositivaRecursiva real x, entero y, entero i, real
7 resultado
8 inicio
9   leer x
10  leer y
11  resultado = potencia(x, y) //Llamada a la función
12  imprimir resultado
13 fin
14
15 //Calcula la y-ésima potencia de un número x
16 funcion potencia(real x, entero y): real
17 inicio
18   si y = 0
19     retornar 1 //Caso base
20   sino
21     retornar x * potencia(x, y - 1) //Llamada recursiva
22   fin si
23 fin funcion
```

Cálculo de Pi

Ahora veremos un ejemplo en el cual podemos ver el uso del pensamiento algorítmico para solucionar un problema que ha existido por milenios.

Desarrollar un algoritmo que permita calcular el valor de π mediante el cálculo de la fórmula de Leibniz:

$$\sum_{n=0}^{\infty} \frac{(-1)^n}{2n+1} = \frac{\pi}{4}$$

El algoritmo deberá solicitar un número $k > 0$, que representará el término hasta el cual se deberá realizar el cálculo.

Análisis

Primero debemos despejar a π de la fórmula de Leibniz, multiplicando por 4 ambos lados de la ecuación:

$$4 \sum_{n=0}^{\infty} \frac{(-1)^n}{2n+1} = \pi$$

Ahora nuestro problema se convierte en calcular una sumatoria desde 0 hasta un número k proporcionado por el usuario. Para ello podemos usar una estructura repetitiva que calcule la sumatoria, y luego multiplicar este resultado por 4 para hallar el valor aproximado de π .

Diseño

En esta fórmula se tiene una fracción, en la cual el denominador se puede calcular mediante una operación aritmética sencilla. No obstante, el numerador requiere realizar una potencia, la cual deberá ser calculada mediante multiplicaciones sucesivas como se mostró en un ejemplo anterior⁴.

Sin embargo, al revisar la fórmula, en este caso se puede encontrar un camino más eficiente. La función del elemento $(-1)^n$ consiste en *alternar* el signo del término i . Observe que si i es par, $(-1)^i$ da como resultado 1, mientras que si i es impar, $(-1)^i$ da como resultado -1. Aplicando este razonamiento, podemos usar una estructura de decisión para determinar el signo del símbolo, y ahorrarnos el cálculo de la potencia.

```

leer k
suma = 0
para i = 0, i<=k, i = i + 1
    num = 1
    si i mod 2 != 0 //i es impar
        num = -1
    fin si
    suma = suma + (num / ((2 * i) + 1))
fin para

```

⁴La mayoría de lenguajes de programación ofrecen subrutinas para calcular la potencia, pero en este momento estamos suponiendo que tales subrutinas no existen y las debemos calcular usando multiplicaciones sucesivas.

Algoritmo

A partir del diseño realizado, se puede crear el algoritmo que permite calcular el valor *aproximado* de π hasta determinado término. La implementación del algoritmo se presenta en el Código 3.28.

Código 3.28: Cálculo iterativo de π

```
1 /*
2  * Cálculo de Pi usando la fórmula de Leibniz
3  */
4 algoritmo calculoPiLeibniz
5 real pi, real suma
6 real num
7 entero k, entero i
8 inicio
9   leer k
10  suma = 0
11  para i = 0, i<=k, i = i + 1
12    num = 1
13    si i mod 2 != 0 //i es impar
14      num = -1
15    fin si
16    suma = suma + (num / ((2 * i) + 1))
17  fin para
18  pi = 4 * suma
19  imprimir pi
20 fin
```

Ahora vamos a *modularizar* el cálculo de π en una función que recibe como parámetro el número del término hasta el cual realizamos la aproximación (Ver Código 3.29).

Código 3.29: Cálculo iterativo de π usando una función

```
1 /*
2  * Cálculo de Pi usando una función
3  */
4 algoritmo calculoPiLeibniz
5 entero k
6 real pi
7 inicio
8   leer k
9   pi = calcularPiHasta(k)
10  imprimir pi
11 fin
12 /* Calcula la aproximación de Pi hasta un término definido*/
13 funcion calcularPiHasta(entero termino):real
14 real suma
15 entero i, num
16 inicio
17   suma = 0
18   para i = 0, i<=termino, i = i + 1
19     num = 1
20     si i mod 2 != 0 //i es impar
```

```

21     num = -1
22     fin si
23     suma = suma + (num / ((2 * i) + 1))
24     fin para
25     retornar 4 * suma
26 fin funcion

```

Máximo común divisor (MCD)

El siguiente es un problema clásico que se encuentra en la mayoría de textos de algoritmia y programación.

Construir un algoritmo que dados dos números enteros positivos a y b , halle e imprima su máximo común divisor.

Análisis

Por definición, el común divisor entre dos números a y b es aquel número entero cuyo residuo de la división entre a y b es cero. Entonces, el máximo común divisor será el mayor de todos los divisores enteros comunes entre a y b .

Se debe tener las siguientes consideraciones:

- El mínimo común divisor de cualquier número mayor que cero es 1. En caso que no exista un divisor común mayor, este será el *caso base*.
- Todo número mayor que cero es divisor de sí mismo.
- El máximo común divisor de 0 con cualquier número x es el mismo número.

A continuación se presentan algunos ejemplos de MCD:

```

mcd (5, 10) = 5
mcd (5, 2) = 1
mcd (48, 60) = 12
mcd (56, 42) = 14
mcd (x, x) = x
mcd (x, n * x) = x

```

Diseño

Partiendo del análisis realizado, podemos diseñar un algoritmo que permita hallar el máximo común divisor entre dos números. Primero vamos a descartar los casos base:

- Si alguno de los números son menores que 1, el mcd es 1.
- Si los números son iguales, el mcd es el mismo número.

Construyamos esta primera aproximación de algoritmo:

```
1 leer a, b
2 si a = 1 o b = 1
3   mcd = 1
4 sino
5   si a = b
6     mcd = a
7   sino
8     //Calcular el mcd de otra forma.
9   fin si
10 fin si
```

Existen diferentes métodos para calcular el mcd, pero el más sencillo (aunque no óptimo) desde el punto de vista algorítmico consiste en buscar todos los posibles divisores comunes de a y b de forma iterativa, y seleccionar el mayor de ellos.

Siguiendo esta aproximación, deberíamos partir del *caso base*, es decir, del mínimo común divisor entre los dos números, que por defecto es la unidad (el número 1). Luego, probamos con 2, 3, etc. hasta un número x . El número x será el máximo común divisor. *Esta solución se deja como ejercicio para el lector.*

Por definición, x no podrá ser mayor que ninguno de los dos números, para poder ser un común divisor. De hecho, el valor de x será igual o menor al menor de los dos números a y b .

Este último razonamiento nos da una pista interesante: Podemos pensar en una segunda estrategia de solución, en la cual no comenzamos en 1, sino en el número más grande que tiene la posibilidad de ser el máximo divisor común de a y b . Este número resulta ser precisamente a o b *el que sea menor* de los dos.

Entonces, nuestro problema puede ser solucionado usando la siguiente estrategia: Comenzaremos por un número k , que será el menor entre los dos números a y b . Si este número es divisor entero de a y b , hemos hallado el MCD. En caso contrario, decrementamos k en 1 y seguimos verificando. Al encontrar el primer divisor común, terminamos la iteración. *En el peor de los casos, debemos llegar hasta 1.*

Algoritmo

De acuerdo con el diseño anterior, luego de descartar los casos base comenzamos a buscar el MCD a partir de un número k , que es el menor entre los valores de a y b . Ese proceso lo realizamos en una estructura repetitiva *hacer mientras*, en cuya condición se usa tanto el valor límite de k (el peor caso, es decir cuando $k = 1$), y una bandera, que nos sirve para detener la iteración *en el primer* divisor común encontrado (Ver Código 3.30).

Código 3.30: Máximo común divisor

```

1  /*
2  * Calcula e imprime el máximo común divisor de dos números a y b
3  */
4  algoritmo buscarMCD
5  entero a, b, mcd
6  entero k
7  booleano encontrado
8  inicio
9  leer a, b
10
11  si a = 1 o b = 1
12    mcd = 1
13  sino
14    si a = b
15      mcd = a
16    sino
17      k = a
18      si b < k
19        k = b
20    fin si
21    encontrado = falso
22    hacer
23      si (a mod k = 0) y (b mod k) = 0
24        mcd = k
25        encontrado = verdadero
26      fin si
27      k = k - 1
28    mientras k > 1 y encontrado = falso
29    si encontrado = falso
30      mcd = 1
31    fin si
32  fin si
33  fin si
34  imprimir "El MCD de ", a , " y ", b , " es ", mcd
35 fin

```

Observe que en la estructura repetitiva se puede omitir la variable bandera *encontrado*, si se asigna 0 como valor de k una vez que se ha encontrado el primer divisor. Esto también causaría que el ciclo terminase.

Por último, se debe validar si existe o no un MCD, ya que en caso de no existir, se debe tomar el caso base, es decir, que el MCD es 1.

3. EJERCICIOS DE FUNDAMENTACIÓN

La solución usando una función que encuentre el MCD entre dos números se muestra en el Código 3.31.

Código 3.31: Máximo común divisor usando funciones

```
1 /*
2  * Calcula e imprime el máximo común divisor de dos números
3  * usando una función
4  */
5 algoritmo buscarMCD
6 entero a, b, resultado
7 entero k
8 inicio
9   leer a, b
10  resultado = mcd(a, b)
11  imprimir resultado
12 fin
13
14 funcion mcd(entero a, entero b): entero
15 booleano encontrado
16   //Caso base, b es cero
17   si b = 0
18     retornar a
19   fin si
20   //Caso base, a es cero
21   si a = 0
22     retornar b
23   fin si
24   //Caso base, a o b son 1, por lo tanto el mcd es 1
25   si a = 1 o b = 1
26     retornar 1
27   fin si
28   //Caso base, los números son iguales, retornar cualquiera
29   si a = b
30     retornar a
31   fin si
32   //Iterar para encontrar el mcd a partir del menor entre a y b
33   k = a
34   si b < k
35     k = b
36   fin si
37
38   encontrado = falso
39   hacer
40     si (a mod k = 0) y (b mod k) = 0
41       retornar k
42     fin si
43     k = k - 1
44   mientras k > 1
45
46   //Por defecto, no existe mcd diferente de 1
47   retornar 1
48 fin funcion
```


Hace más de dos milenios se inventó un algoritmo muy eficiente para hallar el máximo común divisor entre dos números, que es conocido actualmente como el *Algoritmo de Euclides*. Se basa en la idea que si tenemos dos números positivos u y v , y u es mayor que v , entonces el máximo común divisor será un número entre v y $u - v$.

Una pequeña modificación del algoritmo basada en el operador módulo se presenta en Sedgewick and Wayne (2011) y se muestra en el Código 3.32. Si tenemos dos números positivos p y q , se puede usar el operador *módulo*, el cual permite obtener el residuo de la división entera entre p y q . Si el residuo es 0, el máximo común divisor entre p y q es precisamente q . En caso contrario, almacenamos en p el valor de q , y en q almacenamos el residuo de la división entre p y q .

Código 3.32: Algoritmo de Euclides

```

1  /*
2  * Calcula e imprime el máximo común divisor de dos números
3  * usando el algoritmo de Euclides
4  */
5  algoritmo mcdEuclides
6  entero a, b, resultado
7  inicio
8  leer a
9  leer b
10 resultado = mcd(a, b) //Invocar la función mcd
11 imprimir resultado
12 fin
13
14 //Calcula el mcd entre dos números enteros positivos
15 funcion mcd(entero a, entero b):entero
16 entero r
17 inicio
18
19 //Caso base
20 si b = 0
21     retornar a
22 fin si
23 hacer
24     r = a mod b
25     si r = 0
26         retornar b //b es divisor de a, mcd encontrado
27     fin si
28     //Buscar el mcd entre b y a - b (a mod b)
29     a = b
30     b = r
31 mientras a != 0
32     retornar b //No encontrado, en el peor de los casos b = 1
33 fin funcion

```

La solución recursiva del algoritmo de Euclides es *más elegante* aún, y se presenta en el Código 3.33.

Código 3.33: Algoritmo de Euclides en versión recursiva

3. EJERCICIOS DE FUNDAMENTACIÓN

```
1 /*
2  * Calcula e imprime el máximo común divisor de dos números
3  * usando el algoritmo recursivo de Euclides
4  */
5 algoritmo mcdEuclides
6 entero a, b, resultado
7 inicio
8 leer a
9 leer b
10 resultado = mcd(a, b) //Invocar la función mcd
11 imprimir resultado
12 fin
13
14 //Calcula el mcd entre dos números enteros positivos
15 funcion mcd(entero a, entero b):entero
16 inicio
17 si b = 0
18     retornar a
19 fin si
20 retornar mcd(b, a mod b)
21 fin funcion
```

Estudiantes que aprueban una asignatura

Ahora realizaremos una aplicación de algoritmos a cálculos numéricos, esta vez para calcular la definitiva de un conjunto de notas de estudiantes y realizar una selección.

Construir un algoritmo que lea los datos de código y tres notas parciales (n1, n2 y n3) de un conjunto de estudiantes. Para cada estudiante, se deberá calcular e imprimir su nota definitiva, teniendo en cuenta que la primera nota parcial aporta el 35 por ciento a la nota definitiva, la segunda nota aporta el 35 por ciento y la última nota aporta el 30 por ciento restante.

No se conoce por anticipado el número de estudiantes, por lo cual el algoritmo deberá continuar solicitando datos de estudiantes, hasta que se especifique cero como código de estudiante.

También se deberá imprimir el número de estudiantes que aprueban la asignatura, es decir, que su nota definitiva sea mayor que 3.0.

Además se deberá **validar** que los datos ingresados para las notas parciales de cada estudiante se encuentre en el rango de 0.0 a 5.0.

Análisis

Este problema es más interesante, y tiene algunos elementos que debemos analizar.

Se deben leer cuatro datos (`codigo`, `n1`, `n2` y `n3`). El conjunto de estos datos representa *la información de un estudiante*. Con ellos se debe calcular e imprimir el código (que ya lo tenemos), y la nota definitiva. Esta debe ser calculada de acuerdo con lo especificado en el problema.

En este caso la nota final no se calcula como el promedio de las tres notas parciales (es decir sumando `n1`, `n2` y `n3` y dividiendo por 3), sino que cada nota aporta un porcentaje `P` a la nota final.

Recordemos que por definición un *porcentaje* es en realidad una fracción, en la cual el denominador es 100. Por lo tanto, calcular el `P%` de un número es igual a multiplicar al número por la fracción (`P / 100`).

Por ejemplo, si se tiene un valor `x` digamos, 20, y se desea calcular el 50 por ciento (50%) de este valor, se debería realizar el siguiente cálculo:

$$y = x * (50 / 100)$$

Entonces y tomaría el valor de 10, que es el 50% de 20.

En el planteamiento ya nos están indicando los porcentajes que aporta cada nota parcial a la nota final (35%, 35% y 30%, respectivamente), con lo cual podemos definir la operación que calcula la nota definitiva de la siguiente forma:

$$nf = (n1 * 0.35) + (n2 * 0.35) + (n3 * 0.3)$$

En esta expresión, 0.35 equivale al 35% (35/100), y 0.3 equivale al 30% (30/100)

Ya podemos solucionar el problema de leer los datos de un estudiante, calcular su nota definitiva para finalmente imprimir el código y la nota obtenida. La solución se presenta en el Código 3.34.

Código 3.34: Cálculo de la nota definitiva de un estudiante

```
1 leer cod
2 leer n1, n2, n3
3 nf = (n1 * 0.35) + (n2 * 0.35) + (n3 * 0.3)
4 imprimir cod, nf
```

Diseño

Ahora debemos leer los datos de un número *indeterminado* de estudiantes. La única información que nos ofrece el planteamiento del problema es que se debe continuar *hasta que se especifique cero como código del estudiante*. Este planteamiento es similar algunos presentados anteriormente. Como se muestra en el Código 3.35, se debe usar el código del estudiante para controlar la condición de la estructura repetitiva.

3. EJERCICIOS DE FUNDAMENTACIÓN

Código 3.35: Cálculo de la nota definitiva de varios estudiantes

```
1 leer cod // Primera lectura. Si cod = 0, no se entra al mientras
2   en caso contrario, se entrará al mientras
3 mientras cod != 0
4   leer n1, n2, n3
5   nf = (n1 * 0.35) + (n2 * 0.35) + (n3 * 0.35)
6   imprimir cod, nf
7   leer cod // Siguiente lectura, que se realiza justo antes de
8   // evaluar la condición del mientras
9 fin mientras
```

Observe que al usar una estructura *mientras hacer* se deben realizar dos lecturas: la primera lectura se realiza *antes* de entrar a la estructura, y garantiza que sólo entramos a ella si el dato proporcionado es válido, en este caso, si el código es diferente de cero.

Una vez dentro de la estructura, **debemos** realizar otra lectura, que significa que vamos a procesar los datos para *el siguiente* estudiante. Esta lectura generalmente se realiza *justo antes* de terminar la estructura (es decir, antes del `fin mientras`), debido a que es lo último que se hace antes de regresar al inicio del *mientras* y evaluar de nuevo la condición.

En este caso se está cumpliendo lo explicado de las estructuras repetitivas: *Si queremos que un ciclo termine en algún momento, debemos **afectar** una o más variables que aparecen en la condición de la estructura repetitiva*. Si esto no se realiza, la estructura nunca dejará de repetirse.

Existen básicamente dos formas de **afectar** (modificar) una variable: mediante una instrucción de *lectura* o una instrucción de *asignación*, en la cual la variable aparece a la izquierda.

También se puede usar la estructura repetitiva *hacer mientras*, como se muestra a continuación. Tenga cuidado, porque en el siguiente algoritmo falta algo.

```
1 hacer
2   leer cod // La lectura se realiza una vez, dentro del ciclo
3   leer n1, n2, n3
4   nf = (n1 * 0.35) + (n2 * 0.35) + (n3 * 0.35)
5   imprimir cod, nf
6 mientras cod != 0
```

En este caso, se debe prestar especial atención, debido a que por definición la estructura *hacer mientras* se ejecuta *por lo menos* una vez, lo cual hace que no sea necesario leer el código *antes* de entrar a la estructura. Pero, ¿qué pasa si el usuario, en la primera lectura, escribe 0 como código la primera vez?

La estructura *hacer mientras* se ejecutará, y sólo al final evaluará la condición. Para el usuario, este algoritmo no tendrá sentido, debido a que aún si él especifica 0 como código, igual se le pedirán las tres notas. Entonces, se debe realizar una validación, que permita verificar si el usuario ingresó 0 en el código antes de leer las tres notas y calcular la definitiva.

```

1 algoritmo estudiantesQueAprueban
2 entero cod
3 real n1, n2, n3, nf
4 inicio
5   hacer
6     leer cod // La lectura se realiza una vez, dentro del ciclo
7     si cod != 0 // Solo pedir notas y calcular si codigo no es cero
8       leer n1, n2, n3
9       nf = (n1 * 0.35) + (n2 * 0.35) + (n3 * 0.35)
10      imprimir cod, nf
11    fin si
12  mientras cod != 0
13 fin

```

Posteriormente, se debe incluir la lógica necesaria para ir contando los estudiantes cuya nota final *nf* es igual o mayor que 3.0 (Ver Código 3.36). Esto se puede realizar inicializando una variable *aprueban* en cero, y cada vez que se determine que un estudiante aprueba la asignatura, se incrementa esta variable en 1. Esta verificación se realiza con una estructura condicional.

Código 3.36: Cálculo de la nota definitiva, sin validar datos

```

1 n = 0
2 leer cod
3 mientras cod != 0
4   leer n1, n2, n3
5   nf = (n1 * 0.35) + (n2 * 0.35) + (n3 * 0.35)
6   imprimir cod, nf
7   si nf >= 3.0
8     aprueban = aprueban + 1
9   fin si
10  leer cod
11 fin mientras
12 imprimir "El numero de estudiantes que aprueban es ", aprueban

```

La última parte de la solución del problema consiste en **validar** las notas ingresadas por el usuario, es decir, verificar y *posiblemente* evitar que el usuario ingrese notas inválidas. Una nota solo es válida si se encuentra en el rango especificado en el planteamiento del problema, es decir, entre 0.0 y 5.0.

Pero, ¿cómo podemos *evitar* que el usuario ingrese notas inválidas, si es él quien tiene el control de los datos que ingresa?. La respuesta es precisamente *evitando* que el algoritmo *continúe* su ejecución si cualquiera de las notas no es válida. Esto se puede lograr con una estructura repetitiva (Ver Código 3.37), la cual sólo se terminará cuando el usuario haya ingresado una nota en el rango esperado.

Código 3.37: Validar una nota ingresada por el usuario

```

1 imprimir "Ingrese un numero, entre 0.0 y 5.0"
2 leer n

```

3. EJERCICIOS DE FUNDAMENTACIÓN

```
3 mientras n < 0 o n > 5
4   imprimir "El numero no es valido, ingrese un numero entre 0.0 y 5.0"
5   leer n
6 fin mientras
7 imprimir "El numero ingresado es ", n, "que está entre 0.0 y 5.0"
```

Algoritmo

Ya podemos integrar todos los elementos en la solución, que será dada por el algoritmo implementado en el Código 3.38. En este caso, se usará una función `leerNota`, la cual buscará validar que la nota ingresada se encuentre en el rango correcto. La función se usará tres veces, para leer las tres notas. Se deja como ejercicio incluir los mensajes al usuario cuando ingresa notas inválidas.

Código 3.38: Cálculo de la nota definitiva, validando datos

```
1 /*
2  * Lee el código y tres notas parciales, y calcula la definitiva de
3  * cada estudiante. Imprime el código y la definitiva de cada uno
4  * de ellos y también el total de estudiantes que aprueban la asignatura.
5  */
6 algoritmo estudiantesQueAprueban
7   entero cod, entero aprueban
8   real n1, n2, n3, nf
9   inicio
10    aprueban = 0
11    leer cod
12    mientras cod != 0
13      n1 = leerNota()
14      n2 = leerNota()
15      n3 = leerNota()
16      nf = (n1 * 0.35) + (n2 * 0.35) + (n3 * 0.35)
17      imprimir cod, nf
18      si nf >= 3.0
19        aprueban = aprueban + 1
20      fin si
21      leer cod
22    fin mientras
23
24    imprimir "El numero de estudiantes que aprueban es ", aprueban
25  fin
26  //Permite leer una nota entre 0.0 y 5.0, validando datos
27  funcion leerNota():real
28    real n
29    inicio
30      hacer
31        leer n
32        mientras n < 0 o n > 5
33          retornar n
34    fin funcion
```

Una subrutina, en este caso una función, puede ser invocada cuantas veces se desee. Esto nos permite *modularizar* la solución, definiendo funciones o procedimientos que realizan tareas específicas y cuya ejecución es coordinada por el algoritmo principal.

Estudiantes con nota menor que el promedio

Construir un algoritmo que dados el código y tres notas parciales con porcentajes de 30 %, 30 % y 40 % respectivamente de un número indeterminado de estudiantes, imprima el código y la nota definitiva de los estudiantes cuya nota final es menor que el promedio de las notas finales de todo el curso.

El algoritmo deberá continuar solicitando datos hasta que se especifique 0 como código de estudiante.

Análisis

La parte inicial del planteamiento del problema es sencilla, y se trata de leer los datos de código y tres notas de una serie de estudiantes. En el planteamiento se menciona que debemos continuar solicitando datos hasta que se especifique 0 como código de estudiante.

El problema original puede ser dividido en los siguientes subproblemas:

- Calcular la definitiva de cada estudiante
- Calcular el promedio de todo el curso a partir de las definitivas de todos los estudiantes
- Comparar la nota definitiva de cada estudiante con el promedio calculado

Diseño

Con este análisis inicial podemos comenzar a estructurar la solución. Necesitamos una estructura repetitiva que nos permita leer los datos de los estudiantes y calcular su nota definitiva. Podemos partir del algoritmo construido anteriormente:

```
1 leer codigo
2 mientras codigo != 0
3   leer n1, n2, n3
4   nf = (n1 * 0.3) + (n2 * 0.3) + (n3 * 0.4)
5   leer codigo
6 fin mientras
```

3. EJERCICIOS DE FUNDAMENTACIÓN

Ya tenemos la nota definitiva de *cada* estudiante, ahora ¿cómo calculamos el promedio de todo el curso?.

El promedio de todo el curso será la sumatoria de las definitivas de todos los estudiantes, dividida entre el número total de estudiantes. Pero, ¿cómo podemos calcularlo?. Gracias a que tenemos una estructura repetitiva, podemos definir un acumulador y un contador llamados `total` y `n`:

- La variable `total` tendrá cero como valor inicial, y en cada repetición se le sumará la nota definitiva que hemos calculado.
- La variable `n` también tomará como valor inicial cero, y en cada repetición se le sumará 1. Esta variable nos permitirá *contar* el número de estudiantes.

Al terminar la estructura repetitiva, tendremos los datos necesarios para calcular el promedio total del curso.

```
1 total = 0
2 n = 0
3 leer codigo
4 mientras codigo != 0
5     n = n + 1
6     leer n1, n2, n3
7     nf = (n1 * 0.3) + (n2 * 0.3) + (n3 * 0.4)
8     total = total + nf
9     leer codigo
10 fin mientras
11 promedio = total / n
```

Pero ¿qué sucede si el usuario decidió no ingresar datos, es decir, si decidió ingresar 0 para el código *la primera vez*?. Observe que en este caso no se ejecutan las instrucciones dentro de la estructura repetitiva, por lo tanto `n` nunca se incrementa y tendrá valor 0. Debemos verificar este caso, ya que de lo contrario ocurriría un error matemático (división por cero) al momento de calcular el promedio.

```
1 total = 0
2 n = 0
3 leer codigo
4 mientras codigo != 0
5     leer n1, n2, n3
6     nf = (n1 * 0.3) + (n2 * 0.3) + (n3 * 0.4)
7     total = total + nf
8     n = n + 1
9     leer codigo
10 fin mientras
11
12 si n > 0
13     promedio = total / n
14 sino
```



```

15 imprimir "No se ingresaron datos"
16 terminar // Termina la ejecución del algoritmo
17 fin si

```

Observe además que en este caso se decide *terminar* el algoritmo, debido a que si no se ingresaron datos, no tiene sentido continuar.

Nos queda el último detalle, comparar si la nota definitiva de cada estudiantes es menor que el promedio total del curso. Con los medios empleados hasta ahora esto es *prácticamente imposible*, debido a la estrategia que usamos para leer los datos de los estudiantes.

En la solución actual, usamos una estructura repetitiva para leer los datos de los estudiantes, por lo cual cada vez que repetimos (*iteramos*), procesamos los datos de *un* estudiante (o dicho mejor, del *siguiente* estudiante). Pero, ahora necesitamos de nuevo los datos de *todos* los estudiantes, ya que debemos comparar su nota definitiva con respecto al promedio que acabamos de calcular.

Tenemos dos alternativas: pedir de nuevo los datos al usuario, lo cual no es viable. La otra alternativa consiste en *evitar* que los datos del estudiante *anterior* sean *destruidos* (sobrescritos) al leer los datos del *siguiente* dentro de la estructura repetitiva, es decir, *guardarlos* antes de leer los datos del *siguiente* estudiante.

Este es el momento oportuno para recurrir a los *arreglos*. Como se mencionó anteriormente, un arreglo permite almacenar una *colección*⁵ de datos del mismo tipo. Para este problema, definimos el arreglo y *lo vamos llenando con datos a medida que los leemos*. De esta forma, podemos contar con los datos en cualquier momento posterior en el algoritmo, aún después de calcular el promedio.

```

1 algoritmo menorQueElPromedio
2 entero codigo, n
3 real n1, n2, n3, nf, total, promedio
4 entero codigos[] // Definición de arreglo (1)
5 real definitivas[] // Definición de arreglo (1)
6 entero i // Índice en el arreglo (2)
7 inicio
8
9     total = 0
10    n = 0
11    i = 0
12
13    leer codigo
14    mientras codigo != 0
15        leer n1, n2, n3
16        nf = (n1 * 0.3) + (n2 * 0.3) + (n3 * 0.4)
17        total = total + nf
18
19        codigos[i] = codigo // Salvar el codigo en la posicion actual (2)
20        definitivas[i] = nf // Salvar la definitiva en la posicion actual (2)
21

```

⁵Es un término muy usado en computación que se usa para describir un conjunto de datos

3. EJERCICIOS DE FUNDAMENTACIÓN

```
22  n = n + 1 // Incrementar el numero de estudiantes
23  i = i + 1 // Incrementar la posicion en el arreglo (2)
24  leer codigo
25  fin mientras
26
27  si n > 0
28      promedio = total / n
29  sino
30      imprimir "No se ingresaron datos"
31      terminar // Termina la ejecución del algoritmo
32  fin si
33  fin
```

El uso general de los arreglos se puede resumir de la siguiente forma:

1. Definir el nombre del arreglo, y el tipo de datos que almacenará. Por lo general, en el momento de transcribir el algoritmo a un lenguaje de programación, *es necesario* definir el *tamaño de los arreglos*, es decir, el máximo número de elementos (datos) que estos pueden almacenar. Por ahora no nos preocuparemos de eso, que es un aspecto de *implementación*. En algoritmia suponemos que el arreglo puede crecer automáticamente para almacenar los nuevos datos, a no ser que se especifique lo contrario. En este caso, se define un arreglo para almacenar los códigos, y otro arreglo para almacenar las notas finales. Estos son los datos que nos está pidiendo el problema. De ser necesario, podríamos crear tres arreglos más para almacenar las notas parciales.
2. Insertar elementos en el arreglo. Debido a que en un arreglo se almacenan varios datos, por lo general éstos se guardan usando una estructura repetitiva y *un índice que se incrementa dentro de esta estructura*. La primera posición de un arreglo es 0⁶. Si se almacenan n datos, la última posición almacenada será la $n-1$ (Por ejemplo, para almacenar 10 datos, se usarán las posiciones 0, 1, hasta 9). Debemos asegurarnos de *avanzar* la posición (el índice) en la cual se va a guardar el siguiente dato, de lo contrario, se almacenarán todos en la misma posición por lo cual solo se guardará el último dato insertado. En este caso se usa otra variable, llamada i . Observe que *en este caso* tanto la variable i como la variable n se usan de la misma forma, por lo cual se puede omitir la variable i y usar la variable n para dos propósitos simultáneos: contar el número de estudiantes y servir como posición en el arreglo. Por claridad en el algoritmo, y para fomentar el *principio de responsabilidad única*, se usarán las dos variables.

⁶Sin embargo, en algunos textos de algoritmia la primera posición es 1. Debido a que en la mayoría de lenguajes de programación los arreglos simples comienzan en cero, los trabajaremos así para que sea más sencilla la codificación de los programas

3. (no implementado aún) Usar los datos almacenados en el arreglo, para el propósito del algoritmo. En este caso, debemos comparar cada nota final con el promedio obtenido, para imprimir sólo el código y la nota final de los estudiantes cuya nota definitiva está por debajo del promedio.

Algoritmo

Para construir la solución final del problema, nos falta implementar el paso 3 descrito anteriormente. La comparación se puede realizar con una estructura de decisión simple, como se muestra en el siguiente pseudocódigo.

```

1 algoritmo menorQueElPromedio
2 entero codigo, n
3 real n1, n2, n3, nf, total, promedio
4 entero codigos[]
5 real definitivas[]
6 entero i
7 inicio
8
9     total = 0
10    n = 0
11    i = 0
12
13    leer codigo
14    mientras codigo != 0
15        leer n1, n2, n3
16        nf = (n1 * 0.3) + (n2 * 0.3) + (n3 * 0.4)
17        total = total + nf
18
19        codigos[i] = codigo
20        definitivas[i] = nf
21        n = n + 1
22        i = i + 1
23
24    leer codigo
25 fin mientras
26
27 si n > 0
28     promedio = total / n
29
30     para i = 0, i < n, i = i + 1
31         codigo = codigos[i]
32         nf = definitivas[i]
33         si nf < promedio
34             imprimir codigo, nf
35     fin si
36     fin para
37 sino
38     imprimir "No se ingresaron datos"
39     terminar // Termina la ejecución del algoritmo
40 fin si

```

41 `fin`

Algunos aspectos clave de la solución que se deben considerar son:

- Debido a que el problema nos pide imprimir tanto el código como la definitiva de los estudiantes cuya definitiva está por debajo del promedio, necesitamos *dos* arreglos diferentes. Uno para almacenar el código y el otro almacenará la nota de *cada* estudiante.
- Los arreglos están de cierta forma *enlazados*, debido a que los datos para cada estudiante se almacenan *en la misma posición* dentro del arreglo. Es decir, en la posición 0 de cada arreglo se almacenan los datos para el primer estudiante, en la posición 1 los datos para el segundo, y así sucesivamente.
- En la segunda parte de la solución, se vuelve a usar las variables `codigo` y `nf`, esta vez para *recuperar* (sacar) los datos almacenados en el arreglo.

El problema ya está solucionado, pero vamos a hacer la solución más *elegante*⁷, usando la estrategia de *terminar temprano*. En este caso, vamos a modificar el algoritmo en dos partes:

- Cambiar la estructura *mientras* por una estructura *hacer mientras*. Debido a que *por lo general* en este tipo de estructuras es necesario *verificar* que el dato ingresado la primera vez es válido, podemos aprovechar esta situación para cancelar el ciclo en caso contrario.
- *Terminar temprano* el algoritmo, en caso que no se haya ingresado datos. Esto nos permite estar seguros en etapas posteriores que por lo menos se leyó un dato.

El algoritmo obtenido se presenta en el siguiente pseudocódigo.

```
1 algoritmo menorQueElPromedio
2 entero codigo, n
3 real n1, n2, n3, nf, total, promedio
4 entero codigos[]
5 real definitivas[]
6 entero i
7 inicio
8   total = 0
9   n = 0
10  i = 0
11
12  hacer
13    leer codigo
```

⁷Es una apreciación *subjetiva*, o dicho de otra forma, un capricho del autor

```

14     si codigo = 0
15         cancelar
16     fin si
17     // A este punto se llega solo si codigo no es cero
18     n1 = leerNota()
19     n2 = leerNota()
20     n3 = leerNota()
21     nf = (n1 * 0.3) + (n2 * 0.3) + (n3 * 0.4)
22
23     codigos[i] = codigo
24     definitivas[i] = nf
25
26     total = total + nf
27     n = n + 1
28     i = i + 1
29     mientras verdadero // Cuidado cuidado!
30
31     si n = 0
32         imprimir "No se ingresaron datos"
33         terminar // Termina la ejecución del algoritmo
34     fin si
35
36     //Si llegamos a este punto, hay datos para procesar
37     promedio = total / n
38
39     para i = 0, i < n, i = i + 1
40         codigo = codigos[i]
41         nf = definitivas[i]
42         si nf < promedio
43             imprimir codigo, nf
44         fin si
45     fin para
46 fin
47
48 //Repite la lectura de una nota, hasta que es válida
49 funcion leerNota():real
50 real n
51 inicio
52     //Leer la nota, asegurandose que sea valida
53     hacer
54         leer n
55     mientras n < 0 o n > 5
56     //Retornar la nota leida
57     retornar n
58 fin funcion

```

En esta solución hay una advertencia: La estructura hacer mientras tiene la siguiente estructura:

```

1 hacer
2     ... instrucciones
3 mientras verdadero

```

Pero la condición (verdadero) es siempre verdadera!. Si dentro de las instrucciones de

3. EJERCICIOS DE FUNDAMENTACIÓN

la estructura repetitiva no se *hace algo* para terminar el ciclo, hará que el algoritmo *no termine* nunca (lo cual, desde el punto de vista de usuario se verá como si el programa se encuentra *bloqueado*). Este tipo de estructuras es muy útil, pero ***debe ser usada con cuidado***.

Si se quiere una aproximación *menos arriesgada*, se puede usar la solución con la condición original, la cual se hecho será verdadera *hasta que* el código sea cero. El Código 3.39 presenta la solución considerando todos los elementos analizados.

Código 3.39: Estudiantes con nota final menor que el promedio

```
1 algoritmo menorQueElPromedio
2 entero codigo, n
3 real n1, n2, n3, nf, total, promedio
4 entero codigos[]
5 real definitivas[]
6 entero i
7 inicio
8
9     total = 0
10    n = 0
11    i = 0
12
13    hacer
14        leer codigo
15        si codigo = 0
16            cancelar // ROMPE la estructura repetitiva
17        fin si
18        // A este punto se llega solo si codigo no es cero
19        n1 = leerNota()
20        n2 = leerNota()
21        n3 = leerNota()
22        nf = (n1 * 0.3) + (n2 * 0.3) + (n3 * 0.4)
23
24        codigos[i] = codigo
25        definitivas[i] = nf
26
27        total = total + nf
28        n = n + 1
29        i = i + 1
30
31    mientras codigo != 0
32
33    si n = 0
34        imprimir "No se ingresaron datos"
35        terminar // TERMINA la ejecución del algoritmo
36    fin si
37
38    //Si llegamos a este punto, hay datos para procesar
39    promedio = total / n
40
41    para i = 0, i < n, i = i + 1
42        codigo = codigos[i]
43        nf = definitivas[i]
```

```

44     si nf < promedio
45     imprimir codigo, nf
46     fin si
47   fin para
48 fin
49
50 funcion leerNota():real
51 real n
52 inicio
53   //Leer la nota, asegurandose que sea valida
54   hacer
55     leer n
56   mientras n < 0 o n > 5
57   //Retornar la nota leida
58   retornar n
59 fin funcion

```

Estudiantes con alguna nota menor que el promedio

Podemos hacer el ejercicio anterior aún más interesante, para recordar el concepto de *Tipos compuestos de datos*.

Construir un algoritmo que dados el código y tres notas parciales con porcentajes de 30 %, 30 % y 40 % repectivamente de un número indeterminado de estudiantes, imprima el código, las notas parciales y la nota definitiva de los estudiantes cuya nota final o cualquiera de sus notas parciales sea menor que el promedio de las notas finales de todo el curso.

El algoritmo deberá continuar solicitando datos hasta que se especifique 0 como código de estudiante.

Análisis

¿Qué ha cambiado en el planteamiento del problema?. Si observamos detenidamente, nos damos cuenta que se debe imprimir también las notas parciales, y además la condición para decidir si se imprimen o no las notas de cada estudiante ha cambiado. Ahora también debemos verificar si alguna de las notas parciales también es menor que el promedio.

Para solucionar este problema podemos recurrir de nuevo a los arreglos, y en este caso necesitaremos cinco de ellos (uno para el código, uno para cada nota parcial y uno para la nota definitiva). Por supuesto que es una solución válida, pero nos ofrece la oportunidad de ilustrar un concepto vital en algoritmia y programación denominado *tipo compuesto de datos*.

3. EJERCICIOS DE FUNDAMENTACIÓN

Como en el ejemplo anterior, estos arreglos estarán de cierta forma *enlazados*, de forma que los datos de cada estudiante se almacenen en la misma posición dentro de los cinco arreglos. Dicho de otra forma, la posición *i* de cada arreglo almacenará un dato del estudiante *i*.

Diseño

En cambio, podemos *definir* un nuevo *tipo compuesto de datos*, el cual puede ser usado para *encapsular* la información de un estudiante en un único elemento. Una vez definido, dentro de nuestros algoritmos tendremos la posibilidad de crear nuevas variables con base en este tipo de datos, con la misma facilidad como definimos una variable de un *tipo base* (entero, por ejemplo).

```
1 tipo estudiante
2   entero codigo
3   real n1
4   real n2
5   real n3
6   real nf
7 fin tipo
```

Debido a que necesitamos almacenar los datos de un número indeterminado de estudiantes, podemos usar este nuevo tipo para crear un *arreglo de estudiantes*. En cada posición del arreglo se almacenarán los datos *leídos* de un estudiante, y también el valor *calculado* de la nota definitiva. De esta forma, evitamos la complicación de manejar cinco arreglos de forma simultánea.

codigo: 101 n1: 3.5 n2: 3.8 n3: 4.0 nf: 3.76
Datos de otro estudiante

El arreglo para almacenar los datos de los estudiantes (tipo compuesto) se definirá de la siguiente forma:

```
estudiante datos[]    Se define un arreglo llamado "datos", el cual en cada  
                      posición almacenará todos los datos para un estudiante
```


Algoritmo

Ahora será más sencillo solucionar el problema planteado, mediante el algoritmo implementado en el Código 3.40.

Código 3.40: Estudiantes con alguna nota menor que el promedio

```

1  tipo estudiante // Nombre del nuevo tipo compuesto
2      entero codigo
3      real n1
4      real n2
5      real n3
6      real nf
7  fin tipo
8
9  algoritmo menorQueElPromedio
10 entero codigo, n
11 real n1, n2, n3, nf, total, promedio
12 estudiante estudiantes[]
13 entero i
14 inicio
15
16     total = 0
17     n = 0
18     i = 0
19
20     hacer
21         leer codigo
22         si codigo = 0
23             cancelar
24         fin si
25         n1 = leerNota()
26         n2 = leerNota()
27         n3 = leerNota()
28
29         nf = (n1 * 0.3) + (n2 * 0.3) + (n3 * 0.4)
30
31         estudiantes[i].codigo = codigo
32         estudiantes[i].n1 = n1
33         estudiantes[i].n2 = n2
34         estudiantes[i].n3 = n3
35         estudiantes[i].nf = nf
36
37         total = total + nf
38         n = n + 1
39         i = i + 1
40
41     mientras codigo != 0 // Es verdadera n - 1 veces
42
43     si n = 0
44         imprimir "No se ingresaron datos"
45         terminar
46     fin si
47
48     //Si llegamos a este punto, hay datos para procesar

```

3. EJERCICIOS DE FUNDAMENTACIÓN

```
49 promedio = total / n
50
51 para i = 0, i < n, i = i + 1
52     codigo = estudiantes[i].codigo
53     n1 = estudiantes[i].n1
54     n2 = estudiantes[i].n2
55     n3 = estudiantes[i].n3
56     nf = estudiantes[i].nf
57     si nf < promedio
58         o n1 < promedio
59         o n2 < promedio
60         o n3 < promedio
61 imprimir codigo, n1, n2, n3, nf
62     fin si
63 fin para
64 fin
65
66 funcion leerNota():real
67     real n
68     inicio
69         //Leer la nota, asegurandose que sea valida
70         hacer
71             leer n
72             mientras n < 0 o n > 5
73             //Retornar la nota leida
74             retornar n
75     fin funcion
```

Distancia entre puntos

En siguiente problema nos ofrece de nuevo la posibilidad de usar tipos complejos y funciones.

Construir un algoritmo que dados las coordenadas (x, y) de dos puntos, calcule la distancia entre ellos.

Análisis

La ubicación de un punto en el espacio bidimensional se define por sus coordenadas (x,y), que representa la distancia horizontal y vertical con respecto a un *origen de coordenadas*. La distancia entre dos puntos (x₁, y₁) y x₂, y₂) se calcula a partir de la línea recta que une a los dos puntos, como lo muestra la Figura 3.2.

La fórmula matemática para calcular la distancia entre dos puntos viene dada por la expresión:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

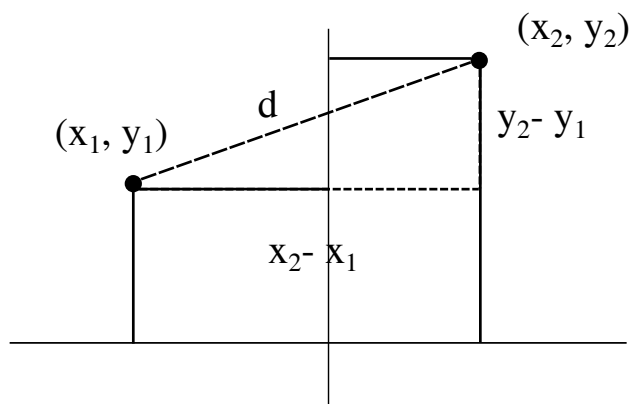


Figura 3.2: Distancia euclidiana entre dos puntos

Diseño

Para solucionar este problema podemos definir un *tipo compuesto* llamado punto, y una rutina que reciba como parámetro los dos puntos y calcule su distancia.

Primero vamos a definir el tipo compuesto de datos, y la subrutina que calcula la distancia entre dos puntos.

```
//Definición del nuevo tipo compuesto
tipo punto
  real x
  real y
fin tipo

/*
 * Calcula y retorna la distancia euclidiana entre dos puntos.
 */
funcion distancia(punto p, punto q): real
real dx, real dy, real d
inicio
  dx = q.x - p.x
  dy = q.y - p.y
  d = raiz_cuadrada( (dx * dx) + (dy * dy))
  retornar d
fin funcion
```

En esta solución se asume que el lenguaje de programación en el cual se codificará el algoritmo tiene alguna subrutina propia que realice la operación de raíz cuadrada.

Algoritmo

Ahora solo queda integrar estos elementos en la solución final del problema, que se muestra en el Código 3.41.

Código 3.41: Distancia euclidiana entre dos puntos

```
//Definición del tipo compuesto
tipo punto
  real x
  real y
fin tipo

/*
 * Calcula la distancia euclidiana entre dos puntos
 */
algoritmo distanciaEntrePuntos
  punto p, punto q
  real d
  inicio
    leer p.x, p.y
    leer q.x, q.y
    d = distancia(p, q)
    imprimir d
  fin

/*
 * Calcula y retorna la distancia euclidiana entre dos puntos.
 */
funcion distancia(punto p, punto q): real
  real dx, real dy, real d
  inicio
    dx = q.x - p.x
    dy = q.y - p.y
    d = raiz_cuadrada( (dx * dx) + (dy * dy))
    retornar d
  fin funcion
```

3.4. Ejercicios propuestos

1. Construir un algoritmo que dados tres números, encuentre el mayor y el menor de ellos.
2. Construir un algoritmo que dados n números, encuentre el mayor y el menor de ellos.
3. Construir un algoritmo que dados tres datos que representan una hora (hora, minuto, segundo) en formato de 24 horas, exprese la misma hora en formato de 12 horas (AM - PM).
4. Construir un algoritmo que dadas dos fechas (año, mes y día), calcule e imprima el número de horas, minutos y segundos transcurridos entre ellas. Tenga en cuenta que

las fechas no se ingresan de forma ordenada, es decir que la primera fecha ingresada puede ser menor, igual o mayor que la segunda.

5. Construir un algoritmo que dados tres puntos, encuentre el par de puntos cuya distancia euclidiana es menor.
6. Construir un algoritmo que dados n puntos, encuentre el más cercano y el más lejano del origen $(0, 0)$.
7. Construir un algoritmo que dados un número real x y un número entero k , calcule e imprima el término k de la siguiente serie: $\sum_{n=0}^{\infty} (-1)^k \frac{x^{(2k+1)}}{(2k+1)!}$
8. Construir un algoritmo que dados n puntos, determine para cada uno de ellos el cuadrante (x positivo / negativo, y positivo / negativo) en el cual se encuentran, e imprima el número total de puntos de cada cuadrante.
9. Construir un algoritmo que dada una letra (carácter), imprima el respectivo carácter en mayúscula. Suponga que los caracteres almacenan su respectivo código ASCII. Solo se deberá permitir que el usuario ingrese letras entre a y z .
10. Construir un algoritmo que dados n caracteres, calcule e imprima el total de letras mayúsculas ingresadas. Suponga que los caracteres almacenan sus respectivos códigos ASCII.
11. Construir un algoritmo que dados los nombres, apellidos, edad y género (femenino = f , masculino = m) de n personas, imprima para cada persona su clasificación por género y edad : niño/niña hasta 13 años, adolescente de 13 a 20 años, adulto/adulta hasta 60 años, anciano/anciana mayor de 60 años.

CAPÍTULO 4

Arreglos

En este capítulo se presentan algunos ejercicios en los cuales se explora el manejo de los arreglos, una de las estructuras de datos básicas que se usan con frecuencia como elementos para almacenar datos de entrada, datos del proceso o los datos que serán proporcionados como salida de los algoritmos. El uso básico de los arreglos comprende adicionar elementos en cada una de sus posiciones, buscar determinados elementos, modificar los elementos almacenados en determinadas posiciones o eliminar elementos.

4.1. Inicialización de arreglos

Una de las primeras tareas que se realizan con arreglos consiste en insertar datos en cada una de sus posiciones, lo cual se conoce como *inicialización* o *llenado*. Los datos que se almacenan en el arreglo pueden ser proporcionados por el usuario o generados por el mismo algoritmo. En los siguientes problemas se presenta el mecanismo básico para inicializar arreglos unidimensionales y bidimensionales.

Inicializar un arreglo unidimensional

Comenzamos con un ejemplo sencillo, que busca explorar el mecanismo básico para inicializar un arreglo unidimensional, conocido también como *vector*.

Construir un algoritmo que inicialice e imprima un arreglo de n elementos, que deberá almacenar en cada posición el índice correspondiente. El número n es proporcionado por el usuario.

Análisis

Lo que nos pide el problema es desarrollar un algoritmo que permita inicializar un arreglo de n elementos como se muestra en la Figura 4.1

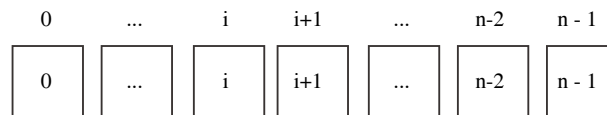


Figura 4.1: Arreglo con índices almacenados en su posición

Observe que debido a que las posiciones del arreglo se enumeran desde cero, en la primera posición se almacenará 0, en la segunda 1, y así sucesivamente. En la última se deberá almacenar $n - 1$. La secuencia de asignación de valores dentro del arreglo (que llamaremos a) será la que se muestra en el Código 4.1.

Código 4.1: Secuencia de asignaciones elemento en su posición

```
1 a[0] = 0 //Almacena 0 en la posición 0
2 a[1] = 1 //Almacena 1 en la posición 1
3 a[2] = 2 //...
4 ...
5 a[i] = i //Almacena i en la posición i
6 a[i + 1] = i + 1
7 ..
8 a[n - 1] = n - 1 //Almacena n - 1 en la posición n - 1
```

Diseño

Para solucionar el problema podemos usar una estructura repetitiva simple para generar los índices del arreglo de 0 a $n - 1$. En el arreglo se almacenará el mismo valor del índice, por lo cual la misma variable que se usa para el índice se puede usar como dato a almacenar en el arreglo.

```
para i = 0, i < n, i = i + 1
    a[i] = i
fin para
```

Algoritmo

El Código 4.2 muestra el proceso para generar un arreglo con los índices almacenados en su posición correspondiente.

Código 4.2: Cada índice en su posición de un arreglo

```
1 algoritmo indiceEnSuPosicion
2 entero a[] //Definición del arreglo
3 entero n, i
4 inicio
5   leer n
6   si n <= 0
7     terminar
8   fin si
9
10  //Guardar cada elemento en su posición
11  para i = 0, i < n, i = i + 1
12    a[i] = i
13  fin para
14
15  //Imprimir el arreglo generado
16  para i = 0, i < n, i = i + 1
17    imprimir a[i]
18  fin para
19 fin
```

En este caso pudimos usar el mismo ciclo que se encarga de guardar cada elemento en su posición para imprimirlo, pero se realiza en otro ciclo para ilustrar las diferentes etapas de solución del problema: inicializar el arreglo, guardar los elementos y finalmente imprimirlos.

Inicializar una matriz

A continuación se presenta un problema de inicialización de una matriz, el cual será de utilidad para comprender cómo se acceden las diferentes posiciones dentro de esta clase de arreglos.

Desarrollar un algoritmo que permita inicializar un arreglo de $n * n$ elementos, usando la disposición de los elementos que se muestra en el siguiente ejemplo, en el cual $n = 5$.

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25

Análisis

En este problema se busca inicializar una matriz de $n * n$ elementos, usando una secuencia ordenada de números, que inicia en 1. Debido a que la matriz tiene $n * n$ elementos, el último número será exactamente $n * n$.

Ahora bien, sabemos que en una matriz se necesitan dos índices para acceder a un elemento. El primer índice se usa para la fila, y el segundo para la columna.

Veamos cómo sería la secuencia de asignaciones necesaria para llenar todos los elementos de la matriz (que llamaremos *m*), si realizáramos una asignación cada vez.

```
m[0][0] = 1 //Fila 0, columna 0
m[0][1] = 2 //Fila 0, columna 1
...
m[0][4] = 5 //Fila 0, columna 4
m[1][0] = 6 //Fila 1, columna 0
m[1][1] = 7 //Fila 1, columna 1
m[1][4] = 10 //Fila 1, columna 4
```

Se puede apreciar que la matriz se llena *por filas*, comenzando por la primera fila, que tiene índice 0. La fila permanecerá constante mientras se llena la matriz en las posiciones $[0,0]$, $[0,1]$, $[0,2]$, hasta llegar a la posición $[0,n-1]$. Luego, se incrementa la fila y se *repite* el proceso. La matriz estará llena cuando se coloque el número $n * n$ en la posición $[n-1,n-1]$.

Diseño

Una primera solución puede partir directamente del análisis realizado. Podemos *generar* los índices de la matriz en el orden requerido, por medio de dos estructuras repetitivas anidadas. La estructura repetitiva externa se usará para la fila, y la estructura repetitiva interna se usará para la columna.

```
entero i, j, n
leer n
para i = 0, i < n, i = i + 1
  //(1)
  //i toma el valor de 0, 1, 2, ... n - 1
  para j = 0, j < n, j = j + 1
    //(2)
    //i permanece constante, j toma el valor de 0, 1, 2, ... n - 1
  fin para
fin para
```

Esto permite que mientras la variable de la estructura repetitiva externa permanezca *constante* para la fila en cada iteración, la variable de la estructura repetitiva interna tome los valores requeridos para cada columna. La secuencia de instrucciones definida en la estructura externa (1) se ejecutará *n* veces, y la secuencia de instrucciones dentro de la

estructura interna (2) se ejecutará n veces por cada iteración de la estructura externa, es decir, se ejecutará un total de $n * n$ veces.

Algoritmo

Para inicializar la matriz se puede usar un *contador* que inicia en 1, y que se va incrementando dentro de la estructura repetitiva anidada (Ver Código 4.3). De esta forma, se incrementará exactamente $n * n$ veces.

Código 4.3: Estructura repetitiva anidada para inicializar una matriz

```

1 algoritmo inicializarMatriz
2 entero m[][]
3 entero n, k, i, j
4 inicio
5   leer n
6   si n <= 0
7     terminar //Verificar dato ingresado
8   fin si
9   k = 1
10  para i = 0, i < n, i = i + 1
11    para j = 0, j < n, j = j + 1
12      m[i][j] = k
13      k = k + 1
14    fin para
15  fin para
16 fin

```

Existe una segunda aproximación que no es fácil visualizar a primera vista, en la cual en lugar de *generar* los índices, se *calculan*. Dado que de todas maneras se debe realizar $n * n$ asignaciones, se puede usar una estructura repetitiva que realice exactamente $n * n$ iteraciones. En cada iteración, se calcula el valor de los índices i, j en los cuales se debe insertar el siguiente valor, como se muestra en el Código 4.4.

Código 4.4: Estructura repetitiva simple para inicializar una matriz

```

1 k = 0
2 mientras k < n * n
3   //Calcular el valor de los índices i y j a partir del valor de k
4   //Almacenar el siguiente valor en la matriz
5   m[i][j] = k
6   k = k + 1 //Incrementar k
7 fin mientras

```

El cálculo de los índices i y j se puede realizar haciendo operaciones simples sobre la variable k :

```

i = k / n //División entera entre k y n, permite calcular la fila
j = k mod n //Residuo entre k y n, permite calcular la columna

```

Se deja al lector comprobar que para $0 < k < n*n$, se cumplen las expresiones presentadas, que permiten calcular la posición $[i,j]$ correspondiente. El Código 4.5 implementa la solución del problema.

Código 4.5: Inicializar una matriz con una sola estructura repetitiva

```
1 algoritmo inicializarMatriz2
2 entero m[][]
3 entero n, k
4 entero i, j
5 inicio
6   leer n
7   si n <= 0
8     terminar //Verificar dato ingresado
9   fin si
10  k = 0
11  mientras k < n * n
12    i = k / n //Calcular fila, division entera
13    j = k mod n //Calcular columna, residuo
14    m[i][j] = k
15    k = k + 1 //Los datos comienzan en 1, incrementar antes
16  fin mientras
17 fin
```

4.2. Búsqueda y selección de elementos

Con frecuencia es necesario buscar uno o varios elementos almacenados en un arreglo, por lo cual debemos conocer las estrategias para realizar estas búsquedas. De forma general, una búsqueda implica que se debe revisar cada posición dentro del arreglo, para determinar si el dato almacenado cumple con la condición deseada.

Construir un algoritmo que dado un arreglo de n números enteros (posiblemente repetidos), determine si un elemento x se encuentra almacenado en el arreglo.

Análisis

Al mencionar que se tiene como entrada un arreglo de n números enteros, se nos está diciendo implícitamente que los datos se han obtenido y almacenado de alguna manera que por el momento no nos interesa, lo importante es que ya contamos con el arreglo de datos.

Lo anterior nos permite alejarnos de los detalles de implementación de la lectura de los datos, y enfocarnos en desarrollar nuestro algoritmo de búsqueda. Por supuesto, al

codificar el algoritmo con el propósito de probarlo, se deberá desarrollar la lógica de programación necesaria para leer los datos y almacenarlos en el arreglo.

Diseño

El algoritmo será implementado como una *función*, que recibe como entrada el arreglo, la cantidad de datos que contiene, y el valor del dato a buscar. Como resultado, la función retornará verdadero si el dato se encuentra dentro del arreglo, o falso en caso contrario. La función se definirá de la siguiente forma:

```
funcion existe(entero x, entero a[], entero n): booleano
```

Dentro de la función haremos uso de una estructura repetitiva, que usaremos para comparar cada elemento almacenado en el arreglo con el dato buscado. En la primera coincidencia, retornaremos verdadero. Si llegamos al final del arreglo y no hemos encontrado coincidencia, podemos retornar falso, ya que el dato buscado no se encuentra en el arreglo.

```
para i = 0, i < n, i = i + 1
    si a[i] = x
        retornar verdadero
    fin si
fin para
retornar falso
```

Algoritmo

El Código 4.6 presenta la función implementada. Se deja como ejercicio construir el algoritmo principal, encargado de leer los n datos y el valor x a buscar, e invocar a la función de búsqueda implementada.

Código 4.6: Búsqueda de un elemento en un arreglo

```
1 /*
2  * Verifica si el elemento x es igual a alguno de los n elementos
3  * almacenados en el arreglo a
4  * Retorna verdadero si el dato es encontrado, falso en caso contrario
5  */
6 funcion existe(entero x, entero a[], entero n): booleano
7 entero i
8 inicio
9     para i = 0, i < n, i = i + 1
10         si a[i] = x
11             retornar verdadero
12         fin si
13     fin para
14     retornar falso
```

4. ARREGLOS

```
15 fin funcion
```

Pero, qué tal si también necesitamos saber *la posición* en la cual se encuentra el elemento?

Construir un algoritmo que dado un arreglo de n números enteros (posiblemente repetidos), encuentre la primera posición en la cual se encuentra un elemento x dado dentro del arreglo. De no encontrarse el elemento, se deberá retornar -1 .

Análisis

Usando el algoritmo del problema anterior, tenemos la solución inmediata. Solo necesitamos retornar el subíndice en cuanto encontremos el elemento, o retornar -1 si el elemento no está en el arreglo.

Diseño

La definición de la función cambia, ya que ahora debemos retornar el subíndice del elemento, si se encuentra, es decir, un número entero.

```
funcion primeraPosicion(entero x, entero a[], entero n): entero
```

Y por supuesto la implementación, ya que se debe retornar la posición del elemento encontrado o -1 si no se encuentra.

```
para i = 0, i < n, i = i + 1
    si a[i] = x
        retornar i
    fin si
fin para
retornar -1
```

Algoritmo

El Código 4.7 presenta la función completa que soluciona el problema planteado.

Código 4.7: Primera posición de un elemento en un arreglo

```
1 /*
2  * Retorna la posición de un elemento dentro de un arreglo
3  * o -1 si no se encuentra
4  */
5 funcion primeraPosicion(entero x, entero a[], entero n): entero
6 entero i
```

```
7 inicio
8   para i = 0, i < n, i = i + 1
9     si a[i] = x
10      retornar i
11   fin si
12 fin para
13 retornar -1
14 fin funcion
```

Selección de elementos del arreglo

En el siguiente problema se muestra el proceso para seleccionar dos datos almacenados en un arreglo: el menor y el mayor.

Construir un algoritmo que dado un arreglo de n elementos, encuentre e imprima el mayor y el menor de los elementos almacenados.

Análisis

En este problema se deben seleccionar el menor y el mayor dato que se encuentren almacenados en un arreglo. Para ello, tenemos que recorrer cada posición, y *comparar* el dato almacenado en la posición actual con *el mayor* u *el menor* dato encontrados hasta ahora.

Diseño

Pero, ¿cuál es el primer número encontrado, a partir del cual se comparan los demás para determinar el mayor y el menor? Si estamos buscando el mayor de los datos, podemos partir del mínimo número entero posible, que sería $-\infty$. De la misma forma, para hallar el menor de los números, deberíamos partir de ∞ .

```
mayor = INFINITO
menor = -INFINITO
para i = 1, i < n, i = i + 1
  si a[i] > mayor
    mayor = a[i]
  fin si
  si a[i] < menor
    menor = a[i]
  fin si
fin para
imprimir mayor, menor
```

Sin embargo, puede ser que estos números no tengan representación en el lenguaje de programación que se elija para codificar el algoritmo. Una aproximación más eficiente consistirá en tomar el primer elemento como *mayor* y como *menor* al mismo tiempo, y por medio de una estructura repetitiva compararlo con los demás elementos. Si encontramos un elemento mayor, tomamos este nuevo elemento como el mayor de todos, y compararemos este con los elementos que faltan. Por complemento, si el elemento es menor, lo tomaremos como el menor y lo compararemos con los elementos faltantes.

Algoritmo

En el Código 4.8 se implementa el algoritmo como una función, la cual encuentra e imprime el menor y el mayor dato. Observe que si en el problema se necesitara usar los datos mayor y menor para otros cálculos, se debería usar otra aproximación, ya que la subrutina implementada no retorna estos valores.

Código 4.8: Imprimir el mayor y el menor dato en un arreglo

```
1 //Imprime el mayor y el menor elemento de un arreglo
2 //Esta subrutina no retorna valor
3 funcion imprimirMayorYMenor(entero a[], entero n): vacío
4 entero mayor
5 entero menor
6 entero i
7 inicio
8   si n = 0
9       imprimir "No hay datos"
10      retornar
11  fin si
12  //Tomar el primer elemento como el mayor y el menor iniciales
13  mayor = a[0]
14  menor = a[0]
15  //Comparar con los demas elementos
16  para i = 1, i < n, i = i + 1
17      //Comparar el elemento actual con el mayor y el menor
18      si a[i] > mayor
19          mayor = a[i] //El nuevo mayor es el elemento actual
20      sino
21          si a[i] < menor
22              menor = a[i] //El nuevo menor es el elemento actual
23      fin si
24  fin para
25  //Imprimir el mayor y el menor
26  imprimir mayor, menor
27 fin funcion
```


Promedio de los datos de un arreglo

El siguiente problema nos permite transformar un arreglo en un dato.

Construir un algoritmo que dado un arreglo de n números, calcule su promedio.

Análisis

Para solucionar este problema podemos usar el mismo algoritmo para calcular el promedio de n números, pero en esta oportunidad los datos de entrada se encuentran almacenados en un arreglo de n elementos. El algoritmo se implementará como una función que puede ser usado en otros algoritmos más complejos:

Diseño

```
funcion promedio(entero a[], entero n): real
```

La subrutina recibirá como parámetros de entrada el arreglo y el número de elementos que contiene, realizará el cálculo del promedio y retornará este valor como un número real.

El cálculo del promedio se realiza usando una estructura repetitiva simple, en la cual se toma cada elemento almacenado en el arreglo y se suma al total. Luego se divide el total calculado entre el número total de datos.

```
suma = 0
para i = 0, i < n, i = i + 1
    suma = suma + a[i]
fin para
promedio = suma / n
```

Algoritmo

El Código 4.9 implementa la solución del problema planteado. Sólo falta por validar el caso en el cual el arreglo está vacío ($n = 0$), en cuyo caso la función retornará 0.

Código 4.9: Promedio de los datos de un arreglo

```
1 //Calcula y retorna el promedio de n datos almacenados en un arreglo
2 funcion promedio(entero a[], entero n): real
3 real suma, real resultado
4 entero i
5 inicio
6 //Valida caso especial
```

4. ARREGLOS

```
7  si n = 0
8      retornar 0
9  fin si
10 suma = 0
11 para i = 0, i < n, i = i + 1
12     suma = suma + i
13 fin para
14 resultado = suma / n
15 retornar resultado
16 fin
```

Par de puntos más cercanos

Ahora vamos a construir un algoritmo que dado un arreglo de puntos, encuentre el par de puntos más cercanos.

Construir un algoritmo que dado un arreglo de n puntos, encuentre e imprima el par de puntos más cercanos entre sí y la distancia que existe entre ellos. Para cada punto se proporcionan las coordenadas (x, y) , y la distancia entre dos puntos se calcula usando la fórmula de Euclides.

Análisis

Para implementar este algoritmo se puede hacer uso del tipo compuesto punto presentado en ejercicios anteriores. Este problema difiere un poco al de encontrar el mayor dato en un arreglo, debido a que no se debe comparar el primer dato con los demás sino *todos los datos* (puntos) entre sí, específicamente la distancia entre todos los posibles pares de puntos. Para este propósito podemos tomar la siguiente aproximación:

```
calcular la distancia entre el punto 0 y el punto 1
calcular la distancia entre el punto 0 y el punto 2
...
calcular la distancia entre el punto 0 y el punto n - 1
calcular la distancia entre el punto 1 y los demás hasta n - 1
...
calcular la distancia entre el punto 2 y los demás hasta n - 1
...
calcular la distancia entre el punto n - 2 y el punto n - 1
```

La menor distancia será el menor dato encontrado.

Diseño

El algoritmo deberá recibir un arreglo de elementos de este tipo de datos y el número de elementos que contiene, y deberá encontrar el par de puntos cuya distancia euclidiana sea menor. Comenzamos definiendo el tipo de datos `punto` y una función `distancia`, que permite calcular la distancia entre un par de puntos.

```
tipo punto
  real x
  real y
fin tipo

funcion distancia(punto p, punto q): real
```

Ahora debemos realizar un proceso para encontrar el par de puntos con menor distancia. Podemos seguir la idea expresada en el análisis, aplicando los siguientes pasos:

1. Tomar la distancia que existe entre el primer punto y todos los demás (el segundo, el tercero, etc. hasta llegar al último), y llevar la cuenta de la menor de ellas. Se realizarán exactamente $n - 1$ comparaciones. A medida que encontramos un par de puntos con menor distancia, almacenamos sus coordenadas en variables auxiliares.
2. Repetimos este proceso, a partir del segundo hasta el penúltimo punto, comparando con los siguientes y actualizando las variables auxiliares si encontramos un par de puntos con distancia menor a la actual. Observe que no es necesario calcular la distancia entre un punto y los anteriores, ya que esta comparación se realizó en una iteración anterior. En la primera repetición se realizarán $n - 2$ comparaciones, en la siguiente $n - 3$ comparaciones, y en la última se realizará una comparación, el penúltimo elemento con el último.

Aplicando este algoritmo, se tendrá el par de puntos con menor distancia. El algoritmo se puede apreciar mejor en la Figura 4.2, en la cual i , j representan los valores que deben tomar los subíndices en cada comparación. Observe que se necesitan dos estructuras repetitivas, una para el índice i , y otra anidada, para el índice j .

El siguiente fragmento de código presenta una posible implementación de esta idea. Ahora bien, cada vez que encontramos la distancia entre un par de puntos, se debe comparar esta distancia con *la menor distancia* obtenida hasta el momento. Sin embargo, ¿inicialmente cuál es la menor distancia?. Podemos *asumir* que la menor distancia es la que existe entre los dos primeros puntos, y a medida que transcurre el algoritmo se actualiza esta distancia (y los puntos correspondientes) si se encuentra un par de puntos más cercanos entre sí.

4. ARREGLOS

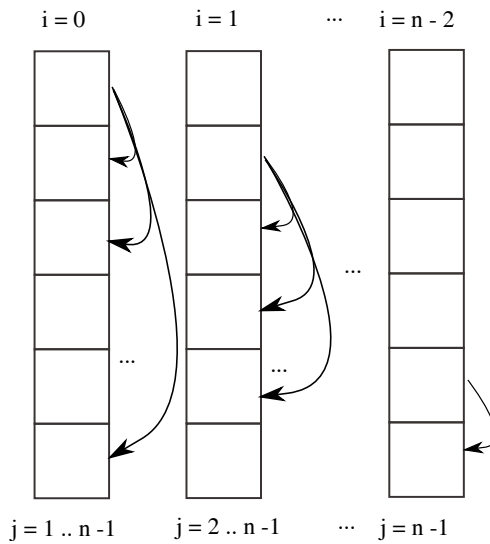


Figura 4.2: Comparación de todos los elemento de un arreglo

```
p.x = datos[0].x
p.y = datos[0].y
q.x = datos[1].x
q.y = datos[1].y
md = distancia(p, q) //Menor distancia inicial

para i = 0, i < n - 1, i = i + 1
  //i toma valores 0, 1, ... n - 2
  para j = i + 1, j < n, j = j + 1
    //j toma valores i + 1, i + 2, ... n - 1
    //comparar la distancia entre a[i] y a[j] con la menor hasta ahora
    d = distancia(a[i], a[j])
    si d < md
      //a[i] y a[j] tienen menor distancia! Guardar a[i] en p y a[j] en q
      md = d
      p.x = a[i].x
      p.y = a[i].y
      q.x = a[j].x
      q.y = a[j].y
  fin para
fin para
```

Este algoritmo básico de comparación nos será de utilidad en ejercicios posteriores, en los cuales también se requiere comparar todos los datos que se encuentran dentro de un arreglo.

Algoritmo

Vamos a construir el algoritmo que soluciona el problema, teniendo en cuenta una última consideración. Debido a que el algoritmo debe encontrar el par de puntos con menor distancia entre sí, es *necesario* que el arreglo tenga por lo menos dos elementos. En caso contrario, no tiene sentido continuar.

El Código 4.10 implementa la solución del problema, para lo cual se ha definido un nuevo tipo de datos punto y dos funciones auxiliares. Se deja como ejercicio construir el algoritmo principal que obtiene los datos del usuario e invoca la rutina para imprimir el par de puntos más cercanos.

Código 4.10: Imprimir el par de puntos con menor distancia entre sí

```

1 tipo punto
2   real x
3   real y
4 fin tipo
5
6 //Imprime el par de puntos mas cercanos
7 //Esta función no retorna un valor
8 funcion imprimirPuntosMasCercanos(punto datos[], entero n): vacío
9   real d //Para calcular la distancia
10  real md //Menor distancia
11  punto p, punto q //Par de puntos mas cercanos
12  entero i, entero j
13  inicio
14    //Validar si existen suficientes puntos
15    si n < 2
16      imprimir "Datos insuficientes"
17      retornar
18    fin si
19
20    //Asumir que los dos primeros puntos son los más cercanos
21    p.x = datos[0].x
22    p.y = datos[0].y
23    q.x = datos[1].x
24    q.y = datos[1].y
25    md = distancia(p, q) //Invocar la función distancia
26
27    //Iterar desde 0 hasta n - 2
28    para i = 0, i < n - 1, i = i + 1
29      //Calcular distancia con los siguientes puntos: i + 1 hasta n - 1
30      //Iterar desde i + 1 hasta n - 1
31      para j = i + 1, j < n, j = j + 1
32        d = distancia(datos[i], datos[j]) //Invocar la función distancia
33        si d < md //Encontramos un par de puntos más cercanos
34          //Actualizar la menor distancia y los puntos más cercanos
35          md = d
36          p.x = datos[i].x
37          p.y = datos[i].y
38          q.x = datos[j].x
39          q.y = datos[j].y

```

4. ARREGLOS

```
40     fin si
41     fin para
42     fin para
43     imprimir p.x, p.y, q.x, q.y, md
44 fin funcion
45
46 //Retorna la distancia entre dos puntos
47 funcion distancia(punto p, punto q): real
48     real dx, real dy
49     inicio
50         dx = q.x - p.x
51         dy = q.y - p.y
52         retornar raiz_cuadrada((dx * dx) + (dy * dy))
53 fin funcion
```

De nuevo, en este algoritmo se supone que existe una función `raiz_cuadrada` que permite obtener la raíz cuadrada de un número. En lenguaje C, por ejemplo, se usa la función `sqrt`.

4.3. Inserción de elementos

Ahora vamos a revisar algunos problemas relacionados con la adición de elementos dentro de un arreglo.

Inserción de elementos al final de un arreglo

Construir una función que permita insertar un nuevo elemento al final de un arreglo de números enteros. La función deberá retornar la cantidad de elementos totales almacenados. Suponga que el arreglo puede crecer automáticamente.

Análisis

Si los índices de los arreglos se enumeran desde cero, antes de la inserción se tendrán ocupadas las posiciones 0 hasta $n - 1$, por lo cual el nuevo elemento se almacenará siempre en la posición n . Como no existe restricción sobre el tamaño del arreglo, podemos asumir que es *seguro* almacenar el nuevo elemento en esta posición (Ver Figura 4.3).

Diseño

Debemos implementar el algoritmo como una función que almacene el elemento y retorne la nueva cantidad de datos almacenados. Si como entrada se recibió n , esta función deberá retornar $n + 1$, ya que insertamos un nuevo elemento.

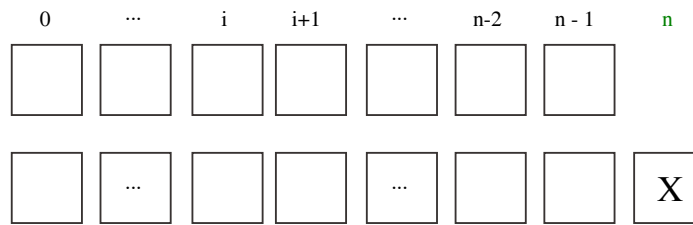


Figura 4.3: Insertar un elemento al final de un arreglo

```
funcion insertarAlFinal(entero a[], entero n, entero x): entero
```

Algoritmo

La inserción en este caso es sencilla, ya que el elemento siempre se almacenará en la posición n , y no tendremos que preocuparnos si existe o no espacio en el arreglo (Ver Código 4.11).

Código 4.11: Insertar un elemento al final del arreglo

```
funcion insertarAlFinal(x, entero a[], entero n): entero
inicio
  a[n] = x
  retornar n + 1
fin funcion
```

Este algoritmo puede ser usado para generar un arreglo dinámicamente, por ejemplo para insertar datos proporcionados por el usuario:

Construir un algoritmo que permita generar dinámicamente un arreglo de n datos, los cuales deberán ser proporcionados por el usuario.

El Código 4.12 implementa un algoritmo que genera dinámicamente un arreglo a partir de los n datos proporcionados por el usuario.

Código 4.12: Generar un arreglo de números proporcionados por el usuario

```
1 algoritmo generarArreglo
2 entero n, entero k, entero x
3 entero a[]
4 inicio
5   //Asegurarse que el valor de n es mayor que cero
6   hacer
7     leer n
8     mientras n <= 0
9
10  k = 0
```

4. ARREGLOS

```
11  mientras k < n
12      leer x
13      k = insertarAlFinal(x, a, k)
14  fin mientras
15 fin
16 /*
17  * Inserta un elemento al final de un arreglo
18  * Se asume que el arreglo puede crecer dinámicamente
19  */
20 funcion insertarAlFinal(x, entero a[], entero n): entero
21 inicio
22     a[n] = x
23     retornar n + 1
24 fin funcion
```

Al final de la ejecución, la variable *k* guardará el número de datos almacenados en el arreglo.

Se puede modificar la solución para que la variable *k* sea modificada por la función, de forma que no se tenga que asignar en cada llamada a la función `insertarAlFinal`. Para lograrlo, se puede pasar *la referencia* de la variable, lo cual permitirá modificarla dentro de la función.

```
1  algoritmo generarArreglo
2  entero n, entero k, entero x
3  entero a[]
4  inicio
5      //Asegurarse que el valor de n es mayor que cero
6      hacer
7          leer n
8          mientras n <= 0
9
10     k = 0
11     mientras k < n
12         leer x
13         insertarAlFinal(x, a, ref k)
14     fin mientras
15 fin
16 /*
17  * Inserta un elemento al final de un arreglo
18  * Se asume que el arreglo puede crecer dinámicamente
19  * Esta rutina no retorna un valor
20  */
21 funcion insertarAlFinal(x, entero a[], ref entero n): vacío
22 inicio
23     a[n] = x
24     *n = *n + 1
25 fin funcion
```


Insertar al final de un arreglo de tamaño limitado

Ahora vamos a solucionar este problema cuando los arreglos tienen un tamaño limitado o no puede crecer dinámicamente.

Construir una función que permita insertar un nuevo elemento al final de un arreglo de números enteros. La función deberá retornar la cantidad de elementos totales almacenados. Suponga que el arreglo sólo puede crecer hasta un límite m dado. En caso de tratar de insertar un elemento más allá de la capacidad del arreglo, se deberá retornar -1 .

Análisis

Podemos partir de la función que permite insertar el elemento a un arreglo, validando antes de insertar si aún queda espacio en el arreglo.

Diseño

En este caso se deben usar dos variables para realizar la inserción: una variable indica la cantidad actual de elementos en el arreglo (n), u la otra indica el límite de capacidad del arreglo (m). Si n es menor que $m - 1$, significa que hay espacio en el arreglo. En caso contrario, el arreglo está lleno y no se pueden insertar más datos.

La función para insertar al final se define de la siguiente forma:

```
funcion insertarAlFinal(entero x, entero a[], entero n, entero m): entero
```

El valor retornado por la función será el número de elementos que se encuentran en el arreglo ($n + 1$), y -1 si no se pudo insertar el nuevo elemento debido a que el arreglo se encuentra lleno.

Algoritmo

La función resultante se muestra en el Código 4.13.

Código 4.13: Insertar al final de un arreglo de tamaño limitado

```
/*
 * Inserta un elemento al final de un arreglo de tamaño limitado.
 * Retorna el número de elementos del arreglo, o -1 si no se pudo
 * insertar el elemento.
 */
funcion insertarAlFinal(entero x, entero a[], entero n, entero m): entero
inicio
```

4. ARREGLOS

```
//Validación inicial de n
si n < 0 o n >= m - 1
    retornar -1
fin si

a[n] = x
retornar n + 1
fin funcion
```

Insertar un elemento en una posición dada

Vamos a estudiar un problema más complejo de inserción de un elemento dentro de un arreglo.

Construir una función que permita insertar un nuevo dato dentro de un arreglo de números enteros que contiene $n \geq 0$ elementos, en una posición $0 \leq i \leq n$. Suponga que el arreglo puede crecer dinámicamente.

Análisis

En este caso, se requiere construir una función que permita insertar un elemento en una posición arbitraria dentro de un arreglo. Debido a que existen datos en el arreglo, se deberán *desplazar* los datos hacia adelante a partir de la posición especificada, para dar espacio al nuevo elemento que se desea insertar. Luego de insertar el nuevo elemento, el arreglo contará con $n + 1$ datos. De nuevo se supone que el arreglo puede almacenar este nuevo dato.

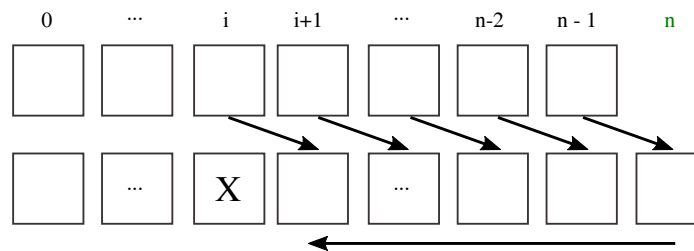


Figura 4.4: Insertar en una posición dada

Diseño

Primero definimos la función para insertar el nuevo dato en el arreglo, de la siguiente forma:

```
funcion insertarEnPosición(entero x, entero a[], entero n, entero i)
```

En la cual x almacena el elemento a insertar, a es el arreglo, n es el número actual de elementos del arreglo y la variable i es la posición en la cual deseamos insertar el nuevo elemento.

Necesitamos una estructura repetitiva que desplace los elementos hacia adelante, y abra espacio al nuevo elemento en la posición especificada. El único caso en el cual no se debe realizar un desplazamiento, es cuando la nueva posición es igual a n . Tenga en cuenta que en el planteamiento se define que el valor de i se encuentra entre 0 y n inclusive, por lo cual no es necesario verificar si la posición especificada es correcta.

Ahora bien, ¿cómo desplazamos los elementos *hacia adelante* en un arreglo? Tenemos dos opciones:

Comenzar moviendo el elemento i a la posición $i + 1$:

```
a[i+1] = a[i]
```

Pero si realizamos esta asignación ¡se pierde el siguiente elemento! Antes de copiar el dato en la siguiente posición, deberíamos *salvar* los datos siguientes por ejemplo en otro arreglo, y luego almacenarlos de nuevo en el arreglo original una posición más adelante.

```
k = 0
para j = i , j < n, j = j + 1
    temp[k] = a[j]
    k = k + 1
fin para

k = 0
para j = i + 1, j <= n, j = j + 1
    a[j] = temp[k]
    k = k + 1
fin para
```

La segunda opción es más práctica y no requiere otro arreglo. Podemos *mover* los elementos hacia adelante, es decir, desplazarlos una posición comenzando desde el último. Sabemos que en un arreglo que contiene n elementos, por lo tanto la última posición que contiene un dato es $n - 1$. Entonces, debemos *mover* el elemento de la posición $n - 1$ a la posición n , el elemento de la posición $n - 2$ a la posición $n - 1$ y así sucesivamente, hasta llegar a la posición $i + 1$. En este punto debemos dejar de desplazar los elementos.

```
j = n
mientras j > i
    a[j] = a[j - 1]
    j = j - 1
fin mientras
```

Algoritmo

En Código 4.14 presenta una implementación del algoritmo para insertar un elemento en una posición dada de un arreglo, en la cual se mueven los elementos desde la posición hacia abajo para abrir espacio al nuevo elemento.

Código 4.14: Insertar un elemento en una posición de un arreglo

```
1 /*
2  * Inserta el elemento x en el arreglo a, en la posición i especificada.
3  * El arreglo contiene n elementos, y contendrá n + 1
4  * elementos luego de la inserción.
5  */
6 funcion insertarEnPosicion(entero x, entero a[], entero n, entero i)
7 entero j, entero k, entero temp
8 inicio
9     //Validar inserción al final
10    si i = n
11        a[i] = x
12        retornar n + 1
13    fin si
14
15    //Desplazar los elementos existentes hacia abajo,
16    //comenzando por el último
17    j = n
18    mientras j > i
19        a[j] = a[j - 1]
20        j = j - 1
21    fin mientras
22
23    //En este punto j = i, la posición del nuevo elemento
24    a[j] = x
25    //Retornar el nuevo número de elementos
26    retornar n + 1
27 fin funcion
```

4.4. Eliminación de elementos

Al eliminar un elemento se *comprime* el arreglo, es decir, se reduce el número de elementos almacenados en 1. El siguiente problema presenta esta situación.

Eliminar un elemento de un arreglo

Construir una función que permita eliminar la primera ocurrencia de un elemento x dentro de un arreglo de $n \geq 0$ elementos. La función deberá retornar la posición en la

cual se eliminó el elemento, o -1 si el elemento especificado no se encontraba dentro del arreglo.

Análisis

Para solucionar este problema, primero debemos encontrar la primera ocurrencia del elemento dentro del arreglo. Una vez hallado, procedemos a *mover* los elementos hacia atrás a partir de la posición en la cual se encontró el elemento, y *borraremos* el último elemento, cambiándolo por un 0, y decrementando el total de elementos almacenados. Si el elemento no se encuentra, la función debe retornar -1, para indicar la condición de error. Observe que la posición del arreglo *no se destruye*, sólo se almacena un valor que no vamos a usar (en este caso 0). Dado que el acceso al arreglo está controlado por una variable que almacena el número de datos que contiene, al decrementarla no *se deberá* acceder a la posición que acabamos de *borrar*.

El proceso de eliminar el elemento x del arreglo que se encuentra en la posición i se ilustra en la Figura 4.5.

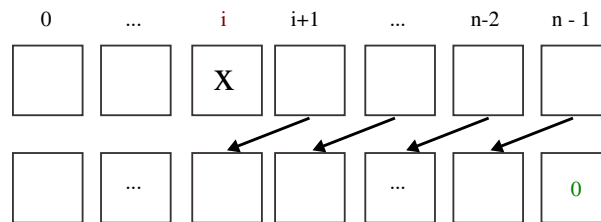


Figura 4.5: Eliminar un elemento de un arreglo

Diseño

Suponiendo que el elemento a eliminar se encuentra en la posición (i), se deberá almacenar en esta posición el dato almacenado en la siguiente posición ($i + 1$) y así sucesivamente, hasta llegar a la última posición ($n - 1$).

```
//Desplazar los elementos hacia atrás
j = i
mientras j < n - 1
    a[j] = a[j + 1]
    j = j + 1
fin para
```

Algoritmo

El Código 4.15 presenta el algoritmo completo. En este caso se adicionó una validación inicial para tratar el caso cuando el arreglo no tiene elementos, es decir, cuando $n = 0$.

Código 4.15: Eliminar un elemento de un arreglo

```
1  /*
2  * Elimina la primera ocurrencia del elemento x dentro del
3  * arreglo a. Retorna la posición del elemento eliminado, o
4  * -1 si el elemento no se encuentra en el arreglo.
5  */
6  funcion eliminarElemento(entero x, entero a[], entero n): entero
7  entero i, entero j
8  inicio
9
10     //Validar arreglo vacío
11     si n = 0
12         retornar -1
13     fin si
14
15     //Buscar el elemento en el arreglo
16     i = 0
17     mientras i < n
18         si a[i] = x
19             //Terminar el ciclo en la primera coincidencia
20             cancelar
21         fin si
22         i = i + 1
23     fin mientras
24
25     //Verificar si el elemento no existe
26     si i = n
27         retornar -1
28     fin si
29
30     //Desplazar los elementos hacia atrás
31     j = i
32     mientras j < n - 1
33         a[j] = a[j + 1]
34         j = j + 1
35     fin para
36
37     //Eliminar el dato de la última posición
38     a[j] = 0
39     retornar i
40 fin funcion
```

4.5. Ejercicios propuestos

1. Desarrollar algoritmos para generar las siguientes matrices de un tamaño proporcionado por el usuario. Para los ejemplos presentados, $n = 5$. En la algunos casos se *debe calcular* los índices para poder insertar el dato correspondiente, y en otros se debe usar estructuras de decisión para determinar *el siguiente trayecto* a llenar: La primera matriz se llena por filas.

1	1	1	1	1
2	2	2	2	2
3	3	3	3	3
4	4	4	4	4
5	5	5	5	5

Columnas iguales. El valor se mantiene constante en la columna.

1	2	3	4	5
1	2	3	4	5
1	2	3	4	5
1	2	3	4	5
1	2	3	4	5

Llenado por columnas. Se desciende por la columna, incrementando el valor.

1	6	11	16	21
2	7	12	17	22
3	8	13	18	23
4	9	14	19	24
5	10	15	20	25

Llenado de serpiente. Se llena por columnas comenzando hacia abajo, alternando la dirección de llenado.

1	10	11	20	21
2	9	12	19	22
3	8	13	18	23
4	7	14	17	24
5	6	15	16	25

4. ARREGLOS

Llenado en diagonal 1. Se debe considera la relación entre fila y columna.

1	2	3	4	5
2	3	4	5	6
3	4	5	6	7
4	5	6	7	8
5	6	7	8	9

Llenado en diagonal 2. Similar a una *onda expansiva*, en cada paso se incrementa 1.

1	2	3	4	5
2	1	2	3	4
3	2	1	2	3
4	3	2	1	2
5	4	3	2	1

Llenado en espiral, $n \geq 5$ e impar. Se llena en cuatro direcciones: izquierda - derecha, arriba - abajo, derecha - izquierda, abajo - arriba. Los límites se van reduciendo dinámicamente.

1	2	3	4	5
16	17	18	19	6
15	24	25	20	7
14	23	22	21	8
13	12	11	10	9

2. Construir un algoritmo que imprima la transpuesta de una matriz dada de n filas por m columnas Sedgewick and Wayne (2011).
3. Desarrollar un algoritmo que permita encontrar la última posición en la cual se encuentra un elemento dentro de un arreglo. En caso de no encontrarse, se deberá retornar -1.
4. Desarrollar un algoritmo que permita determinar el número de veces que un elemento x se encuentra dentro de un arreglo.
5. Desarrollar un algoritmo que permita determinar si los datos de un arreglo se encuentran ordenados en forma ascendente.

6. Desarrollar un algoritmo que permita determinar si los datos de un arreglo se encuentran ordenados en forma descendente.
7. Construir un algoritmo que imprima el total de elementos de un arreglo que se encuentran almacenados en su posición, es decir, que el valor del dato almacenado sea igual a su índice dentro del arreglo.
8. Desarrollar una función que permita insertar un nuevo dato en un arreglo que contiene $n \geq 0$ números enteros, de forma tal que los datos del arreglo queden ordenados de menor a mayor, incluyendo el nuevo elemento insertado. Suponga también que el arreglo crece dinámicamente.
9. Construir una función que permita cambiar la primera ocurrencia de un elemento dado por otro en un arreglo de $n \geq 0$ elementos. El algoritmo deberá retornar la posición en la cual se realizó el reemplazo, o -1 si el elemento no se encontraba en el arreglo.
10. Construir una función que permita cambiar todas las ocurrencias de un elemento por otro en un arreglo de $n \geq 0$ elementos. La función deberá retornar el número de reemplazos realizados, o -1 si el elemento no existía en el arreglo.
11. Construir una función que permita eliminar todas las ocurrencias de un elemento x dentro de un arreglo de $n \geq 0$ elementos. La función deberá retornar el nuevo número de elementos que quedan en el arreglo.
12. Construir una función que permita eliminar el elemento i de un arreglo de $n \geq 0$ elementos. La función no retornará un valor, y deberá borrar el elemento sólo si la posición i proporcionada es válida.
13. Desarrollar una función que dado un arreglo de n elementos, construya otro arreglo con los elementos en orden inverso.
14. Construir una función que dado un arreglo de n elementos, *modifique* el arreglo pasado como parámetro y organice sus elementos en orden inverso.
15. Construir un algoritmo que dado un número n , inicialice un arreglo de n elementos en el cual en la posición $0 \leq i < n$ se almacene la i -ésima potencia de 2.
16. Desarrollar una función que dado un arreglo de n elementos y un dato x , retorne la primera posición en la cual se encuentra el dato en el arreglo, ó -1 si no se encuentra. La búsqueda deberá realizarse *simultáneamente* desde el inicio y el final del arreglo.

4. ARREGLOS

17. Desarrollar una función que dado un arreglo de n elementos y un dato x , retorne la primera posición encontrada del dato en el arreglo, ó -1 si no se encuentra. La búsqueda deberá realizarse *simultáneamente* desde la mitad del arreglo.
18. Desarrollar una función que dado un arreglo de n elementos y un dato x , retorne la primera posición encontrada del dato en el arreglo, ó -1 si no se encuentra. La búsqueda deberá realizarse *simultáneamente* hacia adelante y hacia atrás desde una posición $0 \leq k < n$.
19. Desarrollar una función que dada una matriz de $n * m$ números enteros, calcule e imprima la sumatoria por filas y columnas de los datos.
20. Desarrollar una función que dada una matriz de $n * m$ números enteros, encuentre el mayor elemento almacenado en toda la matriz.
21. Desarrollar un algoritmo que dada una matriz de $n * n$ números enteros, encuentre e imprima el menor dato de cada fila.
22. Desarrollar una función que dada una matriz de $n * m$ y un número i , retorne un arreglo que contiene los elementos de la fila i de la matriz.
23. Desarrollar una función que dada una matriz de $n * m$ y un número j , retorne un arreglo que contiene los elementos de la columna j de la matriz.
24. Construya una función que dada una matriz de $n * m$ números enteros, un número $0 < p \leq n$ y un número $0 < q \leq m$, retorne la *sub matriz* de tamaño $p * q$ de la matriz original.
25. Construir una función que dados dos arreglos de n y m elementos, retorne un arreglo que contiene solo los elementos comunes.
26. Construir una función que dados dos arreglos de n y m elementos, retorne un arreglo que contiene los elementos del primer arreglo que no se encuentran en el segundo.
27. Construir una función que dados dos arreglos de n y m elementos, retorne un arreglo que contiene los elementos repetidos de cada uno de los arreglos. A su vez, el arreglo resultante no deberá contener elementos repetidos.

Cadenas de caracteres

En este capítulo se presentan los problemas más comunes relacionados con el uso básico de cadenas de caracteres. Es necesario tener en cuenta las siguientes consideraciones en los algoritmos presentados:

- Se hará uso extensivo de subrutinas para implementar la lógica de programación que soluciona los problemas, con el propósito de fomentar la creación de algoritmos estructurados.
- Se asumirá que las cadenas pueden crecer dinámicamente, es decir, que no es necesario *reservar* memoria cuando se requiera insertar nuevos datos al final de las mismas.
- Las posiciones de los caracteres dentro de la cadena se manejarán de forma similar a los índices de los arreglos, comenzando en 0.
- Al final de la cadena deberá estar siempre el caracter nulo.
- Las cadenas serán enviadas a las rutinas *por referencia*, sin necesidad de marcar los parámetros como referencias. Esto significa que las funciones pueden modificar el contenido de la cadena, y estos cambios se reflejarán en la cadena original.

5.1. Inicialización de cadenas

El primer paso para usar las cadenas de caracteres consiste en obtener una secuencia de caracteres del usuario, o generarla dentro del mismo algoritmo. La mayoría de lenguajes

de programación ofrecen subrutinas para que el usuario pueda ingresar una secuencia de caracteres hasta que presione la tecla ENTER. El Código 5.1 por ejemplo, permite *saludar* al usuario.

Código 5.1: Saludar al usuario

```
1 algoritmo saludar
2 cadena n
3 inicio
4     imprimir "Por favor escribe tu nombre y presiona ENTER"
5     leer n
6     imprimir "Hola, ", n
7 fin
```

El usuario deberá ingresar una secuencia de caracteres por teclado, y para finalizar deberá pulsar la tecla ENTER. Por lo general, se asume que el usuario *sabe* que debe presionar ENTER para terminar la lectura de datos, pero es conveniente ofrecer las indicaciones completas.

En este algoritmo, la cadena de caracteres quedará almacenada en la variable *n*, y podrá ser usada para realizar algún proceso, o como en este caso, simplemente para enviarla a la pantalla, junto con la cadena de saludo.

Crear cadena a partir de una secuencia de caracteres

En ocasiones es conveniente leer los caracteres individuales y *armar* una cadena de caracteres, en vez de usar las subrutinas proporcionadas por el lenguaje de programación (y la instrucción *leer* en pseudocódigo), debido a que al leer carácter por carácter tenemos la oportunidad de *filtrar* o *descartar* los caracteres que no se consideren convenientes para el algoritmo.

Construir un algoritmo que permita crear e imprimir una cadena de caracteres a partir de los caracteres ingresados uno a uno por el usuario. En la cadena no se deberán almacenar caracteres de puntuación, pero se deberá permitir que el usuario los ingrese. Para indicar el fin de la lectura, el usuario deberá presionar ENTER.

Análisis

En este caso vamos a *filtrar* los datos de entrada. El concepto de filtrar consiste en permitir que el usuario ingrese los datos sin restricciones, pero antes de procesarlos, el algoritmo realiza algunas tareas para eliminar los elementos no deseados. En este caso son los caracteres de puntuación.

Diseño

El problema se puede solucionar con una estructura de reptición simple, dentro de la cual se leerá un caracter y se almacenará en la siguiente posición vacía en la cadena (inicialmente 0). Debido a que asumimos que las cadenas al igual que los arreglos pueden crecer dinámicamente, no tendremos que preocuparnos por *reservar* nuevas posiciones de memoria para almacenar los caracteres ingresados.

Para lograrlo, partimos de una cadena vacía y una variable que manejará el subíndice de la última posición de la cadena (inicialmente 0). A medida que se lean caracteres, se almacenan en la posición *i*, y se incrementa esta variable. El final de la lectura está marcado por el caracter ENTER, el cual representaremos como '\n'.

Dado que toda cadena de caracteres debe terminar con el caracter nulo, debemos adicionarlo al final de todos los caracteres leídos.

```
cad = ""
i = 0
hacer
  leer c //Leer siguiente caracter
  cad[i] = c //Almacenar
  i = i + 1 //Incrementar el subíndice
mientras c != '\n'
cad[i] = nulo //Terminar correctamente la cadena
```

Debemos realizar unos ajustes a este diseño inicial, para adicionar la lógica para evitar los caracteres de puntuación. Dado que nos encontramos dentro de una estructura repetitiva para leer cada caracter, simplemente podemos *omitir* la secuencia que almacena los caracteres y leer un nuevo caracter, mediante una instrucción *continuar*.

```
cad = ""
i = 0
hacer
  leer c
  si c = '.' o c = ':' o
  c = ',' o c = ';'
  //Omitir signos de puntuación
  continuar
fin si
cad[i] = c
i = i + 1
mientras c != '\n'
cad[i] = nulo
```

Solución

El Código 5.2 implementa el algoritmo que soluciona el problema. Se hace uso de una subrutina para leer la cadena, la cual se encarga de filtrar los caracteres correspondientes a

los signos de puntuación. Observe que también se adiciona una validación luego de leer el caracter, para prevenir que ENTER también sea almacenado dentro de la cadena. También se valida si la cadena leída no es vacía, en cuyo caso se termina con el caracter nulo.

Código 5.2: Crea una cadena filtrando caracteres

```
1  /*
2  * Construye una cadena de caracteres ignorando signos de puntuación
3  * a partir de los caracteres leídos por teclado
4  */
5  algoritmo crearCadena
6  cadena s
7  inicio
8  imprimir "Ingrese de caracteres. Para terminar presione ENTER"
9  s = leerSinPuntuacion()
10 imprimir s
11 fin
12 /* Lee una secuencia de caracteres, y retorna una cadena
13 * en la cual se han filtrado los signos de puntuación.
14 */
15 funcion leerSinPuntuacion():cadena
16 caracter c
17 cadena cad
18 entero i
19 inicio
20 i = 0 //Índice inicial para guardar caracteres
21 cad = "" //La cadena comienza vacía
22 hacer
23     leer c
24     si c = '\n' //Se debe terminar la lectura?
25         cancelar
26     fin si
27     si c = ',' o c = ':' o
28     c = ',' o c = ';'
29     continuar //Iterar de nuevo sin almacenar el caracter
30     fin si
31     cad[i] = c
32     i = i + 1
33 mientras c != '\n'
34 si i > 0
35     cad[i] = nulo
36 fin si
37 retornar cad
38 fin
```

En este ejemplo se ilustra el uso simultáneo de las instrucciones *cancelar* y *continuar*. En el primer caso, si el caracter ingresado es ENTER, se debe terminar el ciclo de lectura de datos. En caso contrario, debemos verificar si el caracter leído es un signo de puntuación. De ser así, saltamos las instrucciones faltantes dentro de la estructura repetitiva, y volvemos a leer otro caracter.

Tenga en cuenta que es necesario garantizar que la cadena retornada es válida, por lo cual se debe adicionar el caracter nulo al final una vez que se ha terminado la lectura y

antes de retornar.

Concatenación de cadenas

Construir un algoritmo que lea dos cadenas de caracteres, y construya e imprima una nueva cadena resultado de concatenar las dos cadenas.

Análisis

El término *concatenar* se refiere al proceso de *pegar* dos cadenas de caracteres una tras la otra, para obtener una cadena que contiene los caracteres de la primera cadena seguidos de los caracteres de la segunda, terminando con el caracter nulo.

En algunos lenguajes de programación se cuenta con la subrutina `strcat`, que permite unir dos cadenas. En otros lenguajes es posible usar el operador de adición o el operador *punto* para realizar este proceso.

Diseño

Para solucionar este problema, primero debemos crear una nueva cadena vacía, y copiar los caracteres de la primera cadena en esta nueva cadena hasta encontrar el caracter nulo. Luego debemos tomar los caracteres de la segunda cadena y copiarlos en donde termina la primera cadena, sobrescribiendo el caracter nulo de ser necesario. Por último, debemos terminar la nueva cadena con el caracter nulo.

Comencemos con la copia de la primera cadena. Podemos usar una estructura repetitiva, copiando cada caracter desde la posición 0 hasta llegar al final de la cadena, es decir hasta llegar al caracter nulo.

```
//Copiar los caracteres de s en cad
cad = ""
i = 0
j = 0
mientras s[j] != nulo
    cad[i] = s[j]
    i = i + 1
    j = j + 1
fin mientras
```

Al terminar la copia, la cadena `cad` contendrá todos los caracteres de `s` sin incluir el caracter nulo. Dado que los datos de `s` se copian en la misma posición dentro de `cad`, se puede usar un solo subíndice:

5. CADENAS DE CARACTERES

```
//Copiar los caracteres de s en cad
cad = ""
i = 0
mientras s[i] != nulo
    cad[i] = s[i]
    i = i + 1
fin mientras
```

Ahora la variable *i* almacenará la posición en la cual debemos comenzar a copiar los caracteres de la segunda cadena. Note que el código anterior funciona aún si la cadena *s* se encuentra vacía. En este caso, no se entrará a la estructura repetitiva.

Para copiar la segunda cadena necesitamos dos subíndices: *i*, que almacena la siguiente posición en la nueva cadena, y *j* que comienza en 0, para apuntar las posiciones dentro de la segunda cadena.

```
//Copiar los caracteres de s en cad
cad = ""
i = 0
mientras s[i] != nulo
    cad[i] = s[i]
    i = i + 1
fin mientras

//Copiar los caracteres de t en cad
j = 0
mientras t[j] != nulo
    cad[i] = t[j]
    i = i + 1
    j = j + 1
fin mientras
```

Algoritmo

El Código 5.3 implementa el algoritmo que permite concatenar dos cadenas mediante una subrutina, e imprimir la cadena resultante.

Código 5.3: Concatenar dos cadenas

```
algoritmo concatenarCadenas
cadena s, cadena t, cadena u
inicio
    leer s
    leer t
    u = concatenar(s, t) //Concatena las dos cadenas y retorna el resultado
    imprimir u
fin

/*
 * Concatena las dos cadenas recibidas en una nueva cadena
 */
```



```
* Retorna la cadena resultante
*/
funcion concatenar(cadena s, cadena t)
entero i
entero j
cadena cad
inicio
  i = 0 //Posición inicial para copiar en la nueva cadena
  cad = "" //Comenzar con una cadena vacía

  //Copiar los caracteres de la primera cadena
  mientras s[i] != nulo
    cad[i] = s[i]
    i = i + 1
  fin mientras

  //Copiar los caracteres de la segunda cadena
  j = 0
  mientras t[j] != nulo
    cad[i] = t[j]
    i = i + 1
    j = j + 1
  fin mientras
  //Terminar correctamente la cadena
  si i > 0
    cad[i] = nulo
  fin si
  retornar cad //Retornar el resultado
fin funcion
```

Observe que al terminar de copiar la primera cadena, la variable *i* apunta a la posición en la cual debería ir el caracter nulo de la primera cadena. Este caracter no debe ser copiado, en vez de ello se continúa copiando los caracteres de la segunda cadena en esa posición.

La variable *j* se usa para apuntar al inicio de cada cadena, antes de comenzar a copiar los datos. Es necesario establecerla en 0 nuevamente antes de comenzar a copiar los caracteres de la segunda cadena.

5.2. Consulta, selección y extracción de datos

A continuación se presenta una serie de problemas en los cuales se debe consultar, seleccionar o extraer caracteres de una cadena para llevar a cabo alguna tarea. De forma similar a los arreglos, esta clase de algoritmos requiere recorrer una o varias veces las cadenas de entrada, para encontrar o los caracteres que se seleccionarán y procesarán.

Longitud de una cadena

La primera consulta que se puede realizar sobre una cadena es calcular su longitud, definida como el número de caracteres diferentes a nulo que contienen.

Construir una función que permita calcular la longitud de una cadena de caracteres terminada en nulo.

Análisis

Tenga en cuenta que el *número total* de caracteres de una cadena terminada en nulo es igual a uno más la longitud, debido a que siempre se debe almacenar el caracter nulo al final. Bajo esta premisa, el tamaño total en caracteres de una cadena vacía es 1 (contiene el caracter nulo), y su longitud es 0 (contiene 0 caracteres diferentes de nulo).

Esta subrutina se encuentra en los lenguajes de programación como `strlen`, `len` o `length`.

Diseño

Este problema se puede solucionar de forma sencilla, usando una estructura repetitiva para recorrer la cadena hasta que se encuentre el caracter nulo. Cada vez que se encuentra un caracter diferente de nulo, se incrementará contador, el cual al finalizar la estructura repetitiva contendrá la longitud de la cadena.

```
i = 0
mientras s[i] != nulo
    i = i + 1
fin mientras
```

Algoritmo

La función para calcular la longitud de una cadena se presenta en el Código 5.4.

Código 5.4: Calcular la longitud de una cadena terminada en nulo

```
1 //Calcula y retorna la longitud de una cadena de caracteres
2 funcion longitud(cadena s)
3     entero i
4     inicio
5         i = 0
6         mientras s[i] != nulo
7             i = i + 1
8         fin mientras
9     retornar i
```

```
10 fin funcion
```

Búsqueda de caracteres dentro de una cadena

Para buscar un caracter dentro de una cadena se usa la misma estrategia utilizada para buscar un elemento en un arreglo. Veamos el siguiente problema:

Construir una función que permita obtener la posición de la primera ocurrencia de un caracter *c* dentro de una cadena *s*. Si el caracter especificado no se encuentra en la cadena, se deberá retornar -1.

Esta subrutina se encuentra en los lenguajes de programación como `strpos` o `pos`.

Análisis

El término *ocurrencia* se refiere a el hecho de que un dato *x* se encuentre almacenado en un arreglo. En este caso, el problema nos pide encontrar la posición en la cual se encuentra el elemento dentro de la cadena. En caso de no ser encontrado se retorna la posición -1, la cual no es válida dado que se ha definido que las posiciones en una cadena comienzan en 0. Esto le sirve a los algoritmos que usan la función para *detectar* si el dato especificado se encuentra o no. Sería erróneo retornar 0 en caso de no encontrarse, ya que esta es una posición válida.

Diseño

Para obtener la posición de la primera ocurrencia de un caracter en una cadena puede hacer uso de una estructura repetitiva, que compara uno a uno los caracteres desde el inicio de la cadena, es decir, desde la posición 0. En la primera coincidencia, la función deberá retornar la posición encontrada.

```
i = 0
mientras s[i] != nulo
  si s[i] = c
    retornar i //Retornar inmediatamente
  fin si
fin mientras
```

Si se llega al final de la cadena (al caracter nulo), significa que el caracter no se encuentra en la cadena, por lo tanto se deberá retornar -1.

5. CADENAS DE CARACTERES

```
i = 0
mientras s[i] != nulo
    si s[i] = c
        retornar i
    fin si
    i = i + 1
fin mientras
retornar -1
```

Algoritmo

El Código 5.5 implementa la función pos, que retornará la posición en la cual se encuentra el caracter en la cadena.

Código 5.5: Primera ocurrencia de un caracter en una cadena

```
1 /*
2  * Retorna la primera ocurrencia del caracter c en la cadena
3  * s, ó -1 si el caracter no se encuentra.
4  */
5 funcion pos(cadena s, caracter c): entero
6 entero i
7 inicio
8     i = 0
9     //Recorrer la cadena desde el inicio
10    mientras s[i] != nulo
11        si s[i] = c //Encontrado?
12            retornar i
13        fin si
14        i = i + 1
15    fin mientras
16    retornar -1 //Final de la cadena
17 fin funcion
```

Buscar una cadena dentro de otra

El siguiente es un problema cuya solución se usa con frecuencia al momento de realizar operaciones sobre cadenas.

Construir una función que dadas dos cadenas p y q, determine si p está contenida dentro de q. En caso afirmativo, se deberá retornar la posición en q a partir de la cual se encuentra p. En caso de no encontrarse p se deberá retornar -1.

Esta subrutina se encuentra como substr en los lenguajes de programación.

Análisis

Para solucionar este problema es necesario comparar todos los caracteres de las dos cadenas, verificando si existe la misma secuencia de caracteres de p dentro de q . Se debe tener en cuenta que p no necesariamente se encuentra dentro de q a partir de la primera posición, por lo cual debemos verificar si p se encuentra a partir de la primera posición, luego si se encuentra a partir de la segunda posición, y así sucesivamente.

Diseño

Vamos a solucionar este problema de forma iterativa. Primero verificamos si p se encuentra *a partir de la primera posición* de q . En ese caso, $q[0] = p[0], \dots, q[i] = p[i]$, hasta llegar al final de p .

Para llevar a cabo esta verificación nos ubicamos al inicio de p y q , y usamos una estructura repetitiva para comprobar si todos los caracteres en la misma posición son iguales. Si encontramos un caracter diferente, hemos determinado que p no se encuentra a partir de la primera posición de q . Si llegamos al final de p , entonces podemos decir que se encuentra a partir de la primera posición de q .

También debemos prestar atención de no llegar al final (caracter nulo) de q , dado que en el planteamiento del problema no se especifica que la longitud de q es siempre mayor que la longitud de p . Si llegamos al final de q pero no al final de p , entonces podemos decir que p no se encuentra *dentro* de q porque simplemente no cabe.

```
i = 0
j = i
mientras q[j] != nulo y p[j] != nulo
    si q[j] != p[j] //Caracteres diferentes
        retornar -1 //Son diferentes!
    fin si
    j = j + 1
fin mientras
si p[j] = nulo //Final de p
    retornar i //p se encuentra en q a partir de i
fin si
si q[j] = nulo y p[j] != nulo
    retornar -1 //p no cabe en q
fin si
```

Observe que no se usa la variable i para recorrer la cadena q , ya que es necesario saber la posición *a partir de la cual* se encuentra p . Si usáramos la variable i , no sería posible conocer esta posición.

Ahora bien, la cadena p no necesariamente se encuentra a partir de la primera posición de q , por lo cual debemos *repetir* el proceso anterior, pero ahora avanzando una posición de q y comenzando de nuevo desde el inicio de p .

5. CADENAS DE CARACTERES

Debemos modificar el código anterior, ya que ahora no se van a comparar las mismas posiciones de p y q. Además el proceso de debe seguir repitiendo hasta llegar a un punto de q en el cual p ya no pueda caber.

Primero separaremos los subíndices de p y q en los cuales realizamos la verificación. La variable j se usará en q, y la variable k en p.

```
i = 0 //Inicio de q
k = 0 //Inicio de p
j = i
mientras q[j] != nulo y p[k] != nulo
    si q[j] != p[k] //Caracteres diferentes
        retornar -1 //Son diferentes!
    fin si
    j = j + 1
    k = k + 1
fin mientras
si p[k] = nulo //Final de p
    retornar i //p se encuentra en q a partir de i
fin si
si q[j] = nulo y p[k] != nulo
    retornar -1 //p no cabe en q
fin si
```

Para repetir todo el proceso, podemos *anidar* el algoritmo construido dentro de otra estructura repetitiva, la cual *avance* la posición de q desde la cual se comienza la búsqueda.

```
i = 0 //Inicio de q
mientras q[i] != nulo
    j = i //Buscar en q a partir de la posición i
    k = 0 //Inicio de p
    mientras q[j] != nulo y p[k] != nulo
        si q[j] != p[k] //Caracteres diferentes
            retornar -1 //Son diferentes!
        fin si
        j = j + 1
        k = k + 1
    fin mientras
    si p[k] = nulo //Final de p
        retornar i //p se encuentra en q a partir de i
    fin si
    si q[j] = nulo y p[k] != nulo
        retornar -1 //p no cabe en q
    fin si
    i = i + 1 //Avanzar una posición en q
fin mientras
```

Revisemos el comportamiento de las dos estructuras repetitivas. La primera de ellas se encuentra controlada con la variable i, la cual comienza en 0 y avanza hasta el final de q.

Antes de comenzar la segunda estructura, establecemos j en el valor actual de i, y k en 0. De esta forma, buscaremos la cadena p a partir de la posición i y aplicamos la lógica construida anteriormente.

Si no hemos encontrado la cadena a partir de la posición i , incrementamos esta variable y realizamos de nuevo la búsqueda de p .

Algoritmo

El Código 5.6 implementa la función que soluciona el problema definido.

Código 5.6: Encontrar la primera ocurrencia de una subcadena

```

1  /*
2  * Busca la primera ocurrencia de p dentro de q.
3  * Retorna la posición de la ocurrencia, o -1 si p no se encuentra
4  * dentro de q.
5  */
6  funcion posCadena(cadena p, cadena q): entero
7  entero i, entero j, entero k
8  inicio
9  i = 0
10 mientras q[i] != nulo
11     j = i //Buscar en q a partir de la posición i
12     k = 0 //Inicio de p
13     mientras q[j] != nulo y p[k] != nulo
14         si q[j] != p[k] //Caracteres diferentes
15             retornar -1 //Son diferentes!
16         fin si
17         j = j + 1
18         k = k + 1
19     fin mientras
20     si p[k] = nulo //Final de p
21         retornar i //p se encuentra en q a partir de i
22     fin si
23     si q[j] = nulo y p[k] != nulo
24         retornar -1 //p no cabe en q
25     fin si
26     i = i + 1 //Avanzar una posición en q
27 fin mientras
28 //Llegamos al final de todas las comparaciones, no es subcadena
29 retornar -1
30 fin funcion

```

Es posible optimizar un poco el código para ofrecer una solución más elegante, siendo *positivos* en la estructura repetitiva y usando una condición compuesta (ver Código 5.7). Esto significa que suponemos que las cadenas no han terminado, y que los caracteres que se están verificando son iguales. Luego se verifica cada una de las situaciones por las cuales se terminó la estructura repetitiva.

Código 5.7: Encontrar subcadena

```

1  /*
2  * Busca la primera ocurrencia de la cadena p dentro de una cadena q.
3  * Retorna la posición de la ocurrencia, o -1 si p no se encuentra

```

5. CADENAS DE CARACTERES

```
4  * dentro de q.
5  */
6  funcion posCadena(cadena p, cadena q): entero
7  entero i, entero j, entero k
8  inicio
9      i = 0
10     mientras q[i] != nulo
11         j = i
12         k = 0
13
14         //Recorrer y comparar p con lo que resta de q
15         mientras q[j] != nulo y p[k] != nulo y p[k] = q[j]
16             j = j + 1
17             k = k + 1
18         fin mientras
19
20         si p[k] = nulo //Final de p?
21             retornar i //Es subcadena!
22         fin si
23
24         //Es el final de q y no es el final de p
25         si q[j] = nulo y p[k] != nulo
26             retornar -1 //No tiene sentido seguir buscando
27         fin si
28
29         //Avanzar una posicion en q y volver a empezar las comparaciones
30         i = i + 1
31     fin mientras
32
33     //Llegamos al final de todas las comparaciones, no es subcadena
34     retornar -1
35 fin funcion
```

Paréntesis balanceados

La solución a este problema tiene un amplio espectro de uso, especialmente en intérpretes de código y compiladores.

Desarrollar una función que dada una cadena que contiene solo paréntesis y espacios, determine si los paréntesis se encuentran balanceados.

Análisis

Se considera que una secuencia de paréntesis se encuentra balanceada cuando existe el mismo número de paréntesis de apertura y cierre, y además los paréntesis de cierre se encuentran a la derecha de sus respectivos paréntesis de apertura. Las siguientes son secuencias en las cuales los paréntesis se encuentran balanceados:


```
()  
()  
()  
()  
()  
()  
()
```

Diseño

Una solución a este problema es usada de forma intuitiva cuando estamos escribiendo código para balancear paréntesis, llaves, y otros elementos que marcan posiciones iniciales y finales. La estrategia se resume en el siguiente proceso:

Iniciamos un acumulador en cero. Esto indica que no se ha comenzado a procesar la cadena. Ahora recorreremos la cadena, y cada vez que encontramos un paréntesis de apertura incrementamos el acumulador. Si por el contrario encontramos un paréntesis de cierre, decrementamos el acumulador.

Al finalizar el recorrido de la cadena el acumulador deberá almacenar 0, puesto que debe existir el mismo número de paréntesis de apertura y cierre. Si es mayor que cero, hay más paréntesis de apertura. Si es menor que cero, hay más paréntesis de cierre.

```
i = 0  
cont = 0  
mientras s[i] != nulo  
  si s[i] = '('  
    cont = cont + 1  
  fin si  
  si s[i] = ')'  
    cont = cont - 1  
  fin si  
  i = i + 1  
fin mientras  
si cont = 0  
  retornar verdadero  
sino  
  retornar falso  
fin si
```

El código anterior verifica que se tenga exactamente el mismo número de paréntesis de apertura y cierre en la cadena, pero no permite verificar si estos se encuentran en el orden correcto. Por ejemplo, si la cadena es “)(” o “)()”, al final el contador se encontrará en 0, pero los paréntesis no están correctamente formados.

Por lo tanto, también debemos verificar que el acumulador *en ningún momento* tome valores negativos, lo cual se puede realizar fácilmente modificando el código obtenido hasta ahora:

```
i = 0  
cont = 0
```

5. CADENAS DE CARACTERES

```
mientras s[i] != nulo
  si s[i] = '('
    cont = cont + 1
  fin si
  si s[i] = ')'
    cont = cont - 1
  fin si
  si cont < 0
    retornar falso
  fin si
  i = i + 1
fin mientras
si cont = 0
  retornar verdadero
sino
  retornar falso
fin si
```

Algoritmo

La función que implementa el proceso descrito se presenta en el Código 5.8.

Código 5.8: Verificar paréntesis balanceados

```
1  /*
2  * Retorna verdadero si los paréntesis dentro de la cadena
3  * se encuentran balanceados, falso en caso contrario
4  */
5  funcion balanceados(cadena s)
6  entero i
7  entero cont
8  inicio
9    i = 0
10   cont = 0
11   mientras s[i] != nulo
12     si s[i] = '('
13       cont = cont + 1
14     fin si
15     si s[i] = ')'
16       cont = cont - 1
17     fin si
18     si cont < 0
19       retornar falso
20     fin si
21     i = i + 1
22   fin mientras
23   si cont = 0
24     retornar verdadero
25   fin si
26   retornar falso
27 fin funcion
```

5.3. Algoritmos generales

A continuación se presentará una serie de problemas generales que involucran la creación y modificación de cadenas, y los respectivos algoritmos para solucionarlos.

Cadena inversa

En ocasiones es necesario transformar las cadenas de caracteres, como se plantea en el siguiente problema:

Desarrollar una función que dada una cadena de $n \geq 0$ caracteres, permita obtener una nueva cadena en la cual los caracteres están almacenados en orden inverso.

Esta subrutina se puede encontrar en algunos lenguajes de programación como `rev` o `strrev`.

Análisis

Este problema es relativamente sencillo. Para solucionarlo, primero necesitamos llegar al final de la cadena original, y una vez allí, podemos recorrer hacia atrás la cadena y a medida que se recorre, se adicionan los caracteres en la nueva cadena como se muestra en la Figura 5.1.

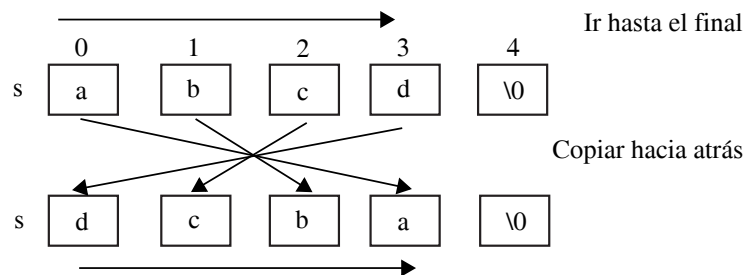


Figura 5.1: Construir la cadena inversa

Diseño

Primero vamos hacia adelante en la cadena original:

```
i = 0
//Ir al final de la cadena original
mientras s[i] != nulo
    i = i + 1
```

5. CADENAS DE CARACTERES

```
fin mientras
```

Ahora regresamos, copiando de atrás hacia adelante en una nueva cadena:

```
t = ""
i = 0
//Ir al final de la cadena original
mientras s[i] != nulo
    i = i + 1
fin mientras

i = i - 1 //s[i] = nulo, retroceder una posición
j = 0 //Posición en la nueva cadena
mientras i >= 0
    t[j] = s[i]
    i = i - 1
    j = j + 1
fin mientras

t[j] = nulo //Terminar correctamente la nueva cadena
```

Solución

La función que permite obtener la cadena inversa se presenta en el Código 5.9. Se han adicionado algunas verificaciones que se explicarán más adelante.

Código 5.9: Función para obtener la cadena inversa

```
/*
 * Retorna una cadena con los caracteres de la cadena
 * original en orden inverso
 */
funcion inversa(cadena s): cadena
cadena t
entero i, entero j
inicio
    t = ""
    i = 0
    //Ir al final de la cadena original
    mientras s[i] != nulo
        i = i + 1
    fin mientras

    si i = 0 //Cadena vacía?
        retornar t //Retornar cadena vacía
    fin si

    i = i - 1 //s[i] = nulo, retroceder una posición
    j = 0 //Posición en la nueva cadena

    /* Sacar los elementos de la cadena original desde el
     * final hacia el principio, y copiar los elementos
```

```

* hacia adelante en la nueva cadena
*/
mientras i >= 0
    t[j] = s[i]
    i = i - 1 //Retroceder en la cadena original
    j = j + 1 //Avanzar en la nueva cadena
fin mientras
t[j] = nulo //Terminar correctamente la nueva cadena
retornar t
fin funcion

```

En la función presentada se verifica si la cadena original era vacía. Si se presenta ese caso, no tiene sentido continuar. En caso contrario, se recorre la cadena original hasta el final. La estructura *mientras* hace que la variable *i* almacene la posición del carácter nulo, por lo cual deberá ser decrementada antes de comenzar la copia. Finalmente la función se retorna la cadena construida.

Eliminar espacios iniciales y finales

Este problema también debe ser solucionado con frecuencia cuando se usan cadenas.

Construir una función que permita obtener una nueva cadena a partir de una cadena de entrada, en la cual se han eliminado los espacios iniciales y finales.

En algunos lenguajes de programación esta subrutina se conoce como `trim`. También es posible encontrar las subrutinas `ltrim` y `rtrim`, que se encargan solo de eliminar los espacios iniciales y finales, respectivamente.

Análisis

En este caso podemos copiar los caracteres de la cadena original a la nueva cadena, avanzando desde el inicio de la cadena original mientras el carácter revisado sea espacio. En cuanto se encuentre un carácter diferente de espacio, se copian los demás caracteres hasta el final de la cadena original.

Esta estrategia nos permite evitar los espacios iniciales, pero no permite evitar los espacios al final, ya que es posible encontrar espacios en el medio de la cadena que no deben ser eliminados.

Luego debemos realizar un recorrido desde el final de la cadena, cambiando los espacios por el carácter nulo hasta encontrar un carácter diferente de espacio.

Diseño

Primero debemos saltar los espacios al inicio de la cadena original:

```
i = 0
mientras s[i] != nulo y s[i] = ' ' //Saltar los espacios iniciales
    i = i + 1
fin mientras
```

Ahora se copian los demás caracteres hasta el final de la cadena:

```
i = 0
mientras s[i] != nulo y s[i] = ' ' //Saltar los espacios iniciales
    i = i + 1
fin mientras

j = 0
mientras s[i] != nulo
    cad[j] = s[i]
    i = i + 1
    j = j + 1
fin mientras
```

Para finalmente eliminar los espacios al final de la nueva cadena:

```
i = 0
mientras s[i] != nulo y s[i] = ' ' //Saltar los espacios iniciales
    i = i + 1
fin mientras

j = 0
mientras s[i] != nulo
    cad[j] = s[i]
    i = i + 1
    j = j + 1
fin mientras

j = j - 1
mientras j > 0 y cad[j] = ' '
    cad[j] = nulo
    j = j - 1
fin mientras
```

Algoritmo

El algoritmo obtenido se presenta en el Código 5.10.

Código 5.10: Eliminar espacios iniciales y finales

```
1 /*
2  * Retorna una nueva cadena a partir de la cadena de
3  * entrada, en la cual se han eliminado los espacios iniciales y finales.
4  */
```

```

5 funcion quitarEspacios(cadena s): cadena
6   cadena cad
7   entero i
8   entero j
9   inicio
10    i = 0
11    cad = ""
12    mientras s[i] != nulo y s[i] = ' ' //Saltar los espacios iniciales
13      i = i + 1
14    fin mientras
15
16    //Verificar si al saltar los espacios llegamos al final de la cadena
17    si s[i] = nulo
18      retornar cad //En ese caso la cadena solo contenía espacios
19    fin si
20
21    //Hay al menos un caracter diferente de espacio
22    j = 0
23    mientras s[i] != nulo
24      cad[j] = s[i]
25      i = i + 1
26      j = j + 1
27    fin mientras
28
29    cad[j] = nulo //Terminar la cadena correctamente
30
31    //Eliminar los espacios finales de la nueva cadena
32    //Retroceder una posición, estamos ubicados en el caracter nulo
33    j = j - 1
34    mientras j > 0 y cad[j] = ' '
35      cad[j] = nulo
36      j = j - 1
37    fin mientras
38    retornar cad //Retornar la cadena obtenida
39 fin funcion

```

La primera estructura repetitiva nos permite *saltar* todos los espacios iniciales, hasta encontrar el final de la cadena o el primer caracter que no es espacio. Luego, debemos verificar si se llegó al final de la cadena, ya que en este caso se debe terminar inmediatamente.

Después se copian todos los caracteres hasta el final de la cadena, incluyendo los espacios finales, ya que pueden existir espacios intermedios que deben ser incluidos. Finalmente, retrocedemos borrando los espacios finales, y retornamos la cadena obtenida.

Dividir cadena

En este problema se creará un arreglo de cadenas a partir de una cadena y un caracter. Esta función se encuentra en algunos lenguajes de programación como `split` o `explode`.

Desarrollar una función que dado un caracter (llamado delimitador) y una cadena de caracteres, divida la cadena original usando el delimitador, y llene un arreglo con las subcadenas obtenidas. El delimitador podrá aparecer varias veces, y en cualquier posición dentro de la cadena. La función recibirá también la referencia del arreglo a llenar, y deberá retornar el número de subcadenas obtenidas.

Análisis

Para solucionar este problema necesitamos crear un nuevo arreglo a partir de las subcadenas extraídas de una cadena que recibimos como parámetro. Las subcadenas están *delimitadas* (separadas) por un caracter que también es enviado a la función que debemos implementar. El proceso se ilustra en la Figura 5.2.

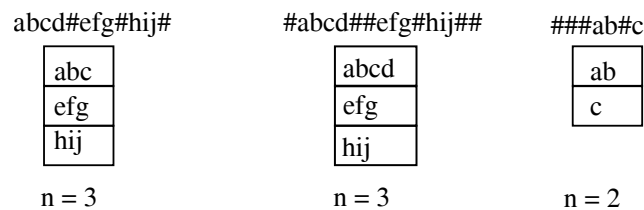


Figura 5.2: Dividir una cadena por un delimitador

El caracter delimitador (# en la Figura 5.2) puede aparecer en cualquier posición de la cadena, y puede encontrarse junto a otros caracteres delimitadores. Incluso es posible recibir una cadena vacía, o que solo contenga delimitadores.

Diseño

La idea consiste en *sacar* las subcadenas que existen, *saltando* (evitando) los caracteres delimitadores. La solución se puede construir con el siguiente proceso general, empezando por la posición inicial de la cadena:

1. Avanzamos *hasta encontrar el primer caracter que no sea delimitador*.
2. Si llegamos al final de la cadena, todos los caracteres revisados son delimitadores, por lo tanto hemos terminado.
3. En caso contrario, nos encontraremos al inicio de una subcadena. Marcamos esta posición como *el inicio de la subcadena*, y avanzamos *hasta encontrar un caracter delimitador o el caracter nulo*. Marcamos la posición hasta la cual hemos llegado como *el fin de la subcadena*.

4. Comparamos las posiciones de inicio y fin de subcadena encontradas en el paso anterior. Si son iguales, quiere decir que no hemos encontrado más subcadenas, por lo tanto debemos terminar el proceso.
5. En caso contrario, hemos encontrado una subcadena. Debemos copiar los caracteres desde la posición inicial hasta la posición final encontradas en el paso 3 dentro de una nueva cadena, y adicionar esta cadena al arreglo de salida.
6. Ahora repetimos todo el proceso desde el paso 1, para encontrar la siguiente subcadena o terminar si no existe otra subcadena.

Este proceso es muy eficiente, ya que solo se recorre la cadena completa una vez, y luego regresamos sólo lo suficiente para copiar la subcadena encontrada. Se recomienda realizar una prueba de escritorio del proceso anterior para comprender su funcionamiento, antes de revisar el algoritmo.

Algoritmo

El proceso descrito en el análisis del problema se implementa en el Código 5.11.

Código 5.11: Dividir una cadena por un delimitador

```

1  /*
2  * Divide una cadena por un delimitador, y llena un arreglo.
3  * Almacena en la referencia entera el número de cadenas obtenidas.
4  */
5  funcion dividir(cadena s, caracter c, cadena a[])
6  cadena cad
7  entero i //Posición de inicio de subcadena
8  entero n //Posición para almacenar la siguiente subcadena
9  entero j, entero k, entero pos //Variables auxiliares
10 inicio
11   i = 0
12   n = 0
13   mientras s[i] != nulo
14     //Saltar los delimitadores
15     mientras s[i] != nulo y s[i] = c
16       i = i + 1
17   fin mientras
18
19   si s[i] = nulo //Hemos llegado al fin de la cadena?
20     cancelar //Terminar, no hay más subcadenas
21   fin si
22
23   //Existe una nueva subcadena.
24
25   j = i //Marcar la posición inicial de la subcadena
26   mientras s[j] != nulo y s[j] != c
27     j = j + 1 //Avanzar hasta encontrar nulo o un delimitador

```

5. CADENAS DE CARACTERES

```
28  fin mientras
29
30  si j = i //No se avanzó?
31      cancelar //No existen más subcadenas.
32  fin si
33
34  /*
35  * Extraer los caracteres y copiarlos en una nueva cadena
36  * La posición inicial es i, la posición final es j - 1
37  */
38  pos = 0
39  cad = ""
40  para k = i, k < j, k = k + 1
41      cad[pos] = s[k]
42      pos = pos + 1
43  fin para
44
45  cad[pos] = nulo //Terminar la nueva cadena correctamente
46
47  a[total] = cad //Almacenar la nueva cadena en el arreglo de salida
48  n = n + 1 //Incrementar el índice en el arreglo
49  i = j //Avanzar a la posición final de la subcadena
50  fin mientras
51  retornar total //Retornar el número de subcadenas obtenidas
52 fin funcion
```

De acuerdo con lo especificado en el planteamiento del problema, dentro del arreglo de salida se almacenan las cadenas obtenidas. Finalmente se retorna el número de subcadenas almacenadas.

Transformar un número en cadena de caracteres

En el siguiente problema se construye una cadena de caracteres a partir de un dato de otro tipo.

Construir una función que retorne la representación como una cadena de caracteres de un número entero dado, en formato decimal.

Esta subrutina se puede encontrar en algunos lenguajes de programación como `tostring` o `itoa`. En otros lenguajes es posible obtener una cadena de caracteres, simplemente concatenando la cadena vacía con el número.

Análisis

Para solucionar este problema, se debe tomar el número y generar los caracteres para cada uno de sus dígitos en decimal. Si bien existen diferentes aproximaciones para encon-

trar una solución, se puede usar un proceso relativamente sencillo, partiendo del hecho que leemos los números de izquierda a derecha.

Diseño

Primero se debe revisar si el número es menor que cero. En caso afirmativo, la cadena comenzará con el carácter -, y el número original se multiplicará por -1 para que sea positivo.

1. Se debe encontrar el mayor dígito del número, que irá más a la izquierda. Esto se puede lograr con una estructura repetitiva, dividiendo sucesivamente por 10 hasta que el resultado de esta división sea cero. En cada repetición se actualiza la potencia correspondiente. Al finalizar este ciclo, se tendrá la potencia de 10 del mayor dígito.
2. Ahora que conocemos la mayor potencia de 10, realizamos el proceso en orden inverso. Dividimos repetidamente el número entre la potencia y calculamos el residuo, el cual será el dígito que debemos guardar.
3. Para el dígito encontrado, encontrar el carácter correspondiente y añadirlo a la cadena de salida.
4. Repetir los pasos 2 y 3 dividiendo la potencia entre 10, y tomando el nuevo número como el residuo entre el número original y la potencia usada. Los pasos deberán ser repetidos mientras la potencia sea mayor que cero.

Algoritmo

El algoritmo que implementa el proceso especificado se presenta en el Código 5.12.

Código 5.12: Transformar un número en cadena de caracteres

```

1 funcion convertirACadena(entero n): cadena
2   cadena cad
3   entero potencia, entero digito, entero i, entero pos
4   entero aux, digito
5   caracter c
6   inicio
7
8   si n = 0
9     cad = "0"
10    retornar cad
11  fin si
12
13  cad = ""
14  pos = 0

```

```
15 si n < 0 //El número es negativo?
16   cad = "-" //Comenzar con -
17   pos = 1
18   n = n * - 1 //Volver positivo
19 fin si
20
21 potencia = 1
22 aux = n
23 mientras aux > 0
24   aux = aux / 10
25   potencia = potencia * 10 //Aumentar la potencia
26 fin mientras
27
28 //Retroceder en una potencia de 10
29 potencia = potencia / 10
30
31 aux = n
32 hacer
33   digito = aux / potencia
34   c = '0' + digito //Caracter ASCII '0' + el digito calculado
35   cadena[pos] = c
36   pos = pos + 1
37   aux = aux mod potencia
38   potencia = potencia / 10
39 mientras potencia > 0
40   cad[pos] = nulo
41 fin funcion
```

Inicialmente se trata el caso base, cuando el número especificado es 0. Luego comenzamos a construir la cadena de salida, verificando el signo del número especificado. Si el número es negativo, se inicia la cadena con - y se avanza la posición en la cual se almacenarán los dígitos como caracteres.

La primera estructura repetitiva permite encontrar la potencia de 10 que corresponderá al primer dígito. Este ciclo encontrará *la siguiente potencia*, por lo cual se deberá retroceder en una potencia de 10.

En la siguiente estructura repetitiva se extrae cada dígito, de mayor a menor realizando la división entera entre el número y la potencia actual de 10. Una vez extraído el dígito, se convierte a caracter ASCII, suponiendo que es posible obtenerlo sumando el caracter '0' al dígito obtenido. Posteriormente se actualizan los valores del número a dividir y la potencia de 10, y se repite el proceso mientras la potencia sea mayor que cero.

5.4. Ejercicios propuestos

1. Construir una función que dado un caracter de *pegamento* y dos cadenas, permita obtener una nueva cadena de caracteres en la cual se *pegan* las dos cadenas usando el caracter especificado.

2. Construir una función dados un arreglo de cadenas y un caracter de *pegamento*, retorne una nueva cadena en la cual se encuentran todas las cadenas del arreglo pegadas con el caracter especificado.
3. Construir una función que retorne el número de ocurrencias de una cadena *p* dentro de una cadena *q*.
4. Construir una función que permita reemplazar la primera ocurrencia de una cadena *p* por otra cadena *r*, dentro de una cadena *q*. La cadena de reemplazo *r* puede ser vacía. No se debe realizar ninguna suposición sobre el tamaño de las tres cadenas.
5. Construir una función que permita dividir una cadena *s* por cualquiera de los caracteres delimitadores especificados en otra cadena *d*. La función deberá llenar un arreglo pasado como parámetro, y retornará el número de subcadenas obtenidas.
6. Construir una función que permita reemplazar todas las ocurrencias de una cadena *p* por otra cadena *r*, dentro de una cadena *q*. De nuevo, la cadena de reemplazo puede ser vacía, y no se deberá realizar ninguna suposición sobre el tamaño de cualquiera de las cadenas.
7. Construir una función que elimine determinados caracteres iniciales y finales de una cadena *p*. Los caracteres que se deben eliminar se especificarán en otra cadena *q*.
8. Construir una función que permita obtener una cadena de caracteres que será la representación en binario de un número entero proporcionado.
9. Construir una función que permita obtener una cadena de caracteres que será la representación en hexadecimal de un número entero proporcionado.
10. Construir una función que dada una cadena de caracteres que contiene cualquier tipo de caracteres (letras, números, símbolos de puntuación, etc.), transforme cualquier letra en mayúsculas en su equivalente en minúsculas. Asuma que los caracteres se representan también como números, por lo cual es posible identificar los rangos de caracteres en mayúsculas y minúsculas en una tabla ASCII y realizar operaciones de suma y/o resta sobre ellos. No considere caracteres con acentos.
11. Construir una función que dada una cadena, transforme a mayúsculas cualquier letra en minúsculas que contenga.
12. Construir una función que determine si una cadena es palíndromo, es decir, si se lee igual de adelante hacia atrás o de atrás hacia adelante.

5. CADENAS DE CARACTERES

13. Construir una función que dadas dos cadenas de caracteres, calcule e imprima la longitud de la cadena más corta.
14. Construir una función que dada una cadena de caracteres y un número n , recorte la cadena al número de caracteres especificado en n . Considere el caracter nulo al final de la cadena.
15. Construir una función que dada una cadena de caracteres, un número n y un caracter c , construya una nueva cadena que contendrá máximo n caracteres de la cadena original, incluyendo el caracter nulo. Si la cadena original tiene menos caracteres, se deberá *rellenar* los espacios faltantes con el caracter especificado.

CAPÍTULO 6

Ordenamiento

En este capítulo revisaremos un problema que ha sido estudiado prácticamente desde el inicio de la computación, y que se usa en prácticamente todas las áreas de aplicación de la informática y la computación en las cuales es necesario procesar datos. Como mencionan Sedgewick and Wayne (2011), se usan algoritmos de ordenamiento en campos tan variados como aplicaciones comerciales para procesar datos, dinámica molecular, astrofísica, lingüística, genómica, predicción del clima, entre otros.

En su sentido más general, el ordenamiento es una operación sobre un conjunto de datos, que permite *organizarlos* de forma que cumplan con determinada condición. Supongamos que se tiene un arreglo a de n elementos. Los elementos se encontrarán ordenados en forma ascendente si para todo $j \geq i$, se cumple que $a[j] \geq a[i]$ (a), y se encontrarán ordenados de forma descendente si $a[j] \leq a[i]$ (b), como se muestra en la Figura 6.1.

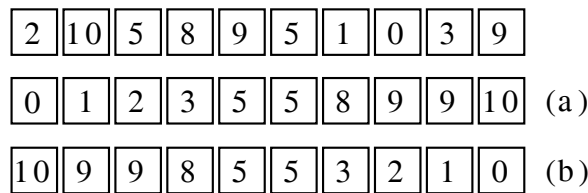


Figura 6.1: Ordenamiento de datos

Un algoritmo de ordenamiento es entonces aquel que toma como entrada un conjunto de datos, y permite organizarlos de forma ascendente o descendente. Es importante tener en cuenta que en la mayoría de libros de texto sólo se presentan algoritmos que permiten organizar los elementos de forma ascendente -de mayor a menor-, ya que para organizarlos en orden contrario se requieren pocas modificaciones al mismo algoritmo.

En las siguientes secciones se presentarán algunos algoritmos *clásicos* para resolver el problema del ordenamiento de datos. En los algoritmos se usará un arreglo a y un número n como entrada, y el proceso permitirá organizar los elementos dentro del arreglo, de forma que se encuentren ordenados en forma ascendente. Al final de cada algoritmo se incluirán algunos comentarios relacionados con su *eficiencia*, la cual se estudia con detalle en cursos avanzados de algoritmos y estructuras de datos.

6.1. Ordenamiento por selección (Selection sort)

La idea detrás del ordenamiento por selección puede ser considerada una de las más intuitivas. Debemos recorrer el arreglo, para buscar el elemento que *debería* ir en la primera posición. Si estamos ordenando el arreglo en forma ascendente, debería ser el menor de todos. Si por el contrario, estamos ordenando el arreglo en forma descendente, debería ser el mayor de todos. Al final del recorrido, se *intercambia* el elemento encontrado (el menor o mayor de todos, según el caso) con el que se encuentre en la primera posición del arreglo. El proceso se muestra en la Figura 6.2.

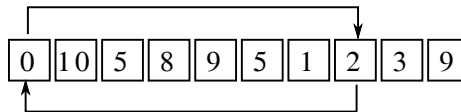


Figura 6.2: Ordenamiento por selección - primera iteración

Ahora que el menor elemento de todos se encuentra en la primera posición, nos *olviamos* de él y repetimos el proceso para ubicar el siguiente menor elemento. Realizamos la búsqueda e intercambiamos el dato en la posición del elemento encontrado con el que se encuentre en la segunda posición del arreglo como se muestra en la Figura 6.3.

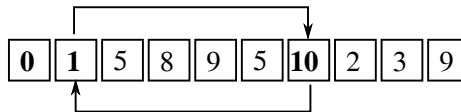


Figura 6.3: Ordenamiento por selección - segunda iteración

El ordenamiento deberá repetirse para ubicar las posiciones faltantes, por lo cual se deberán realizar $n - 1$ iteraciones. En la Figura 6.4 se muestran todos los intercambios realizados para el arreglo de ejemplo.

Implementación del algoritmo

Para implementar el algoritmo por selección se requiere una estructura repetitiva que permita ubicar el elemento $0 \leq i \leq n - 1$ en su respectiva posición. Dentro de esta

2	10	5	8	9	5	1	0	3	9	
0	10	5	8	9	5	1	2	3	9	i=0
0	1	5	8	9	5	10	2	3	9	i=1
0	1	2	8	9	5	10	5	3	9	i=2
0	1	2	3	9	5	10	5	8	9	i=3
0	1	2	3	5	9	10	5	8	9	i=4
0	1	2	3	5	5	10	9	8	9	i=5
0	1	2	3	5	5	8	9	10	9	i=6
0	1	2	3	5	5	8	9	10	9	i=7
0	1	2	3	5	5	8	9	9	10	i=8
0	1	2	3	5	5	8	9	9	10	i=9

Figura 6.4: Todas las iteraciones del ordenamiento por selección

estructura repetitiva, debemos buscar y *seleccionar* el elemento correspondiente, y una vez encontrado lo intercambiamos con el que se encuentre en la posición *i*.

El algoritmo de la búsqueda por selección se presenta en el Código 6.1.

Código 6.1: Ordenamiento por selección

```

1  /* Ordena un arreglo de n elemento usando selection sort*/
2  funcion ordenarSeleccion(entero a[], entero n)
3  entero i, j, posMin, temp
4  inicio
5      //Posicionar el elemento i
6      para i = 0, i < n - 1, i = i + 1
7          //asumir que el menor elemento se encuentra en su posición
8          posMin = i
9          //Buscar el menor en las posiciones restantes
10         para j = i + 1, j < n, j = j + 1
11             si a[j] < a[posMin]
12                 posMin = j //Menor encontrado, recordar su posición
13             fin si
14         fin para
15         //Colocar el menor encontrado en la posición i, intercambiando
16         //con el elemento que se encuentre en esa posición
17         temp = a[i]
18         a[i] = a[posMin]
19         a[posMin] = temp
20     fin para
21 fin funcion

```

Eficiencia

El algoritmo de ordenamiento por selección es uno de los más intuitivos, pero no es muy eficiente en términos de la cantidad de recorridos o comparaciones que realiza. Sin embargo, es un buen punto de partida para adentrarse en el mundo del análisis de algoritmos.

En Sedgewick and Wayne (2011) se presenta un análisis del algoritmo de selección, en el cual muestra que para un arreglo de n elementos se debe realizar $n - 1$ intercambios y aproximadamente $\frac{n^2}{2}$ comparaciones. La siguiente tabla muestra el número de comparaciones necesarias para ordenar un arreglo, para diferentes valores de n .

Tabla 6.1: Número de intercambios y comparaciones de selection sort

Elementos (n)	Intercambios	Comparaciones
5	4	10
10	9	45
20	19	190
50	49	1225
100	99	4950
1000	999	499500
10000	9999	49995000

Una desventaja de *selection sort* consiste en que siempre realizará el mismo número de comparaciones para el mismo número de datos, sin importar qué tan ordenados se encuentren. Como veremos más adelante, algunos algoritmos intentan evitar las comparaciones innecesarias, las cuales ahorrarán tiempo considerable cuando n es relativamente grande.

6.2. Ordenamiento por inserción (Insertion Sort)

Cormen et al. (2009) compara este algoritmo con el proceso que se realiza para ordenar un mazo de cartas de menor a mayor. A diferencia de *selection sort*, no se recorre el arreglo buscando el menor elemento para llevarlo a la posición $0 \leq i \leq n - 1$ que le corresponde. En su lugar, se verifica si todos los elementos a la izquierda del que estamos revisando en la posición i son menores que él.

En caso afirmativo, podemos decir que el arreglo está ordenado *hasta* la posición i . Si por el contrario encontramos algún elemento *mayor* que el que estamos revisando, entonces *desplazamos* los elementos mayores una posición a la derecha, abriendo un espacio, y luego *insertamos* el elemento en el espacio abierto. De esta forma, todos los datos mayores han sido movidos *después* del elemento, y el arreglo seguirá ordenado hasta la posición

en la cual acabamos de insertar el dato. El funcionamiento general de *insertion sort* se muestra en la Figura 6.5.

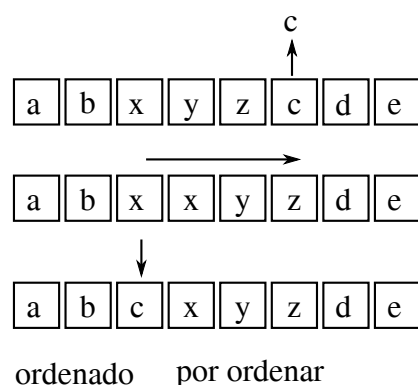


Figura 6.5: Funcionamiento general de *insertion sort*

Veamos cómo funciona el algoritmo *insertion sort* sobre nuestro conjunto de datos de prueba. El elemento en la primera posición *siempre* estará ordenado con respecto a todos los elementos a su izquierda, por lo cual el algoritmo comienza por revisar el elemento de la segunda posición como se muestra en la Figura 6.6.

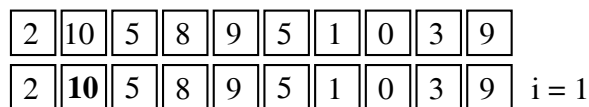


Figura 6.6: Primera iteración de *insertion sort*

En este caso, todos los elementos a la izquierda de la posición $i = 1$ (el número 2) son menores que 10, por lo cual podemos continuar. Al llegar a la tercera posición ($i = 2$), encontramos que debemos mover *todos* los datos mayores que 5 (el número 10) una posición a la derecha, y luego insertar el número 5 en el espacio que abrimos (Ver Figura 6.7).

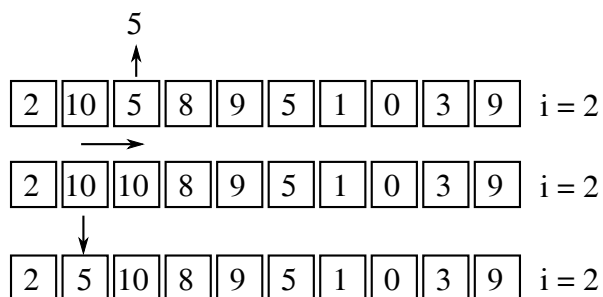


Figura 6.7: Segunda iteración de *insertion sort*

Observe que al mover los elementos hacia la derecha, se *sobreescribe* el valor del elemento en la posición que estamos verificando. Por esta razón se debe almacenar en una variable auxiliar, la cual se usará además para comparar con los elementos a la izquierda hasta encontrar el sitio en el cual debemos abrir el espacio. Cuando hemos terminado de mover todos los elementos mayores a la derecha, colocamos en el espacio correspondiente el valor almacenado en la variable auxiliar.

Al revisar la cuarta posición, se debe mover de nuevo el número 10 a la derecha para abrir un espacio en el cual se insertará el número 8 como se muestra en la Figura 6.8.

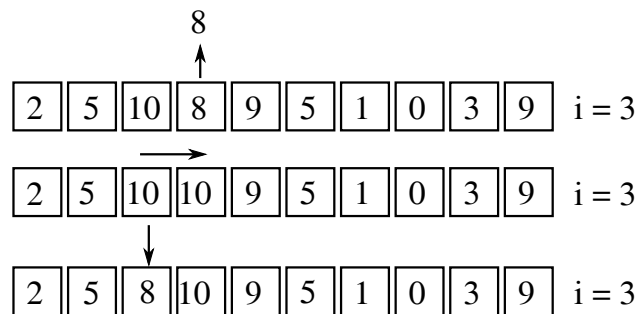


Figura 6.8: Tercera iteración de *insertion sort*

Luego se mueve de nuevo el número 10 a la derecha, y se inserta el número 9 en el espacio abierto. La siguiente iteración (Figura 6.9) es más interesante, debido a que se mueven tres datos a la derecha para abrir espacio al número 5.

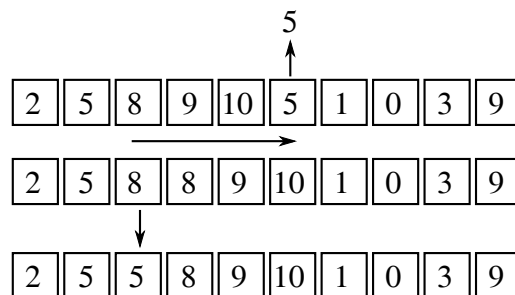


Figura 6.9: Quinta iteración de *insertion sort*

En este paso se puede ver la mejora que representa la inserción con respecto a la selección. No se debe recorrer el arreglo para buscar el menor elemento. En su lugar, sólo debemos retroceder en el arreglo *mientras* los elementos revisados sean mayores que el dato que estamos verificando.

Implementación del algoritmo

El Código 6.2 presenta la implementación del algoritmo *insertion sort*. Al igual que el algoritmo de selección, también se usan dos estructuras repetitivas anidadas. La estructura externa recorre el arreglo de izquierda a derecha, y la estructura anidada realiza los movimientos necesarios para insertar el elemento i de forma ordenada en el arreglo.

Código 6.2: Algoritmo *insertion sort*

```

1  /* Ordena un arreglo de n elementos usando insertion sort */
2  funcion ordenarInsercion(entero a[], entero n)
3  entero i, entero j, entero aux
4  inicio
5      para i = 1, i < n, i = i + 1
6          aux = a[i] //Almacenar el elemento actual
7
8          //Desplazar los mayores que se encuentren a la izquierda
9          j = i
10         mientras j > 0 y a[j - 1] > aux
11             a[j] = a[j - 1]
12             j = j - 1
13         fin mientras
14
15         //Insertar el elemento en su posición
16         a[j] = aux
17     fin para
18 fin funcion

```

Eficiencia

El algoritmo *insertion sort* también se basa en dos estructuras repetitivas anidadas, por lo cual es de esperar que para n datos en el peor de los casos se realicen aproximadamente $\frac{n^2}{2}$ comparaciones. Sin embargo, en Sedgewick and Wayne (2011) se demuestra que en el mejor de los casos, este algoritmo realiza $n - 1$ comparaciones y 0 intercambios cuando los datos se encuentran ordenados. Lo anterior representa una mejora con respecto al algoritmo de selección, en el cual siempre se realizarán $\frac{n^2}{2}$ comparaciones sin importar si los datos se encuentran ordenados o no.

6.3. Shellsort

Este algoritmo inventado por Shell (1959) se basa en el método de ordenamiento por inserción presentado anteriormente. Si recordamos, en *insertion sort* el elemento que estábamos revisando se movía hacia la izquierda buscando su posición en el arreglo, comparando sucesivamente con sus anteriores. *Shellsort* se basa en el mismo concepto, pero en

lugar de mover el elemento una posición hacia la izquierda cada vez, se define un *paso* de salto que se va disminuyendo sucesivamente.

Por ejemplo, si tomamos un paso de salto de $\frac{n}{3}$, el elemento en la posición i se comparará con el elemento almacenado en la posición $i - \frac{n}{3}$. Si es menor, el elemento *saltará* a esa posición. Comenzamos con un paso de salto grande, y en cada iteración se disminuye el paso hasta llegar a 1. Eso significa que en la última iteración, se comporta exactamente como *insertion sort*. Para ese entonces el arreglo se encontrará casi ordenado, gracias a las iteraciones anteriores.

Sin embargo, hasta el momento no se ha encontrado la secuencia de pasos de salto que permita un desempeño óptimo del algoritmo. En Sedgewick and Wayne (2011) se propone un algoritmo adaptado en el Código 6.3 para calcularlo:

Código 6.3: Paso de salto para *shellsort*

```
1 h = 1
2 mientras h < n/3
3   h = 3 * h + 1
4 fin mientras
5 mientras h > 0
6   //Insertion sort de h a n
7   h = h / 3
8 fin mientras
```

Con el algoritmo presentado, se calcula la secuencia 1, 4, 13, 40, 121, etc., hasta llegar al primer valor de h por encima de $\frac{n}{3}$. Después se aplica repetidamente *insertion sort*, en los elementos ubicados desde h hasta n . Si se encuentra algún elemento menor, se mueve h posiciones hacia atrás.

Luego se divide h en 3, y se repite el proceso hasta llegar a $h = 1$. En este momento se aplicará *insertion sort*, pero el arreglo se encontrará casi ordenado. El mejor comportamiento de *shellsort* se hace a medida que n aumenta, debido a que los elementos que se deben mover *saltan* distancias mayores.

Veamos el funcionamiento de *shellsort* con nuestros datos de prueba. En este caso, $n = 10$, por lo cual se usará la secuencia de pasos de salto será 4, 1. En la primera pasada (con $h = 4$) se revisarán los elementos desde 4 hasta 9, y se moverán los números 5, 1, 0, 3 y 9 como se muestra en la Figura 6.10.

En la siguiente pasada con $h = 1$, básicamente se realiza *insertion sort* sobre el arreglo. Se deja como ejercicio analizar los movimientos que se realizan hasta obtener el arreglo ordenado.

La Tabla 6.2 muestra la secuencia de pasos de salto para valores grandes de n .

h = 4

2	10	5	8	9	5	1	0	3	9	
2	10	5	8	9	5	1	0	3	9	9 no salta
2	5	5	8	9	10	1	0	3	9	5 salta 4 posiciones
2	5	1	8	9	10	5	0	3	9	1 salta 4 posiciones
2	5	1	0	9	10	5	8	3	9	0 salta 4 posiciones
2	5	1	0	3	10	5	8	9	9	3 salta 4 posiciones
2	5	1	0	3	9	5	8	9	10	9 salta 4 posiciones

Figura 6.10: Primera pasada de *shellsort*

Tabla 6.2: Pasos de salto de shellsort para diferentes valores de n

n	Paso inicial	Secuencia de pasos de salto
100	40	40, 13, 4, 1
1000	364	364, 121, 40, 13, 4, 1
10000	9841	9841, 3280, 1093, 364, 121, 40, 13, 4, 1
100000	88573	88573, 29524, 9841, 3280, 1093, 364, ..., 4, 1

Implementación del algoritmo

El Código 6.4 muestra una implementación de *shellsort* construida a partir del programa en Java que se presenta en Sedgewick and Wayne (2011).

Código 6.4: Pseudocódigo de shellsort

```

1  /* Ordena un arreglo de n datos usando shellsort */
2  funcion shellsort
3  entero i, entero j, entero k, entero h, entero aux
4  inicio
5    //Calcular el paso de salto inicial
6    h = 1
7    mientras h <= n/3
8      h = (3 * h) + 1
9    fin mientras
10   mientras h > 0
11     //Insertion sort de h a n, con salto h
12     para i = h, i < n, i = i + 1
13       aux = a[i]
14       j = i
15       mientras j >= h y a[j - h] > aux
16         a[j] = a[j - h]
17         j = j - h
18     fin mientras

```

```
19     a[j] = aux
20     fin para
21     h = h / 3 //Disminuir el paso de salto
22     fin mientras
23 fin funcion
```

Eficiencia

A primera vista *shellsort* no parece ser muy diferente de *insertion sort*. De hecho, ahora se tienen tres estructuras repetitivas anidadas en vez de dos, por lo cual el tiempo de ejecución debería incrementarse. Sin embargo, el ciclo externo es la fortaleza del algoritmo. A medida que el tamaño del arreglo crece su eficiencia mejora, ya que precisamente el ciclo externo nos permite que los elementos *salten* una mayor cantidad de posiciones en vez de moverlos una posición a la izquierda. Cuando llegamos al último paso de salto ($h = 1$), el arreglo se encontrará casi ordenado, por lo cual no será necesario desplazar muchos elementos.

6.4. Ordenamiento por mezcla (Mergesort)

Según Knuth (1998), este algoritmo propuesto por John von Newmann en los años 40 fue uno de los primeros algoritmos creados para realizar ordenamiento por de datos por computador. La idea general es la siguiente: Si se tienen dos secuencias *ordenadas* de datos, es posible *mezclarlas* para obtener una nueva secuencia en la cual los datos *también se encuentran ordenados*. Para ello, se toma sucesivamente el elemento más pequeño que se encuentre en las dos subsecuencias y se lleva al final de la nueva secuencia, que se encuentra inicialmente vacía. Si se terminan los elementos de alguna de las dos secuencias, los elementos restantes de la otra se colocan al final de la secuencia de salida.

Al final tendremos una nueva secuencia, con los datos *combinados* de las dos secuencias de entrada, como se muestra en la Figura 6.11.

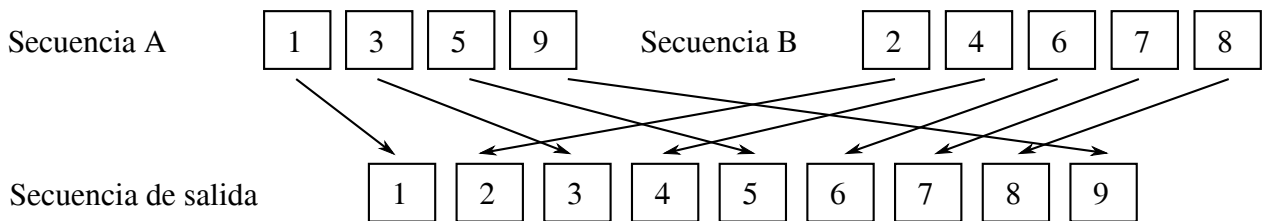


Figura 6.11: Mezcla de dos secuencias ordenadas

El Código 6.5 implementa una función que permite mezclar dos secuencias ordenadas.

Código 6.5: Mezcla de dos secuencias ordenadas

```

1  /*
2  * Crea una secuencia ordenada de n + m elementos
3  * a partir de dos secuencias ordenadas a y b
4  * de n y m elementos, respectivamente
5  */
6  funcion mezclar(
7      entero r[], //Secuencia resultado, inicialmente vacía
8      entero a[], //Primera secuencia
9      entero n, //Tamaño de la primera secuencia
10     entero b[] //Segunda secuencia
11     entero m //Tamaño de la segunda secuencia
12 )
13 entero i, entero j, entero k
14 inicio
15     i = 0
16     j = 0
17     k = 0
18
19     //Tomar el menor elemento de a ó b y ponerlo en r
20     mientras i < n y j < m
21         si a[i] < b[j]
22             r[k] = a[i]
23             i = i + 1
24         sino
25             r[k] = b[j]
26             j = j + 1
27         fin si
28         k = k + 1
29     fin mientras
30
31     /*
32     * El ciclo anterior se termina cuando se agotan
33     * los elementos de a y/o los elementos de b
34     */
35     //Si quedan elementos en a, ponerlos al final de r
36     mientras i < n
37         r[k] = a[i]
38         i = i + 1
39         k = k + 1
40     fin mientras
41
42     //Si quedan elementos en b, ponerlos al final de r
43     mientras j < m
44         r[k] = b[j]
45         j = j + 1
46         k = k + 1
47     fin mientras
48 fin funcion

```

Ahora supongamos que las dos secuencias ordenadas que deseamos mezclar se encuentran contiguas (una tras otra) en un arreglo, y que la mezcla se almacene en el mismo arreglo, como se muestra en la Figura 6.12.

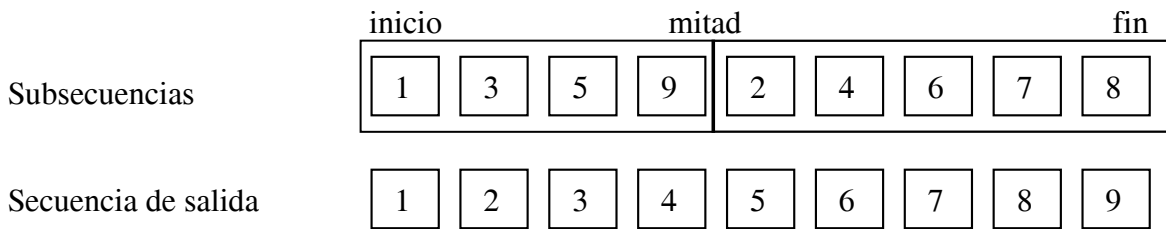


Figura 6.12: Mezcla de dos subsecuencias

En este caso es conveniente copiar todos los datos a un arreglo *auxiliar*. Dado que conocemos las posiciones iniciales y finales de cada secuencia, podemos cambiar nuestra función para mezclar las dos secuencias y almacenarlas en el mismo arreglo (Ver Código 6.6).

Código 6.6: Mezcla de subsecuencias en el mismo arreglo

```

1  /*
2  * Mezcla dos subsecuencias contiguas en el mismo arreglo, y almacena la
3  * secuencia ordenada a partir de la posición en la que se
4  * encontraba la primera subsecuencia
5  *
6  */
7  funcion mezclar(
8      entero a[], //Arreglo
9      entero posInicial, //Inicio de la primera subsecuencia
10     entero mid, //Fin de la primera e inicio de la segunda subsecuencia
11     entero posFinal //Fin de la segunda subsecuencia
12 )
13     entero aux[] //Arreglo auxiliar
14     entero i, entero j, entero k
15     inicio
16
17     //Copiar las subsecuencias en un arreglo auxiliar
18     para i = posInicial, i <= posFinal, i = i + 1
19         aux[i] = a[i]
20     fin para
21
22     i = posInicial //Inicio de la primera subsecuencia
23     j = mid + 1 //Inicio de la segunda subsecuencia
24     k = posInicial //Posición para almacenar la secuencia resultante
25
26     //Mezclar subsecuencias
27     mientras i <= mid y j <= posFinal
28         si aux[i] < aux[j]
29             a[k] = aux[i]
30             i = i + 1
31         sino
32             a[k] = aux[j]
33             j = j + 1
34         fin si
35         k = k + 1
36     fin mientras

```

```

37
38 //Copiar los elementos restantes, si existen
39
40 mientras i <= mid
41   a[k] = aux[i]
42   i = i + 1
43   k = k + 1
44 fin mientras
45
46 mientras j <= posFinal
47   a[k] = aux[j]
48   j = j + 1
49   k = k + 1
50 fin mientras
51 fin funcion

```

Observe que en este algoritmo se debe cumplir que *inicio* ≤ *mid* ≤ *fin*. Es decir, las dos subsecuencias deben ser contiguas y la posición *mid* debe encontrarse entre *inicio* y *fin*, pero no necesariamente en la mitad.

Implementación del algoritmo

La implementación de *mergesort* sobre un arreglo de *n* elementos se basa en construir dos subsecuencias ordenadas contiguas, y luego mezclarlas dentro del mismo arreglo usando la función *mezclar* definida anteriormente.

El ordenamiento se puede implementar como una función recursiva, la cual recibe como parámetro las posiciones iniciales y finales del arreglo a ordenar (Ver Código 6.7). Esta función toma las posiciones inicial y final, y calcula la posición media. Luego, se invoca a sí misma para ordenar el arreglo entre la posición inicial y la posición media, y de nuevo para ordenar el arreglo entre la posición media y la posición final. Cuando se han creado estas dos subsecuencias, invoca la función *mezclar*, que se encarga de unir las subsecuencias construidas para crear la secuencia ordenada entre la posición inicial y la posición final.

Código 6.7: Pseudocódigo recursivo para mergesort

```

1 /*
2  * Ordena un arreglo de n elementos
3  */
4 funcion mergesort(entero a[], entero n)
5 inicio
6   ordenar(a, 0, n - 1)
7 fin funcion
8
9 /*
10 * Ordena y mezcla de forma recursiva un arreglo desde
11 * una posición inicial a una posición final
12 *

```

6. ORDENAMIENTO

```
13 */
14 funcion ordenar(
15     entero a[],
16     entero posInicial,
17     entero posFinal)
18     entero mid
19 inicio
20     //Caso base: la posición final es la misma posición inicial
21     si posFinal = posInicial
22         retornar
23     fin si
24
25     //Caso general: Ordenar de forma recursiva y mezcla
26     //Calcula la posición media
27     mid = posInicial + ((posFinal - posInicial)/2)
28
29     //Crea dos subsecuencias ordenadas contiguas de forma recursiva
30     ordenar(a, posInicial, mid) //Ordena desde posInicial hasta mid
31     ordenar(a, mid + 1, posFinal) //Ordena desde mid + 1 hasta posFinal
32
33     //Mezcla las subsecuencias creadas
34     mezclar(a, posInicial, mid, posFinal)
35 fin funcion
```

En este pseudocódigo la función *mergesort* es simplemente una *fachada* que realiza la primera llamada a *ordenar*. En otras palabras, esta función se encarga de *iniciar* el proceso recursivo de ordenamiento. Recordemos que cuando se invoca una función recursiva, esta se invoca a sí misma hasta llegar al caso base. En este momento, se *retorna* a la llamada anterior, y así sucesivamente hasta retornar al punto en el cual se realizó la primera llamada.

La primera llamada a *ordenar* divide el arreglo en dos partes contiguas entre la posición inicial y la posición final, y luego se invoca a sí misma dividiendo de nuevo el arreglo en dos partes contiguas y así sucesivamente, hasta que la posición inicial y la posición final sean iguales. En este momento se llega al caso base, debido a que el resultado de ordenar un solo elemento en su posición no requiere ninguna operación. Al retornar a la llamada anterior, se realiza la mezcla entre dos elementos contiguos, luego cuatro, etc., hasta crear la subsecuencia ordenada con los elementos de la primera mitad.

Luego se continúa ordenando la segunda mitad del arreglo original, mediante llamadas recursivas que ordenan secuencias contiguas de dos elementos contiguos, cuatro, etc., hasta completar la segunda subsecuencia. Finalmente, se mezclan la dos subsecuencias para obtener el arreglo completo ordenado.

Las funciones recursivas se definen e implementan con relativa facilidad, pero ofrecen una dificultad particular al momento de realizar el seguimiento de su ejecución. Veamos la secuencia de llamadas recursivas necesaria para ordenar nuestro arreglo de 10 elementos, en la cual se muestra las posiciones que reciben como parámetro las funciones *ordenar* y

mezclar.

Los números entre paréntesis representan el nivel de la llamada recursiva, siendo 0 la primera llamada que se realiza desde la función *mergesort* y los números de 1 a 4 las llamadas recursivas que realiza la función *ordenar*. Los números entre corchetes muestran la secuencia de llamadas que se realizan a la función *mezclar*, la que efectivamente lleva a cabo el ordenamiento mediante mezclas de las subsecuencias contiguas de 2, 4, hasta n elementos.

```
a = [ 2 10 5 8 9 5 1 0 3 9 ]
mergesort(a, 10)
(0) : ordenar 0 9
    (1) : ordenar 0 4
        (2) : ordenar 0 2
            (3) : ordenar 0 1
                (4) : ordenar 0 0 : Caso base 0=0
                (4) : ordenar 1 1 : Caso base 1=1
                (4) : mezclar 0 0 1 [1]
                    [ 2 10 5 8 9 5 1 0 3 9 ]
            (3) : ordenar 2 2 : Caso base 2=2
            (3) : mezclar 0 1 2 [2]
                [ 2 5 10 8 9 5 1 0 3 9 ]
        (2) : ordenar 3 4
            (3) : ordenar 3 3
            (3) : Caso base 3=3
            (3) : ordenar 4 4
            (3) : Caso base 4=4
            (3) : mezclar 3 3 4 [3]
                [ 2 5 10 8 9 5 1 0 3 9 ]
        (2) : mezclar 0 2 4 [4]
            [ 2 5 8 9 10 5 1 0 3 9 ]
    (1) : ordenar 5 9
        (2) : ordenar 5 7
            (3) : ordenar 5 6
                (4) : ordenar 5 5
                (4) : Caso base 5=5
                (4) : ordenar 6 6
                (4) : Caso base 6=6
                (4) : mezclar 5 5 6 [5]
                    [ 2 5 8 9 10 1 5 0 3 9 ]
            (3) : ordenar 7 7 : Caso base 7=7
            (3) : mezclar 5 6 7 [6]
```

6. ORDENAMIENTO

```
          [ 2 5 8 9 10 0 1 5 3 9 ]
(2) : ordenar 8 9
      (3) : ordenar 8 8
      (3) : Caso base 8=8
      (3) : ordenar 9 9
      (3) : Caso base 9=9
      (3) : mezclar 8 8 9   [7]
          [ 2 5 8 9 10 0 1 5 3 9 ]
(2) : mezclar 5 7 9 [8]
      [ 2 5 8 9 10 0 1 3 5 9 ]
(1) : mezclar 0 4 9 [9]
      [ 0 1 2 3 5 5 8 9 9 10 ]
a = [ 0 1 2 3 5 5 8 9 9 10 ]
```

Una observación cuidadosa de la secuencia de mezclas realizadas nos permite crear una versión iterativa (no recursiva) de *mergesort*. Primero podemos realizar todas las mezclas de dos elementos contiguos, luego de cuatro, etc., hasta llegar a una mezcla de n elementos.

El Código 6.8 muestra la implementación del algoritmo iterativo, adaptado del código en Java mostrado en Sedgewick and Wayne (2011).

Código 6.8: Mergesort iterativo

```
1 mergesortIterativo(entero a[], entero n)
2     entero paso
3     entero posInicial, entero posFinal, entero mid
4 inicio
5     //Realizar mezclas sucesivas de 2, 4, ... hasta n elementos
6     paso = 1
7     mientras (paso < n)
8         posInicial = 0
9         posFinal = posInicial + (paso * 2) - 1
10        //Ajustar al fin del arreglo
11        si posFinal > n - 1
12            posFinal = n - 1
13        fin si
14        mientras posInicial < n - paso
15            mid = posInicial + paso - 1
16            mezclar(a, posInicial, mid, posFinal)
17            posInicial = posInicial + (paso * 2)
18            posFinal = posInicial + (paso * 2) - 1
19            //Ajustar al fin del arreglo
20            si posFinal > n - 1
21                posFinal = n - 1
22            fin si
23        fin mientras
24        //Duplicar el paso
25        paso = paso * 2
26    fin mientras
27 fin funcion
```

A continuación se muestra la secuencia de mezclas que se realizan en la versión iterativa, para nuestro arreglo de prueba de 10 elementos.

```
a = [ 2 10 5 8 9 5 1 0 3 9 ]

mezclar 0 0 1
[ 2 10 5 8 9 5 1 0 3 9 ]
mezclar 2 2 3
[ 2 10 5 8 9 5 1 0 3 9 ]
mezclar 4 4 5
[ 2 10 5 8 5 9 1 0 3 9 ]
mezclar 6 6 7
[ 2 10 5 8 5 9 0 1 3 9 ]
mezclar 8 8 9
[ 2 10 5 8 5 9 0 1 3 9 ]
mezclar 0 1 3
[ 2 5 8 10 5 9 0 1 3 9 ]
mezclar 4 5 7
[ 2 5 8 10 0 1 5 9 3 9 ]
mezclar 0 3 7
[ 0 1 2 5 5 8 9 10 3 9 ]
mezclar 0 7 9
[ 0 1 2 3 5 5 8 9 9 10 ]

a = [ 0 1 2 3 5 5 8 9 9 10 ]
```

Eficiencia

Mergesort es un algoritmo muy eficiente, y permite organizar N datos en un tiempo proporcional a $N * \log N$ (Sedgewick and Wayne, 2011). Sin embargo, tiene como desventaja que requiere un arreglo auxiliar de N elementos para almacenar los elementos mientras se realiza la mezcla. Si el número de datos no es tan grande, o si el límite de memoria no es muy bajo, no existe problemas en usarlo.

Una forma de optimizar *mergesort* es cambiar el método de ordenamiento cuando el tamaño de la subsecuencia es pequeño (15 o menos elementos), en cuyo caso es mejor usar el ordenamiento por inserción o selección. En pseudocódigo esto equivale a modificar la función *ordenar* para verificar el tamaño de la secuencia. En caso de ser menor igual a 15, se invocará *insertion sort*, y en caso contrario se realizará la llamada recursiva.

6.5. Ordenamiento rápido (Quicksort)

El algoritmo de ordenamiento rápido (Hoare, 1962) se encuentra entre los más usados en la actualidad. Es uno de los muchos aportes de su autor a la computación, los cuales le han llevado a recibir diferentes reconocimientos a lo largo de su vida (Jones, 2012).

Quicksort es similar a *mergesort* en el sentido de dividir el arreglo y ordenar sus partes, estrategia que se conoce como *divide y vencerás*. Sin embargo, se usa una aproximación diferente: En vez de ordenar y fusionar las partes, *quicksort* primero busca y ubica un elemento (llamado *pivote*) en su posición, y luego ordena de forma recursiva los elementos ubicados a la izquierda y derecha del *pivote*.

¿Qué significa que un elemento se encuentre en su posición?. Un elemento k dentro de un arreglo de tamaño n se encontrará en su posición si todos los elementos en las posiciones $0 \leq i < k$ son menores que él y todos los elementos en las posiciones $k < j < n$ son mayores o iguales, sin importar si se encuentran ordenados. Este concepto se ilustra en la Figura 6.13.

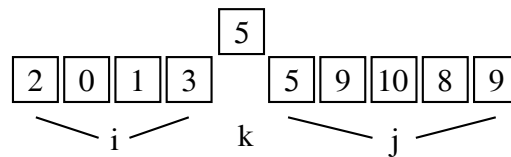


Figura 6.13: Pivote en *quicksort*

Entonces el problema de ordenar un arreglo se reduce a buscar y ubicar un elemento en su posición, y usarlo como *pivote* para ordenar de forma recursiva la lista de elementos a su izquierda y su derecha. Este proceso es llamado *particionar* o *porcionar* el arreglo. Inicialmente se ubicará el primer pivote (dato) en su posición, y de forma recursiva se genera un nuevo pivote en la izquierda y se ubicarán los demás elementos. Luego, se realizará el mismo proceso en la lista derecha: se ubica un pivote, y se ordenan los elementos de su izquierda y su derecha. Las llamadas recursivas terminarán si la sub-lista de elementos a ordenar se encuentra vacía o tiene un solo elemento.

A pesar de la aparente sencillez del planteamiento, implementar la lógica necesaria para buscar y ubicar *el mejor* pivote no es tan fácil como parece. Debemos tener en cuenta que se desea *ubicar* un elemento en su posición, pero esta no es conocida de antemano. Es decir, no se trata de *seleccionar* el elemento que se encuentra en la mitad del arreglo sin ordenar, sino que quisiéramos *encontrar* el elemento que *debería* estar ubicado en la mitad del arreglo cuando los datos estén ordenados.

Debido a que el proceso de ubicar el pivote puede complicarse a tal grado que afecte el desempeño del algoritmo, el autor en su artículo original propone seleccionar un elemento que estará dentro de la lista a ordenar. Un método sencillo consiste en usar cualquier ele-

mento en la lista (el primero, el elemento del medio o el último, o una posición aleatoria). Después de elegir el pivote se tratará de encontrar la posición que le corresponde en del arreglo ordenado.

El primer paso para encontrar la posición en la cual debería ubicarse el pivote es el siguiente: se usan dos índices o referencias para apuntar al inicio y al final de la lista a ordenar, que llamaremos i e j , respectivamente. En caso de usar el primer elemento como pivote, podemos avanzar i a la siguiente posición del arreglo.

El índice i se avanza desde el inicio de la lista hacia el final, tratando de ubicar el primer elemento *mayor* que el pivote (en otras palabras, se avanza el índice i *mientras* el elemento en la posición i sea menor o igual que el pivote). Ahora es el turno del índice j . Lo movemos hacia atrás, hasta encontrar el primer elemento almacenado en la posición i que sea *menor* o igual que el pivote.

En la Figura 6.14 se muestra cómo se mueven estos dos índices para la lista de datos de prueba usada hasta ahora, **tomando como pivote el primer elemento del arreglo**.

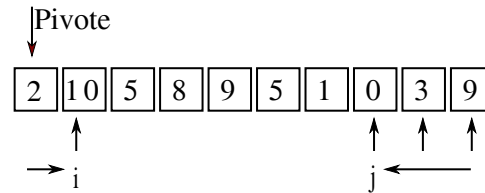


Figura 6.14: Movimientos de los índices en la partición

Observe detenidamente la relación entre los elementos apuntados por los índices i e j . El número 0 se encuentra *a la derecha* del número 10 ($j > i$). Pero 0 *es menor* que 10 ($a[j] < a[i]$), por lo cual debería estar *a su izquierda* (aunque no necesariamente a su lado) en el arreglo ordenado. Debemos entonces *intercambiar* los valores almacenados en las posiciones i e j para *acercarlos a sus posiciones finales*, como se muestra en la Figura 6.15.

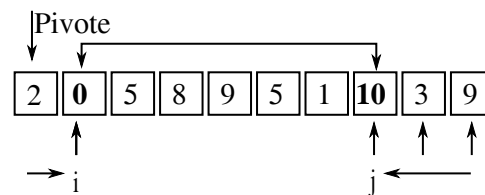


Figura 6.15: Intercambio de valores en la partición

Ahora observe la relación entre los elementos intercambiados y el pivote. El elemento usado como pivote *deberá* ubicarse en alguna posición intermedia entre el número en la posición i y el número en la posición j (0 y 10, respectivamente). Lo podríamos colocar *justo a la derecha* del número 0, o *justo a la izquierda* del número 10, y estaría ordenado

con respecto a ellos. Sin embargo, podemos aprovechar esta *pasada* para seguir buscando números *mayores* y *menores* que el pivote (el número 2), avanzando de nuevo el índice i hacia adelante tratando de ubicar otro dato mayor que el pivote, y el índice j tratando de ubicar otro dato menor que el pivote como se muestra en la Figura 6.16.

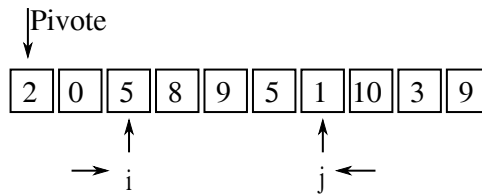


Figura 6.16: Siguiendo movimiento de los índices en la partición

En este caso sólo movimos el índice i una posición a la derecha, y el índice j una posición a la izquierda. Dependiendo de los datos, es posible que un índice se mueva más que otro, o que alguno no se mueva. De nuevo, intercambiamos los datos encontrados, para acercarlos a su posición final en el arreglo ordenado como se aprecia en la Figura 6.17.

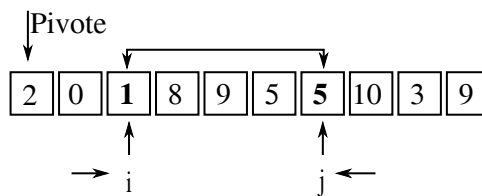


Figura 6.17: Siguiendo intercambio de valores en la partición

Si continuamos avanzando el índice i hacia adelante y el índice j hacia atrás realizando los intercambios correspondientes, llegará un momento en que los índices i y j *se cruzarán*, es decir, se cumplirá que $j < i$. En otra feliz coincidencia, esto sucede en la siguiente repetición del proceso que se está llevando a cabo con los datos de prueba, lo cual se muestra en la Figura 6.18.

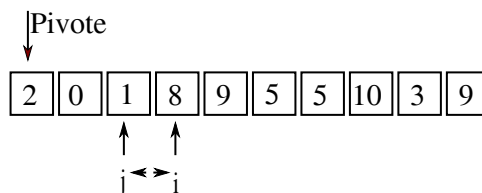


Figura 6.18: Cruce de índices en la partición

Observe además que en este caso se cumple lo mencionado anteriormente con respecto al movimiento de los índices: el índice i se movió una posición hacia la derecha, mientras que el índice j se movió 4 posiciones hacia la izquierda.

Analicemos con más detalle la Figura 6.18. Ya hemos realizado todos los intercambios posibles en relación con el pivote actual (el número 2). Desde la posición inicial del arreglo hasta la posición j se encuentran los elementos *menores o iguales* que el pivote, y a partir de la posición i se encuentran los elementos *mayores* que el mismo. Ahora bien, ¿a qué posición debemos *mover* el pivote (el número 2), de forma que todos los elementos a su izquierda sean menores y todos los elementos a su derecha sean mayores o iguales?

Una alternativa salta a la vista: usando la idea uno de los algoritmos de ordenamiento presentados anteriormente, podemos *mover* una posición hacia la izquierda los elementos *menores* que el pivote, y ubicar al pivote en el espacio abierto como se ilustra en la Figura 6.19.

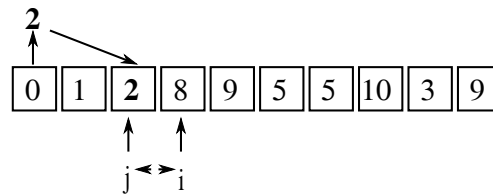


Figura 6.19: Mover pivote a su posición

Sin embargo esta alternativa no es óptima, debido a que si los elementos menores que el pivote son muchos, se tienen que realizar muchas operaciones para mover cada elemento una posición hacia la izquierda.

Existe una mejor alternativa, que consiste en *intercambiar* el pivote con el elemento que se encuentra en la posición j . Por definición, este es *el primer elemento menor que el pivote de derecha a izquierda*, y también *el último elemento menor que el pivote de izquierda a derecha*. Entonces, podemos intercambiarlo con el pivote, lo cual siempre implicará mover solo dos elementos de su posición, sin importar el número de datos intermedios. Este movimiento se muestra en la Figura 6.20.

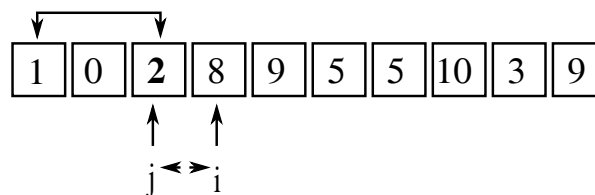


Figura 6.20: Mover pivote a su posición con un intercambio

Ahora tenemos un arreglo en el cual todos los elementos a la izquierda del número 2 almacenado en la tercera posición son menores que él, y todos los elementos a su derecha son mayores o iguales. Usando el proceso de particionado descrito anteriormente, podemos ordenar de forma recursiva el arreglo de dos elementos ubicado a la izquierda del

número, y el arreglo de siete elementos ubicado a su derecha. Esto se deja como ejercicio para el lector.

Implementación del algoritmo

La clave del algoritmo *quicksort* es la función de particionado, la cual se encarga de buscar y ubicar un pivote en su posición. Si bien existen varias implementaciones en diferentes textos, una de las más simples es la presentada por Brassard and Bratley (2006). El Código 6.9 presenta la implementación de la función *particionar* de *quicksort*, la cual se basa en el pseudocódigo que presenta el autor mencionado.

Código 6.9: Función de particionado para quicksort

```
1 /*
2  * Ubica un elemento en su posición (pivote), los
3  * elementos menores que el pivote a la izquierda y los
4  * elementos mayores o iguales a la derecha
5  * Retorna la posición en la cual se ubicó el pivote
6  */
7 funcion particionar(entero a[],
8                     entero posInicial,
9                     entero posFinal): entero
10 entero i, entero j, entero x, entero tmp
11 inicio
12     //Tomar el primer elemento como pivote
13     x = a[posInicial]
14     i = posInicial
15     j = posFinal
16     //Buscar el primer elemento mayor que x
17     mientras i < posFinal y a[i] <= x
18         i = i + 1
19     fin mientras
20
21     //Buscar el primer elemento menor o igual que x
22     mientras a[j] > x
23         j = j - 1
24     fin mientras
25
26     //Intercambiar y avanzar, hasta que haya cruce
27     mientras i < j
28         tmp = a[i]
29         a[i] = a[j]
30         a[j] = tmp
31
32     //Avanzar una posición a la izquierda y la derecha
33     //Y buscar posibles intercambios con respecto al pivote
34
35     i = i + 1
36     j = j - 1
37
38     //Buscar el siguiente elemento mayor que x
```

```

39  mientras a[i] <= x
40      i = i + 1
41  fin mientras
42
43  //Buscar el siguiente elemento menor o igual que x
44  mientras a[j] > x
45      j = j + 1
46  fin mientras
47  fin mientras
48  /*
49   * Ubicar el pivote en su posición (j), intercambiándolo con
50   * el elemento que se encuentra en esa posición
51   */
52  tmp = a[posInicial]
53  a[posInicial] = a[j]
54  a[j] = tmp
55
56  //Retornar la ubicación del pivote (j)
57  retornar j
58 fin funcion

```

Ahora definiremos una función auxiliar llamada ordenar (ver Código 6.10), la cual se encarga de ubicar un pivote entre una posición inicial y una posición final, y organizar de forma recursiva los elementos a la izquierda y derecha del pivote. La función principal *quicksort* será una fachada que realiza la primera llamada a la función recursiva.

Código 6.10: Pseudocódigo para el algoritmo quicksort

```

1  /*
2   * Ordena en forma ascendente un arreglo de n elementos usando el algoritmo
3   * quicksort
4   */
5  funcion quicksort(entero a[], entero n)
6  inicio
7      //Primera llamada a la función recursiva
8      ordenar(a, 0, n-1)
9  fin funcion
10
11 /*
12 * Ordena de forma recursiva un arreglo entre una posición inicial
13 * y una posición final, empleando el método de particionado de quicksort.
14 */
15 funcion ordenar(entero a[], entero posInicial, entero posFinal)
16 entero p
17 inicio
18     //Los límites son válidos?
19     si posFinal > posInicial
20
21         //Ubicar y obtener el pivote
22         p = particionar(a, posInicial, posFinal)
23
24         //Ordenar de forma recursiva los elementos a la izquierda del pivote
25         ordenar(a, posInicial, p - 1)
26

```

```
27 //Ordenar de forma recursiva los elementos a la derecha del pivote
28 ordenar(a, p + 1, posFinal)
29 fin si
30 fin funcion
```

Eficiencia

Quicksort es considerado el algoritmo de ordenamiento general con mejor desempeño promedio ($N\log N$), y sigue siendo usado en la actualidad con bastante frecuencia. Sin embargo, posee algunas desventajas. Por ejemplo, si el pivote encontrado resulta estar en la primera posición de la lista, se realizará una llamada recursiva *vacía* sobre los elementos de la izquierda (ninguno) y otra llamada para ordenar los elementos a la derecha. Si el pivote resulta estar siempre en la primera (o en la última) posición, cada llamada recursiva solo reducirá el problema en un elemento, de forma similar a *selection sort* o *insertion sort*.

Otra desventaja de *quicksort* es que en su versión original intercambia dos valores iguales, lo cual, cuando el número de datos es grande y existe una gran cantidad de números repetidos, causa un decremento en su desempeño.

Quicksort ha sido estudiado por años y décadas, y buscando mejorar sus resultados se han creado otros algoritmos como el *heapsort*, el *bucketsort* y recientemente el *timsort*. Este último es usado para realizar algunas tareas que requieren ordenamiento en el sistema operativo Android, así como en los lenguajes Java y Python. También se han creado algoritmos que reúnen características de varios algoritmos, denominados *algoritmos híbridos*, y otros que aprovechan las posibilidades de la *programación en paralelo*. No obstante, *quicksort* sigue siendo uno de los algoritmos preferidos, debido a su rapidez y elegancia.

6.6. Ejercicios propuestos

1. Realizar la prueba de escritorio de los algoritmos presentados para ordenar las siguientes secuencias de números:

```
[ 0 2 5 3 1 7 9 11 20 2 ]
[ 3 1 4 1 5 9 2 6 5 3 5 8 9 ]
[ 1 7 4 0 9 4 8 8 2 4 ]
[ 13 19 9 5 12 8 7 4 11 2 6 21 ]
[ 12 11 10 9 8 7 6 5 4 3 2 1 ]
[ 1 2 3 4 5 6 7 8 9 10 ]
[ 5 5 5 5 5 5 5 5 5 5 ]
```

```
[ 12 8 3 14 9 18 11 16 19 6 17 20 10 7 13 5 15 2 1 4 ]  
[ -6 16 8 -4 5 17 2 15 7 -3 -8 1 -5 -7 11 19 13 ]  
[ 8 -6 0 4 3 12 -10 -9 6 14 13 20 ]
```

2. Proponer una mejora a *quicksort* en la cual no se realice el intercambio del elemento, cuando los datos almacenados en las dos posiciones son iguales.

Bibliografía

- ASCII (2015). Ascii table [en línea]. Disponible en: <<http://www.asciitable.com>>. Fecha de consulta: 23 Marzo 2015.
- Bell, D. (2005). *Software Engineering for students*. Pearson Education.
- Brassard, G. and Bratley, P. (2006). *Fundamentos de Algoritmia*. Pearson Education, Madrid, 2 edition.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2009). *Introduction to Algorithms, Third Edition*. The MIT Press.
- Hoare, C. A. R. (1962). Quicksort. *The Computer Journal*, 5(1):10–16.
- Jones, C. (2012). C. Anthony (“Tony”) R. Hoare. [en línea]. Disponible en: <<http://goo.gl/L5x1Eg>>. Fecha de consulta: 23 Marzo 2015.
- Joyanes Aguilar, L. (2008). *Fundamentos de programación*. McGraw-Hill Interamericana de España S.L., 4 edition.
- Joyanes Aguilar, L. and Zahonero, I. (2005). *Programación en C: metodología, algoritmos y estructuras de datos*. McGraw-Hill, 2 edition.
- Knuth, D. E. (1997). *The Art of Computer Programming, Vol. 1: Fundamental Algorithms*. Addison-Wesley Professional, Reading, Mass, 3 edition.
- Knuth, D. E. (1998). *The Art of Computer Programming, Vol. 3: Sorting and Searching*. Addison-Wesley Professional, Redwood City, CA, USA, 2 edition.
- Nassi, I. and Shneiderman, B. (1973). Flowchart techniques for structured programming. *SIGPLAN Not.*, 8(8):12–26.
- Nisan, N. and Schocken, S. (2005). *The Elements of Computing Systems: Building a Modern Computer from First Principles (History of Computing S.)*. The MIT Press.

Nisan, N. and Schocken, S. (2015). From NAND to Tetris: Building a modern from first principles [en línea]. Disponible en: <<http://www.nand2tetris.org/>>. Fecha de consulta: 23 Marzo 2015.

Oviedo F., A. (2004). *Diseño Estructurado de Algoritmos*.

Oviedo Regino, E. (2015). *Lógica de programación Orientada a Objetos*. Ecoe Ediciones, 1 edition.

RAE (2015). Algoritmo - Real Academia Española [en línea]. Disponible en: <<http://goo.gl/JYsJAS>>. Fecha de consulta: 23 Marzo 2015.

Sedgewick, R. and Wayne, K. (2011). *Algorithms, 4th Edition*. Addison-Wesley.

Shell, D. L. (1959). A high-speed sorting procedure. *Communications of the ACM*, 2(7):30–32.

Índice analítico

- algoritmo, 1
 - concepto, 1
 - de búsqueda, clasificación o selección, 3
 - de construcción o producción de datos, 2
 - de decisión, 2
 - de optimización, 4
 - prueba de, 18
 - representación de, 4
 - de transformación, 3
- arreglo
 - bidimensional, 37
 - unidimensional, 35
- condiciones, 41
 - compuestas, 43
 - operador and, 42
 - operador or, 42
- estructuras algorítmicas, 43
 - continuar, 63
 - de decisión, 44
 - mientras hacer, 50
 - anidadas, 58
 - cancelar, 60
 - de selección, 49
 - hacer mientras, 53
 - para hacer, 56
- terminar, 61
- instrucciones, 25
 - asignación, 28
 - de incremento y decremento, 31
 - de lectura de datos, 27
 - salida por pantalla, 26
- ordenamiento, 199
 - por inserción, 202
 - por mezcla, 208
 - por selección, 200
 - rápido, 216
 - shell, 205
- post - incremento, 53
- pre - incremento, 53
- programa de computador, 6
- referencias o apuntadores, 39
- subrutinas, 64
 - concepto, 65
 - estructura general, 66
 - paso de parámetros, 70
 - recursivas, 69
- tabla ascii, 35
- tipo compuesto de datos, 39
- tipos simples de datos, 33
 - arreglos, 35

cadena de caracteres, 38

caracteres, 34

tipos numéricos, 34

variable, 32